China Daheng Group, Inc. Beijing Image Vision Technology Branch

C SDK
Programmer's Guide

Version：V3.1.0
Date：2026-05-07

DAHENG IMAGING 大恒图像

Notice

All rights reserved. No parts of this manual may be used or reproduced, in any forms or by any means, without prior written permission of China Daheng Group, Inc. Beijing Image Vision Technology Branch.

The right is also reserved to modify or change any parts of this manual in the future without prior notification.

All other trademarks are the properties of their respective owners.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Web: www.daheng-imaging.com/en

Sales Email: isales@daheng-imaging.com

Sales Tel: +86 10 8282 8878-8081

Support Email: isupport@daheng-imaging.com

Contents

1. Camera's Work Flow 1
1.1. Overall working flow 1
1.2. Camera control flow 2
1.3. DQBuf acquisition flow (zero copy) 3
1.4. DQAllBufs acquisition flow (zero copy, Linux only) 4
1.5. Capture callback flow 5
1.6. Offline events get flow 6
1.7. Remote device events get flow 7
2. Programming Guide 8
2.1. Build Programming Environment 8
2.1.1. Windows 8
2.1.2. Linux 11
2.1.3. macOS 14
2.2. Quick Guide 14
2.2.1. Initialization and uninitialization GxIAPI runtime library 14
2.2.2. Enumerate cameras and get information 15
2.2.3. Get Interface Information 16
2.2.4. Get Interface Object 16
2.2.5. Configurate the camera IP address 16
2.2.6. Open and close the camera 17
2.2.7. Camera control function 18
2.2.8. DQBuf image acquisition 21
2.2.9. CallBack image acquisition 23
2.2.10. Reconnect 25
2.2.11. Properties Setting UI 29
2.3. Use the Gigabit Camera to Debug the Programs 32
3. Camera Function Attribute Specification 34
3.1. Device Control 34
3.1.1. Device Information 34
3.1.2. Device Control 35
3.2. Image Format 36
3.2.1. ROI 36
3.2.2. Image Resolution 38
3.2.3. Data Format 40
3.2.4. Test Images 42
3.2.5. Sensor Shutter Mode 43
3.3. Acquisition Control 43
3.3.1. Acquisition 43
3.3.2. Trigger 48
3.3.3. Exposure 51
3.3.4. Transfer Control 53
3.3.5. Acquisition Burst Mode 54
3.3.6. Frame Store Mechanism 55

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

3.3.7. Frame Rate Control 56
3.3.8. Exposure Mode 56
3.3.9. Exposure Overlap Time Max 57
3.3.10. Multi-gray Control (MGC) Mode 58
3.3.11. Frame Information 58
3.4. Digital I/O 62
3.4.1. Pin Control 62
3.4.2. The I/O Control of the USB2.0 Camera 63
3.5. Counter and Timer Control 65
3.5.1. Timer 65
3.5.2. Counter 66
3.6. Analog Control 67
3.6.1. Gain 67
3.6.2. Black Level 69
3.6.3. White Balance 70
3.7. Transport Layer Control 72
3.7.1. PayloadSize 72
3.7.2. IP Configuration 73
3.7.3. Estimate the Bandwidth 73
3.7.4. The Heartbeat Timeout Time of the Device 74
3.7.5. Packet Size 74
3.7.6. Packet Delay 75
3.7.7. Link Speed 75
3.8. Bandwidth Control 75
3.9. Event Control 76
3.10. LUT (Look-up Table) Control 77
3.11. User Configuration 78
3.12. Set Camera's IP Address 79
3.12.1. Configuration Static IP Address 79
3.12.2. Force IP 80
3.13. Other Functions 80
3.13.1. Auto Exposure/ Auto Gain Related Function 80
3.13.2. Dead Pixel Correction 84
3.13.3. ADCLevel (DigitalShift) 84
3.13.4. Blanking Control 86
3.13.5. User Data Encryption Area 87
3.13.6. User Data Area 87
3.13.7. Remove Parameter Limits 88
3.13.8. Flat Field Correction 89
4. Local Device Control 91
4.1. Related Attribute 91
4.2. Sample Code 91
4.3. Precautions 91
5. Flow Layer Control 92
5.1. Statistical Attribute 92

5.1.1. Related Attribute 92
5.1.2. Precautions 92
5.1.3. Sample Code 92
5.2. Control Parameters 92
5.2.1. Related Parameters 92
5.2.2. Precautions 94
5.2.3. Sample Code 94
6. Functions Affected by Camera Model 95
7. GxIAPI Library Definitions 96
7.1. Type 96
7.1.1. Data Type 96
7.1.2. Handle Type 96
7.1.3. Callback Function Type 97
7.2. Constant 97
7.2.1. GX_STATUS_LIST 97
7.2.2. GX_FRAME_STATUS 98
7.2.3. GX_ACCESS_MODE 99
7.2.4. GX_OPEN_MODE 99
7.2.5. GX_IP_CONFIGURE_MODE_LIST 99
7.2.6. GX_RESET_DEVICE_MODE 100
7.2.7. GX_TL_TYPE_LIST 100
7.2.8. GX_NODE_ACCESS_MODE 100
7.2.9. GX_DEVICE_CLASS 101
7.2.10. GX_LOG_TYPE_LIST 101
7.2.11. GX_WINDOWS_ID 102
7.2.12. GX_WINDOWS_SHOW_MODE 102
7.2.13. GX_CFA_METHOD 103
7.2.14. GX_IMAGE_FORMAT_TYPE 103
7.2.15. GX_VIDEO_FORMAT_TYPE 103
7.3. Structure 104
7.3.1. GX_OPEN_PARAM 104
7.3.2. GX_FRAME_DATA_EX 104
7.3.3. GX_REGISTER_STACK_ENTRY 105
7.3.4. GX_INTERFACE_INFO 106
7.3.5. GX_DEVICE_INFO 108
7.3.6. GX_INT_VALUE 111
7.3.7. GX_FLOAT_VALUE 111
7.3.8. GX_STRING_VALUE 112
7.3.9. GX_ENUM_VALUE 112
7.3.10. GX_GIGE_ACTION_COMMAND_RESULT 113
7.3.11. GX_SAVE_IMAGE_INFO 113
7.3.12. GX_RECORD_PARAM 114
7.4. Interfaces 114
7.4.1. Get SDK Version Information 114
7.4.2. Initialization and Cancellation 115

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

7.4.3. emStatus = GXGetDataStreamHandleFromDev (hDevice, 1, &hDS);Device Parameter Settings 126
7.4.4. Device Functional Interface 140
7.4.5. Device Acquisition Interface 153
7.4.6. Device Event Interface 173
7.4.7. Image and Video Saving Interface 187
7.5. Not recommended Definitions 192
7.5.1. Type 192
7.5.2. Constants 192
7.5.3. Structure 193
7.5.4. Interface 199
8. Image Processing Interface Description 239
8.1. Type 239
8.1.1. Data Type 239
8.2. Constant 239
8.2.1. Status code 239
8.2.2. Pixel Bayer format 240
8.2.3. Bayer conversion type 240
8.2.4. Valid data bit 241
8.2.5. The actual image bit depth 241
8.2.6. The image mirror and flip type 242
8.2.7. Color image RGB order 242
8.2.8. Image Format Conversion Handle 242
8.2.9. IMAGE_CONVERT_DECLARE_HANDLE(DX_IMAGE_FORMAT_CONVERT_HANDLE);System Parameter 242
8.2.10. Flat Field Correction Handle 243
8.2.11. Decompression Handle 243
8.3. Structure 243
8.3.1. Monochrome image process function set structure 243
8.3.2. Color image process function set structure 243
8.3.3. Flat field correction process function set structure 244
8.3.4. Color correction UserSet process function set struct 245
8.3.5. Static defect correction function set structure 245
8.3.6. Flat field correction function set structure 246
8.4. Interfaces 247
8.4.1. DxRaw12PackedToRaw16 247
8.4.2. DxRaw10PackedToRaw16 248
8.4.3. DxRaw8toRGB24 249
8.4.4. DxRaw8toRGB24Ex 250
8.4.5. DxRotate90CW8B 252
8.4.6. DxRotate90CCW8B 253
8.4.7. DxRotate90CW16B 254
8.4.8. DxRotate90CCW16B 255
8.4.9. DxBrightness 256
8.4.10. DxContrast 258
8.4.11. DxSharpen24B 260

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

8.4.12. DxSharpenMono8 262
8.4.13. DxSaturation 262
8.4.14. DxGetWhiteBalanceRatio 264
8.4.15. DxAutoRawDefectivePixelCorrect 266
8.4.16. DxRaw16toRaw8 267
8.4.17. DxRGB48toRGB24 268
8.4.18. DxRaw16toRGB48 270
8.4.19. DxRaw8toARGB32 271
8.4.20. DxGetContrastLut 272
8.4.21. DxGetGammatLut 274
8.4.22. DxImageImprovment 275
8.4.23. DxARGBImageImprovment 279
8.4.24. DxImageImprovmentEx 282
8.4.25. DxImageMirror 286
8.4.26. DxImageMirror16B 287
8.4.27. DxGetLut 288
8.4.28. DxCalcCCParam 289
8.4.29. DxRaw8ImgProcess 291
8.4.30. DxMono8ImgProcess 294
8.4.31. DxGetFFCCoefficients 296
8.4.32. DxFlatFieldCorrection 297
8.4.33. DxCalcUserSetCCParam 298
8.4.34. DxImageFormatConvert 300
8.4.35. DxImageFormatConvertCreate 304
8.4.36. DxImageFormatConvertDestroy 304
8.4.37. DxImageFormatConvertSetOutputPixelFormat 305
8.4.38. DxImageFormatConvertSetAlphaValue 306
8.4.39. DxImageFormatConvertSetInterpolationType 307
8.4.40. DxImageFormatConvertGetBufferSizeForConversion 308
8.4.41. DxImageFormatConvertGetOutputPixelFormat 309
8.4.42. DxImageFormatConvertSetValidBits 310
8.4.43. DxImageFormatConvertSet3DcalibParam 310
8.4.44. DxImageFormatConvertSetYStep 314
8.4.45. DxStaticDefectCorrection 314
8.4.46. DxCalcCameraLutBuffer 315
8.4.47. DxReadLutFile 316
8.4.48. DxSetSystem 318
8.4.49. DxGetSystem 318
8.4.50. DxFFCCalculate 318
8.4.51. DxFFCCreate 322
8.4.52. DxFFCDestroy 322
8.4.53. DxFFCGetCoefficientsSize 323
8.4.54. DxFFCSetFrameCount 324
8.4.55. DxDecompression 325
8.4.56. DxDecompressionCreate 327
8.4.57. DxDecompressionDestroy 327

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

8.5. Function.....328
8.5.1. Image Quality Enhancement Function 328
9. Revision History 337

1. Camera's Work Flow

1.1. Overall working flow

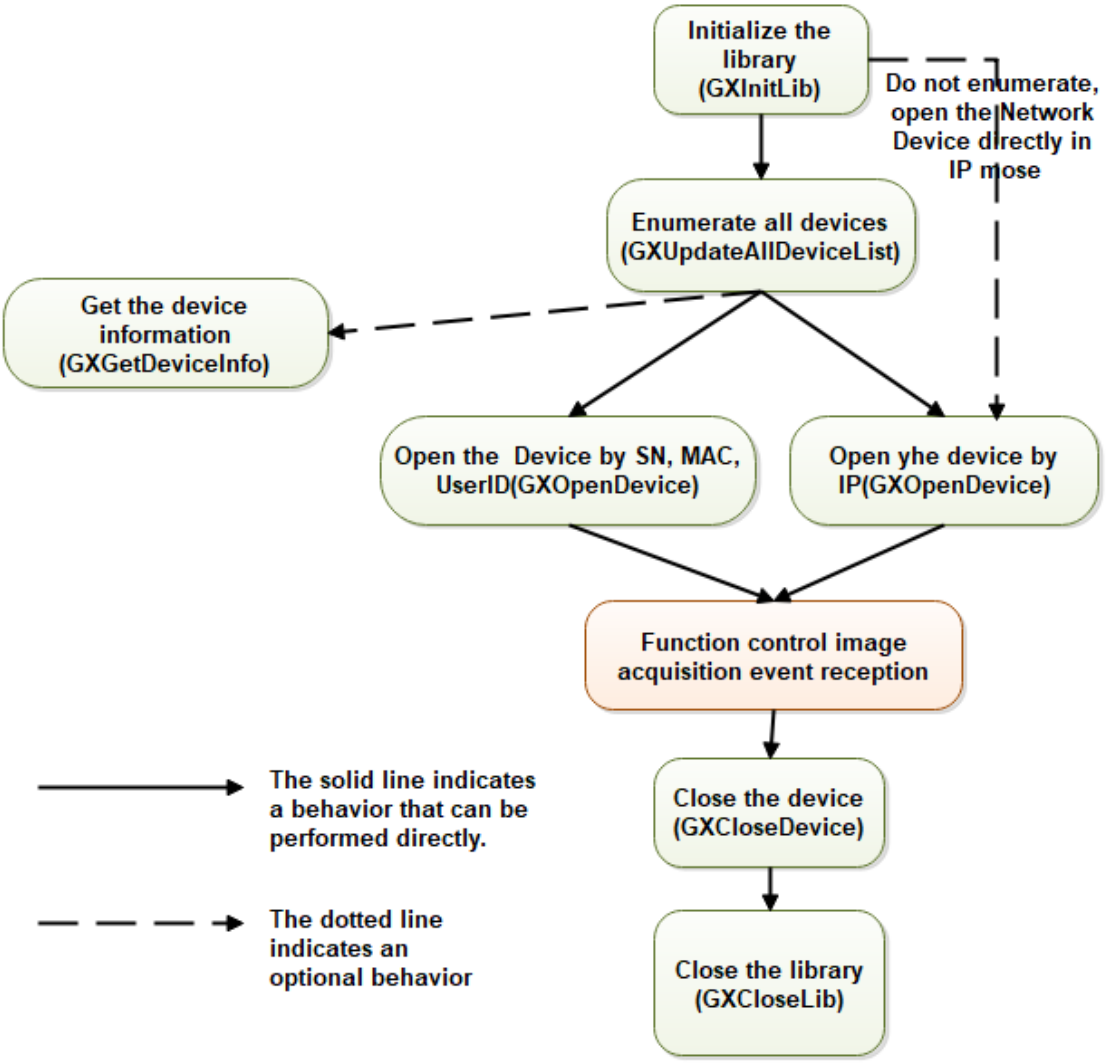


Figure 1-1 Overall working flow

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

1.2. Camera control flow

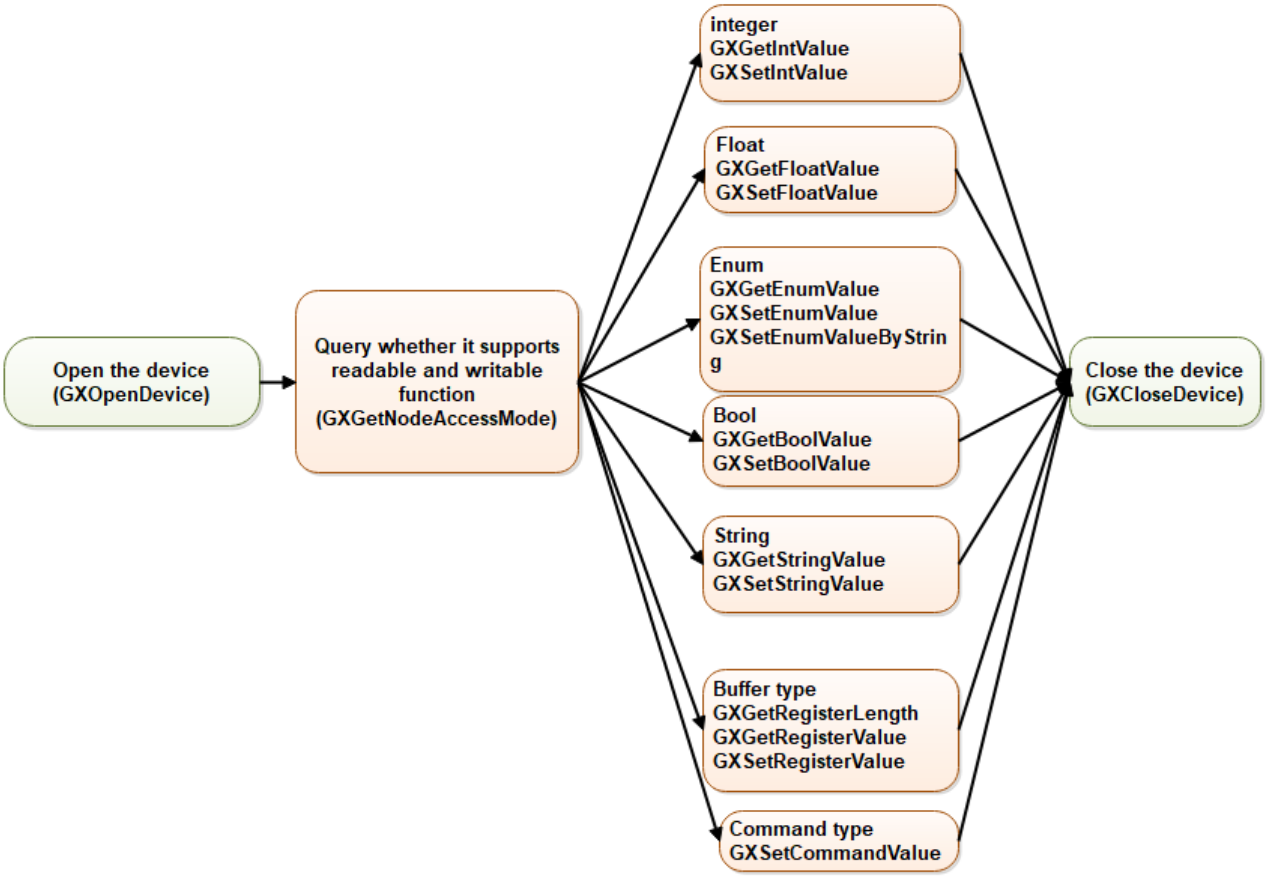


Figure 1-2 Camera control flow

1.3. DQBuf acquisition flow (zero copy)

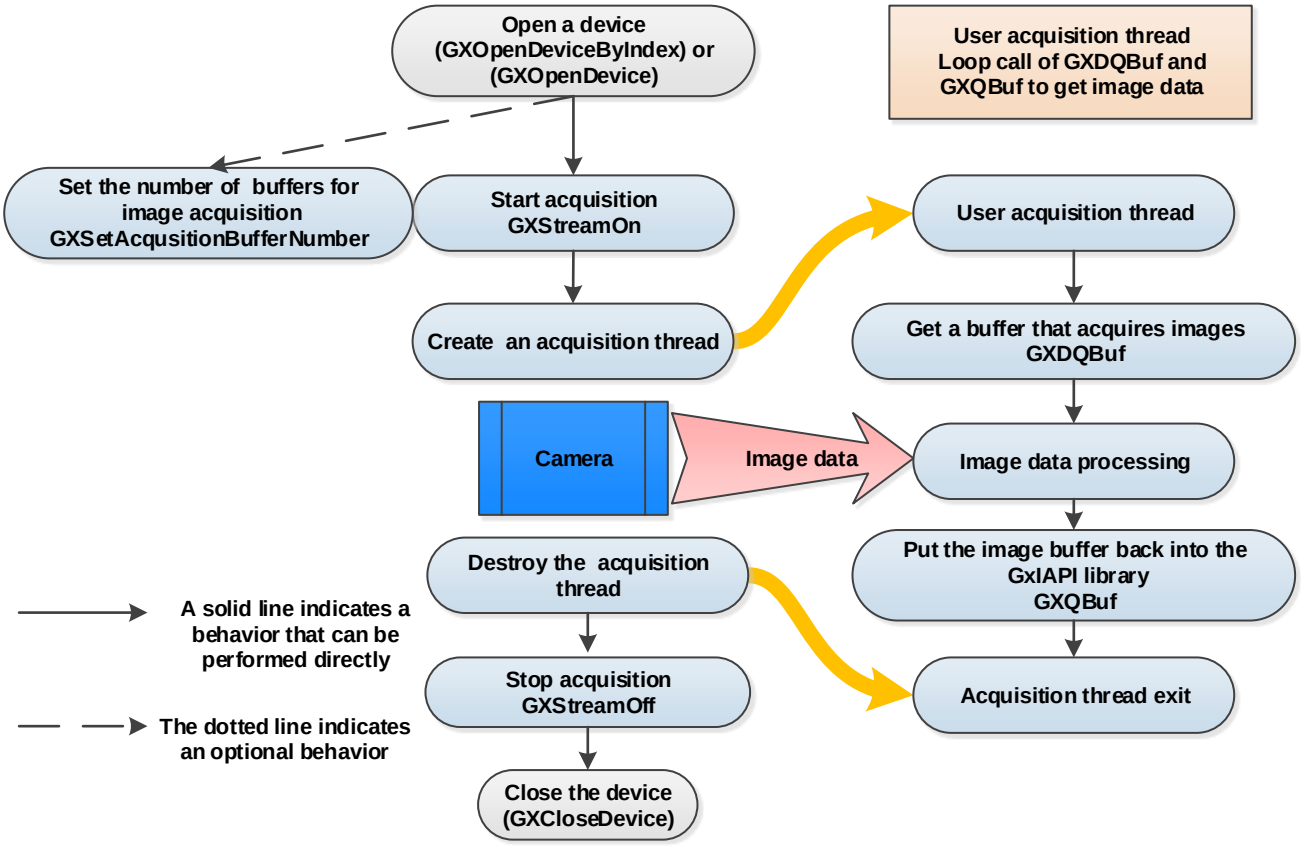


Figure 1-3 DQBuf acquisition flow

Precautions:

1) The user acquisition thread must be started after the start of the acquisition and be destroyed before the acquisition is stopped.

2) Windows currently does not support the GXStreamOn()/GXStreamOff() interface. Use GXSetCommandValue (hDevice, "AcquisitionStart")/GXSetCommandValue (hDevice, "AcquisitionStop") instead.
3) After calling the GXQBuf() interface and putting the image buffer back into the GxIAPI library, you can no longer access the image buffer pointer.
4) The GXStreamOff() interface will put all image buffers obtained by GXDQBufEx back into the GxIAPI library, and then users can no longer access these image buffers.
5) The above flowchart demonstrates the way to start the thread for image acquisition, or you can call GXDQBufEx() directly to acquire images without creating a thread.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

1.4. DQAllBufs acquisition flow (zero copy, Linux only)

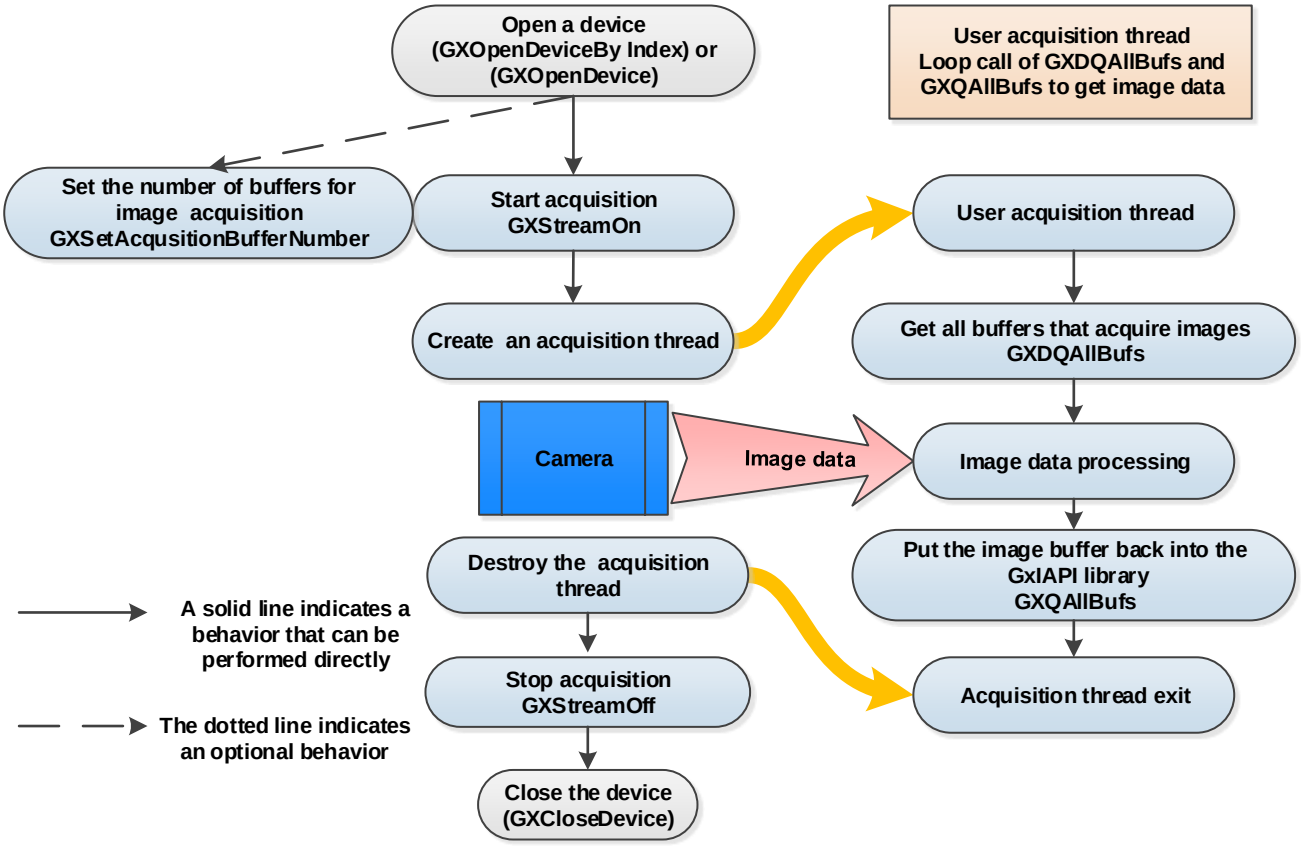


Figure 1-4 DQAllBufs acquisition flow

Precautions:

1) The user acquisition thread must be started after the start of the acquisition and be destroyed before the acquisition is stopped;
2) The GXStreamOff() interface will put all image buffers obtained by GXQAllBufs() back into the GxIAPI library, and then users can no longer access these image buffers;
3) The above process demonstrates the way to start the thread for image acquisition, or you can call GXDQAllBufs() to acquire images without creating a thread.

Recommended scenarios:

1) In the case of having delay with image processing or display, the delay can be alleviated by only processing or displaying the latest image in the array.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

1.5. Capture callback flow

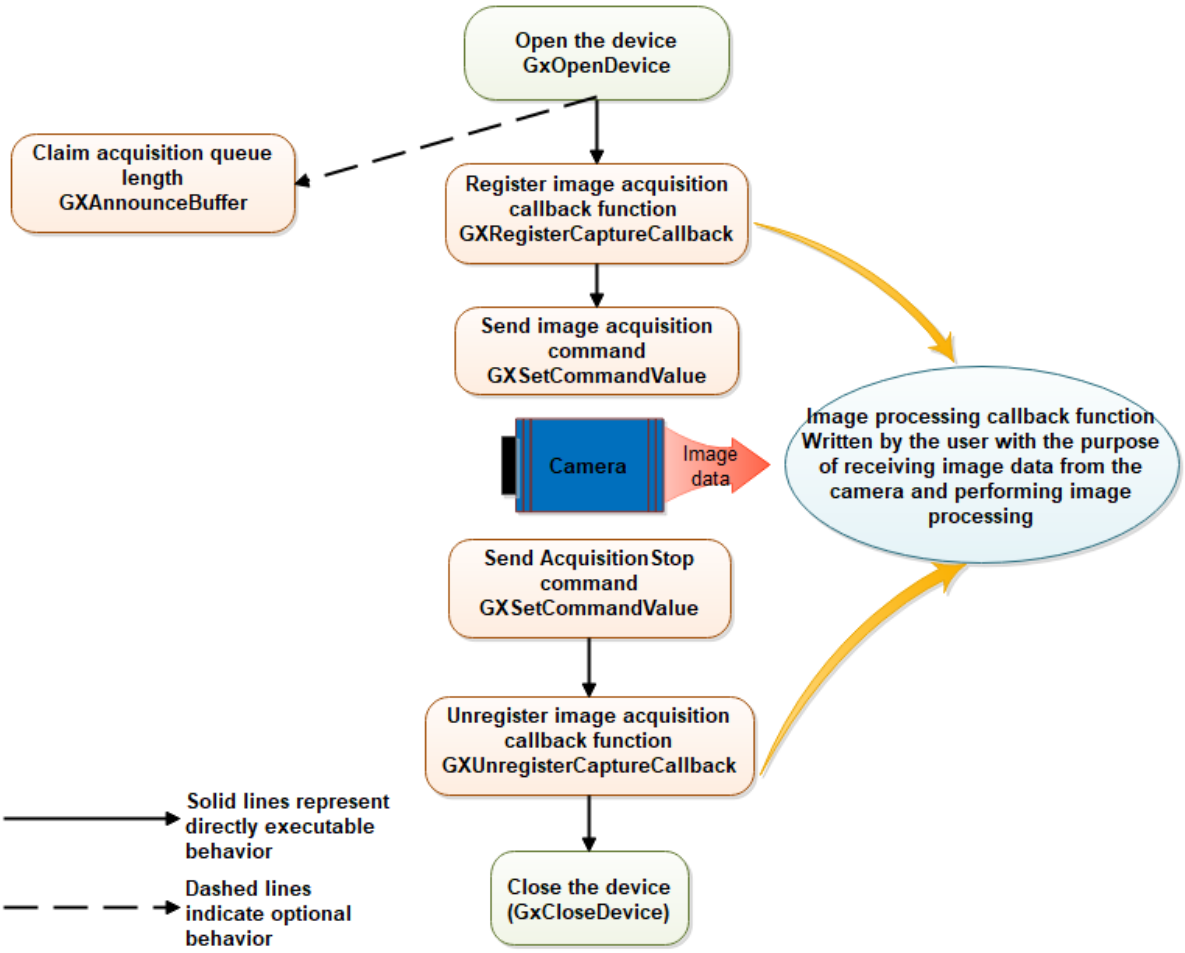


Figure 1-5 Capture callback flow

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

1.6. Offline events get flow

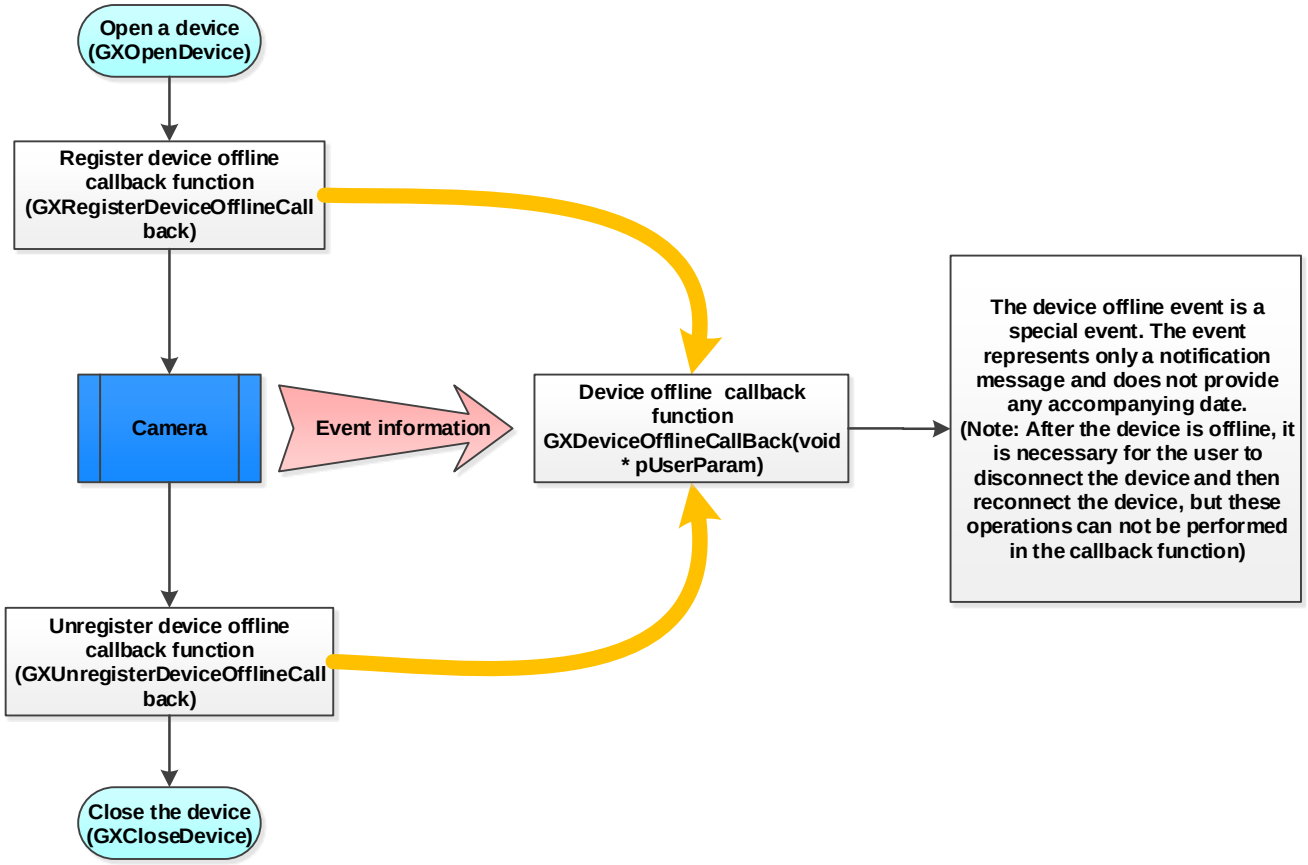


Figure 1-6 Offline events get flow

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

1.7. Remote device events get flow

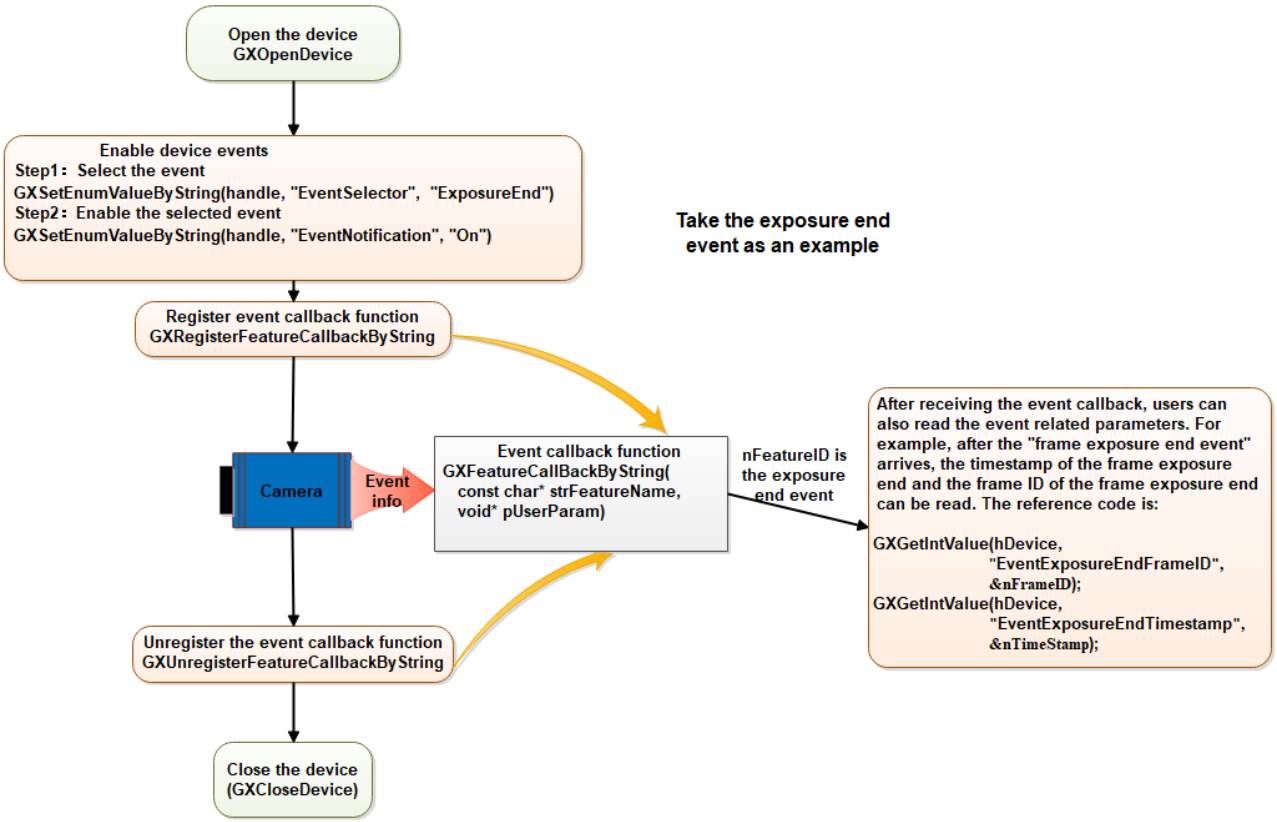


Figure 1-7 Remote device events get flow

2. Programming Guide

2.1. Build Programming Environment

2.1.1. Windows

2.1.1.1. C

The GxIAPI and DxImageProc are the general programming interface and image processing algorithm interface produced by the software department of Daheng Imaging. The interfaces are suit for all the Daheng's cameras that support the GenICam protocol. Under the installation directory of the installation packet ./ Development/VC SDK, the folder contains GxIAPI.h, DxImageProc.h and GxIAPI.lib, DxImageProc.lib files and some simple sample programs.

- GxIAPI.dll, DxImageProc.dll Dynamic link library files
- GxIAPI.lib, DxImageProc.lib Static link library files
- GxIAPI.h, DxImageProc.h Interfaces and macro declaration header files

When installing the install packet, the installation paths of the .dll files are added to the system environment variable "path", so your applications will automatically search the .dll file from the environment variable and load it.

When creating the project with the GxIAPI or DxImageProc, the .h header file and .lib static link library are needed, and then you can call and compile it successfully.

2.1.1.1.1. Configuration the VC6 programming environment

Click **Project->Settings** in the menu, pop-up the **project settings** window, click on the **C/C++** tab, set the **Category** as **Preprocessor**, fill in the .h files directory path address (based on the actual installation directory) in the **Additional include directories**, see Figure 2-1:

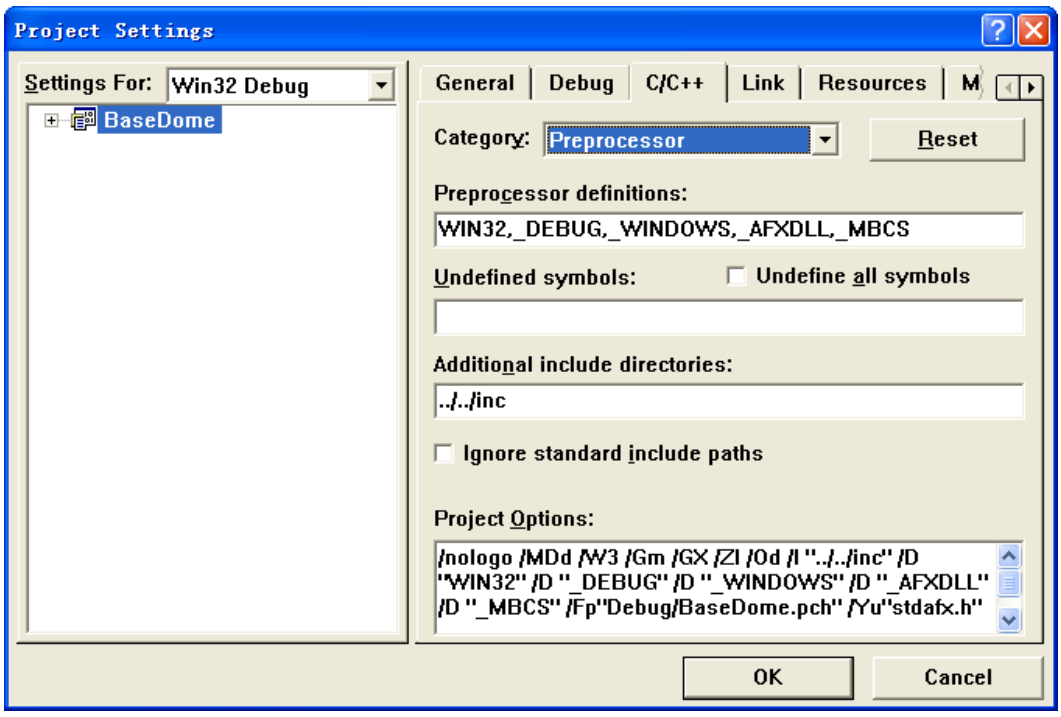


Figure 2-1

Select **Link** tab, set **Category** as **General**, fill **GxIAPI.lib** and **DxImageProc.lib** in the **Object/library modules**, and see Figure 2-2:

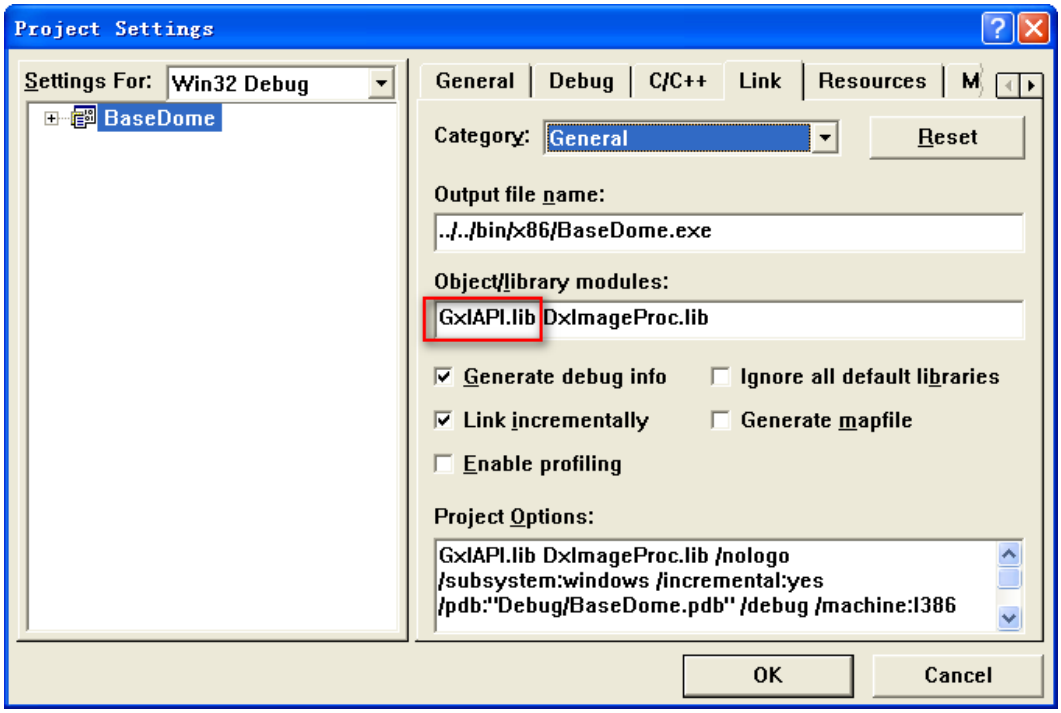


Figure 2-2

Still stay in **Link** tab, set **Category** as **Input**, and fill in the directory path address (based on the actual installation directory) of the **GxIAPI.lib** and **DxImageProc.lib** in the **Additional library path**, and see Figure 2-3:

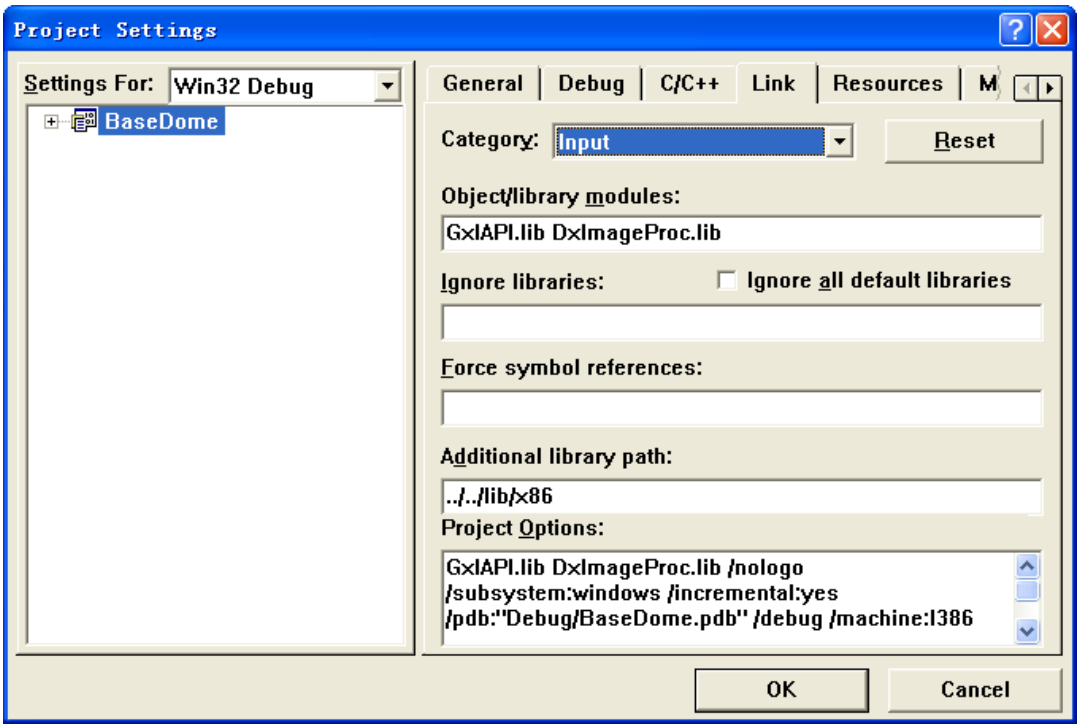


Figure 2-3

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

2.1.1.1.2. Configuration the Visual Studio programming environment

Select the project which created by yourself in the solution resource management window, then click **project->properties** in the menu, pop-up the **Property page** window. Select **Configuration Properties->C/C++->General**, fill in the directory path address (based on the actual installation directory) of the **GxIAPI.h** and **DxImageProc.h** in the **Additional Include Directories**, and see Figure 2-4:

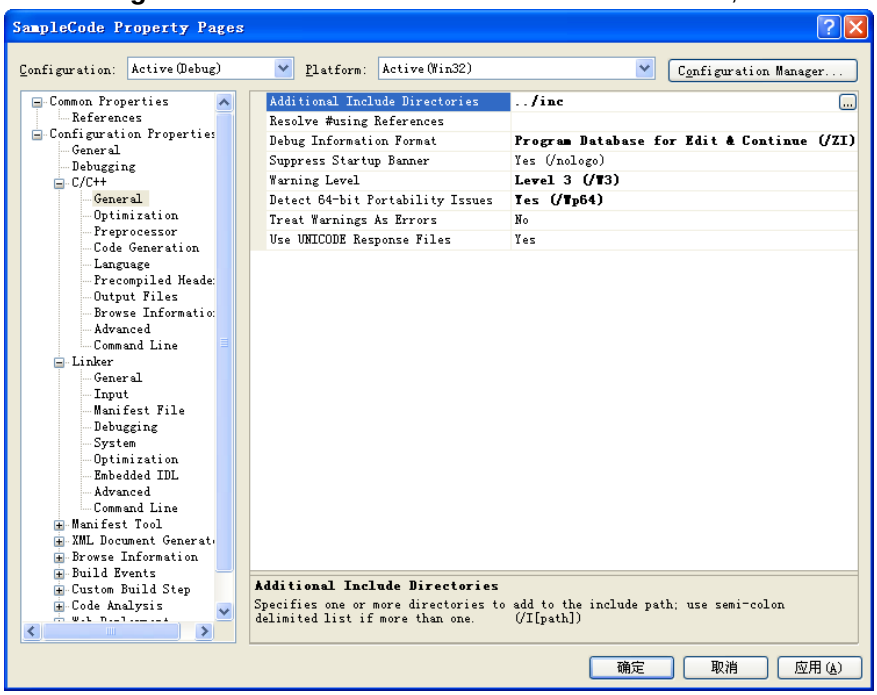


Figure 2-4

Select **Configuration Properties->Linker->General**, fill in the directory path address (based on the actual installation directory) of the **GxIAPI.lib** and **DxImageProc.lib** in the **Additional Library Directories**, and see Figure 2-5:

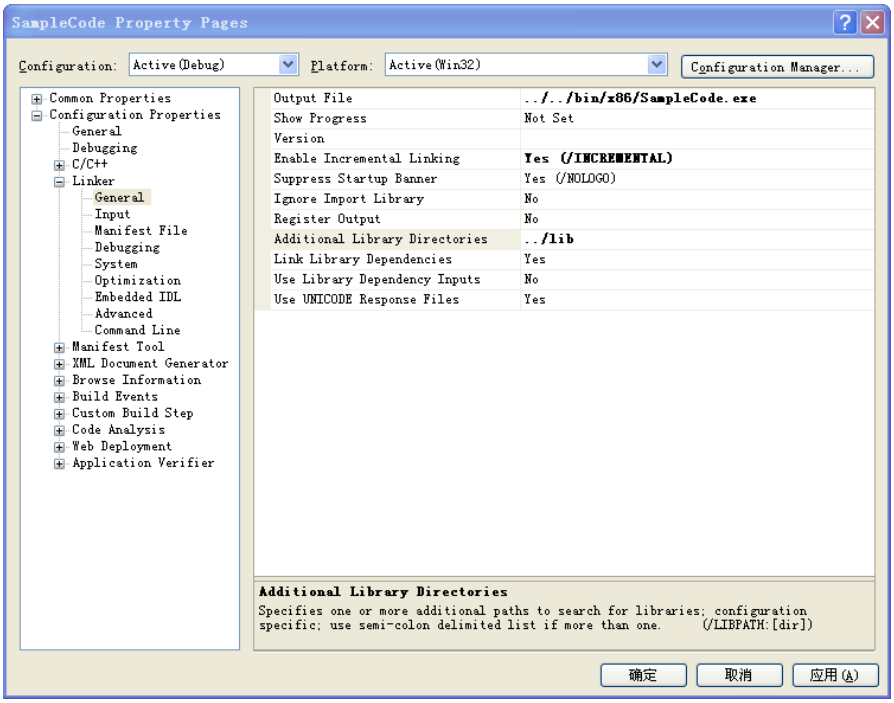


Figure 2-5

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Select **Configuration Properties->Linker->Input**, fill in the **GxIAPI.lib** and **DxImageProc.lib** in the **Additional Dependencies**, and see Figure 2-6:

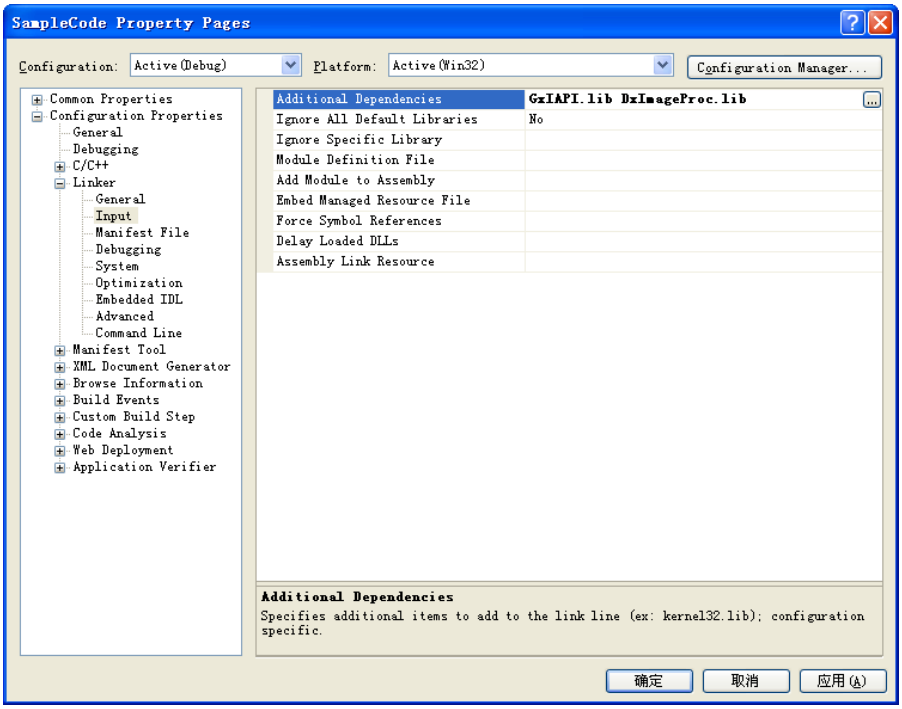


Figure 2-6

2.1.2. Linux

2.1.2.1. Recommended configuration

1) Support Linux kernel version: >= 3.5.0 (32 and 64 bit)
   g++ version: >= 4.8
   glibc version: >= 2.17
2) Recommended system release version: Ubuntu 16.04 or above, CentOS7, CentOS8

2.1.2.2. Console program

2.1.2.2.1. Makefile

1) Before compiling the program, you need to copy the GxIAPI. h and DxImageProc. h header files from the include directory of the SDK library to the project directory, or add the include directory of the SDK library to the search path by the "- I" parameter.
2) When linking the application, you must link to the libgxiapi.so (-lgxiapi).

There are a series of sample programs in the Linux package, the following contents are the generic Makefile contents of the sample programs.

The red mark: samplename is the name of the compile and output program.

```
# Makefile for sample program
.PHONY      : all clean

# the program to build
NAME        := samplename

# Build tools and flags
CXX         := g++
LD          := g++
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
SRCS        := $(wildcard *.cpp)
OBJS        := $(patsubst %cpp, %o, $(SRCS))
CPPFLAGS    := -w -I./

LDFLAGS     := -lgxiapi -lpthread

all         : $(NAME)

$(NAME)     : $(OBJS)
$(LD) -o $@ $^ $(CPPFLAGS) $(LDFLAGS)

%.o     : %.cpp
$(CXX) $(CPPFLAGS) -c -o $@ $<

clean       :
$(RM) *.o $(NAME)
```

2.1.2.3. Qt-based sample program

The following describes the installation method and configuration steps of the qtcreator integrated development environment.

2.1.2.3.1. Install qtcreator

a) Installation
sudo apt-get install qtcreator
b) Run
Run qtcreator at the terminal

2.1.2.3.2. Configuring the environment under qtcreator (based on Qt Creator 3.5.1)

Step 1: After running the program, the display interface is as shown in Figure 2-7. Select **Open Project** and select the project file of the project to be opened in the pop-up dialog box.

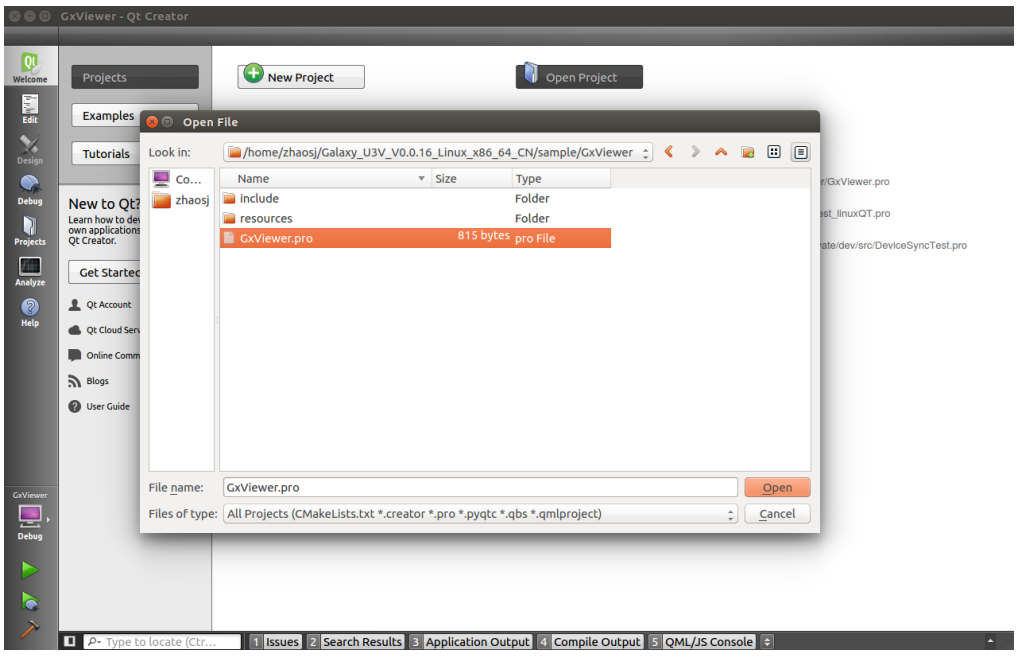


Figure 2-7

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Step 2: When opening the project, if the project cannot be opened and the Kit related contents are prompted, you need to check the configuration of the qtcreator. Select **Tool -> Options -> Build & Run -> Qt Versions** to see if qmake is detected. If it is not detected automatically, you need to load it manually, as shown in Figure 2-8. This step can be ignored if the project can be opened normally.

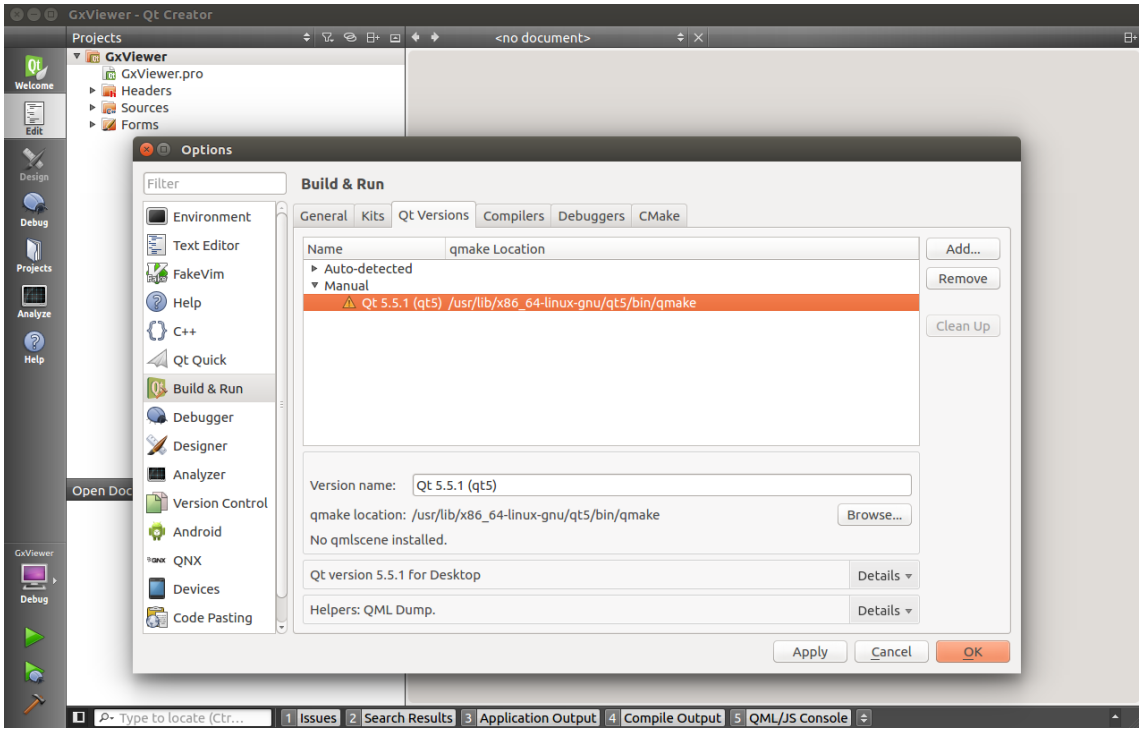


Figure 2-8

Step 3: After qmake loads successfully, add qmake to the Qt Version of the build options you want to use in the **Tool -> Options -> Build & Run -> Kits** tab, as shown in Figure 2-9. This step can be ignored if the project can be opened normally.

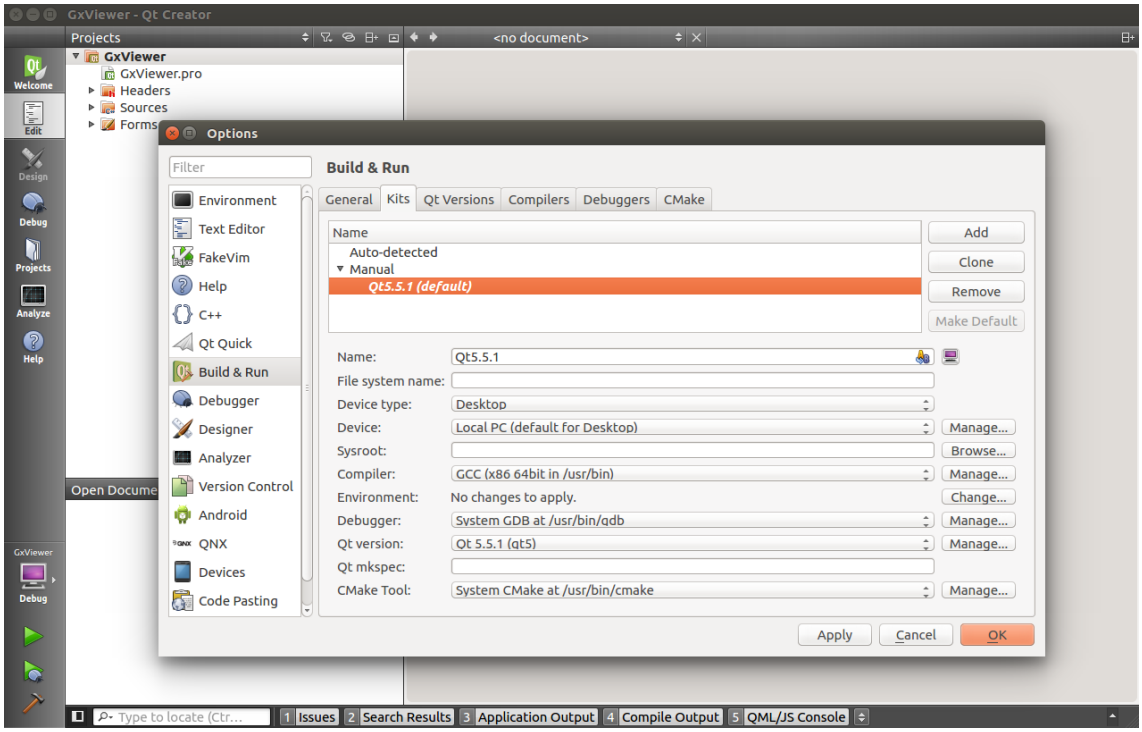


Figure 2-9

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Step 4: After the project is successfully opened, compile and run.

2.1.3. macOS

2.1.3.1. Development Environment Installation

- Install daheng macOS sdk

1) Double-click the package with suffix .pkg in the provided dmg to install.
2) Click Continue and select the disk to install. The galaxy. framework is installed in the/Library/Frameworks/directory of the disk.
3) Authentication.
4) Keep clicking Continue until installation finished.

2.1.3.2. Sample program based on the Xcode

The sample program in the installation package can be compiled and run normally under Xcode version 11.3 (11C29). The installation method and configuration steps of Xcode (Xcode11.3) integrate development environment are as follows.

- MacOS platform installs Xcode based on Xcode 11.3

1) Installation

Search Xcode in APP Store and then click Install button.

2) Operation

Operate Xcode in LaunchPad.

2.1.3.3. Xcode (Xcode11.3 based) configuration environment

Step 1: After operation, create a new project through "File->New->Project", and select "macOS->Command Line Tool".

Step 2: Add header files and library files to the project. First add the galaxy.framework to the project, the path of the galaxy.framework is in the /Library/Frameworks/. Right-click the project name, click "Add Files to…", you can access the path of galaxy.framework by the shortcut key Command+Shift+G. Add galaxy.framework to the project, and by the same way add the GxIAPI.cti and libGxIAPI.dylib in the /Library/Frameworks/galaxy.framework/Versions/A/Libraries/ to the project.

Step 3: After loading the Framework and library file successfully, click the project name, select "TestProgram->BuildSettings->Linking->Runpath Search Paths", and add /Library/Frameworks/ to this item (the directory of galaxy.framework).

Step 4: Reference the header file to write a simple program, compile and run it, and the successful interface call proves that the Xcode environment configuration is successful.

2.2. Quick Guide

- Initialization and uninitialization GxIAPI runtime library
- Enumeration cameras and get the information
- Configurate the camera IP address
- Open and close the camera
- Camera control function
- DQBuf image acquisition
- Callback image acquisition
- Automatic reconnection

2.2.1. Initialization and uninitialization GxIAPI runtime library

Before using the GxIAPI interface (except GXCloseLib() / GXGetLastError()), you must call the GXInitLib() to initialize the library. Before exiting the application, you also need to call GXCloseLib() to release resources in order to corresponding with GXInitLib(). All the sample codes in this manual have been initialized and uninitialized GxIAPI library.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Sample Code:

```
#include "GxIAPI.h"

int main(int argc, char* argv[])
{
    GX_STATUS emstatus = GX_STATUS_SUCCESS;

    //Calls GXInitLib() at the start location to initialize resources.
    emstatus = GXInitLib();
if (emstatus != GX_STATUS_SUCCESS)
{
    return 0;
    }

    //Uses GxIAPI.
    //Calls GXCLoseLib() at the end of the program to release resources.
    emstatus = GXCloseLib();

    return 0;
}
```

2.2.2. Enumerate cameras and get information

You can call GXUpdateAllDeviceList() interface to enumerate the current available device, return the number of the device, and then use GxGetDeviceInfo() to get the device's basic information without opening the camera.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32DeviceNum = 0;

emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
if ((emStatus == GX_STATUS_SUCCESS) && (ui32DeviceNum > 0))
{
    for (uint32_t  i = 0; i < ui32DeviceNum; ++i)
    {
        GX_DEVICE_INFO stDeviceInfo;
        GXGetDeviceInfo(i, &stDeviceInfo);
    }
}
```

If you want to enumerate the specified types of cameras, you can also use GxUpdateAllDeviceListEx() to get the network information of the device.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32DeviceNum = 0;

emStatus = GXUpdateAllDeviceListEx(GX_TL_TYPE_GEV, &ui32DeviceNum,
                                   1000);
if ((emStatus == GX_STATUS_SUCCESS) && (ui32DeviceNum > 0))
{
    GX_DEVICE_INFO stDeviceInfo;
    unsigned int nDeviceIp;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    //Obtain the network IP information of the first device
    emStatus = GXGetDeviceInfo(1, &stDeviceInfo);
    nDeviceIp = stDeviceInfo.DevInfo.stGEVDevInfo.nCurrentIp;
}
```

2.2.3. Get Interface Information

Users obtain interface information by calling GXGetInterfaceInfo() interface, with a parameter type of GX_INTERFACE_INFO.

Sample Code:

```
//Get Interface information
GX_INTERFACE_INFO stInterfaceInfo = NULL;
GX_STATUS emStatus = GXGetInterfaceInfo(1 , &stInterfaceInfo );
```

2.2.4. Get Interface Object

Users obtain the Interface handle by calling GXGetInterfaceHandle() interface, where the parameter type is integer and the index starts counting from 1.

Sample Code:

```
//Get Interface pointer
GX_IF_HANDLE hIFHandle = NULL;
GX_STATUS emStatus = GXGetInterfaceHandle(1, &hIFHandle);
```

2.2.5. Configurate the camera IP address

1) For GigE cameras, you can call GXGigEIpConfiguration() to configurate the IP address for the camera, and the IP address set in this way is still valid after the camera has been rebooted. Calling this function requires the MAC address of the destination camera, which can be got by calling GxGetDeviceIPInfo().

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

const char* chMAC = "00-21-49-00-00-00";
const char* chIpAddress = "192.168.10.10";
const char* chSubnetMask = "255.255.255.0";
const char* chDefaultGateway = "192.168.10.2";
const char* chUserID = "Daheng Imaging";
GX_IP_CONFIGURE_MODE emIpConfigureMode = GX_IP_CONFIGURE_STATIC_IP;

//This example is described in persistent IP configuration mode,
//and other IP configuration modes are similar.
//Sets to persistent IP configuration mode.
emstatus = GXGigEIpConfiguration(chMAC, emIpConfigureMode,
                                 chIpAddress, chSubnetMask,
                                 chDefaultGateway, chUserID);
```

2) You can also call GXGigEForceIp() to configurate the camera IP address, but the IP address set in this way may effective only for this use, when the camera rebooted, the camera may use the previous IP address. Calling this function requires the MAC address of the destination camera, which can be got by calling GxGetDeviceIPInfo().

Sample Code:

```
GX_STATUS emstatus = GX_STATUS_SUCCESS;
const char* chMAC = "00-21-49-00-00-00";
const char* chIpAddress = "192.168.10.10";
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    const char* chSubnetMask = "255.255.255.0";
    const char* chDefaultGateway = "192.168.10.2";

    //Force Ip.
    emStatus = GXGigEForceIp(chMAC, chIpAddress, chSubnetMask,
                             chDefaultGateway);
```

2.2.6. Open and close the camera

After calling the GXUpdateAllDeviceList() interface to enumerate the device, if the return value of the device number is greater than 0, it means that there have devices can be used currently, you can call the GxOpenDevice() interface to open them.

Sample Code:

```
    GX_STATUS emstatus = GX_STATUS_SUCCESS;
    uint32_t nDeviceNum = 0;
    GX_OPEN_PARAM stOpenParam;

    emStatus = GXUpdateAllDeviceList(&nDeviceNum, 1000);
    if ((emStatus == GX_STATUS_SUCCESS) && (nDeviceNum > 0))
    {
        GX_DEV_HANDLE  hDevice = NULL;

       //Open the first device in the enumeration list.
       //Assuming that the user enumerates three available devices, the
       //user can set the pszContent field of the stOpenParam parameter
       //to 1, 2, 3.
       stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
       stOpenParam.openMode   = GX_OPEN_INDEX;
       stOpenParam.pszContent = "1";

       //Opens the device via a serial number.
       //stOpenParam.openMode = GX_OPEN_SN;
       //stOpenParam.pszContent = "EA00010002";

       //Opens the device via an IP address.
       //stOpenParam.openMode = GX_OPEN_IP;
       //stOpenParam.pszContent = "192.168.40.35";

       //Opens the device via a MAC address.
       //stOpenParam.openMode = GX_OPEN_MAC;
       //stOpenParam.pszContent = "54-04-A6-C2-7C-2F";

       emstatus = GXOpenDevice(&stOpenParam, &hDevice);

       //Operates the device: control and acquisition.
       //..

       //Closes the Device.
       emstatus = GXCloseDevice(hDevice);

    }
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

The GxOpenDevice() can open the camera by specifying access mode (exclusive, control, etc.), opening mode (Via IP, SN, MAC, Index, etc.). See the explanation for the GxOpenDevice() interface later.

2.2.7. Camera control function

The GalaxyView.exe provides sample codes for feature type data node read & write. In the demo program menu bar: click the arrow below the demonstration program to open the property document window, as shown in the follow:

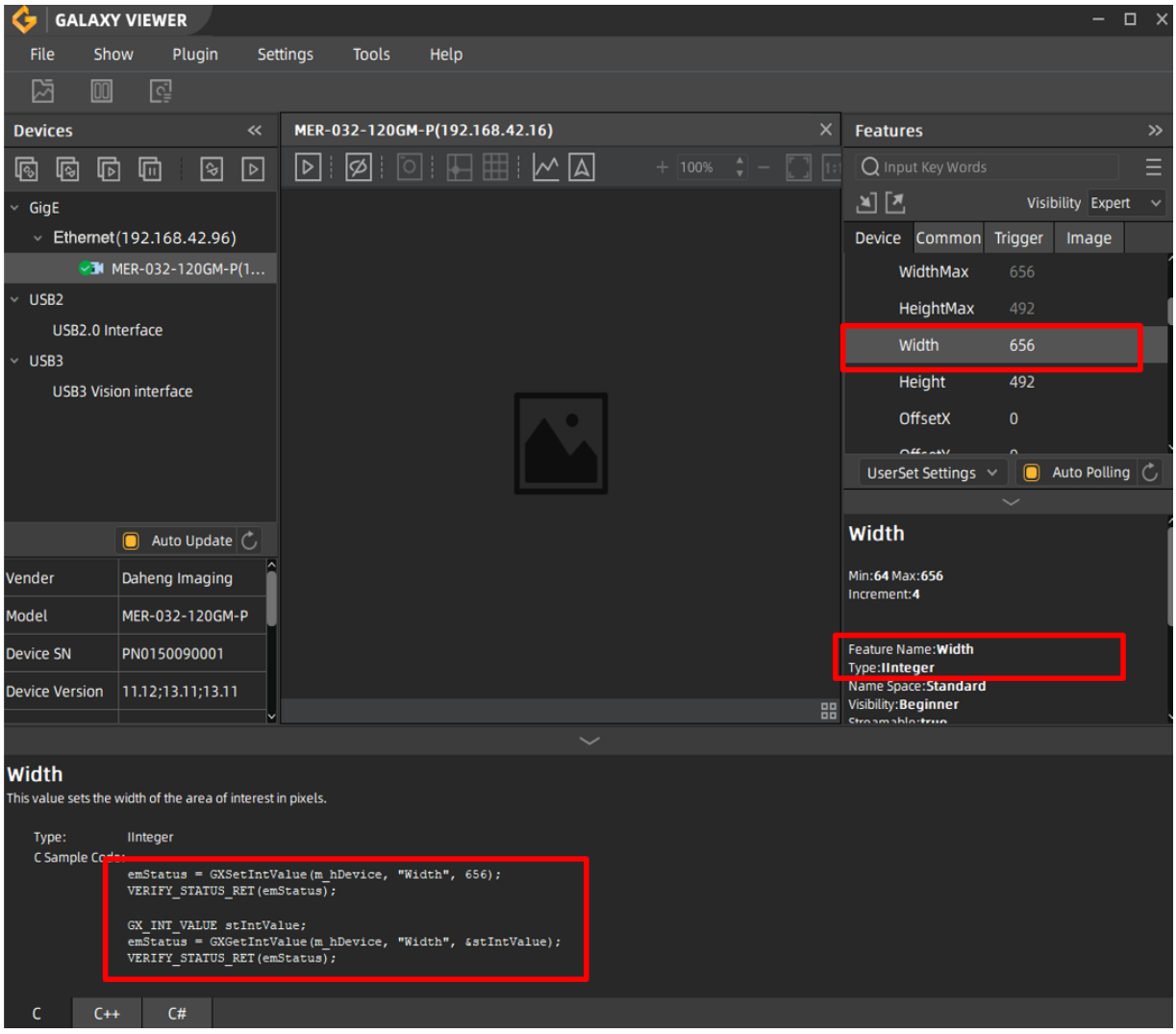


Figure 2-10

In the view of feature document, three development languages are available. Switch to C Tab to get the code for reading and writing user-specified feature in C language. The code can be directly copied to the user's development project.

The following describes the access interfaces for different types of features.

Integer type data node related interfaces:

GXGetIntValue(): Read values, minimum, maximum, step size.

GXSetIntValue(): Set the int value.

Sample Code:

```
//The value Width corresponding to the Feature Name in
//the above figure is the second parameter of the interface
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
//The value IInterger corresponding to Type indicates that
//the attribute is an integer corresponding to GX_ INT_ Value
//Read the image width

GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_INT_VALUE stIntValue;
int64_t i64Width = 0;

//Get the current value of image width
emStatus = GXGetIntValue(hDevice, "Width", &stIntValue);
i64Width = stIntValue.nCurValue;

//Set the image width to the minimum value
int64_t i64Value = stIntValue.nMin;
emstatus = GXSetIntValue(hDevice, "Width", i64Value);
```

Float type data node related interfaces:

GXGetFloatValue(): Read values, minimum, maximum, step size.

GXSetFloatValue(): Set the float value.

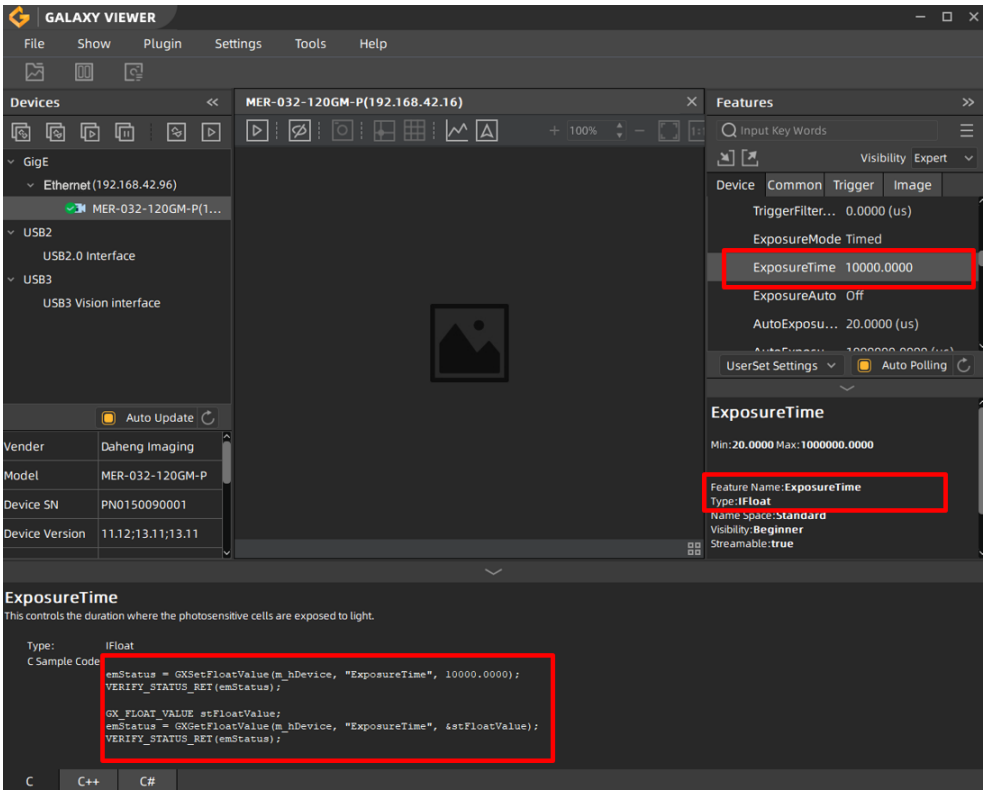


Figure 2-11

Sample Code:

```
//The value ExposeTime corresponding to the Feature Name in the
//above figure is the second parameter of the interface
//The value IFloat corresponding to Type indicates that the
//attribute is a floating-point shape corresponding to
//GX_FLOAT_Value
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
//Read exposure time
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_FLOAT_VALUE stFloatValue;

//Read the current value of exposure time
emStatus = GXGetFloatValue(hDevice, "ExposureTime", &stFloatValue);
double dExposureTime = stFloatValue.dCurValue;

//Set exposure time
double dValue = 3000;
emStatus =GXSetFloatValue(hDevice, "ExposureTime", dValue)
```

Enumerate type data node related interfaces:

GXGetEnumValue(): Read the number, value, and the description of enumeration items .

GXSetEnumValue(): Set the enumerate value.

GXSetEnumValueByString(): Set the enumerate value.

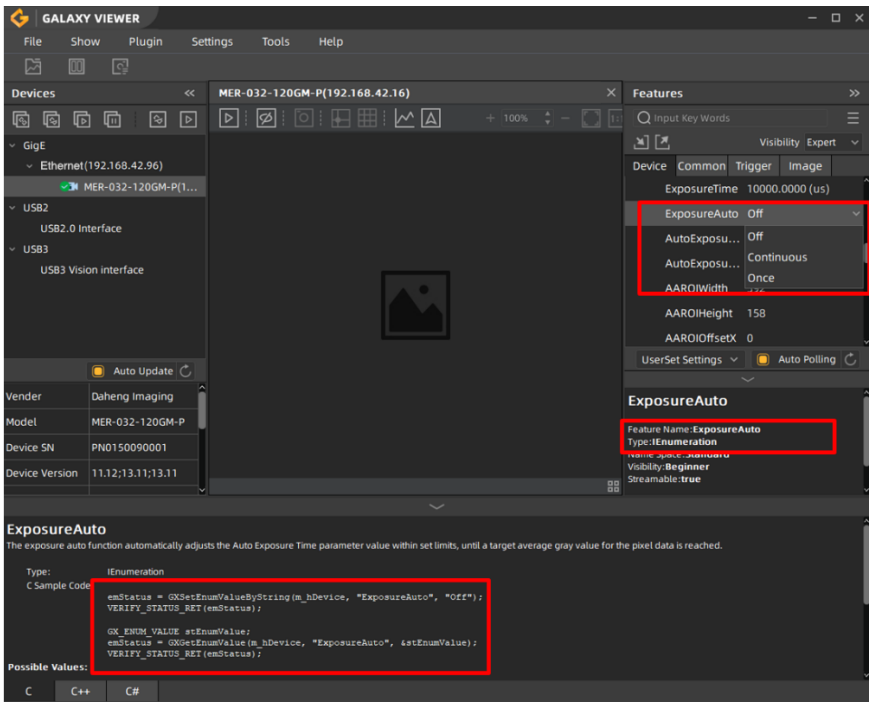


Figure 2-12

Sample Code:

```
//The value corresponding to the Feature Name in the above figure
//is the second parameter of the interface
//The value IEnumeration corresponding to Type indicates that the
//property is an enumeration corresponding to GX_ ENUM_ Value

//Read auto exposure
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_ENUM_VALUE stEnumValue;

//Read the current enumeration value of automatic exposure
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    emStatus = GXGetEnumValue(hDevice, "ExposureAuto", &stEnumValue);
    int64_t i64ExposureAuto = stEnumValue.stCurValue.nCurValue;

    //Set automatic exposure
    int64_t i64Value = GX_EXPOSURE_AUTO_CONTINUOUS;
    emStatus = GXSetEnumValue(hDevice, "ExposureAuto", i64Value);

    //Set the drop-down list in the automatic exposure image to
    //correspond to the third parameter of this interface
    emstatus = GXSetEnumValueByString(hDevice,"ExposureAuto",
                                      "Continuous");
```

Boolean type data node related interfaces:

GXGetBoolValue(): Get the bool value.

GXSetBoolValue(): Set the bool value.

String type data node related interfaces:

GXGetStringValue(): Read the current value, the length of the current value of the string (bytes), and the maximum length of the string (bytes).

GXSetStringValue(): Set the string value.

Register type data noderelated interfaces:

GXGetRegisterLength(): Query the length of block data, use this length to apply for memory, and then call GXGetRegisterValue() to read the data.

GXGetRegisterValue(): Read the register value.

GXSetRegisterValue(): Set the register value.

Command type data node related interfaces:

GXSetCommandValue(): Send command.

2.2.8. DQBuf image acquisition

After the GXOpenDevice() interface is called to open the device, the GXStreamOn() interface can be called in sequence to enable the stream acquisition and remote device acquisition. In this case, GXDQBufEx() and GXQBufEx() interfaces *(*GXDQAllBufsEx(), GXQAllBufs() are similarly used, Linux only) can be used to continuously acquire images. After calling the GXDQBufEx() interface, you can get an image buffer and do your image processing, and calling the GXQBufEx() interface, put the buffer back into the capture queue.

Sample Code:

```
    //------------------------------
    //The GXDQBufEx interface acquires one frame of image at a time. This
    //sample
    //code demonstrates how to use these interfaces to get a frame of image.
    //------------------------------
    #include "GxIAPI.h"
```

```
int main(int argc, char* argv[])
{
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    uint32_t ui32DeviceNum = 0;

    //Initializes the library.
    emStatus = GXInitLib();
    if (emStatus != GX_STATUS_SUCCESS)
    {
        return 0;
      }

    //Enumerate device list
    emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
    if ((emStatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
    {
        return 0;
    }

    //Opens the device.
    GX_OPEN_PARAM stOpenParam;
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emstatus = GXOpenDevice(&stOpenParam, &hDevice);
    if (emstatus == GX_STATUS_SUCCESS)
    {
        //Define the incoming parameters of GXDQBufEx
        GX_FRAME_BUFFER pFrameBuffer;

        //Start acquisition
        #ifdef __linux__
        emStatus = GXStreamOn(hDevice);
        #else
        emStatus = GXSetCommandValue(hDevice, "AcquisitionStart");
        #endif
        if (emStatus == GX_STATUS_SUCCESS)
        {
            //Calling GXDQBufEx to retrieve a frame of image
            emStatus = GXDQBuf(hDevice, &pFrameBuffer, 1000);
            if (emStatus == GX_STATUS_SUCCESS)
            {
                if (pFrameBuffer-> nStatus == GX_FRAME_STATUS_SUCCESS)
                {
                //Image acquisition successful
                //Process images
                }

                //Call GXQBufEx to put the image buf back into the library
                //and continue acquire images
                emStatus = GXQBufEx(hDevice, pFrameBuffer);
            }
```

```
        }
        //Stop acquisition
        #ifdef __linux__
        emStatus = GXStreamOff(hDevice);
        #else
        emStatus = GXSetCommandValue(hDevice, "AcquisitionStop");
        #endif
    }
    emStatus = GXCloseDevice(hDevice);
    emStatus = GXCloseLib();
    return 0;
}
```

2.2.9. CallBack image acquisition

Terms:

- Image processing callback function: the user-defined image processing function, the GxIAPI specifies the return value of the function, formal parameter and etc. (see the GxIAPI library specification->type->Callback Function Type->GXCaptureCallBack()).
- Registered callback function: The user calls the GxRegisterCaptureCallbackEx() interface to pass the pointer of the user-defined image processing callback function to GxIAPI library (see GxRegisterCaptureCallbackEx() interface for details).
- Unregistered callback function: The user calls the GxUnregisterCaptureCallback() interface to notify the GxIAPI library to release the callback function pointer which registered by the user.

Sample Code:

```
#include "GxIAPI.h"

//Image processing callback function.
static void GX_STDC OnFrameCallbackFun(GX_FRAME_DATA_EX* pFrame)
{
    if (pFrame->status == GX_FRAME_STATUS_SUCCESS)
    {
        //Do some image processing operations.
    }
    return;
}

int main(int argc, char* argv[])
{
    GX_STATUS     emstatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    GX_OPEN_PARAM stOpenParam;
    uint32_t      uiDeviceNum = 0;

    //Initializes the library.
    emstatus = GXInitLib();
    if (emstatus != GX_STATUS_SUCCESS)
    {
        return 0;
    }

    //Updates the enumeration list for the devices.
```

```
    emstatus = GXUpdateAllDeviceList(&uiDeviceNum, 1000);
    if ((emstatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
    {
        return 0;
    }
    //Opens the device.
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode   = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emstatus = GXOpenDevice(&stOpenParam, &hDevice);
    if (emstatus == GX_STATUS_SUCCESS)
    {
        //Setting device's property of GevSCPSPacketSize to improve
        //the acquisition performance of the network camera
        bool bImplementPacketSize = false;
        uint32_t ui32PacketSize     = 0;

        //Determine whether the device supports GevSCPSPacketSize
        GX_NODE_ACCESS_MODE emAccessMode;
        emStatus = GXGetNodeAccessMode(phDevice,"GevSCPSPacketSize",
                                       &emAccessMode);
        bImplementPacketSize = (emAccessMode == GX_NODE_ACCESS_MODE_RW)
                               ? true : false;
        if (bImplementPacketSize)
        {
            //Get Optimal PacketSize
            emstatus = GXGetOptimalPacketSize (hDevice, &ui32PacketSize);

            //Set the Optimal PacketSize to GevSCPSPacketSize
            emstatus = GXSetIntValue(hDevice, "GevSCPSPacketSize",
                                     ui32PacketSize);
        }
        //Registers image processing callback function.
        emstatus = GXRegisterCaptureCallbackEx (hDevice, NULL,
                                                OnFrameCallbackFun);

        //Sends a start acquisition command.
        emStatus = GXSetCommandValue(hDevice, "AcquisitionStart");

        //----------------------
        //
        //In this interval, the image will be returned to the user via the
        //OnFrameCallbackFun interface.
        //
        //----------------------

        //Sends a stop acquisition command.
        emstatus = GXSetCommandValue(hDevice, "AcquisitionStop");

        //Unregisters image processing callback function.
        emStatus = GXUnregisterCaptureCallback(hDevice);
    }
    emstatus = GXCloseDevice(hDevice);
```

```
        emstatus = GXCloseLib();
        return 0;
    }
```

You can also call the GxGetImageEx() interface to get images, but this method cannot be used with the callback mode simultaneously (see the GxGetImageEx() interface for details).

2.2.10. Reconnect

2.2.10.1. Procedure

GigE Vision camera and USB3 Vision camera provides local device event, including: device offline event, device disconnection event and device reconnection event. The procedure for obtaining events is as follows:

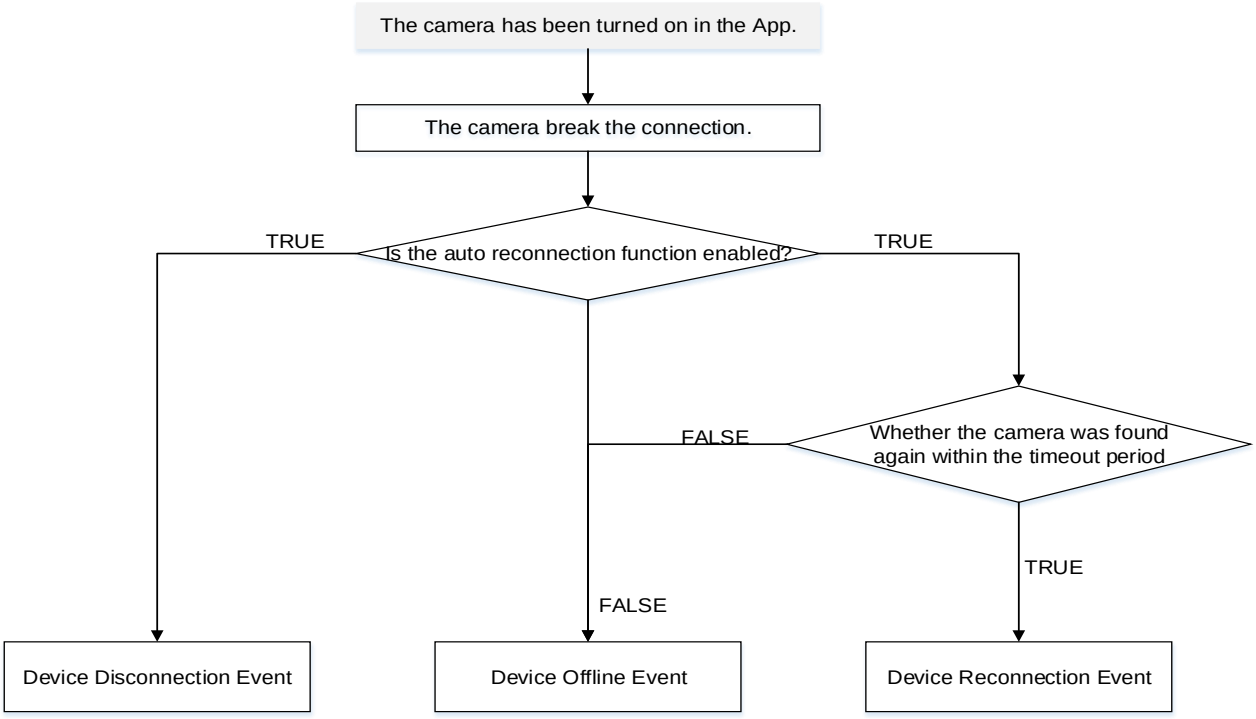


Figure 2-13 Offline and Reconnection Procedure

If a device offline event is registered, the device offline event will be received directly when the automatic reconnection function is not enabled.

If a device offline event or a device disconnection event is registered, an event in the following situations will be received when the automatic reconnection function is enabled:

1. As soon as the device breaks the connection, the API will call the device offline event callback function.
2. If the camera can be found within the timeout period, the API will call the device reconnection event callback function.
3. If the camera can not be found within the timeout period, the API will call the device disconnection event callback function.

Note: If IP address of the camera is changed, the auto reconnection function will not work in this case.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

2.2.10.2. Supported Range

	Auto Reconnection		Restore Image Capture	Restore Parameter	
	ON	OFF		Thought UserSet	Auto
GigE Vision Transport Layer Socket	✓	✓	✓	✓	×
USB3 Vision Transport Layer	✓	✓	✓	✓	×
GxIAPI C	✓	✓	✓	✓	×
GxIAPI C++	✓	✓	✓	✓	×
GxIAPI C#	✓	✓	✓	✓	×
GxIAPI Python	✓	✓	✓	✓	×

If the user wish to enable the automatic reconnection function, so that the API automatically helps the user to restore the capture status after the camera has successfully reconnected, the user must manually save the current parameters as UserSet0 before starting the capture and set the parameter group to UserSet0. Otherwise, if the width, height, pixel format, and other properties of the camera's image differ before and after reconnection, the capture may encounter exceptions, such as image distortion or program crashes.

Take a network camera as an example. When the device is offline but not powered off, the parameters will not change. In this scenario, users can choose not to set UserSet0 as the startup parameter group.

2.2.10.3. Attribute Node

Users can control the auto reconnection function and set the reconnection timeout period by using the relevant attribute nodes in the local device layer.

Model	Attribute Node	Explanation
LocalDevice/DeviceAutoConnection	EnableAutoConnection	Enable auto reconnection.
	DeviceConnectStatus	Device connection status.
	AutoConnectCounts	Auto reconnection count.
	AutoConnectTimeout	Auto reconnection timeout period, units ms.

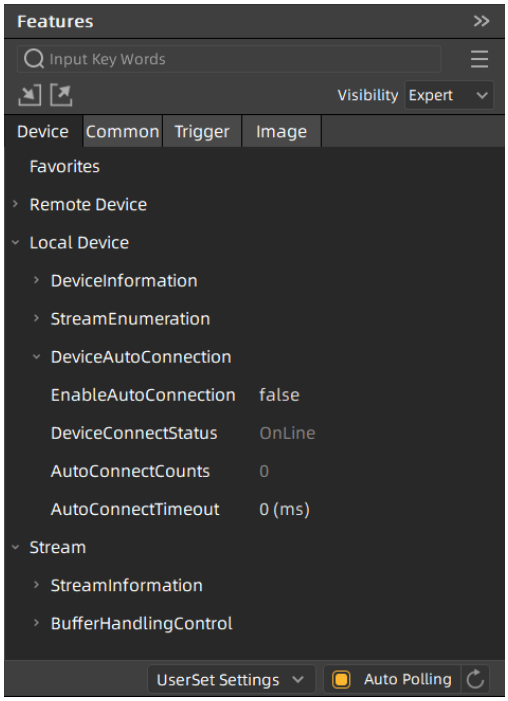


Figure 2-14 Auto Reconnection Attribute Nodes

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

2.2.10.4. Code Sample

GigE Vision camera and USB3 Vision camera provides device reconnection event and device disconnection event mechanism; users can get the device reconnection event and device disconnection event by calling function, and register the event after open the device.

Obtaining reconnection events involves the following steps:

1) Enable the reconnection function.
2) Register reconnection events.
3) Activate the callback function after the device reconnects.
4) Unregister reconnection events.

Obtaining disconnection events involves the following steps:

1) Enable the reconnection function.
2) Register disconnection events.
3) Activate the callback function after the device disconnects.
4) Unregister disconnection events.

Note: Be sure to unregister events before closing the device.

Closing the device is not allowed within the reconnection callback function, otherwise an illegal calling exception will be raised.

Closing the device is not allowed within the disconnection callback function, otherwise an illegal calling exception will be raised.

```
#include "GxIAPI.h"

//Reconnection callback function.
static void GX_STDC OnDeviceReconnectCallbackFun(void *pUserParam)
{
      //Upon receiving a reconnection notification, users perform the
      //necessary operations…

    return;
}

// Device disconnection callback function
static void GX STDC OnDeviceDisconnectCallbackFun(void *pUserParam)
{
    //Upon receiving a reconnection notification, users perform the
    //necessary operations…
    return;
}

int main(int argc, char *argv[])
{
    GXInitLib();

    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    GX_OPEN_PARAM stOpenParam;
    uint32_t ui32DeviceNum = 0;

    emStatus = GXUpdateDeviceList(&ui32DeviceNum, 1000);
    if ((emStatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    {
        GXCloseLib();
        return 0;
    }

    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emStatus = GXOpenDevice(&stOpenParam, &hDevice);

    if (emStatus == GX_STATUS_SUCCESS)
    {
        //Device reconnection mechanism is activated.
        GX_LOCAL_DEV_HANDLE hLocalDevice;
        GXGetLocalDeviceHandleFromDev(hDevice, &hLocalDevice);
        emStatus = GXSetBoolValue(hLocalDevice, "EnableAutoConnection",
true);
        if (emStatus != GX_STATUS_SUCCESS)
        {
            GXCloseDevice(hDevice);
            GXCloseLib();
            return 0;
        }
        // Set the timeout period.
        emStatus = GXSetIntValue(hDevice, "AutoConnectTimeout", 500);
        if (emStatus != GX_STATUS_SUCCESS)
        {
            GXCloseDevice(hDevice);
            GXCloseLib();
            return 0;
        }

        //Register reconnection callback function.
        emStatus = GXRegisterDeviceReconnectCallback(hDevice, NULL,
                                                     OnDeviceReconnectCallba
ckFun);
        if (emStatus != GX_STATUS_SUCCESS)
        {
            GXCloseDevice(hDevice);
            GXCloseLib();
            return 0;
        }

        //Register disconnection callback function.
        emStatus = GXRegisterDeviceDisconnectCallback(hDevice, NULL,
                                                      OnDeviceDisconnectCall
backFun);
        if (emStatus != GX_STATUS_SUCCESS)
        {
            GXCloseDevice(hDevice);
            GXCloseLib();
            return 0;
        }

        // ......

        //Unregister reconnection event callback.
        emStatus = GXUnregisterDeviceReconnectCallback(hDevice);
        if (emStatus != GX_STATUS_SUCCESS)
        {
            GXCloseDevice(hDevice);
```

```
            GXCloseLib();
            return 0;
        }
        //Unregister reconnection event callback.
        emStatus = GXUnregisterDeviceDisconnectCallback(hDevice);
        if (emStatus != GX_STATUS_SUCCESS)
        {
            GXCloseDevice(hDevice);
            GXCloseLib();
            return 0;
        }
    }

    GXCloseDevice(hDevice);
    GXCloseLib();
    return 0;
}
```

2.2.10.5. Notes

(1) If the camera successfully reconnects, residual frames may be generated during the reconnection process.

(2) When using the auto-reconnection function, it is strongly recommended to use the default parameter group and avoid any parameter configuration. If a non-default parameter group is used, modifying ROI settings may cause the camera to fail to resume image acquisition after reconnection. If the auto-reconnection enable status is modified during disconnection, the camera may still attempt to reconnect.

(3) Upon successful reconnection, the camera's transmission data will be reset to the initial acquisition state, including the delivered frame count and residual frames being cleared.

2.2.11. Properties Setting UI

GxIAPI provides a properties setting window UI based on WxWidgets, supporting four programming languages: C, C++, C#, and Python. It is a pop-up window and is non-blocking by default.

2.2.11.1. Feature Description

Feature controller	Feature explanation
Search	After entering keywords of the property node in the search bar, all nodes containing those keywords within the camera will be filtered. Clicking the corresponding item will navigate to the settings location for that node.
ExpandAll / CollapseAll	Click ExpandAll to display all nodes in the properties tree. And click CollapseAll to collapse the nodes in the properties tree.
Auto Polling	Tick Auto Polling to activate the Polling function. When properties within the camera changed, the corresponding UI elements in the properties tree will update accordingly. Automatic Auto Polling are disabled by default.
Visibility	Visibility levels are categorised as Beginner, Expert, and Guru. Selecting the corresponding visibility level will filter the relevant property nodes within the properties tree.
Properties tree	The properties tree displays camera device information, stream information, local configuration information, and frame grabber information in sequence. Corresponding sub-items may be selected for configuration.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

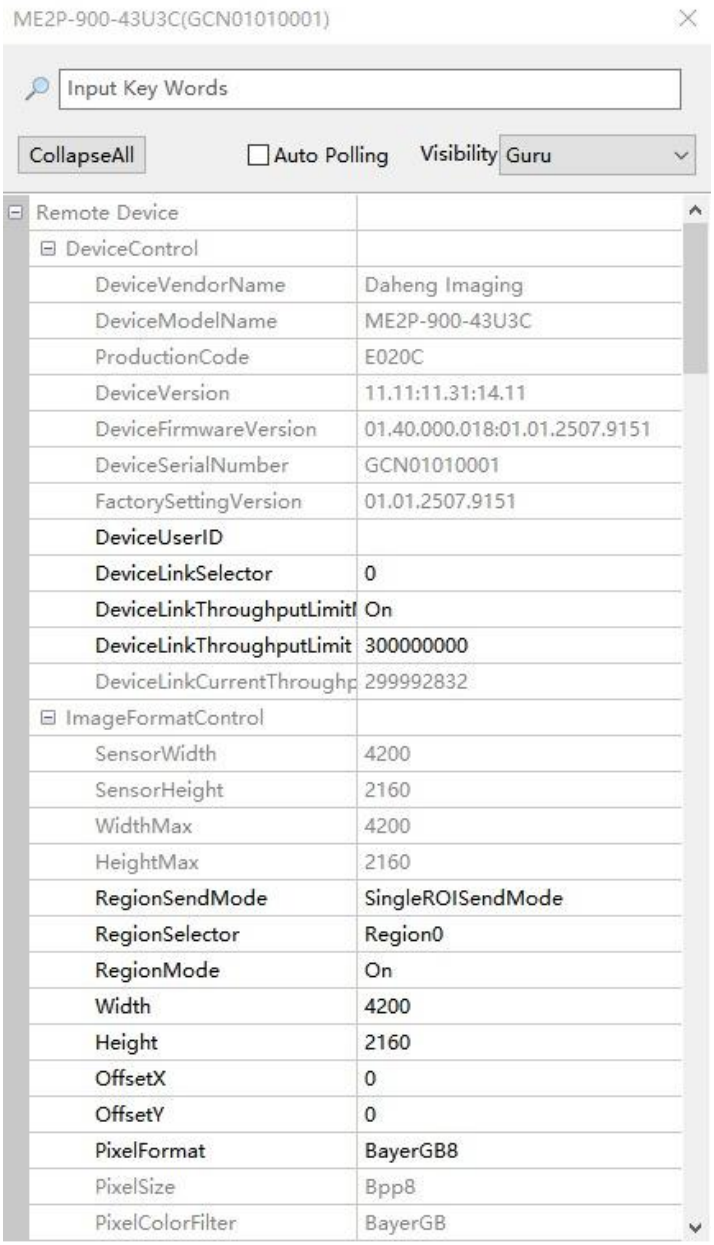


Figure 2-15 Properties Setting UI

2.2.11.2. Sample Code

Steps to display the properties setting UI:

(1) Create the handle of properties Window.

(2) Set the display position of the Properties window.

(3) Set the shown mode of the Properties window (The BLOCK_SHOW_MODE setting is only configured within the console environment. Do not configure it within the GUI application, as doing so may result in undefined behavior).

(4) Set the display title of the Properties window. If no title is set, the default display will be the user's custom name. Should neither the custom name nor any title be configured, the camera's model number and serial number will be displayed as the title.

(5) Destroy the properties window.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```c
#include "GxIAPI.h"

int main(int argc, char *argv[])
{
    GXInitLib();

    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    GX_OPEN_PARAM stOpenParam;
    uint32_t ui32DeviceNum = 0;

    emStatus = GXUpdateDeviceList(&ui32DeviceNum, 1000);
    if ((emStatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
    {
        GXCloseLib();
        return 0;
    }

    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emStatus = GXOpenDevice(&stOpenParam, &hDevice);

    if (emStatus == GX_STATUS_SUCCESS)
    {
       // Declare window operation handle.
       GX_OPERATE_HANDLE phOperateWnd = NULL;

       //1. Create the handle of window operation.
       emStatus = GXCreateWnd(phDevice, GX_WND_PROP, NULL, &phOperateWnd);
       if (emStatus != GX_STATUS_SUCCESS)
        {
            GXCloseDevice(hDevice);
            GXCloseLib();
            return 0;
        }

       //2. The Properties window is positioned relative to the top-left
       // corner of the screen, with a width and height of 700 pixels.
       emStatus = GXSetShowPosition(phOperateWnd, 0, 0, 700, 700);
       if (emStatus != GX_STATUS_SUCCESS)
        {
            GXDestroyWnd(phOperateWnd);
            GXCloseDevice(hDevice);
            GXCloseLib();
            return 0;
        }


       //3. Set the properties window display mode to block (set
       // BLOCK_SHOW_MODE when the programme is a console
       // application. Do not set for GUI programmes).
       emStatus = GXSetShowMode(phOperateWnd, BLOCK_SHOW_MODE);
       if (emStatus != GX_STATUS_SUCCESS)
        {
            GXDestroyWnd(phOperateWnd);
            GXCloseDevice(hDevice);
            GXCloseLib();
            return 0;
        }
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
        //4. Set the title of the properties window.
        emStatus = GXSetWndTitle(phOperateWnd, "title");
        if (emStatus != GX_STATUS_SUCCESS)
         {
             GXDestroyWnd(phOperateWnd);
            GXCloseDevice(hDevice);
            GXCloseLib();
            return 0;
         }

        //5. Display the window.
        emStatus = GXShowWnd(phOperateWnd, true);
        if (emStatus != GX_STATUS_SUCCESS)
         {
             GXDestroyWnd(phOperateWnd);
            GXCloseDevice(hDevice);
            GXCloseLib();
            return 0;
         }

        //6. Destroy the window.
        emStatus = GXDestroyWnd(phOperateWnd);
        if (emStatus != GX_STATUS_SUCCESS)
         {
             GXDestroyWnd(phOperateWnd);
            GXCloseDevice(hDevice);
            GXCloseLib();
            return 0;
         }
    }

    GXCloseDevice(hDevice);
    GXCloseLib();
    return 0;
}
```

2.2.11.3. Notes

(1) This properties window does not support saving or loading the coefficients of pixel level FFC. Should you require this functionality, please utilise GalaxyView or the FFC plugin.

(2) The blocking display mode (BLOCK_SHOW_MODE) is only applicable when invoked within a console framework. Should your application be a GUI programme (such as Qt, MFC, Winforms, etc.), please refrain from invoking it, as doing so may result in undefined behaviour.

2.3. Use the Gigabit Camera to Debug the Programs

Windows users are likely to encounter a situation that the device offline caused by a heartbeat timeout when using the Visual Studio development platform to debug a gigabit camera in debug mode. The application must be at a fixed time interval to send the heartbeat package to the device. If the device does not receive the heartbeat packages, you can think that the current connection has been disconnected, thus no longer accepting any commands sent from the application.

When the user runs the application program successfully, the underlying library will send heartbeat package normally, to maintain the connection state with the device, but if the user sets breakpoints in the application program, when the program runs to the breakpoint, the debugger will suspend all threads,

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

including the thread which sends heartbeat packages, so when you debugging the program code in the Debug mode, the thread will not send heartbeat packages to the device.

You can increase the heartbeat timeout time to resolve this problem. Under debugging mode, the gigabit transfer layer software will set the timeout time as 5 minutes automatically when the device is opened. If the environment variable **GIGE_HEARTBEAT_TIMEOUT** is added in the system, and the value of it is greater than 0, then the heartbeat timeout value of the device will automatically be set to the value of the environment variable.

You can modify the heartbeat timeout by two ways. The first way is to add the following codes to the program when the device is opened.

```
//The hDevice is the device handle. The device has been opened via the
//GXOpenDevice interface.

//Sets the heartbeat time to 5 minutes.
GXSetIntValue(hDevice, "GevHeartbeatTimeout", 300000);
```

The second way is to add environment variables **GIGE_HEARTBEAT_TIMEOUT** to the system and set the value greater than 0, then use the application program to open the device and the heartbeat timeout value will automatically become the value of environment variable. Note that you only need to add this environment variable to the developed system, both the debug and the release application will work.

Note: If you set the heartbeat timeout very long, when the program is ended, it still not calls the GXCloseDevice() interface to close the device, or not call the GXCloseLib() interface to release resources, this will cause the device cannot be reset during a heartbeat time, which leads to fail when you try to open the device again. This problem can be solved by resetting the device. There are two methods to reset the device:

1) Select “Reset Device” directly in the IP Configurator.

2) Reset the device by the GXGigEResetDevice() interface.

Reset device: It is equivalent to powering off and powering up the device, and the programs in the camera are completely reloaded.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

3. Camera Function Attribute Specification

3.1. Device Control

3.1.1. Device Information

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
DeviceVendorName	STRING	RO
DeviceModelName	STRING	RO
DeviceFirmwareVersion	STRING	RO
DeviceVersion	STRING	RO
DeviceSerialNumber	STRING	RO
FactorySettingVersion	STRING	RO
DevicePHYVersion	STRING	RO
DeviceISPFirmwareVersion	STRING	RO
DeviceUserID	STRING	RW

- Sample Code

```
// To get the name of the device's vendor, for example
//Get the vendor name
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_STRING_VALUE stVendorNameValue;
const char* pszVendorName = NULL;

emStatus = GXGetStringValue(hDevice, "DeviceVendorName",
&stVendorNameValue);

pszVendorName = stVendorNameValue.strCurValue;

//Get custom name
GX_STRING_VALUE stUserIDValue;
const char* strUserID = NULL;

emStatus = GXGetStringValue(hDevice, "DeviceUserID",
&stUserIDValue);
pszUserID = stUserIDValue.strCurValue;

// Set user defined name
emstatus = GXSetStringValue(hDevice, "DeviceUserID", "TestUserID");
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

3.1.2. Device Control

- Terms

Device reset: Restores the device to the initial state and the device is powered on again. After the call is completed, the host will lose its connection to the current device. And because the interface can be set only when the device is open, after the call is completed, the developer needs to actively call the GXCloseDevice() interface to close the device to release the corresponding memory resources.

Timestamp tick frequency: Read-only information, which represents the frequency of timestamp counter, and its value is 125000000Hz.

Timestamp latch: Latchs the current timestamp value, that is, the time value taken from the start of the device power-on to the call of the timestamp latch command. The time value needs to be read through the "timestamp latch value".

Reset timestamp: Resets the timestamp counter and recount from 0.

Reset timestamp latch: First latchs the current timestamp value, then resets the timestamp counter and recounts from 0.

Device temperature selector: Selects the location within the device, where the temperature will be measured.

Device temperature: Device temperature in degrees Celsius (℃).

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
DeviceReset	COMMAND	WO
TimestampTickFrequency	INT	RO
TimestampLatch	COMMAND	WO
TimestampReset	COMMAND	WO
TimestampLatchReset	COMMAND	WO
TimestampLatchValue	INT	RO
DeviceTemperatureSelector	ENUM	RW
DeviceTemperature	FLOAT	RO

- Sample Code

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//Send reset timestamp command.
emStatus = GXSetCommandValue(hDevice,"TimestampReset");

//Send device reset command.
emStatus = GXSetCommandValue(hDevice,"DeviceReset");

//Get device temperature selector.
GX_ENUM_VALUE  stValue;
emStatus = GXGetEnumValue(hDevice, "DeviceTemperatureSelector",
                          &stValue);
const char *pszString = stValue.stCurValue.strCurSymbolic;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
//Set device temperature selector.
emStatus=GXSetEnumValueByString(hDevice,
                                "DeviceTemperatureSelector",
                                "Sensor");

//Get device temperature.
GX_FLOAT_VALUE stTemperatureValue;
double dCurValue = 0;
emStatus = GXGetFloatValue(hDevice, "DeviceTemperature",
                           &stTemperatureValue );
dCurValue = stTemperatureValue.dCurValue;
```

3.2. Image Format

3.2.1. ROI

- Terms

ROI: Region of interest, a configurable rectangle selected area related to the sensor of the camera, the camera just output the data of selected area, and the data beyond the area will be ignored.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
SensorWidth	INT	RO
SensorHeight	INT	RO
WidthMax	INT	RO
HeightMax	INT	RO
Width	INT	RW
Height	INT	RW
OffsetX	INT	RW
OffsetY	INT	RW

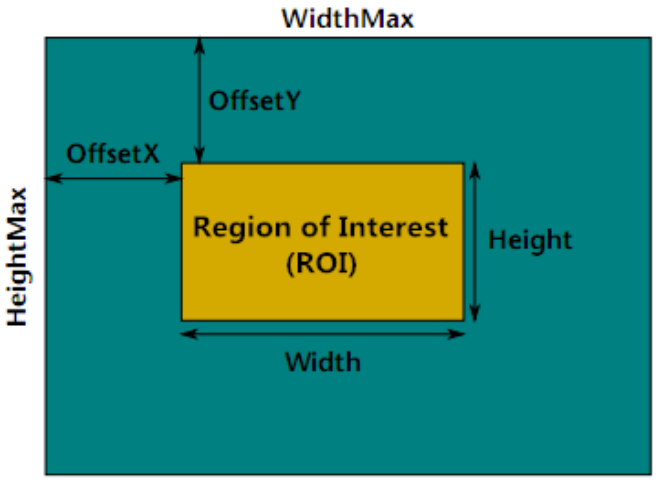


Figure 3-1 The related parameters of ROI

- Effect images



Figure 3-2 The original image



Figure 3-3 The image after ROI

- Sample Code

```
//Sets an area with an offset of (0,0) and a size of 600x400.
GX_STATUS emStatus = GX_STATUS_SUCCESS;
int64_t i64Width= 600;
int64_t i64Height= 400;
int64_t i64OffsetX = 0;
int64_t i64OffsetY = 0;
emStatus = GXSetIntValue(hDevice, "Width", i64Width);
emStatus = GXSetIntValue(hDevice, "Height", i64Height);
emStatus = GXSetIntValue(hDevice, "OffsetX", i64OffsetX);
emStatus = GXSetIntValue(hDevice, "OffsetY", i64OffsetY);
```

- Precautions

1) There are two pairs of attributes that affect the maximum ROI size: “SensorWidth” and “SensorHeight”.

The width and height of the sensor determine the effective resolution of the image sensor, and also determine the total of the pixels available, and the values are fixed. In the default mode (no Binning, no Decimation, no ROI), the image size is equal to “SensorWidth” * “SensorHeight”.

“WidthMax” and “HeightMax” determine the maximum size of the current ROI available, and the maximum ROI width can be affected by binning or decimation. In the default mode (no Binning, no Decimation):

“WidthMax” * “HeightMax” = “SensorWidth” * “SensorHeight”.

2) In order to ensure that the ROI is valid, the four attributes of ROI need to follow the relational formula, as follows:

“OffsetX” + “Width” <= “WidthMax” (the maximum width of the current image)

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

"OffsetY" + "Height" <= "HeightMax" (the maximum height of the current image)

Two formulas above illustrate the maximum of four attributes are dynamic changes, for example, if you adjust the value of the width, then the adjustable maximum value of the OffsetX will be affected, the correlation has been achieved in the GxIAPI.

3.2.2. Image Resolution

- Terms

Binning and Decimation directly affect the camera sensor, before generating the image, the sub-sample processing operation has completed. These two functions can improve the frame rate by modifying the resolution of the horizontal or the vertical direction. Compared to ROI, the ROI function is to cut the field of view, but the Binning and Decimation are handled with the whole image, do not affect the field of view.

1) Binning: It is an image output mode. Two types of Binning are available: horizontal Binning and vertical Binning. In this mode, the charges of adjacent pixels are added according to the Sum mode selected by the user, or the average of adjacent pixel charges is taken. The value is output in one-pixel mode. The advantage of using Binning is that several pixels can be combined for use as one pixel, which increases the camera's response to light, output speed, and reduce resolution. When Binning is used for both rows and columns, the aspect ratio of the image does not change. When using 2:2 Binning, the resolution of the image will be reduced by 75%.
2) Decimation: The pixel skip output, picking the Nth (horizontal or vertical) pixel for output, and the other pixels are ignored.
3) Horizontal Binning mode: This mode has two modes: Sum and Average. In Sum mode, adjacent charges are added together and then output in one-pixel mode. When the pixel value is greater than the maximum value, the maximum value is taken. In Average mode: the adjacent charges are added together and averaged.
4) Vertical Binning mode: Like horizontal Binning mode.
5) DecimationLineNumber: The line number of decimation, For details: *MERCURY2 USB3 Vision Cameras User Manual section 8.3*.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
BinningHorizontal	INT	RW
BinningVertical	INT	RW
DecimationHorizontal	INT	RW
DecimationVertical	INT	RW
BinningHorizontalMode	ENUM	RW
BinningVerticalMode	ENUM	RW
DecimationLineNumber	INT	RW
SensorDecimationHorizontal	INT	RW
SensorDecimationVertical	INT	RW

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

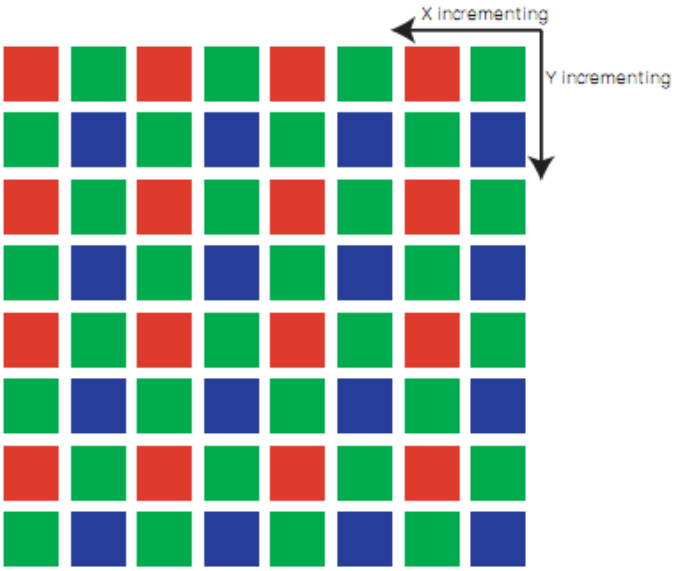


Figure 3-4 Original image

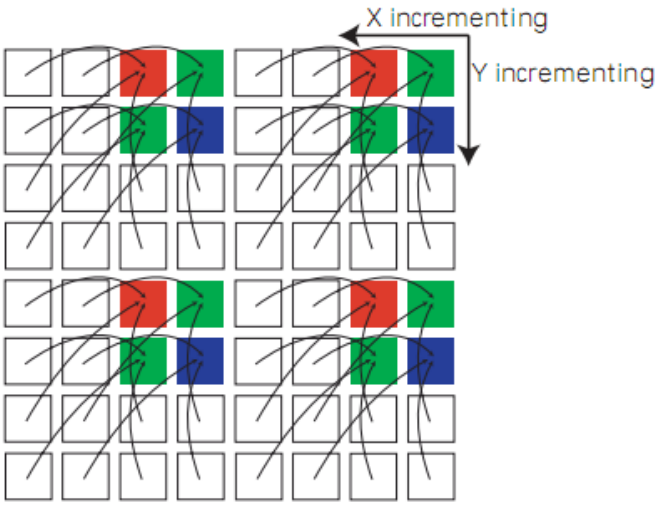


Figure 3-5 Binning (2x2) processing

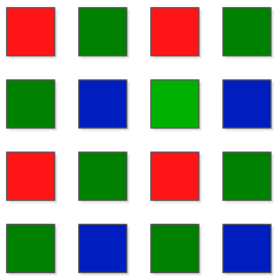


Figure 3-6 The Binning processing result

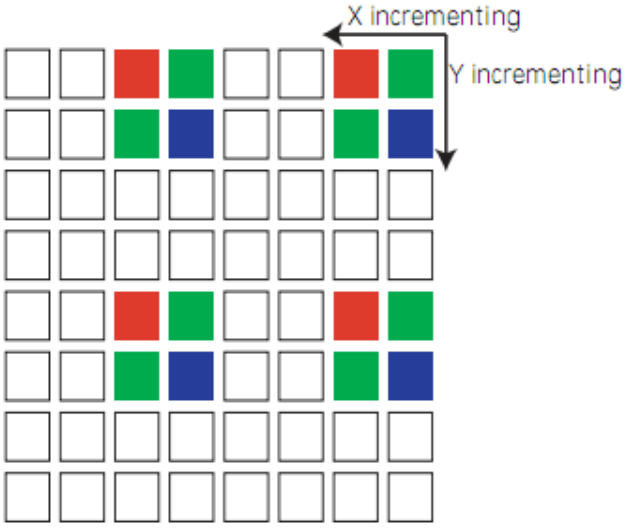


Figure 3-7 Decimation（2x2）processing

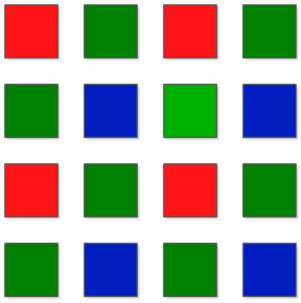


Figure 3-8 The decimation processing result

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

- Sample Code

```
//Configures a 2×2 Binning and 2×2 Decimation.
GX_STATUS emStatus = GX_STATUS_SUCCESS;
int64_t i64BinningH = 2;
int64_t i64BinningV = 2;
int64_t i64DecimationH = 2;
int64_t i64DecimationV = 2;

//Set horizontal and vertical Binning mode to Sum mode.
emStatus = GXSetEnumValueByString(hDevice, "BinningHorizontalMode",
                                  "Sum");
emStatus = GXSetEnumValueByString(hDevice, "BinningVerticalMode",
                                  "Sum");
emStatus = GXSetIntValue(hDevice, "BinningHorizontal", i64BinningH);
emStatus = GXSetIntValue(hDevice, "BinningVertical", i64BinningV);
emStatus = GXSetIntValue(hDevice, "DecimationHorizontal",
                         i64DecimationH);
emStatus = GXSetIntValue(hDevice, "DecimationVertical",
                         i64DecimationV);
```

- Precautions

3.2.3. Data Format

- Terms

1) Data bit depth: The data bit depth represents that the data bits of each pixel gray value occupied. For example, 8 bits of data represent a gray value range of 0 to 255. The RAW data format converts the captured optical signal into a digital signal by a CMOS or CCD image sensor without any compression. RAW8 indicates that the output image data bit is 8 bits, and RAW12 indicates that the output image data bit is 12 bits.

2) Bayer color conversion: The Bayer type is the formatting of RAW image data, see Figure 3-9, when the image data is processed or displayed, it needs to convert it to a 24 bits RGB real color image data. A simple interpolation algorithm is as follows:

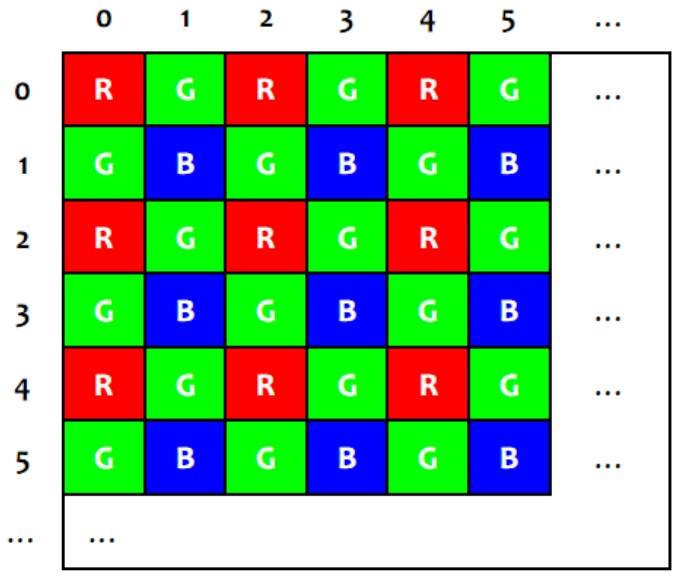


Figure 3-9 Bayer format

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

One simple transformation is as follows:

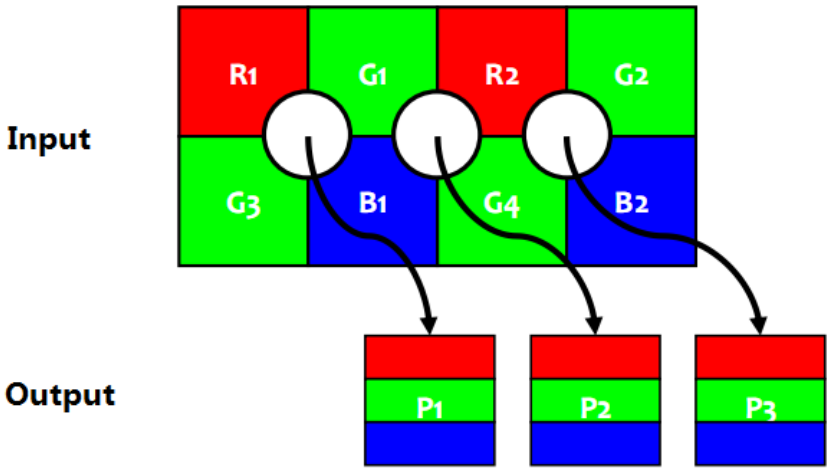


Figure 3-10 Bayer conversion

Pixel 1 (P1)

$$P1_{Red} = R1$$

$$P1_{Green} = \frac{G1 + G3}{2}$$

$$P1_{Blue} = B1$$

Pixel 2 (P2)

$$P2_{Red} = R2$$

$$P2_{Green} = \frac{G1 + G4}{2}$$

$$P2_{Blue} = B1$$

Pixel 3 (P3)

$$P3_{Red} = R2$$

$$P3_{Green} = \frac{G2 + G4}{2}$$

$$P3_{Blue} = B2$$

Figure 3-11

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
PixelSize	ENUM	RO
PixelColorFilter	ENUM	RO
PixelFormat	ENUM	RW

Data Format: The PixelFormat is 8Byte, 32bit. It gathers various of information together to form a single amount of information. The highest two bytes represents the color type (color or monochrome), the followed two bytes represent the data bit depth (8bit, 10bit, etc.), and the lowest four bytes represent the coding sequence number.

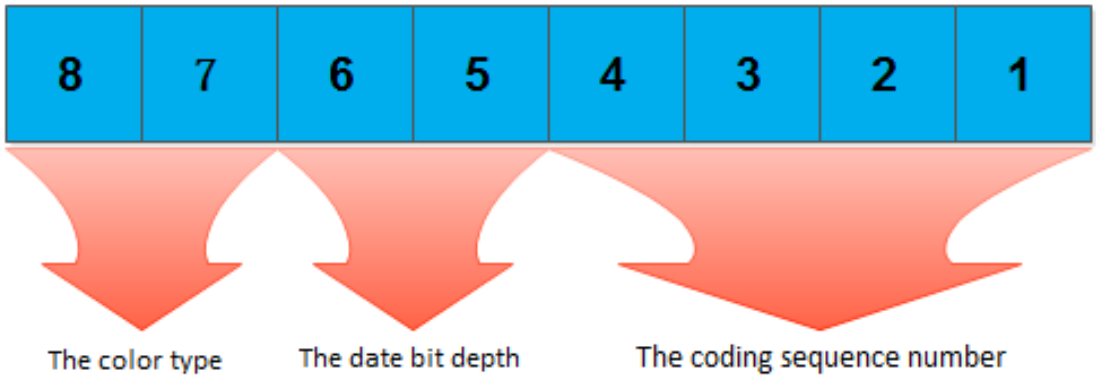


Figure 3-12 Data format

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

- Sample Code

```
//Use the GXGetEnumValue interface
//Query the "PixelFormat" type supported by the current camera
//Please refer to the interface descriptions, which are omitted here.
GX_STATUS emstatus = GX_STATUS_SUCCESS;

//Reads the current pixelformat.
GX_ENUM_VALUE stPixelFormat;
int64_t i64PixelFormat = 0;
emStatus = GXGetEnumValue(hDevice, "PixelFormat", &stPixelFormat);
i64PixelFormat = stPixelFormat.stCurValue.nCurValue;

//Sets the pixelformat to the bayer format of the BG type.
emstatus = GXSetEnumValueByString(hDevice, "PixelFormat", "BayerBG10");

//Reads the current pixelsize.
GX_ENUM_VALUE stPixelSize;
int64_t i64PixelSize = 0;
emStatus = GXGetEnumValue(hDevice, "PixelSize", &stPixelSize);
i64PixelSize = stPixelSize.stCurValue.nCurValue;

//Reads the current colorfilter.
GX_ENUM_VALUE stColorFilter;
int64_t i64ColorFilter = 0;
emStatus = GXGetEnumValue(hDevice, "PixelColorFilter",
                          &stColorFilter);
i64ColorFilter = stColorFilter.stCurValue.nCurValue;
```

3.2.4. Test Images

- Terms

Test Images: The GigE Vision series cameras support three test images: gray gradient test image, moving vertical stripe test image, and moving diagonal gray gradient test image; The USB3 Vision series cameras support three test images: gray gradient test image, moving diagonal gray gradient test image, and static diagonal gray gradient test image; The Pallas series cameras support three test images: gray gradient test image, moving diagonal gray gradient test image, and static diagonal gray gradient test image.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
TestPatternGeneratorSelector	ENUM	RW
TestPattern	ENUM	RW

- Sample Code

```
GX_STATUS emstatus = GX_STATUS_SUCCESS;

//Querie what type of test images the current camera supports. See
//the usage of the GXGetEnumValue interface.
//This example assumes that the current camera supports all types of test
//images.
//Sets to scroll vertical stripes test image.
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    emstatus = GXSetEnumValueByString(hDevice, "TestPattern",
                                      "VerticalLineMoving");
    //Closes the test image function.
    emstatus = GXSetEnumValueByString(hDevice, "TestPattern", "Off");
```

3.2.5. Sensor Shutter Mode

- Terms

Sensor shutter mode: Refer to the way in which image data is captured and processed. Depending on the design, the imaging sensor can support different shutter modes, and there are three optional values available. Global: All pixels start exposing at the same time, and the exposure time is the same. Rolling: All pixels have the same exposure time, but the start time of pixels in each row is different. GlobalReset: All pixels start exposing at the same time, but the exposure time of pixels in each row is different.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
SensorShutterMode	ENUM	RW

- Sample Code

```
    GX_STATUS emstatus = GX_STATUS_SUCCESS;

    //Sets the sensor shutter mode to Global.
    emstatus = GXSetEnumValueByString(hDevice, "SensorShutterMode",
                                      "Global");
```

3.3. Acquisition Control

3.3.1. Acquisition

- Terms

1) Acquisition mode: To set the image generation mode of the camera. It mainly defines the number of the images in an acquisition period of the camera. An acquisition period means that from send start acquisition command to send stop acquisition command.
2) Acquisition frame count: When the acquisition mode is multi-frame, the parameter determines the number of the images in an acquisition period of the camera.
3) Acquisition speed level: To control the frame rate of the camera. The larger the acquisition speed level, the greater the frame rate; the smaller the acquisition speed level, the smaller the frame rate.
4) Acquisition burst frame count: Acquire multiple frames of images by a single trigger. For example, if the “Acquisition burst frame count” parameter is set to three, the camera automatically outputs three images after the trigger.
5) Acquisition status selection: The acquisition status is used to determine if the camera is waiting for a trigger signal. There are two modes: FrameTriggerWait and AcquisitionTriggerWait. FrameTriggerWait: After selecting this status, you can determine whether the camera is waiting for the frame start trigger signal by querying the acquisition function status. AcquisitionTriggerWait: After selecting this status, you can determine whether the camera is waiting for the trigger signal in the multi-frame acquisition status by querying the acquisition status function. The function is only used in the trigger mode and has no effect on the continuous acquisition mode.
6) Acquisition status: This function code is used together with the acquisition status selection function. For details, please refer to the acquisition status selection. During non-acquisition period (before the acquisition is started, after the acquisition is stopped), the value of the query has no meaning in the non-trigger mode.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
AcquisitionMode	ENUM	RW
AcquisitionStart	COMMAND	WO
AcquisitionStop	COMMAND	WO
AcquisitionFrameCount	INT	RW
AcquisitionSpeedLevel	INT	RW
AcquisitionBurstFrameCount	INT	RW
AcquisitionStatusSelector	ENUM	RW
AcquisitionStatus	BOOL	RO

- Control Workflow

(About the concept of trigger mode, please ref the next section--Trigger)

Non-trigger + Continous acquisition mode:

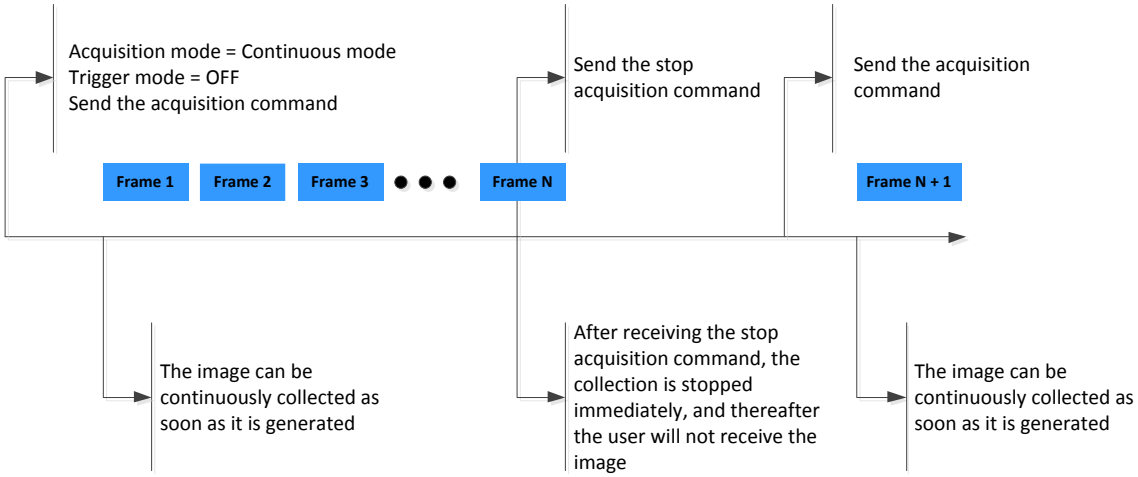


Figure 3-13 Non-trigger + Continous acquisition mode

Trigger + Continous acquisition mode:

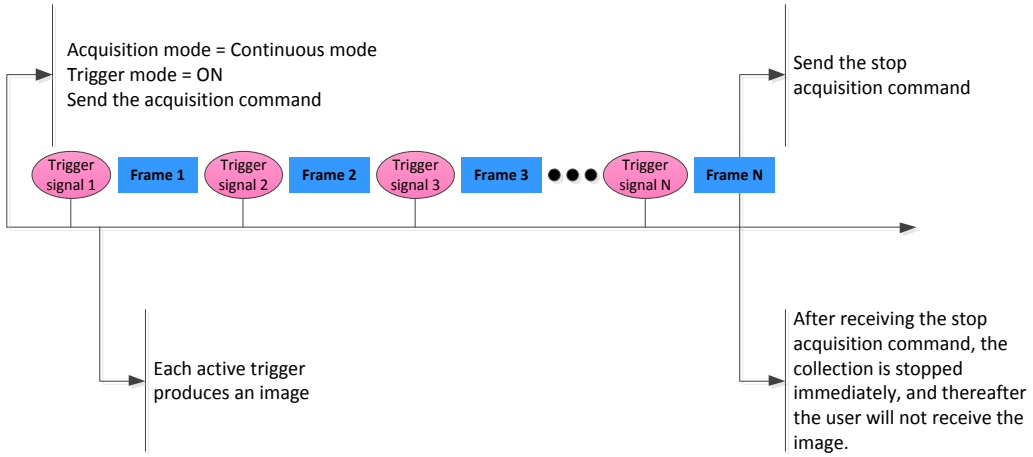


Figure 3-14 Trigger + continous acquisition mode

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Non-trigger + Multi-frame acquisition mode:

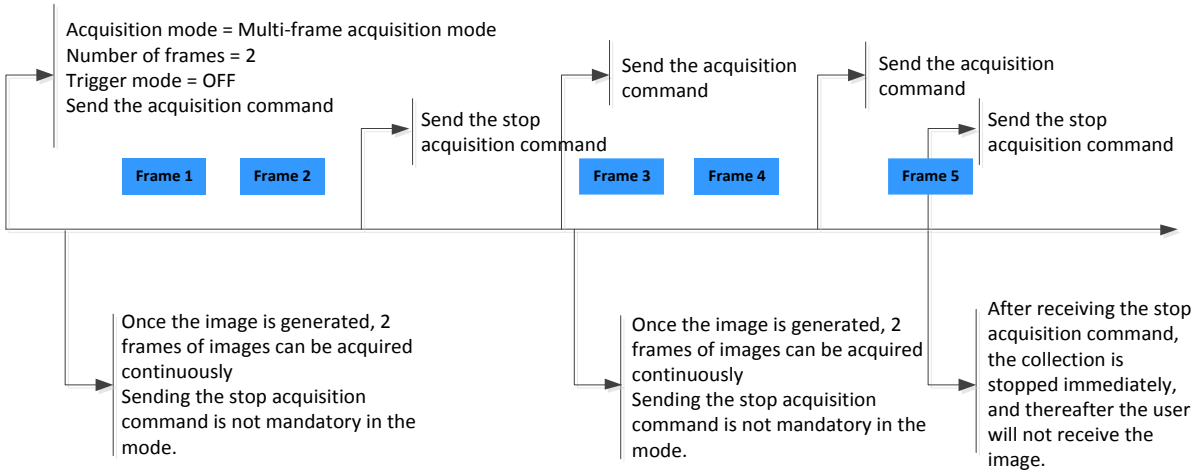


Figure 3-15 Non-trigger + multi-frame acquisition mode

Trigger + Multi-frame acquisition mode:

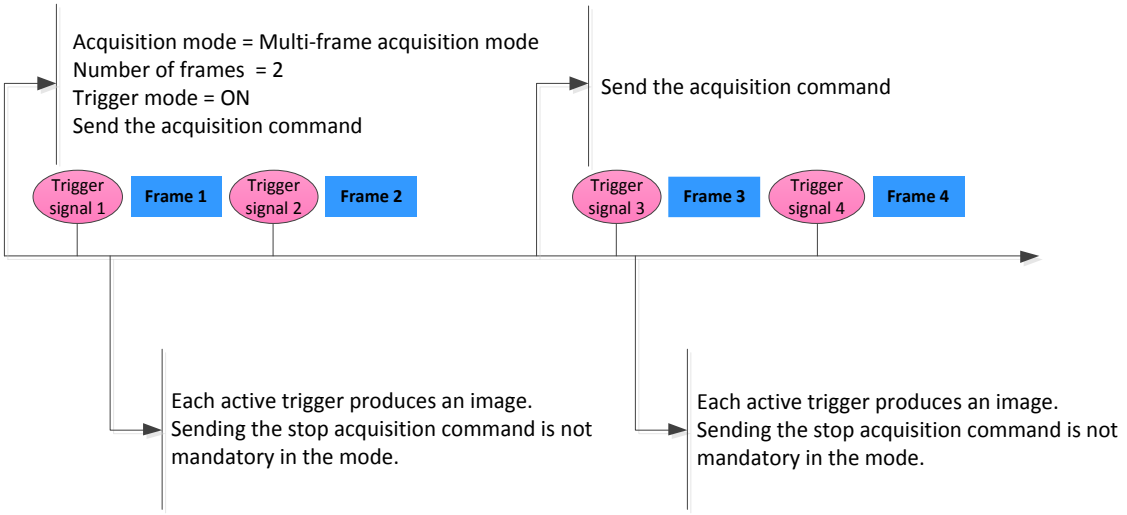


Figure 3-16 Trigger + multi-frame acquisition mode

Non-trigger + Single frame acquisition mode:

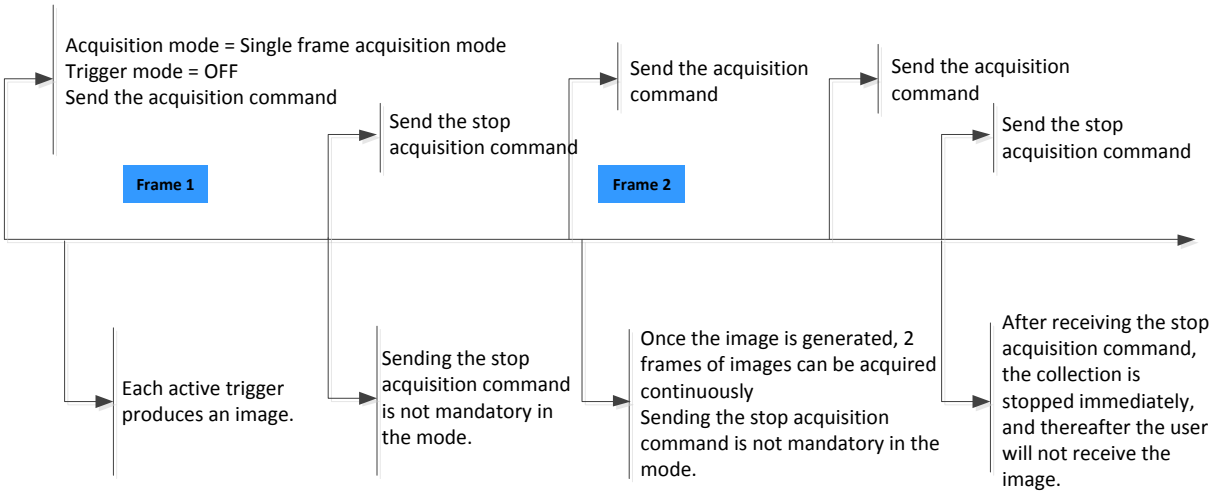


Figure 3-17 Non-trigger + single frame acquisition mode

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Trigger + Single frame acquisition mode:

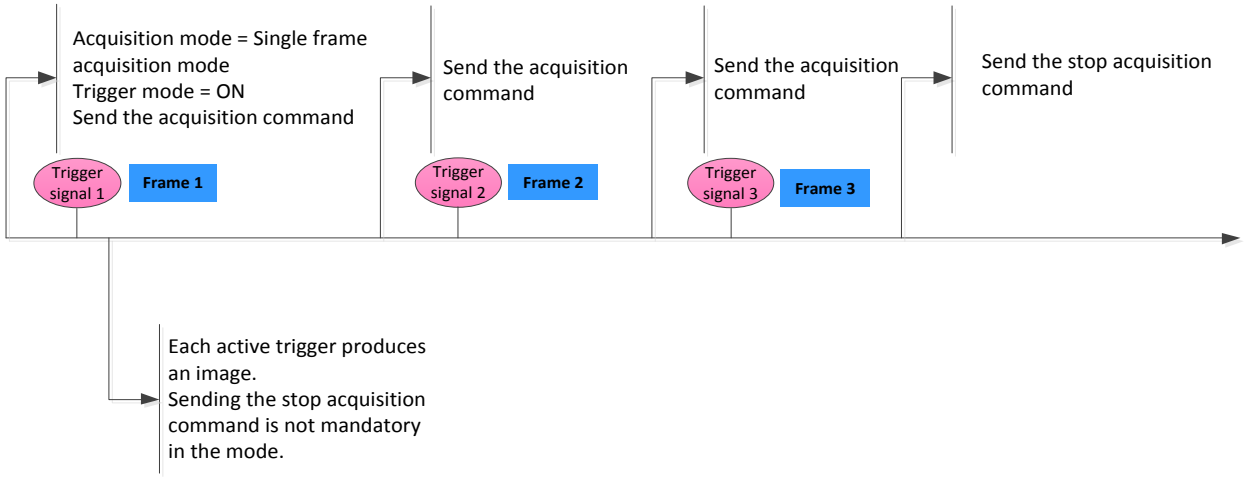


Figure 3-18 Trigger + single frame acquisition mode

- Sample Code

```
#include "GxIAPI.h"

//Image processing callback function.
static void GX_STDC OnFrameCallbackFun(GX_FRAME_DATA_EX* pFrame)
{
    if (pFrame->status == 0)
    {
        //Performs some image processing operations.
    }
    return;
}
int main(int argc, char* argv[])
{
    GX_STATUS emstatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    GX_OPEN_PARAM stOpenParam;
    uint32_t ui32DeviceNum = 0;

    //Initializes the library.
    emstatus = GXInitLib();
    if (emstatus != GX_STATUS_SUCCESS)
    {
        return 0;
    }

    //Updates the enumeration list for the devices.
    emstatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
    if ((emstatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
    {
        return 0;
    }

    //Opens the device.
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    stOpenParam.openMode = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emStatus = GXOpenDevice(&stOpenParam, &hDevice);
    if (emStatus == GX_STATUS_SUCCESS)
    {
        //Setting device's property of GevSCPSPacketSize to improve the
        //acquisition performance of the network camera.
        bool bImplementPacketSize = false;
        uint32_t ui32PacketSize = 0;

        //Determine whether the device supports GevSCPSPacketSize.
       GX_NODE_ACCESS_MODE emAccessMode;
       emStatus=GXGetNodeAccessMode(hDevice, "GevSCPSPacketSize",
                                    &emAccessMode);
       bImplementPacketSize=(emAccessModem = GX_NODE_ACCESS_MODE_RW)
                            ? true : false;

        if (bImplementPacketSize)
       {
            //Get Optimal PacketSize.
            emStatus = GXGetOptimalPacketSize (hDevice, &ui32PacketSize);

            //Set the Optimal PacketSize to GevSCPSPacketSize.
            emStatus = GXSetIntValue(hDevice, "GevSCPSPacketSize",
                                     ui32PacketSize);
       }
        //Set acquisition mode. Default acquisition mode for General camera is
        //continuous mode.
        //emstatus = GXSetEnumValueByString(hDevice, "AcquisitionMode",
                                            "Continuous");

        //Registers image processing callback function.
        emStatus = GXRegisterCaptureCallbackEx(hDevice, NULL,
                                               OnFrameCallbackFun);

        //Send a start acquisition command.
        emStatus = GXSetCommandValue(hDevice, "AcquisitionStart");
        //----------------------
        //
        //In this interval, the image will be returned to the user via the
        //OnFrameCallbackFun interface.
        //
        //----------------------

        //Send a stop acquisition command.
        emStatus = GXSetCommandValue(hDevice, "AcquisitionStop");

        //Unregisters image processing callback function.
        emSstatus = GXUnregisterCaptureCallback(hDevice);
        }
        emStatus  = GXCloseDevice(hDevice);
        emStatus  = GXCloseLib();
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
        return 0;
}
```

- Precautions

3.3.2. Trigger

3.3.2.1. General Function

- Terms

1) Trigger Source: Select the source of the trigger signal. Software trigger or hardware trigger.
2) Trigger Mode: To determine whether the trigger signal is valid or not. On-valid; OFF-invalid.
3) Trigger Polarity: The activate mode of trigger. Rising edge valid or falling edge valid.
4) Software Command: Simulation the trigger signal with software.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
TriggerMode	ENUM	RW
TriggerSoftware	COMMAND	WO
TriggerActivation	ENUM	RW
TriggerSwitch	ENUM	RW
TriggerSource	ENUM	RW
TriggerSelector	ENUM	RW
TriggerDelay	FLOAT	RW

- Sample Code

```
#include "GxIAPI.h"

//Image processing callback function.
static void GX_STDC OnFrameCallbackFun(GX_FRAME_DATA_EX* pFrame)
{
    if (pFrame->status == 0)
    {
        //Performs some image processing operations.
    }
    return;
}

int main(int argc, char* argv[])
{
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    GX_OPEN_PARAM stOpenParam;
    uint32_t ui32DeviceNum = 0;

    //Initializes the library.
    emStatus = GXInitLib();
    if (emStatus != GX_STATUS_SUCCESS)
    {
        return 0;
```

```
    }

    //Updates the enumeration list for the devices.
    emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
if ((emStatus != GX_STATUS_SUCCESS)||(ui32DeviceNum<= 0))
    {
        return 0;
    }

    //Opens the device.
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emStatus = GXOpenDevice(&stOpenParam, &hDevice);
    if (emStatus == GX_STATUS_SUCCESS)
    {
        //Setting device's property of GevSCPSPacketSize to improve
        //the acquisition performance of the network camera
        bool bImplementPacketSize = false;
        uint32_t ui32PacketSize = 0;

        //Determine whether the device supports GevSCPSPacketSize
        GX_NODE_ACCESS_MODE emAccessMode;
         emStatus = GXGetNodeAccessMode(hDevice,"GevSCPSPacketSize",
                                        &emAccessMode);

        bImplementPacketSize = (emAccessMode == GX_NODE_ACCESS_MODE_RW)
                               ? true : false;

        if (bImplementPacketSize)
        {
            //Get Optimal PacketSize
            emStatus = GXGetOptimalPacketSize (hDevice, &ui32PacketSize);

            //Set the Optimal PacketSize to GevSCPSPacketSize
            emStatus = GXSetIntValue (hDevice, "GevSCPSPacketSize",
                                      ui32PacketSize);
        }
        //Sets the trigger mode to ON.
        emStatus =GXSetEnumValueByString(hDevice, "TriggerMode", "On");

        //Sets the trigger activation mode to the rising edge.
         emStatus = GXSetEnumValueByString(hDevice, "TriggerActivation",
                                           "RisingEdge");


        //Registers image processing callback function.
        emStatus = GXRegisterCaptureCallbackEx(hDevice, NULL,
                                           OnFrameCallbackFun);

        //Sends a start acquisition command.
        emStatus =GXSetCommandValue(hDevice,"AcquisitionStart");
```

```
            //----------------------
            //
            //In this interval, the image will be returned to the user via the
            //OnFrameCallbackFun interface.
            //----------------------

            //Sends a stop acquisition command.
            emStatus = GXSetCommandValue(hDevice, "AcquisitionStop");

            //Unregisters image processing callback function.
            emStatus = GXUnregisterCaptureCallback(hDevice);
        }
        emStatus  = GXCloseDevice(hDevice);
        emStatus  = GXCloseLib();
        return 0;
    }
```

- Precautions

3.3.2.2. Advanced Function

- Terms

1) Trigger filtering value at rising edge: If the pulse width at the rising edge is smaller than the trigger filtering value, then the camera will not handle the trigger signal. When the pulse width at the rising edge is larger or equal to the trigger filtering value, it will generate the valid trigger signal.
2) Trigger filtering value at falling edge: If the pulse width at the falling edge is smaller than the trigger filtering value, then the camera will not handle the trigger signal. When the pulse width at the falling edge is larger or equal to the trigger filtering value, it will generate the valid trigger signal.

- Related Parameters

Attribute Name	Attribute Type	Accessible Properties
TriggerFilterRaisingEdge	FLOAT	RW
TriggerFilterFallingEdge	FLOAT	RW

- Trigger waveform figure

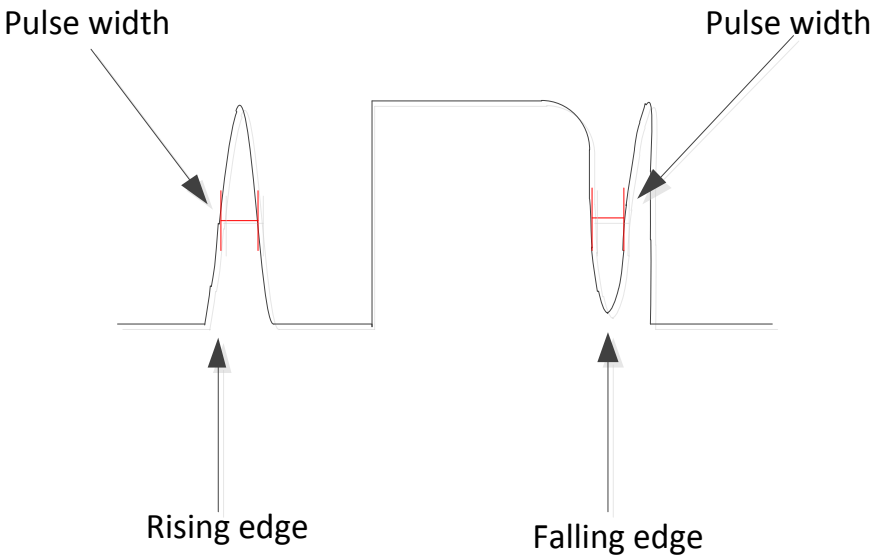


Figure 3-19 Trigger waveform

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

- Sample Code

```
//Gets the settings range of rising edge filter.
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_FLOAT_VALUE stRaisingRange;
emStatus = GXGetFloatValue(hDevice, "TriggerFilterRaisingEdge",
                           &stRaisingRange);

//Sets the rising edge filter to the minimum value.
emStatus  = GXSetFloatValue(hDevice, "TriggerFilterRaisingEdge",
                            stRaisingRange.dMin);


//Sets the rising edge filter to the maximum value.
emStatus  = GXSetFloatValue(hDevice, "TriggerFilterRaisingEdge",
                            stRaisingRange.dMax);

//Gets the current rising edge filter value.
double dValueUp = stRaisingRange.dCurValue;

//Gets the settings range of falling edge filter.
GX_FLOAT_RANGE fallingRange;
emStatus  = GXGetFloatValue(hDevice, "TriggerFilterFallingEdge",
                            &stFallingRange);

//Sets the falling edge filter to the minimum value.
emStatus  = GXSetFloatValue(hDevice, "TriggerFilterFallingEdge",
                            stFallingRange.dMin);

//Sets the falling edge filter to the maximum value.
emStatus  = GXSetFloatValue(hDevice, "TriggerFilterFallingEdge",
                            stFallingRange.dMax);

//Gets the current falling edge filter value.
double dValueDown = stFallingRange.dCurValue;
```

- Precautions

3.3.3. Exposure

- Terms

1) Exposure Mode: Different modes of exposure, in Timed mode, you can use “ExposureTime” or “ExposureAuto”.
2) Exposure Time Mode: Different modes of exposure time, including Standard mode and UltraShort mode. “ExposureAuto” is only available in Standard mode.
3) Exposure Time: That is shutter time, refers to the time interval of the sensor shutter from open to close, in the interval the object can be captured in the sensor. If you reduce the exposure time, the image will darker, increase the exposure time, the image will brighter.
4) Exposure Delay: The exposure delay function can effectively solve the strobe delay problem. Most strobes have a delay of at least tens of microseconds from trigger to light. When the camera is working in a small exposure mode, the fill light effect of the strobe will be affected. The exposure delay is achieved by the strobe signal and the delay of the actual exposure starting.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
ExposureMode	ENUM	RW
ExposureTimeMode	ENUM	RW
ExposureTime	FLOAT	RW
ExposureAuto	ENUM	RW
ExposureDelay	FLOAT	RW

- Effect images



Figure 3-20 Original image



Figure 3-21 Reduce exposure time



Figure 3-22 Increase exposure time

- Sample Code

```
//Gets the adjustment range of exposure time.
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_FLOAT_VALUE stShutterRange;
double dExposureValue = 2.0;

emStatus = GXGetFloatValue(hDevice, "ExposureTime",
                           &stShutterRange);

//Sets the exposure time to the minimum.
emStatus = GXSetFloatValue(hDevice, "ExposureTime",
                           stShutterRange.dMin);

//Sets the exposure time to the maximum.
emStatus = GXSetFloatValue(hDevice, "ExposureTime",
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
                                stShutterRange.dMax);

    //Sets the exposure mode to continuous automatic exposure
    emStatus = GXSetEnumValueByString(hDevice, "ExposureAuto",
                                      "Continuous");
    //Sets the exposure delay to 2μs
    emStatus = GXSetFloatValue(hDevice, "ExposureDelay",
                               dExposureValue);

    //Sets the exposure time mode to UltraShort mode
    emStatus = GXSetEnumValueByString(hDevice, "ExposureTimeMode",
                                      "UltraShort");
```

- Precautions

When the external light source is sunlight or direct current (DC) light source, no special requirements for the exposure time, but when the external light source is alternating current (AC) light source, the exposure time must synchronize with the external light source (under 50Hz light source, the exposure time must be a multiple of 1/100s. Under 60Hz light source, the exposure time must be a multiple of 1/120s).

3.3.4. Transfer Control

- Terms

When multiple cameras are connected to the host by switches, if trigger these cameras to acquire images at the same time, some data may lost because the large instantaneous bandwidth of the switch and the storage capacity is limited. So, you need to use frame transfer delay to avoid this problem.

In trigger mode, by setting the transmission control mode as "User Control", when the camera receives software trigger commands or hardware trigger signal and completes the image acquisition, the images will be stored in the camera inside the frame memory, waiting for the host to send "start transfer" command, and then the camera will transfer the images to the host. The transfer delay time is determined by the host. When multi-cameras are triggered simultaneously, you can set different transmission delays for each camera, to avoid the large instantaneous bandwidth of the switch.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
TransferControlMode	ENUM	RW
TransferOperationMode	ENUM	RW
TransferStart	COMMAND	WO
TransferBlockCount	INT	RW

- Sample Code

```
    GX_STATUS emStatus  = GX_STATUS_SUCCESS;

    //The premise must be to ensure that the trigger mode is open.
    emStatus = GXSetEnumValueByString(hDevice, "TriggerMode", "On");

    //Set the transfer control mode to user control mode.
    emStatus = GXSetEnumValueByString(hDevice, "TransferControlMode",
                                      "UserControlled");

    //Set the transfer operation mode to the specified transfer frame mode.
    emStatus = GXSetEnumValueByString(hDevice, "TransferOperationMode",
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
                                        "MultiBlock");

    //Set the number of output frames per command.
    emStatus = GXSetIntValue(hDevice, "TransferBlockCount", 1);

    //Send a software trigger signal (or external trigger) after the start
   //of acquisition.
    emStatus = GXSetCommandValue(hDevice, "AcquisitionStart");
    emStatus = GXSetCommandValue(hDevice, "TriggerSoftware");

    //Sent a transfer command after the image is triggered.
    emStatus = GXSetCommandValue(hDevice, "TransferStart");
```

- Precautions

The transfer control function can only work in trigger mode.

3.3.5. Acquisition Burst Mode

- Terms

Acquisition burst mode setting is only applicable to trigger mode, which includes standard (GigE or USB3 interface is limited by the bandwidth) and high speed (GigE or USB3 interface is not limited by the bandwidth) mode. Acquisition is conducted according to the maximum acquisition capacity of the sensor, and the frame rate is generally greater than the current transmission capacity. It is mainly used in conjunction with the UserControlled mode (one of the transmission control mode).

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
AcquisitionBurstMode	ENUM	RW

- Sample Code

```
    GX_STATUS emStatus = GX_STATUS_SUCCESS;

    //1.Set trigger mode as 14 frame per acquisition
    emStatus = GXSetEnumValueByString(hDevice, "TriggerSelector",
                                      "FrameStart");
    emStatus = GXSetEnumValueByString(hDevice, "TriggerMode", "On");
    emStatus = GXSetIntValue(hDevice, "AcquisitionFrameCount", 14);

    //2.Set Acquisition burst mode as high speed.
    emStatus = GXSetEnumValueByString(hDevice, "AcquisitionBurstMode",
                                      "HighSpeed");

    //3.Set frame buffer as 14.
    emStatus = GXSetIntValue(hDevice, "FrameBufferCount", 14);

    //4.Set transfer control mode as UserControlled,and transfer block
    //as 14.
    emStatus = GXSetEnumValueByString(hDevice, "TransferControlMode",
                                       "UserControlled");
    emStatus = GXSetEnumValueByString(hDevice, "TransferOperationMode",
                                      "MultiBlock");
    emStatus = GXSetIntValue(hDevice, "TransferBlockCount", 14);

    //5.Start acquisition, do 1, 2, 3, 4 step before finishing
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
//acquisition and after opening the camera.
emStatus = GXSetCommandValue(hDevice, "AcquisitionStart");

//6.Send trigger command after acquisition (software or hardware
//trigger).
emStatus = GXSetCommandValue(hDevice, "TriggerSoftware");

//7.Send start transfer command (whether to deliver this command
//in time or later depends on your need).
//Note: Do next trigger after finishing data transfer this time.
emStatus = GXSetCommandValue (hDevice, "TransferStart");
```

- Precautions

Because of the transfer control mode is under user control, especially when multiple frames are transmitted at one time, the transmission is carried out according to the maximum transmission speed of the interface. Due to the different performance of the NIC, frame dropping may occur during long operation, so the IPG (Inter Packet Gap, default value: 0) needs to be increased. The IPG settings depend on the following factors:

- Frame settings in a single transmission
- Interval between two transmissions (100ms or above recommended)
- Packet Size (8164 recommended, NIC jumbo frame mode)

3.3.6. Frame Store Mechanism

- Terms

Frame Store Coverage: When the average bandwidth of the data written in the internal frame is greater than the average bandwidth of the data read from it, then the frame store will be full. If the frame store is full, the image data will be overwritten.

Frame Buffer: The frame count of the camera.

Frame Buffer Flush: Clear the previous data of frame buffer.

1) In trigger mode, the frame buffer flush command will be ignored while in acquisition process.
2) The frame buffer flush command will be ignored while in passive data transmission process.
3) Able to use when trigger mode is on and in UserControlled (Transmission control mode)

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
FrameBufferOverwriteActive	BOOL	RW
FrameBufferCount	INT	RW
FrameBufferFlush	COMMAND	WO

- Sample Code

```
GX_STATUS emStatus = GX_EMSTATUS_SUCCESS;

//Enables the frame store coverage.
emStatus = GXSetBoolValue(hDevice, "FrameBufferOverwriteActive", true);

//Disables the frame store coverage.
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
emStatus = GXSetBoolValue(hDevice, "FrameBufferOverwriteActive", false);

//Set trigger mode as ON.
emStatus = GXSetEnumValueByString(hDevice, "TriggerMode", "On");

//Set transfer control mode as UerControled.
emStatus = GXSetEnumValueByString(hDevice, "TransferControlMode",
                                  "UserControlled");

//Send frame buffer flush command (Be careful to use this command
//while in need).
EmStatus = GXSetCommandValue(hDevice, "FrameBufferFlush");
```

3.3.7. Frame Rate Control

- Terms

1) Acquisition frame rate mode: The function is to control whether to open the frame rate control mode. "on" is to start the frame rate control and "off" is to disable the frame rate control. For more details, please ref GxIAPI.h.
2) Acquisition frame rate: Desired acquisition frame rate.
3) Current acquisition frame rate: Acquisition frame mode in actual operation.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
AcquisitionFrameRateMode	ENUM	RW
AcquisitionFrameRate	FLOAT	RW
CurrentAcquisitionFrameRate	FLOAT	RO

- Sample Code

```
GX_STATUS emStatus = GX_EMSTATUS_SUCCESS;
//Enable the frame rate adjustment mode.
emStatus = GXSetEnumValueByString(hDevice, "AcquisitionFrameRateMode",
                                  "On");
//Set the acquisition frame rate. Assuming the setting is 10.0.
//users can set this value according to actual needs.
emStatus = GXSetFloatValue(hDevice, "AcquisitionFrameRate", 10.0);
```

- Precautions

If the value of " AcquisitionFrameRate " was set too large, beyond the actual operation ability of the camera, the camera will run at the maximum frame rate it can achieve. That is the value of the current acquisition frame rate (“ CurrentAcquisitionFrameRate ”). (The frame rate is affected by the exposure time, ROI and etc.)

3.3.8. Exposure Mode

- Terms

1) Timed: Timed exposure mode, the length of exposure is defined by the value of the camera's ExposureTime setting. This mode is available on all camera models.
   Timed exposure mode can be used in any trigger mode.
2) TriggerWidth: TriggerWidth exposure mode, the length of exposure is defined by the width of the hardware trigger signal, so this mode only takes effect in the external trigger mode. This is useful if

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

you intend to vary the length of exposure for each captured frame.

a) If rising edge triggering is enabled, exposure starts when the trigger signal rises and continues until the trigger signal falls.

b) If falling edge triggering is enabled, exposure starts when the trigger signal falls and continues until the trigger signal rises.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
ExposureMode	ENUM	RW

- Sample Code

```
GX_EMSTATUS emStatus = GX_EMSTATUS_SUCCESS;

//Sets the exposure mode.
emStatus = GXSetEnumValueByString(hDevice, "ExposureMode",
                                  "TriggerWidth");
// Sets the Exposure Overlap Time Max.
emStatus = GXSetFloatValue(hDevice, "ExposureOverlapTimeMax",
                           621.0);
```

- Precautions

If the TriggerWidth exposure mode is enabled, do not send trigger signals at too high a rate. Otherwise, trigger signals will be ignored.

In the TriggerWidth exposure mode, If the Exposure Overlap Time Max parameter is available, set it to the smallest exposure time you intend to use.

3.3.9. Exposure Overlap Time Max

- Terms

1) Exposure Overlap Time Max: You can use Exposure Overlap Time Max to increase the camera's frame rate. With overlapping image acquisition, the exposure of a new image begins while the camera is still reading out the sensor data of the previous image.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
ExposureOverlapTimeMax	FLOAT	RW

- Sample Code

```
GX_STATUS emStatus = GX_EMSTATUS_SUCCESS;

 //Sets the exposure mode.
emStatus = GXSetEnumValueByString(hDevice, "ExposureMode",
                                  "TriggerWidth");
 //Sets the Exposure Overlap Time Max.
emStatus = GXSetFloatValue(hDevice, "ExposureOverlapTimeMax",
                           621.0);
```

- Precautions

Exposure Overlap Time Max is only available in TriggerWidth exposure mode.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

3.3.10. Multi-gray Control (MGC) Mode

- Terms

1) Multi-gray Control: The gray value of different frames can set by the parameters setting. Configurable parameters include MGC ExposureTime and MGC Gain.

2) Multi-gray Control Mode: Including Off mode, TwoFrame mode and FourFrame mode.

3) Off mode: Disables Multi-gray Control mode.

4) TwoFrame mode: Two groups of different MGC ExposureTime and MGC Gain can be set. After setting all the parameters and starting acquisition, the camera will output the images with the different gray value alternately.

5) FourFrame mode: Four groups of different MGC ExposureTime and MGC Gain can be set. After setting all the parameters and starting acquisition, the camera will output the images with the different gray value cyclically.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
MultiGrayControlMode	ENUM	RW
MGCSelector	INT	RW
MGCExposureTime	FLOAT	RW
MGCGain	FLOAT	RW

- Sample Code

```
GX_STATUS emStatus = GX_EMSTATUS_SUCCESS;

//Sets the TwoFrames mode.
emStatus=GXSetEnumValueByString(hDevice,"MultiGrayControlMode",
                                "TwoFrame");
//Sets the first frame MGC ExposureTime and MGC Gain.
emStatus = GXSetIntValue(hDevice, "MGCSelector", 0);
emStatus = GXSetFloatValue(hDevice, "MGCExposureTime", 400000.0);
emStatus = GXSetFloatValue(hDevice, "MGCGain", 2.0);

//Sets the second frame MGC ExposureTime and MGC Gain.
emStatus = GXSetIntValue(hDevice, "MGCSelector", 1);
emStatus = GXSetFloatValue(hDevice, "MGCExposureTime", 1000.0);
emStatus = GXSetFloatValue(hDevice, "MGCGain", 1.0);
```

3.3.11. Frame Information

3.3.11.1. Principle

When the frame information is activated, the camera will append a chunk immediately after the returned capturing image cache, which contains the ID, length, and frame information content that the user wishes to obtain.

The captured image is placed at the beginning as the first data chunk, and the additional chunks are added sequentially as chunks 2 and 3.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

The sample of a collection of data chunks containing preprocessed image data chunks and additional frame information chunks is shown as the following schematic diagram.



Figure 3-23 Image Frame Information Structure Schematic Diagram

When manually parsing, parse backwards from the end to the beginning. First read a 32-bit length, then read a 32-bit ID, and immediately after that read the frame information content which length corresponding to the length read in the first step.

3.3.11.2. Enabling and Obtaining the Supported Types of Frame Information

- For specific available frame information, please refer to the chunk data control section in the property tree of GalaxyView, as shown in the following:

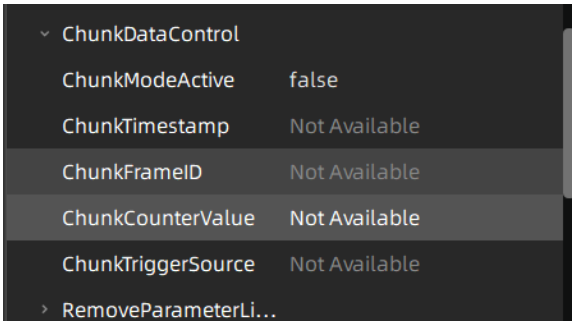


Figure 3-24 Network Frame Information Nodes

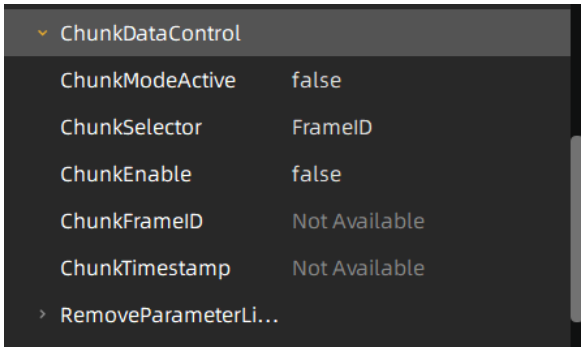


Figure 3-25 U3 Frame Information Nodes

- Enable frame information

1. Set the ChunkModeActive to true.

Code Sample:

```
//Frame information is activated.
GX_NODE_ACCESS_MODE emAccess;
status = GXGetNodeAccessMode(phDevice, "ChunkModeActive", &emAccess);
bool bImplementChunkMode =
    (emAccess == GX_NODE_ACCESS_MODE_RW) ? true : false;
if (bImplementChunkMode)
{
    emStatus = GXSetBoolValue(phDevice, "ChunkModeActive", true);
}
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

2. If the chunkselector node is supported by the camera, set it to the following optional feature items, i.e. the feature you want to enable:
   - FrameID
   - Timestamp
   - CounterValue
3. Enable the selected chunk type by setting the ChunkEnable to true.
4. Repeat steps 2 and 3 for each frame information that needs to be enabled.

Note: Steps 2 and 3 above can only be set if supported by the camera.

Code Sample:

```
// If the camera supports frame information selector, the single item frame
// information will be activated.
status = GXGetNodeAccessMode(phDevice, "ChunkSelector", &emAccess);
bool bImplementChunkSelector =
    (emAccess == GX_NODE_ACCESS_MODE_RW) ? true : false;
if (bImplementChunkSelector)
{
    emStatus = GXSetEnumValueByString(phDevice, "ChunkSelector", "FrameID");
}

status = GXGetNodeAccessMode(phDevice, "ChunkEnable", &emAccess);
bool bImplementChunkEnable =
     (emAccess == GX_NODE_ACCESS_MODE_RW) ? true : false;
if (bImplementChunkSelector)
{
    emStatus = GXSetBoolValue(hDevice, "ChunkEnable", true);
}
```

3.3.11.3. Get Frame Information through the Frame Information Handle Type

➢ Get the image structure GX_FRAME_CALLBACK_PARAM which Contains the frame information handle by acquisition with callback.

Code Sample:

```
//Acquisition with callback interface
static void GX_STDC OnFrameCallbackFun(GX_FRAME_CALLBACK_PARAM* pFrame)
{
    if (pFrame-> status == GX_FRAME_STATUS_SUCCESS)
    {
        //Get the frame information ID through frame information handle
        GX_INT_VALUE stValueChunkFrameID;
        GX_STATUS emStatus = GXGetIntValue(
                (GX_CHUNK_DATA_HANDLE)
                pFrameData->pChunkDataHandle,
                "ChunkFrameID",
                &stValueChunkFrameID);
        if (emStatus == GX_STATUS_SUCCESS)
        {
            cout << "ChunkFrameID : %d" << stValueChunkFrameID.nCurValue;
        }
        else{
```

```
            //Through GXGetLastError interface obtain detailed information about
            //the error if fault.
        }
    }
    return;
}
```

- Get the image structure GX_FRAME_DATA_EX which Contains the frame information handle through acquisiting single frame GXGetImageEx().

Code Sample:

```
X_INT_VALUE stPayLoadSize;

// Get the image buffer size, then dynamically allocate memory.
emStatus = GXGetIntValue(hDevice, "PayloadSize", &stPayLoadSize);
if(emStatus==GX_STATUS_SUCCESS&&stPayLoadSize.nCurValue>0)
    {
    // Define the input parameters for GXGetImageEx.
          GX_FRAME_DATA_EX stFrameData;

       // Apply for a buffer based on the size of the acquired image fuffer
    //m_nPayLoadSize
             stFrameData.pImgBuf= malloc((size_t)stPayLoadSize.nCurValue);

    // After sending the start capture command, obtain image information
    // through GXGetImageEx.
    emStatus = GXGetImageEx(hDevice, &stFrameData, 100);


   //Get frame information ID through frame information handle.
   GX_INT_VALUE stValueChunkFrameID;
   GX_STATUS emStatus = GXGetIntValue((GX_CHUNK_DATA_HANDLE)
                  stFrameData.pChunkDataHandle,
                  "ChunkFrameID",
                  &stValueChunkFrameID);
   if (emStatus == GX_STATUS_SUCCESS)
   {
      cout << "ChunkFrameID : %d" << stValueChunkFrameID.nCurValue;
   }
   else{
          //Through GXGetLastError interface obtain detailed information about
          //the error if fault.
   }
```

- Get the image structure GX_FRAME_DATA_EX which Contains the frame information handle through acquisiting single frame GXDQBufEx().

Code Sample:

```
GX_INT_VALUE stPayLoadSize;

// Get the image buffer size, then dynamically allocate memory.
emStatus = GXGetIntValue(hDevice, "PayloadSize", &stPayLoadSize);
if(emStatus==GX_STATUS_SUCCESS&&stPayLoadSize.nCurValue>0)
    {
    /Define the input parameters of GXDQBufEx
    GX_FRAME_DATA_EX pFrameBuffer;
```

```
        // Apply for a buffer based on the size of the acquired image fuffer
        // m_nPayLoadSize.
            stFrameData.pImgBuf= malloc((size_t)stPayLoadSize.nCurValue);

 //After sending the start acquisition command, obtain image information
 //through GXDQBufEx.
            emStatus = GXDQBufEx(hDevice, &pFrameBuffer, 1000);


// Get the frame information ID through the frame information handle.
 emStatus = GXGetIntValue((GX_CHUNK_DATA_HANDLE)
             stFrameBuffer.pChunkDataHandle,
             "ChunkFrameID",
             &stValueChunkFrameID);
if (emStatus == GX_STATUS_SUCCESS)
{
   cout << "ChunkFrameID : %d" << stValueChunkFrameID.nCurValue;
}
else{
       //Through GXGetLastError interface obtain detailed information
       //about the error if fault.
}
```

3.4. Digital I/O

3.4.1. Pin Control

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
LineSelector	ENUM	RW
LineMode	ENUM	RW
LineInverter	BOOL	RW
LineSource	ENUM	RW
LineStatus	BOOL	RO
LineStatusAll	INT	RO
UserOutputSelector	ENUM	RW
UserOutputValue	BOOL	RW
PulseWidth	FLOAT	RW

- Sample Code

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//For example, the pin is selected as Line2.
emStatus = GXSetEnumValueByString(hDevice, "LineSelector", "Line2");

//Sets the pin direction to output mode.
emStatus = GXSetEnumValueByString(hDevice, "LineMode", "Output");
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
//Optional operation: pin level inversion.
//emStatus = GXSetBoolValue(hDevice, "LineInverter", true);

//You can set the output source to the strobe. The code is as follows.
emStatus = GXSetEnumValueByString(hDevice, "LineSource", "Strobe");

//You can set the output source to the user-defined output.
//The code is as follows.
emStatus = GXSetEnumValueByString(hDevice, "LineSource",
                                  "UserOutput0");

//You can set the output source to a user-defined output value.
//The code is as follows.
emStatus = GXSetEnumValueByString(hDevice, "UserOutputSelector",
                                  "UserOutput0");
emStatus = GXSetBoolValue(hDevice, "UserOutputValue", true);

//Gets the status of the line2 pin (The current pin is selected as line2).
bool bLineStatus = true;
emStatus = GXGetBoolValue(hDevice, "LineStatus", &bLineStatus);

//Gets the status of all pins.
GX_INT_VALUE stAllLineStatus;
int64_t i64AllLineStatus = 0;
emStatus = GXGetIntValue(hDevice, "LineStatusAll",
                         &stAllLineStatus);

i64AllLineStatus  = stAllLineStatus.nCurValue;
```

3.4.2. The I/O Control of the USB2.0 Camera

The IO control of USB2.0 is special, and the "pins control" operation is not the same operation process as shown in the previous section. The IO control of USB2.0 involves the following three functions:

- Terms

1) User IO output mode: There are two modes: strobe mode and user-defined mode. In the strobe mode, the camera sends trigger signals to activate the strobe (the trigger signal has rising edge and falling edge). In user-defined mode, you can set the camera's output level to meet the special demand, such as control the LED lamp (the output level is high or low).

2) Output signal polarity: When the output signal mode is strobe, the polarity means rising edge or falling edge, and when the output signal mode is user-defined, the polarity means high level or low level.

3) Strobe switch: The switch just works on the strobe mode. When the switch is set to open, it will output strobe signal, if the switch is set to close, it will not output strobe signal.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties	Description
UserOutputMode	ENUM	RW	USB2.0 camera specific
UserOutputValue	BOOL	RW	For USB2.0 cameras, this value represents the polarity of the output signal. when the output

			signal mode is strobe mode, it represents the rising or falling edge. When the output signal mode is user-defined, it represents high or low level
StrobeSwitch	ENUM	RW	USB2.0 camera specific

- Control Flow Chart

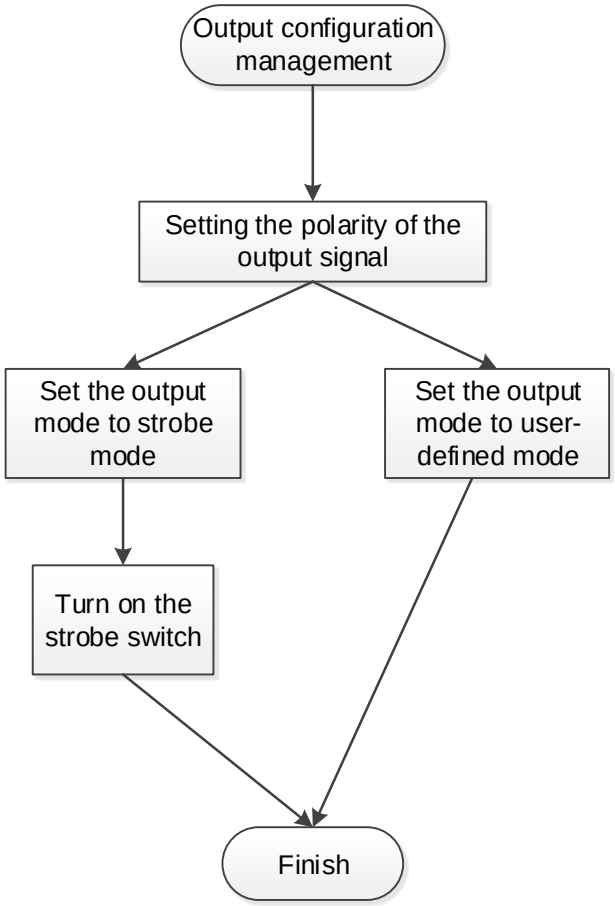


Figure 3-26 IO control flowchart

- Sample Code

```
#include"GxIAPI.h"

int main(int argc, char* argv[])
{
   GX_STATUS emStatus = GX_STATUS_SUCCESS;
   GX_DEV_HANDLE hDevice = NULL;
   GX_OPEN_PARAM stOpenParam;
   uint32_t ui32DeviceNum = 0;

   //Initializes the library.
   emStatus = GXInitLib();
   if (emStatus!= GX_STATUS_SUCCESS)
   {
      return 0;
   }

   //Updates the enumeration list for the devices.
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
    if ((emStatus != GX_STATUS_SUCCESS)||(ui32DeviceNum<= 0))
    {
        return 0;
    }

    //Opens the device.
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emStatus = GXOpenDevice(&stOpenParam, &hDevice);
    if (emStatus == GX_STATUS_SUCCESS)
    {
        //Sets the user-defined output polarity to high.
        emStatus = GXSetBoolValue(hDevice, "UserOutputValue" ,true);

        //Sets the output mode to the strobe mode.
        emStatus = GXSetEnumValueByString(hDevice, "UserOutputMode",
                                          "Strobe");

        //Turn on the strobe switch.
        emStatus = GXSetEnumValueByString(hDevice, "StrobeSwitch",
                                          "On");
    }
    emStatus = GXCloseDevice(hDevice);
    GXCloseLib();
    return 0;
}
```

- Precautions

3.5. Counter and Timer Control

3.5.1. Timer

- Terms

1) Timer: Used to measure the duration of internal or external signals.

2) Timer duration: When the timer reaches the timer duration, the timer stops counting until a new trigger signal is generated or a timer reset command is sent.

3) Timer delay time: Set the duration of the delay to apply at the reception of a trigger before starting the timer.

4) Timer trigger source: Set the internal signal that triggers the selected timer.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
TimerSelector	ENUM	RW
TimerDuration	FLOAT	RW
TimerDelay	FLOAT	RW
TimerTriggerSource	ENUM	RW

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

- Sample Code

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//For example, select Timer1 as the timer.
emStatus=GXSetEnumValueByString(hDevice, "TimerSelector",
                                "Timer1");
//Set the timer duration to 100us, the user can set according to
//the actual situation.
emStatus = GXSetFloatValue(hDevice, "TimerDuration", 100.0);

//Set the timer delay to 10us, the user can set according to the
//actual situation.
emStatus = GXSetFloatValue(hDevice, "TimerDelay", 10.0);

//Set the trigger source to "Exposure Start".
emStatus = GXSetEnumValueByString(hDevice, "TimerTriggerSource",
                                  "ExposureStart");
```

3.5.2. Counter

- Terms

1) Counter: Used to record internal events, I/O external events, or clock tick counts.

2) Counter event trigger source: Select the events that will be the source to increment the counter. Currently only one mode "Frame Start" is supported.

3) Counter reset source: Select the signals that will be the source to reset the counter.

4) Counter reset signal polarity: Activation mode of the counter reset source signal. It is divided into rising edge valid or falling edge valid.

5) Counter reset: Reset the selected counter. Only valid when set counter reset source to "Software".

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
CounterSelector	ENUM	RW
CounterEventSource	ENUM	RW
CounterResetSource	ENUM	RW
CounterResetActivation	ENUM	RW
CounterReset	COMMAND	WO
CounterTriggerSource	ENUM	RW
CounterDuration	INT	RW
TimerTriggerActivation	ENUM	RW

- Sample Code

```
GX_STATUS status = GX_STATUS_SUCCESS;
//For example, select Counter1 as the counter.
emStatus = GXSetEnumValueByString(hDevice, "CounterSelector",
                                  "Counter1");

//Sets the counter event trigger source to "Frame Start".
emStatus = GXSetEnumValueByString(hDevice, "CounterEventSource",
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
                                    "FrameStart");

    //Sets the reset source to "Software".
    emStatus = GXSetEnumValueByString(hDevice, "CounterResetSource",
                                    "Software");

    //Sets the reset signal polarity to rising edge valid.
    emStatus = GXSetEnumValueByString(hDevice, "CounterResetActivation",
                                    "RisingEdge");

    //Sends counter reset command. Only valid when set counter reset
     //source to "Software".
    emStatus = GXSetCommandValue(hDevice, "CounterReset");

    //Sets the trigger source to software trigger.
    emStatus = GXSetEnumValueByString(hDevice, "CounterTriggerSource",
                                    "Software");

    //Sets the counter duration value.
    emStatus = GXSetEnumValueByString(hDevice, "CounterDuration", 50);

    //Sets the counter to rising edge trigger.
    emStatus = GXSetEnumValueByString(hDevice, "TimerTriggerActivation",
                                    "RisingEdge");
```

3.6. Analog Control

3.6.1. Gain

The gain is a multiplication factor for improving the value of the pixels, and the effect is to increase the brightness of the image. In certain conditions (external environments, etc.) the sensor does not have the desired saturation, so the gain is adjusted. Of course, improving gain also increases noise, so improving the gain does not improve the dynamic range of actual pixels. So, when you need to improve the image brightness, you should first consider to adjust the exposure time, only when adjusting the exposure time can not meet the requirements, then you can adjust the gain.

- Terms

1) Gain channel selection: You should select the channel before adjusting the gain, there are four channels: ALL, Red, Green, and Blue.
2) Gain control mode: Manual adjustment and automatical adjustment.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
GainSelector	ENUM	RW
Gain	FLOAT	RW
GainAuto	ENUM	RW

- Effect images



Figure 3-27 Original image



Figure 3-28 Reduce gain



Figure 3-29 Increase gain

- Sample Code

```
//Manual gain control-----------------------
//Use GXGetEnumValue interfaces to query "GainSelector" type
//supported by the current camera.
//Please refer to the interface descriptions, which are omitted here
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//Assumes that the current camera support "GainSelector" and "All"

//Selects the gain channel type.
emStatus = GXSetEnumValueByString(hDevice, "GainSelector",
                                  "All");
//emStatus = GXSetEnumValueByString(hDevice, "GainSelector", "Red");
//emStatus = GXSetEnumValueByString(hDevice, "GainSelector", "Green");
//emStatus = GXSetEnumValueByString(hDevice, "GainSelector", "Blue");
//Gets the adjustment range of the gain.
GX_FLOAT_VALUE stGainRange;
emStatus = GXGetFloatValue(hDevice, "Gain", &stGainRange);

//Sets the gain to the minimum.
emStatus = GXSetFloatValue(hDevice, "Gain", stGainRange.dMin);

//Sets the gain to the maximum.
emStatus = GXSetFloatValue(hDevice, "Gain", stGainRange.dMax);
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
//Sets the auto gain adjustment to continuous mode.
emStatus = GXSetEnumValueByString(hDevice, "GainAuto", "Continuous");
```

- Precautions

For USB2.0 camera, the unit of the gain is not "dB", it is the value of the register, and it has an adjustment range, the larger the value, the larger the magnification of the gain.

3.6.2. Black Level

- Terms

1) Black Level: The black level is to define the reference level of the image data. Adjust the black level will not affect the amplification of the signal. It only shifts the signal up and down. Up the black level, the image is brighter, down the black level, the image is darker.
2) Control Mode of Black Level: Manual adjustment and automatic adjustment.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
BlackLevelSelector	ENUM	RW
BlackLevel	FLOAT	RW
BlackLevelAuto	ENUM	RW

- Effect Images



Figure 3-30 Original image



Figure 3-31 Reduce black level



Figure 3-32 Increase black level

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

- Sample Code

```
//Manual black level adjustment -----------------------
//Use GXGetEnumValue interface to query the "BlackLevelSelector" type
//supported by the current camera
//Please refer to the interface descriptions, which are omitted here

//Assumes that the current camera supports "BlackLevelSelector" and "All"
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//Selects the black level channel type.
emStatus = GXSetEnumValueByString(hDevice, "BlackLevelSelector",
                                  "All");

//emStatus= GXSetEnumValueByString(hDevice, "BlackLevelSelector",
//                                 "Red");
//emStatus= GXSetEnumValueByString(hDevice, "BlackLevelSelector",
//                                 "Green");
//emStatus= GXSetEnumValueByString(hDevice, "BlackLevelSelector",
//                                 "Blue");

//Gets the range of the black level.
GX_FLOAT_RANGE stBlackLevelRange;
emStatus = GXGetFloatValue(hDevice, "BlackLevel",
                           &stBlackLevelRange);

//Sets the black level to the minimum.
emStatus = GXSetFloatValue(hDevice, "BlackLevel",
                           stBlackLevelRange.dMin);

//Sets the black level to the maximum.
emStatus = GXSetFloatValue(hDevice, "BlackLevel",
                           stBlackLevelRange.dMax);


//Automatic black level adjustment--------------------------
//Sets to continuous automatic black level mode.
emStatus = GXSetEnumValueByString(hDevice, "BlackLevelAuto",
                                  "Continuous");
```

- Precautions

3.6.3. White Balance

- Terms

1) White Balance: Under different color temperatures, the object's color may change. Especially the white objects. Indoors, the white objects look with orange tonal under the tungsten light which is of a low color temperature, under that light condition, the image will yellow shift; if in the blue sky which is of high color temperature, the image will be bluer. In order to minimize the external light's impact and to restore the real color of the object, color correction is required, to achieve the correct color balance, known as the white balance adjustment.
2) White Balance Control Mode: Manual adjustment and automatical adjustment.
3) Auto White Balance Light Environment: You should set this value by the light environment of the camera, it is better to the white balance effect.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

4) Auto White Balance ROI: Auto White Balance function use the image data of the "white dot" area (ROI) to calculate the white balance coefficient, and then use the coefficient to handle the components of the image, in order to make the R/G/B component the same in the ROI. The auto white balance function is only available on color sensors.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
BalanceRatioSelector	ENUM	RW
BalanceRatio	FLOAT	RW
BalanceWhiteAuto	ENUM	RW
AWBLampHouse	ENUM	RW
AWBROIOffsetX	INT	RW
AWBROIOffsetY	INT	RW
AWBROIWidth	INT	RW
AWBROIHeight	INT	RW
LightSourcePreset	ENUM	RW

- Effect Images



Figure 3-33 Image color shift under D65 environment



Figure 3-34 After white balance adjustmen

- Sample Code

```
//The manule white balance adjustment------------------------
//Use GXGetEnumValue interface to query "BalanceRatioSelector" type
//supported by the current camera
//Please refer to the interface description, which is omitted here

//Assume that the current camera supports "BalanceRatioSelector" and "Red"
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//Selects the white balance channel.
emStatus = GXSetEnumValueByString(hDevice, "BalanceRatioSelector",
                                  "Red");
//emStatus= GXSetEnumValueByString(hDevice, "BalanceRatioSelector",
//                                 "Green");
//emStatus= GXSetEnumValueByString(hDevice, "BalanceRatioSelector",
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
//                                      "Blue");

//Gets the range of the white balance adjustment.
GX_FLOAT_VALUE stRatioRange;
emStatus = GXGetFloatValue(hDevice, "BalanceRatio", &stRatioRange);

//Sets the white balance ratio to the minimum.
emStatus = GXSetFloatValue(hDevice, "BalanceRatio", stRatioRange.dMin);

//Sets the white balance ratio to the maximum.
emStatus = GXSetFloatValue (hDevice, "BalanceRatio", stRatioRange.dMax);

//Sets the automatic white balance ROI (the user can modify the parameters
//according to their own need).
emStatus = GXSetIntValue(hDevice, "AWBROIWidth", 100);
emStatus = GXSetIntValue(hDevice, "AWBROIHeight", 100);
emStatus = GXSetIntValue(hDevice, "AWBROIOffsetX", 0);
emStatus = GXSetIntValue(hDevice, "AWBROIOffsetY", 0);

//Auto white balance setting--------------------------
//Sets the lighting environment for the automatic white balance, such as
//the current camera in the fluorescent environment.
emStatus = GXSetEnumValueByString(hDevice, "AWBLampHouse",
                                  "Fluorescence");

//Sets to continuous automatic white balance mode.
emStatus = GXSetEnumValueByString(hDevice, "BalanceWhiteAuto",
                                  "Continuous");

//Light source preset setting-------------------------
//Sets Light source preset to Daylight-6500K
emStatus = GXSetEnumValueByString(hDevice, "LightSourcePreset",
                                  "Daylight6500K");
```

- Precautions

If the automatic white balance function fails, it may be because the ROI of the automatic white balance is too small to find the white spots.

3.7. Transport Layer Control

3.7.1. PayloadSize

- Terms

PayloadSize: This parameter indicates the size (unit: bytes) of each image data, which outputting from the camera. According to this value, you can allocate the buffer size to capture images. When the frame information is closed, the output image does not attach the frame information, and the size of the payloadSize represents the image size. When the frame information is opened, the output image comes with the frame information, and then the size of the payloadSize is equal the image size and the frame information size.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
PayloadSize	INT	RO

- Sample Code

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_INT_VALUE stValue;
int64_t nValue = 0;
emStatus = GXGetIntValue(hDevice, "PayloadSize", &stValue);
i64Value = stValue.nCurValue;
```

- Precautions

When the user allocates the buffer for image data, it is strongly recommended that the user use the value of payloadSize.

3.7.2. IP Configuration

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
GevCurrentIPConfigurationLLA	BOOL	RO
GevCurrentIPConfigurationDHCP	BOOL	RW
GevCurrentIPConfigurationPersistentIP	BOOL	RW

- Sample Code

```
GX_STATUS status = GX_STATUS_SUCCESS;
//This example is described in persistent IP configuration mode,
//and other IP configuration modes are similar.
//Sets the IP configuration mode to persistent IP mode.
emStatus = GXSetBoolValue(hDevice,
                          "GevCurrentIPConfigurationPersistentIP",
                          true);
```

3.7.3. Estimate the Bandwidth

- Terms

Estimate the bandwidth: The network bandwidth required to transfer image data under the current condition of the camera.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
EstimatedBandwidth	INT	RO

- Sample Code

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
//Reads the current estimated bandwidth.
GX_INT_VALUE stValue;
int64_t i64Value = 0;
emStatus  = GXGetIntValue(hDevice, "EstimatedBandwidth", &stValue);
i64Value = stValue.nCurValue;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

3.7.4. The Heartbeat Timeout Time of the Device

- Terms

The heartbeat timeout time of the device: If the device does not receive the GVCP command packet from the host in this time, it will disconnect with the host.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
GevHeartbeatTimeout	INT	RW

- Sample Code

```
GX_STATUS emStatus  = GX_STATUS_SUCCESS;
//Reads the current heartbeat timeout time of the device.
GX_INT_VALUE stTimeValue = 0;
emStatus = GXGetIntValue(hDevice,"GevHeartbeatTimeout",
                         &stTimeValue);

//Reads the settable range of the device's heartbeat timeout time.
//Please refer to the GXGetIntValue interface for use.
//Sets the heartbeat timeout time of the device.
emStatus = GXSetIntValue(hDevice, "GevHeartbeatTimeout",
                         stTimeValue.nCurValue);
```

- Precautions

It is strongly recommended that the user use the default value unless special situation allow the user to modify the value based on the application environment.

3.7.5. Packet Size

- Terms

Packet Size: It means that the GigE Vision camera transfers the network packet size of the stream channel data to the host, unit: bytes.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
GevSCPSPacketSize	INT	RW

- Sample Code

```
GX_STATUS emStatus  = GX_STATUS_SUCCESS;

//Reads the packet size.
GX_INT_VALUE stValue;
int64_t i64PacketSize = 0;
emStatus  = GXGetInt(hDevice, GX_INT_GEV_PACKETSIZE, &nPacketSize);
i64PacketSize = stValue.nCurValue;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

3.7.6. Packet Delay

- Terms

Packet delay: To control the transfer image data bandwidth of the GigE Vision camera. The packet delay is the number of idle clocks that are inserted between adjacent network packets. Increase the packet delay will reduce the camera's bandwidth usage, and may also reduce the frame rate of the camera.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
GevSCPD	INT	RW

- Sample Code

```
GX_STATUS emStatus  = GX_STATUS_SUCCESS;
//Reads the packet delay
int64_t i64PacketDelay = 0;
GX_INT_VALUE stValue;
emStatus = GXGetIntValue(hDevice, "GevSCPD", &stValue);
i64PacketDelay = stValue.nCurValue;
```

3.7.7. Link Speed

- Terms

Link Speed: The current network environment is gigabit network or 100M Ethernet.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
GevLinkSpeed	INT	RO

- Sample Code

```
GX_STATUS emStatus  = GX_STATUS_SUCCESS;

//Reads the link speed of the current network environment
GX_INT_VALUE stValue;
int64_t i64Value = 0;
emStatus = GXGetIntValue(hDevice, "GevLinkSpeed", &stValue);

i64Value = stValue.nCurValue;
```

3.8. Bandwidth Control

- Terms

The USB3 Vison series camera supports bandwidth control function, to control the bandwidth upper limit of a single device. The bandwidth of the current device will not change when the link bandwidth limit of the device is greater than the bandwidth of the current device. When the link bandwidth limit of the device is less than the current device, the bandwidth of the current device will be reduced to the limit of the link bandwidth of the device. The current device acquisition bandwidth can be read from the camera.

For example: The current acquisition bandwidth of the device is 35000000Bps, the device link bandwidth limit is 40000000Bps, then the current acquisition device bandwidth is still 35000000Bps; if the current acquisition device bandwidth is 70000000Bps, the device link bandwidth limit is 40000000Bps, then the current acquisition device bandwidth is still 40000000Bps.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
DeviceLinkSelector	INT	RW
DeviceLinkThroughputLimitMode	ENUM	RW
DeviceLinkThroughputLimit	INT	RW
DeviceLinkCurrentThroughput	INT	RO

- Sample Code

```
GX_STATUS emStatus  = GX_STATUS_SUCCESS;

//Enables the bandwidth limit mode.
emStatus = GXSetEnumValueByString(hDevice,
                                  "DeviceLinkThroughputLimitMode", "On");
//Reads the current bandwidth value.
GX_INT_VALUE stLinkThroughputVal;
emStatus = GXGetIntValue(hDevice, "DeviceLinkCurrentThroughput",
                         &stLinkThroughputVal);

//Sets the limiting value of the bandwidth.
int64_t i64LinkThroughputLimitVal = 40000000;
emStatus = GXSetIntValue(hDevice, "DeviceLinkThroughputLimit",
                         i64LinkThroughputLimitVal);
```

- Precautions

When the device bandwidth limit mode is opened or the bandwidth of the device is changed before, you should stop the acquisition.

3.9. Event Control

- Terms

When event notification is set to "on", the camera can generate an "event" and transfer a related event message to the computer whenever a specific situation has occurred. For MER-G series camera, the camera can generate and transfer events for the following types of situations:

- The camera has ended exposure
- An image block is discarded
- The trigger signal overflow
- The image frame buffer is not empty
- The event array is overrun

Every event has a corresponding enable status, and in default all the events' enable status are disable.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
EventSelector	ENUM	RW
EventNotification	ENUM	RW

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

EventExposureEnd	INT	RO
EventExposureEndTimestamp	INT	RO
EventExposureEndFrameID	INT	RO
EventBlockDiscard	INT	RO
EventBlockDiscardTimestamp	INT	RO
EventBlockDiscardFrameID	INT	RO
EventOverrun	INT	RO
EventOverrunTimestamp	INT	RO
EventFrameStartOvertrigger	INT	RO
EventFrameStartOvertriggerTimestamp	INT	RO
EventFrameStartOvertriggerFrameID	INT	RO
EventBlockNotEmpty	INT	RO
EventBlockNotEmptyTimestamp	INT	RO
EventBlockNotEmptyFrameID	INT	RO
EventInternalError	INT	RO
EventInternalErrorTimestamp	INT	RO
EventFrameBurstStartOvertrigger	INT	RO
EventFrameBurstStartOvertriggerFrameID	INT	RO
EventFrameBurstStartOvertriggerTimestamp	INT	RO
EventFrameStartWait	INT	RO
EventFrameStartWaitFrameID	INT	RO
EventFrameStartWaitTimestamp	INT	RO
EventFrameBurstStartWait	INT	RO
EventFrameBurstStartWaitTimestamp	INT	RO
EventFrameBurstStartWaitFrameID	INT	RO

- Sample Code

See GXRegisterFeatureCallbackByString() interface description.

3.10. LUT (Look-up Table) Control

- Terms

LUT: The look-up table is a mapping table for pixel value conversions, which corresponds to pixel value one by one, and is used to change the output of each pixel value. It can be used for the conversion of high bit depth images to low bit depth images, and can also be used for simple pixel value conversion of the same bit depth images. The look-up table gives the user maximum flexibility to implement linear and non-linear conversion of pixel values (such as logarithmic and exponential corrections) without consuming valuable computing resources in the system.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
LUTSelector	ENUM	RW
LUTValueAll	BUFFER	RW
LUTEnable	BOOL	RW
LUTIndex	INT	RW
LUTValue	INT	RW

- Sample Code

See GXGetRegisterValue(), GXSetRegisterValue() interfaces description.

- Precautions

3.11. User Configuration

This section describes the control of all parameter configurations for the camera, allowing the user to save or load the vendor parameter group or user parameter group.

- Terms

1) Parameter type: The user selects a type of parameters (default, userset0) to save or load.

2) Load parameters: Load the selected parameters to the camera.

3) Save parameters: Save the camera's current parameters to the user selected parameter group.

4) Set parameters: Set the parameter group used when the device is powered on.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
UserSetSelector	ENUM	RW
UserSetLoad	COMMAND	WO
UserSetSave	COMMAND	WO
UserSetDefault	ENUM	RW

Sample Code

```
    //Use GXGetEnumValue interface to query "UserSetSelector" type
    //supported by the current camera.
    //Please refer to the interface descriptions, which are omitted here.

   //Assume that the current camera supports two parameter groups, such
   //as the vendor default parameter group, and user parameter group 0.
    GX_STATUS emStatus = GX_STATUS_SUCCESS;

   //Chooses to load the vendor default parameter group.
    emStatus = GXSetEnumValueByString(hDevice, "UserSetSelector",
                                      "Default");
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
emStatus = GXSetCommandValue(hDevice, "UserSetLoad");

//Chooses to load the user parameter group 0.
emStatus = GXSetEnumValueByString(hDevice, "UserSetSelector",
                                  "UserSet0");
emStatus = GXSetCommandValue(hDevice, "UserSetLoad");

//Sets the startup parameter group to the vendor parameter group.
emStatus = GXSetEnumValueByString(hDevice, "UserSetDefault",
                                  "Default");
```

- Precautions

3.12. Set Camera's IP Address

3.12.1. Configuration Static IP Address

- Terms

1) Static IP: Configuration the camera IP in a static IP mode.
2) DHCP: Configure the camera IP with DHCP, which is the IP allocated by the DHCP server.
3) LLA: Configure the camera IP in LLA (Link-Local Address) mode.

- Related Parameters

`GX_IP_CONFIGURE_STATIC_IP`: Configuration the camera IP in a static IP mode.

`GX_IP_CONFIGURE_DHCP`: Configure the camera IP with DHCP, which is the IP allocated by the DHCP server.

`GX_IP_CONFIGURE_LLA`: Configure the camera IP in LLA (Link-Local Address) mode.

`GX_IP_CONFIGURE_DEFAULT`: Configure the camera IP in the default mode. In this mode, the camera will enable all the three configurations available, but still configure the camera in a static IP mode.

- Sample Code

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//This is the MAC sample address. The actual MAC address of the
//camera can be obtained from the GXGetDeviceInfo interface.
char chArrMAC[] = "00-21-49-00-00-00";

char chArrIpAddress[] = "192.168.10.10";
char chArrSubnetMask[] = "255.255.255.0";
char chArrDefaultGateway[] = "192.168.10.2";
char chArrUserID[] = "Daheng Imaging";
GX_IP_CONFIGURE_MODE emIpConfigureMode = GX_IP_CONFIGURE_STATIC_IP;

//This example is described in persistent IP configuration mode,
//and other IP configuration modes are similar.
//Sets to static IP configuration mode.
emStatus = GXGigEIpConfiguration(chArrMAC, emIpConfigureMode,
                                 chArrIpAddress, chArrSubnetMask,
                                 chArrDefaultGateway, chArrUserID);
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

- Precautions

1) Before calling this interface, you must do enumeration operation first, and to perform this operation when the camera is not open.
2) When you select the GX_IP_CONFIGURE_STATIC_IP and GX_IP_CONFIGURE_DEFAULT parameters to configuration the camera IP, the parameters pointers to the IP address, subnet mask, default gateway etc. are not NULL.
3) When you select the GX_IP_CONFIGURE_LLA and GX_IP_CONFIGURE_DHCP parameters to configuration the camera IP, the parameters pointers to the IP address, subnet mask, default gateway etc. can be NULL.
4) The maximum length of user-defined name (UserID) is 16 characters. Configuration cameras IP in any way, the user-defined name (UserID) parameter pointers can be NULL.

3.12.2. Force IP

- Terms

Force IP: Use the Force IP mode can temporarily change the camera's IP address, only for the use of the camera, which will restore the original IP when the camera has rebooted.

- Sample Code

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//This is the MAC sample address. The actual MAC address of the
//camera can be obtained from the GXGetDeviceIPInfo interface.
char chArrMAC[] = "00-21-49-00-00-00";
char chArrIpAddress[] = "192.168.10.10";
char chArrSubnetMask[] = "255.255.255.0";
char chArrDefaultGateway[] = "192.168.10.2";

//ForceIp
emStatus = GXGigEForceIp(chArrMAC, chArrIpAddress, chArrSubnetMask,
                         chArrDefaultGateway);
```

- Precautions

Before calling this interface, you must do enumeration operation first, and to perform this operation when the camera is not open.

3.13. Other Functions

3.13.1. Auto Exposure/ Auto Gain Related Function

During the acquisition process, the camera can automatically adjust the gain value and exposure time within a certain range according to the change of the ambient light to maximize the user's desired gray level. This is the automatic gain and automatic exposure function. By default, the camera calculates and adjusts the brightness according to the entire image, making the image brightness in the area of interest the expected value.

The automatic gain and auto exposure functions of the variable area are very flexible, and the user can set the desired value of the image brightness according to different application scenarios. In some applications where the backlight or the local brightness difference of the image is large, the user can delineate the region of interest as needed. The camera will intelligently select the gain and exposure adjustment ratio according to the set value to ensure the best image quality. In addition, the user can also

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

set parameters such as the upper and lower limits of the exposure time adjustment and the upper and lower limits of the gain adjustment.

3.13.1.1. Expectation Gray Value

- Terms

Expectation Gray Value: The expectation of gray is a basic parameter of automatic exposure and automatic gain adjustment, and the final expectation of automatic function adjustment is to achieve the expectation gray value that the user set.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
ExpectedGrayValue	INT	RW

- Sample Code

```
//Gets the adjustment range of the expectation gray value.
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_INT_Value stGrayValueRange;
emStatus = GXGetIntValue(hDevice, "ExpectedGrayValue",
                         &stGrayValueRange);

//Sets the gray value to the minimum.
emStatus=GXSetIntValue(hDevice, "ExpectedGrayValue",
                       stGrayValueRange.nMin);

//Sets the gray value to the maximum.
emStatus=GXSetIntValue(hDevice, "ExpectedGrayValue",
                       stGrayValueRange.nMax);
```

- Precautions

3.13.1.2. Light Environment

- Terms

Light Environment: The light environment is the external work environment of the camera, which is divided into daylight, 50Hz AC light source and 60Hz AC light source. The automatic function can adapt better according to the external illumination condition.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
AALightEnvironment	ENUM	RW

- Sample Code

```
//Use GXGetEnumValue interface to query "AALightEnvironment" type
//supported by the current camera

//Please refer to the interface descriptions, which are omitted here.
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//Selects the type of the gain channel.
emStatus = GXSetEnumValueByString(hDevice, "AALightEnvironment",
                                  "NatureLight");
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
//emStatus = GXSetEnumValueByString(hDevice, "AALightEnvironment",
//                                  "AC60HZ");
```

- Precaution

3.13.1.3. Statistical Area

- Terms

Statistical Area: The camera supports automatic function statistics of variable area, in default, the camera to statistics the whole image to achieve the expectation gray value. The statistical area function allows the user to automatically select a part of the area to execute auto-function automatically, which is more flexible.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
AAROIWidth	INT	RW
AAROIHeight	INT	RW
AAROIOffsetX	INT	RW
AAROIOffsetY	INT	RW

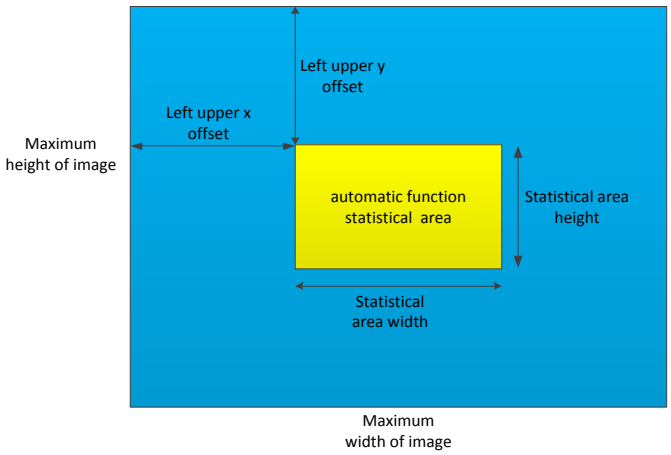


Figure 3-35 Statistical Area

- Sample Code

```
//Gets the adjustment range.
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_INT_VALUE stROIWidthRange;
GX_INT_VALUE stROIHeightRange;
GX_INT_VALUE stROIXRange;
GX_INT_VALUE stROIYRange;
emStatus = GXGetIntValue(hDevice, "AAROIWidth", &stROIWidthRange);
emStatus = GXGetIntValue(hDevice, "AAROIHeight", &stROIHeightRange);
emStatus = GXGetIntValue(hDevice, "AAROIOffsetX", &stROIXRange);
emStatus = GXGetIntValue(hDevice, "AAROIOffsetY", &stROIYRange);

//Sets the statistics area to the whole image.
emStatus = GXSetIntValue(hDevice, "AAROIWidth", stROIWidthRange.nMax);
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
emStatus = GXSetIntValue(hDevice, "AAROIHeight", stROIHeightRange.nMax);
emStatus = GXSetIntValue(hDevice, "AAROIOffsetX", 0);
emStatus = GXSetIntValue(hDevice, "AAROIOffsetY", 0);
```

- Precautions

1) The statistical area only provides a statistical range for automatic exposure and automatic gain calculation, then the camera will calculate to get the adjustment parameters according to the data within the scope.
2) To ensure the effectiveness of the AAROI, the four-attribute range of AAROI can be ref as the following formulas:

   0 <AAROIOffsetX <= Width (Max. Width of the image)-AAROIWidth

   0 <AAROIOffsetY <= Height (Max. Height of the image)-AAROIHeight

   0 <AAROIWidth <= Width (Max. Width of the image)-AAROIOffsetX

   0 <AAROIHeight <= Height (Max. Height of the image)-AAROIOffsetY

3.13.1.4. Auto Adjustment Range

- Terms

1) Auto Exposure Range: When the automatic exposure mode is turned on, the camera can adjust the exposure time dynamically in a range to adapt the current environment, and the user can adjust the range.
2) Auto Gain Range: When the automatic gain mode is turned on, the camera can adjust the gain dynamically in a range to adapt the current environment, and the user can adjust the range.
3) Auto Exposure: A work mode of the camera. In this mode, the camera automatically adjusts the exposure time in a range to achieve the best image quality according to the environment.
4) Auto Gain: A work mode of the camera. In this mode, the camera automatically adjusts the gain in a range to achieve the best image quality according to the environment.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
AutoGainMin	FLOAT	RO
AutoGainMax	FLOAT	RO
AutoExposureTimeMin	FLOAT	RW
AutoExposureTimeMax	FLOAT	RW

- Sample Code

```
//Gets the adjustment range.
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_FLOAT_VALUE stAutoGainMinRange;
GX_FLOAT_VALUE stAutoGainMaxRange;
GX_FLOAT_VALUE stAutoExposureMinRange;
GX_FLOAT_VALUE stAutoExposureMaxRange;
emStatus = GXGetFloatValue(hDevice, "AutoGainMin",
                           &stAutoGainMinRange);
emStatus = GXGetFloatValue(hDevice, "AutoGainMax",
                           &stAutoGainMaxRange);
emStatus = GXGetFloatValue(hDevice, "AutoExposureTimeMin",
                           &stAutoExposureMinRange);
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    emStatus = GXGetFloatValue(hDevice, "AutoExposureTimeMax",
                               &stAutoExposureMaxRange);

    //Sets the boundary value for the adjustment range.
    emStatus = GXSetFloatValue(hDevice, "AutoGainMin",
                               stAutoGainMinRange.dMin);
    emStatus = GXSetFloatValue(hDevice, "AutoGainMax",
                               stAutoGainMinRange.dMax);
    emStatus = GXSetFloatValue(hDevice, "AutoExposureTimeMin",
                               stAutoShutterMinRange.dMin);
    emStatus = GXSetFloatValue(hDevice, "AutoExposureTimeMax",
                               stAutoShutterMaxRange.dMax);
```

- Precautions

In order to ensure the correct legality of automatic functional range values, some special processing is done for automatic function tunable:

For "AutoGainMin" (the minimum value of auto gain), the minimum value itself has the adjusting range, the maximum of the range must not greater than the current value of the maximum auto gain value. Similarly, for "AutoGainMax" (the maximum value of auto gain), this value also has adjustable range, the minimum of the range must not smaller than the current value of the minimum auto gain value.

3.13.2. Dead Pixel Correction

- Terms

Dead Pixel: There may be very few bad pixels or dead pixels on the sensor surface, so that the actual color cannot be correctly reflected. When for interpolation calculation, these dead pixels will also pollute the surrounding colors. In order to solve this problem, the manufacturer adopts the software method. The image is preprocessed to eliminate dead pixels.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
DeadPixelCorrect	ENUM	RW

- Sample Code

```
    //Enables automatic dead pixel correction algorithm.
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    emStatus = GXSetEnumValueByString(hDevice, "DeadPixelCorrect", "On");

    //Disables automatic dead pixel correction algorithm.
    emStatus =GXSetEnumValueByString(hDevice, "DeadPixelCorrect", "Off");
```

- Precautions

3.13.3. ADCLevel (DigitalShift)

- Terms

Assuming the current camera is 10 bits or 12 bits output, you need to select 8 bits from the 10 bits or 12 bits to display the image. When the camera output image is not 8 bits, this function can be used to select which 8 bits image to output.

In some devices, the ADCLevel is called digital shift, that is DigitalShift.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

If the current camera output 10 bits, there are three conversion level:

Level0: 0~7bit

Level1: 1~8bit

Level2: 2~9bit

If the current camera output 12 bits, there are five conversion level:

Level0: 0~7bit

Level1: 1~8bit

Level2: 2~9bit

Level3: 3~10bit

Level4: 4~11bit

ADCLevel: The lower the level, the brighter the image and the louder the noise; the higher the level, the darker the image and the lower the noise.

DigitalShift: The lower the value, the darker the image and the lower the noise; the larger the value, the brighter the image and the louder the noise.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
ADCLevel	INT	RW
DigitalShift	INT	RW

- Effect images (If the camera output 10 bits, and the user select 8 bits to output)



Figure 3-36 Select Level0 (0~7bit)



Figure 3-37 Select Level1 (1~8bit)



Figure 3-38 Select Level2 (2~9bit)

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

- Sample Code

```
//Gets the range of the ADC conversion level (The sample also applies
//to the bit output "DigitalShift")
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_INT_VALUE stADCLevelRange;
emStatus = GXGetIntValue(hDevice, "ADCLevel", &stADCLevelRange);

//Sets the ADC conversion level to the minimum.
emStatus = GXSetIntValue(hDevice, "ADCLevel", stADCLevelRange.nMin);

//Sets the ADC conversion level to the maximum.
emStatus = GXSetIntValue (hDevice, "ADCLevel",
                          stADCLevelRange.nMax);
```

- Precautions

If the current camera's output is 10 bits, and the user's camera setting is 10 bits, then there is no need to select 8 bits. At this time, the ADCLevel function is disable. Only when the camera's output is 10 bits, and the user's camera need to output 8 bits, you will need to select which 8 bits.

3.13.4. Blanking Control

- Terms

1) Horizontal blanking (line blanking): In the scan process of converting light signal to electrical signals, scanning always starts from the top left corner of the image and horizontal scan forward, scanning point also move down at a slower rate. When the scanning point reach the right side of the image, the scanning point quickly returned to the left, and to restart the second line scan below the starting line of the first line. The return processes of the lines between is referred to as horizontal blanking (HBlank).
2) Vertical blanking (field blanking): A complete image scan signal consisting of a sequence of line signals separated by horizontal blanking intervals, referred to as a frame. After scanning a frame, the scanning point is returned from the lower right corner of the image to the upper left corner of the image to start a new frame scanning. This time interval is called vertical blanking, also called field blanking (VBlank).

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
HBlanking	INT	RW
VBlanking	INT	RW

- Sample Code

```
//Gets the range of the horizontal blanking
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_INT_VALUE stHBRange;
emStatus = GXGetIntValue(hDevice, "HBlanking", &stHBRange);
//Sets the value of the horizontal blanking to the minimum
emStatus = GXSetIntValue(hDevice, "HBlanking", stHBRange.nMin);

//Sets the value of the horizontal blanking to the maximum
emStatus = GXSetIntValue(hDevice, "HBlanking", stHBRange.nMax);

//Gets the range of the vertical blanking
GX_INT_VALUE stVBRange;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    emStatus = GXGetIntValue(hDevice, "VBlanking", &stVBRange);
    //Sets the value of the vertical blanking to the minimum
    emStatus = GXSetIntValue(hDevice, "VBlanking", stVBRange.nMin);

    //Sets the value of the vertical blanking to the maximum
    emStatus = GXSetIntValue(hDevice, "VBlanking", stVBRange.nMax);
```

- Precautions

Increasing the horizontal or vertical blanking will reduce the frame rate, conversely, it increases the frame rate.

3.13.5. User Data Encryption Area

- Terms

The user data encryption area is designed to protect the user's self-owned intellectual property rights. Users can define it by themselves. When the user data encryption area can be accessed, the ordinary parameters are used, if the user data encryption area is not accessible, the user must use a secret key to open it. The users can be more closely tied to their software through the data encryption area, then to improve the difficulty of decryption and protect their self-owned intellectual property.

The user data encryption area can be accessed when the camera leaves the factory.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
UserPassword	STRING	RW
VerifyPassword	STRING	RW
UserData	BUFFER	RW

- Sample Code

```
    Please contact technical support for specific use (support@daheng-imaging.com).
```

- Precautions

Using the encryption area can improve some difficulty of decryption only, can not ensure that the program don't be cracked, so Daheng company does not bear any legal liability for the loss of the safe.

3.13.6. User Data Area

- Terms

The user data area is a data area reserved for the user. The area is readable and writable, and is 16K bytes in total. It is divided into 4 data segments, and each data segment is 4K bytes. When reading and writing data segments, you first need to set the serial number of the data segment, and then read and write the set data segment according to the data and data size. The data is saved to the camera flash area immediately after being written.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
DataFieldSelector	ENUM	RW
DataFieldValue	BUFFER	RW

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

- Sample Code

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
//Applies for a 4K bytes buffer, which needs to be written by the user.
size_t nLength = 4096;
uint8_t *pui8SetBuffer = new uint8_t[nLength];

GX_NODE_ACCESS_MODE emAccessMode = GX_NODE_ACCESS_MODE_NI;
emStatus = GXGetNodeAccessMode(hDevice, "DataFieldSelector",
                               &emAccessMode);
if (emAccessMode != GX_NODE_ACCESS_MODE_NI)
{
    //Sets the Flash data area to number 1.
    emStatus = GXSetEnumValueByString(hDevice, "DataFieldSelector",
                                      "DataField_1");
    //Sets the value of the user data area.
    emStatus = GXSetRegisterValue(hDevice, "DataFieldValue",
                                  pui8SetBuffer, nLength);
    //Gets the value of the user data area.
    emStatus = GXGetRegisterValue(hDevice, "DataFieldValue",
                                  pui8SetBuffer, &nLength);
}
else
{
    //Sets the value of the user data area.
    emStatus = GXSetRegisterValue(hDevice, "DataFieldValueAll",
                                  pui8SetBuffer, nLength);
    //Gets the value of the user data area.
    emStatus = GXGetRegisterValue(hDevice, "DataFieldValueAll",
                                  pui8SetBuffer, &nLength);
}

//After operating
if(NULL != pui8SetBuffer)
{
    delete[] pui8SetBuffer;
    pui8SetBuffer = NULL;
}
```

- Precautions

3.13.7. Remove Parameter Limits

- Terms

By setting the remove parameter limits, you can make the range of some features with parameter limits larger, and the supported range will not cause the camera's image or function to be abnormal. Features that support remove parameter limits are white balance, exposure, gain, auto exposure, auto gain, sharpness, black level.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
RemoveParameterLimit	ENUM	RW

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

- Sample Code

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//Sets remove parameter limits to ON.
emStatus = GXSetEnumValueByString(hDevice, "RemoveParameterLimit",
                                  "On");

//Sets remove parameter limits to OFF.
emStatus = GXSetEnumValueByString(hDevice, "RemoveParameterLimit",
                                  "Off");
```

- Precautions

3.13.8. Flat Field Correction

- Terms

Shadow: The inconsistent response of each pixel of the imaging device results in inconsistent brightness between the center and the four edges, as well as the color deviation caused by the insufficient incidence angle of the LENS in the periphery, which is generally appeared as the color inconsistency between the center and the surrounding area. In order to solve this phenomenon, software processing is used to pre-process the image to eliminate shadows.

- Related Attribute

Attribute Name	Attribute Type	Accessible Properties
FlatFieldCorrection	ENUM	RW
FFCLoad	BUFFER	RO
FFCSave	BUFFER	WO

- Sample Code

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
//Sets flat field correction to ON.
emStatus = GXSetEnumValueByString(hDevice, "FlatFieldCorrection",
                                  "On");
//Gets the length of flat field correction parameters.
size_t nSize = 0;
emStatus = GXGetRegisterLength(hDevice, "FFCLoad", &nSize);

//Applies memory for saving flat field correction parameters.
char *pszBuffer = new char[nSize];

//Loads the flat field correction parameters from the device.
emStatus = GXGetRegisterValue(hDevice, "FFCLoad",
                              (uint8_t *)pszBuffer, &nSize);

//Saves the flat field correction parameters to the device.
emStatus = GXSetRegisterValue(hDevice, "FFCSave" ,
                             (uint8_t *)pszBuffer, nSize);

//Sets flat field correction to OFF.
emStatus = GXSetEnumValueByString(hDevice, "FlatFieldCorrection",
                                  "Off");
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
//After operation
If (NULL != pszBuffer)
{
    delete[] pszBuffer;
    pszBuffer = NULL;
}
```

- Precautions

4. Local Device Control

4.1. Related Attribute

Attribute Name	Attribute Type	Accessible Properties
DeviceCommandTimeout	INT	RW
DeviceCommandRetryCount	INT	RW
EnableAutoConnection	BOOL	RW
DeviceConnectStatus	ENUM	RO
AutoConnectCounts	INT	RO
AutoConnectTimeout	INT	RW

4.2. Sample Code

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//Sets the number of times the command is retried.
uint64_t ui64CommandRetryCount = 0;
emStatus = GXSetIntValue(hDevice, "DeviceCommandRetryCount",
                         ui64CommandRetryCount);
```

4.3. Precautions

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

5. Flow Layer Control

5.1. Statistical Attribute

5.1.1. Related Attribute

Attribute Name	Attribute Type	Accessible Properties
StreamAnnouncedBufferCount	INT	RO
StreamDeliveredFrameCount	INT	RO
StreamLostFrameCount	INT	RO
StreamIncompleteFrameCount	INT	RO
StreamDeliveredPacketCount	INT	RO
StreamResendPacketCount	INT	RO
StreamRescuedPacketCount	INT	RO
StreamResendCommandCount	INT	RO
StreamUnexpectedPacketCount	INT	RO
StreamMissingBlockIDCount	INT	RO

5.1.2. Precautions

During the acquisition process, the user can read these parameters at any time to observe the current driver's acquisition status.

5.1.3. Sample Code

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_DS_HANDLE hDataStream = 0;
emStatus = GXGetDataStreamHandleFromDev (hDevice, 1, &hDataStream);

//Reads the number of frames received.
uint64_t ui64GetFrameCount = 0;
GX_INT_VALUE stValue;
emStatus = GXGetIntValue(hDataStream, "StreamDeliveredFrameCount",
                         &stValue);
ui64GetFrameCount = stValue.nCurValue;
```

5.2. Control Parameters

5.2.1. Related Parameters

- GigE Vision camera related attribute

Attribute Name	Attribute Type	Accessible Properties
MaxPacketCountInOneBlock	INT	RW
MaxPacketCountInOneCommand	INT	RW
ResendTimeout	INT	RW
MaxWaitPacketCount	INT	RW

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

ResendMode	ENUM	RW
BlockTimeout	INT	RW
MaxNumQueueBuffer	INT	RW
StreamBufferHandlingMode	ENUM	RW

“MaxPacketCountInOneBlock”: The maximum resend packet count of a block, that is the maximum resend packet of a frame, if a frame image has send too much resend packets more than the value, then it is not to send the resend command, but directly determine the frame is an incomplete frame. If the value is 0, it means the function is closed.

“MaxPacketCountInOneCommand”: The maximum packet count of one resend process. The packet is judged lost in the receiving process. If the count is too much greater than the value, then it is not to send the resend command, but directly determine the frame is an incomplete frame. If the value is 0, it means the function is closed.

“ResendTimeout”: The timeout time of resent packets. After the retransmission command is sent, the time to wait for the resend packet arrives. If the resend packet does not arrive in this time, it may be an incomplete frame.

“MaxWaitPacketCount”: The maximum count of waiting packets. If a frame has not be transferred completed, the next frame is coming, and the next frame image has received N packets, the previous frame still not transferred completed, then stop the previous frame, and make it as an incomplete frame. If the value is 0, it means the function is closed.

“ResendMode”: The mode of resend, it controls the resend switch.

“BlockTimeout”: The timeout time of data block. The total time of frame receives. If the receive time is longer than the value, you can take the frame as an incomplete one.

“MaxNumQueueBuffer”: The maximum buffer count of the acquisition queue, it can adjust the number of the buffers which involved in data receiving underlying. When many high-resolution cameras acquisition simultaneously fail, you can reduce the value of this parameter appropriately, so that more cameras can acquie at the same time. If the value is too small in a high-resolution acquisition mode, some frames may be lost. Please contact technical support for specific usage.

“StreamBufferHandlingMode”: The Buffer processing mode that the current stream channel may support. GigE Vision supports three modes. The definition and functions of the three modes are as follows:

“StreamBufferHandlingMode” (OldestFirst): The default value. The image buffer follows the first-in-first-out principle. After all the buffers are filled, the new image data will be discarded until the user completes the processing of the buffer that has filled the image data. The typical application is to receive each frame of images acquired by the camera without losing frames. In order to achieve no frame loss, the speed of image data transmission and processing need to be as fast as possible (at least less than the frame period).

“StreamBufferHandlingMode” (OldestFirstOverwrite): Follow the first-in-first-out principle. The difference from the OldestFirst mode is that when all the buffers are filled, the SDK will automatically discard one frame of image buffer with the oldest timestamp to receive new image data. The typical application is that it does not require receiving each frame of images acquired by the camera, and the image data transmission and processing speed is slow.

“StreamBufferHandlingMode” (NewestOnly): In this mode, the user always receives the latest image received by the SDK. Each time the SDK receives a new frame of image data, it will automatically discard the image with the old timestamp. Therefore, when the user's image processing is not timely or the speed is slow, frame loss will occur. In the main applications, the real-time requirements of image acquisition and

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

display are high, and it is not required to receive each frame of images acquired by the camera. However, depending on the camera's frame rate, memory cache, transmission speed, and user applications, there may be a delay between the latest image received by the SDK and the latest image exposed by the camera.

- USB3 Vision camera related attribute

Attribute Name	Attribute Type	Accessible Properties
StreamTransferSize	INT	RW
StreamTransferNumberUrb	INT	RW
StreamBufferHandlingMode	ENUM	RW

“StreamTransferSize”: The size of the transmission data block. For USB3 Vision series camera, it means the size of every transmission data block.

“StreamTransferNumberUrb”: The number of transmission data block. This parameter limits the data block number of single camera mapping to the system kernel actually. The default value is 64, if the value is too small, it will affect the transmission efficiency; However, the number of total data block which can be mapping to the system kernel is limited. When multiple cameras are used together, you can reduce the value appropriately, and increase the number of acquisition device at the same time.

“StreamBufferHandlingMode”: The Buffer processing mode that the current stream channel may support. USB3 Vision supports the same mode as GigE Vision.

5.2.2. Precautions

The above control parameters can only be modified after stopping acquisition, and the control parameters are not allowed to be modified in the acquisition process.

5.2.3. Sample Code

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_DS_HANDLE hDataStream = 0;
emStatus = GXGetDataStreamHandleFromDev (hDevice, 1, &hDataStream);

//Sets the timeout time for resend.
uint64_t ui64ResendTimeout = 128;
emStatus = GXSetIntValue(hDataStream, "ResendTimeout",
                         ui64ResendTimeout);
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

6. Functions Affected by Camera Model

Camera Model: MER-040-60UM/C	
1	[UC/UM] Adjust the value of "Width" will update the current value and minimum value of the "HBlanking" automatically. Formulas are as follows: HBlanking_min=MAX(-21,0x236-Width+1); HBlanking_cur=MAX(HBlanking_cur, HBlanking_min);
2	[UM] Both "BinningHorizontal" and "BinningVertical" have interconnected relationships.

Table 6-1 MER-040-60UM/C

Camera Model: MER-500-7UM/C	
1	When adjusting the Binning function, the horizontal and vertical value of Decimation must be 0. When adjusting the Decimation function, the horizontal and vertical value of Binning must be 0.
2	The horizontal parameter value of Binning must not be 2, it may be 0, 1, 3.

Table 6-2 MER-500-7UM/C

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

7. GxIAPI Library Definitions

7.1. Type

7.1.1. Data Type

Name	Description
Int8_t	8-bit signed integer
Int16_t	16-bit signed integer
Int32_t	32-bit signed integer
Int64_t	64-bit signed integer
uint8_t	8-bit unsigned integer
uint16_t	16-bit unsigned integer
uint32_t	32-bit unsigned integer
uint64_t	64-bit unsigned integer

7.1.2. Handle Type

Name	Description
GX_PORT_HANDLE	Universal handle
GX_IF_HANDLE	Interface handle
GX_DEV_HANDLE	Device handle: This handle is used for control and acquisition functions, which can be obtained through GXOpenDevice
GX_DS_HANDLE	DataStream handle
GX_LOCAL_DEV_HANDLE	Local device layer handle
GX_EVENT_CALLBACK_HANDLE	Device's event callback handle: register device related event callback functions, such as device disconnection callback functions
GX_FEATURE_CALLBACK_BY_STRING_HANDLE	Device attribute update callback handle
GX_FEATURE_CALLBACK_HANDLE	Device attribute update callback handle: Obtained when registering device attribute update callback function

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

GX_WIND_HANDLE	The handle of passing in parent window. Setting it to NULL indicates pop-up display. (Currently only supports setting to NULL)
GX_OPERATE_HANDLE	The handle of operation window, which can be obtained through the interface GXCreateWnd.
GX_RECORDER_HANDLE	The handle of saving videos, which can be obtained through the interface GXCreateRecorder.

7.1.3. Callback Function Type

Name	Description
typedef void (GX_STDC * GXCaptureCallBack) (**GX_FRAME_CALLBACK_PARAM** *pFrameData)	Capture callback function type
typedef void (GX_STDC *GXDeviceOfflineCallBack) (void *pUserParam)	Device offline callback function type
typedef void (GX_STDC *GXFeatureCallBack) (GX_FEATURE_ID_CMD nFeatureID, void *pUserParam)	Device attribute update callback function type
typedef void (GX_STDC *GXFeatureCallBackByString) (const char* pszFeatureName, void *pUserParam)	Device's feature callback function type
typedef void (GX_STDC *GXDeviceReconnectCallBack) (void *pUserParam)	Device reconnection callback function type
typedef void (GX_STDC *GXDeviceDisconnectCallBack) (void *pUserParam)	Device disconnection callback function type

7.2. Constant

7.2.1. GX_STATUS_LIST

```
typedef enum GX_STATUS_LIST
{
    GX_STATUS_SUCCESS              = 0,
    GX_STATUS_ERROR                = -1,
    GX_STATUS_NOT_FOUND_TL         = -2,
    GX_STATUS_NOT_FOUND_DEVICE     = -3,
    GX_STATUS_OFFLINE              = -4,
    GX_STATUS_INVALID_PARAMETER    = -5,
    GX_STATUS_INVALID_HANDLE       = -6,
    GX_STATUS_INVALID_CALL         = -7,
    GX_STATUS_INVALID_ACCESS       = -8,
    GX_STATUS_NEED_MORE_BUFFER     = -9,
    GX_STATUS_ERROR_TYPE           = -10,
    GX_STATUS_OUT_OF_RANGE         = -11,
    GX_STATUS_NOT_IMPLEMENTED      = -12,
    GX_STATUS_NOT_INIT_API         = -13,
    GX_STATUS_TIMEOUT              = -14
}GX_STATUS_LIST;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Name	Description
GX_STATUS_SUCCESS	Success
GX_STATUS_ERROR	There is an unspecified internal error that is not expected to occur
GX_STATUS_NOT_FOUND_TL	The TL library cannot be found
GX_STATUS_NOT_FOUND_DEVICE	The device is not found
GX_STATUS_OFFLINE	The current device is in an offline status
GX_STATUS_INVALID_PARAMETER	Invalid parameter. Generally, the pointer is NULL or the input IP and other parameter formats are invalid
GX_STATUS_INVALID_HANDLE	Invalid handle
GX_STATUS_INVALID_CALL	The interface is invalid, which refers to software interface logic error
GX_STATUS_INVALID_ACCESS	The function is currently inaccessible or the device access mode is incorrect
GX_STATUS_NEED_MORE_BUFFER	The user request buffer is insufficient: the user input buffersize during the read operation is less than the actual need
GX_STATUS_ERROR_TYPE	The type of FeatureID used by the user is incorrect, such as an integer interface using a floating-point function code
GX_STATUS_OUT_OF_RANGE	The value written by the user is crossed
GX_STATUS_NOT_IMPLEMENTED	This function is not currently supported
GX_STATUS_NOT_INIT_API	There is no call to initialize the interface
GX_STATUS_TIMEOUT	Timeout error

7.2.2. GX_FRAME_STATUS

```
enum GX_FRAME_STATUS_LIST
{
    GX_FRAME_STATUS_SUCCESS          = 0,
    GX_FRAME_STATUS_INCOMPLETE       = -1
};
typedef int32_t GX_FRAME_STATUS;
```

Name	Description
GX_FRAME_STATUS_SUCCESS	Normal frame
GX_FRAME_STATUS_INCOMPLETE	Incomplete frame

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

7.2.3. GX_ACCESS_MODE

```
typedef enum GX_ACCESS_MODE
{
    GX_ACCESS_READONLY      =2,
    GX_ACCESS_CONTROL       =3,
    GX_ACCESS_EXCLUSIVE      =4
} GX_ACCESS_MODE;
typedef int32_t GX_ACCESS_MODE_CMD;
```

Name	Description
GX_ACCESS_READONLY	Open the device in read-only mode
GX_ACCESS_CONTROL	Open the device in controlled mode
GX_ACCESS_EXCLUSIVE	Open the device in exclusive mode

7.2.4. GX_OPEN_MODE

```
typedef enum GX_OPEN_MODE
{
    GX_OPEN_SN              =0,
    GX_OPEN_IP              =1,
    GX_OPEN_MAC             =2,
    GX_OPEN_INDEX           =3,
    GX_OPEN_USERID          =4
} GX_OPEN_MODE;
typedef int32_t GX_OPEN_MODE_CMD;
```

Name	Description
GX_OPEN_SN	Opens the device via a serial number
GX_OPEN_IP	Opens the device via an IP address
GX_OPEN_MAC	Opens the device via a MAC address
GX_OPEN_INDEX	Opens the device via a serial number (Start from 1, such as 1, 2, 3, 4…)
GX_OPEN_USERID	Opens the device via user defined ID

7.2.5. GX_IP_CONFIGURE_MODE_LIST

```
typedef enum GX_IP_CONFIGURE_MODE_LIST
{
    GX_IP_CONFIGURE_DHCP ,
    GX_IP_CONFIGURE_LLA ,
    GX_IP_CONFIGURE_STATIC_IP ,
    GX_IP_CONFIGURE_DEFAULT
}GX_IP_CONFIGURE_MODE_LIST;
typedef int32_t GX_IP_CONFIGURE_MODE;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Name	Description
GX_IP_CONFIGURE_DHCP	Enable the DHCP mode to allocate the IP address by the DHCP server
GX_IP_CONFIGURE_LLA	Enable the LLA mode to allocate the IP addresses
GX_IP_CONFIGURE_STATIC_IP	Enable the static IP mode to configure the IP address
GX_IP_CONFIGURE_DEFAULT	Enable the default mode to configure the IP address

7.2.6. GX_RESET_DEVICE_MODE

```
typedef enum GX_RESET_DEVICE_MODE
{
    GX_MANUFACTURER_SPECIFIC_RESET          = 0x2
}GX_RESET_DEVICE_MODE;
typedef int32_t GX_RESET_DEVICE_MODE;
```

Name	Description
GX_MANUFACTURER_SPECIFIC_RESET	Device reset. It is equivalent to powering off and powering up the device

7.2.7. GX_TL_TYPE_LIST

```
typedef enum GX_TL_TYPE_LIST
{
    GX_TL_TYPE_UNKNOWN      = 0,
    GX_TL_TYPE_USB          = 1,
    GX_TL_TYPE_GEV          = 2,
    GX_TL_TYPE_U3V          = 4,
    GX_TL_TYPE_CXP          = 8
}GX_TL_TYPE_LIST;
typedef int32_t GX_TL_TYPE;
```

Name	Description
GX_TL_TYPE_UNKNOWN	Unknown device
GX_TL_TYPE_USB	USB2.0
GX_TL_TYPE_GEV	GEV
GX_TL_TYPE_U3V	U3V
GX_TL_TYPE_CXP	CXP

7.2.8. GX_NODE_ACCESS_MODE

```
typedef enum GX_NODE_ACCESS_MODE
{
    GX_NODE_ACCESS_MODE_NI      = 0,
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    GX_NODE_ACCESS_MODE_NA        = 1,
    GX_NODE_ACCESS_MODE_WO        = 2,
    GX_NODE_ACCESS_MODE_RO        = 3,
    GX_NODE_ACCESS_MODE_RW        = 4,
    GX_NODE_ACCESS_MODE_UNDEF = 5
}GX_NODE_ACCESS_MODE;
```

Name	Description
GX_NODE_ACCESS_MODE_NI	Not implemented
GX_NODE_ACCESS_MODE_NA	Not available
GX_NODE_ACCESS_MODE_WO	Write only
GX_NODE_ACCESS_MODE_RO	Read only
GX_NODE_ACCESS_MODE_RW	Write and read
GX_NODE_ACCESS_MODE_UNDEF	Undefined

7.2.9. GX_DEVICE_CLASS

```
enum GX_DEVICE_CLASS_LIST
{
    GX_DEVICE_CLASS_UNKNOWN          = 0,
    GX_DEVICE_CLASS_USB2             = 1,
    GX_DEVICE_CLASS_GEV              = 2,
    GX_DEVICE_CLASS_U3V              = 3,
    GX_DEVICE_CLASS_SMART            = 4,
    GX_DEVICE_CLASS_CXP              = 5
};
typedef    int32_t    GX_DEVICE_CLASS;
```

Name	Description
GX_DEVICE_CLASS_UNKNOWN	Unknown device
GX_DEVICE_CLASS_USB2	USB2.0 device
GX_DEVICE_CLASS_GEV	GigE Vision device
GX_DEVICE_CLASS_U3V	USB3 Vision device
GX_DEVICE_CLASS_SMART	Smart device
GX_DEVICE_CLASS_CXP	CXP device

7.2.10. GX_LOG_TYPE_LIST

```
enum GX_LOG_TYPE_LIST
{
    GX_LOG_TYPE_OFF          = 0x00000000,
    GX_LOG_TYPE_FATAL        = 0x00000001,
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    GX_LOG_TYPE_ERROR       = 0x00000010,
    GX_LOG_TYPE_WARN        = 0x00000100,
    GX_LOG_TYPE_INFO        = 0x00001000,
    GX_LOG_TYPE_DEBUG       = 0x00010000,
    GX_LOG_TYPE_TRACE       = 0x00100000
};
typedef    uint32_t    GX_LOG_TYPE_LIST;
```

Name	Description
GX_LOG_TYPE_OFF	Do not generate any type of log
GX_LOG_TYPE_FATAL	Generate FATAL type logs
GX_LOG_TYPE_ERROR	Generate ERROR type logs
GX_LOG_TYPE_WARN	Generate WARNING type logs
GX_LOG_TYPE_INFO	Generate INFO type logs
GX_LOG_TYPE_DEBUG	Generate DEBUG type logs
GX_LOG_TYPE_TRACE	Generate TRACE type logs

7.2.11. GX_WINDOWS_ID

```
typedef enum GX_WINDOWS_ID
{
    GX_WND_PROP = 0,
}GX_WINDOW_TYPE;
```

Name	Description
GX_WND_PROP	The display window type as properties tree.

7.2.12. GX_WINDOWS_SHOW_MODE

```
typedef enum GX_WINDOWS_SHOW_MODE
{
    NON_BLOCK_SHOW_MODE = 0,
    BLOCK_SHOW_MODE = 1,
}GX_WINDOWS_SHOW_MODE;
```

Name	Description
NON_BLOCK_SHOW_MODE	Displaying the properties window in non-blocking mode is the default behaviour. No configuration is required in GUI applications.
BLOCK_SHOW_MODE	Displaying the properties window in blocking mode is not modal behaviour. Upon setting this mode, the programme blocks at the GXShowWnd interface. This

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

	mode is only applicable when configured in console mode.

7.2.13. GX_CFA_METHOD

```
typedef enum GX_CFA_METHOD
{
GX_CFA_METHOD_QUICK = 0,
GX_CFA_METHOD_BALANCE = 1,
GX_CFA_METHOD_OPTIMAL = 2,
}GX_CFA_METHOD;
```

Name	Description
GX_CFA_METHOD_QUICK	Quick.
GX_CFA_METHOD_BALANCE	Balance.
GX_CFA_METHOD_OPTIMAL	Optimal.

7.2.14. GX_IMAGE_FORMAT_TYPE

```
typedef enum GX_IMAGE_FORMAT_TYPE
{
GX_IMAGE_FORMAT_JPEG = 0, ///<JPEG
GX_IMAGE_FORMAT_PNG,///<PNG
GX_IMAGE_FORMAT_TIFF,///<TIFF
GX_IMAGE_FORMAT_RAW,///<RAW
GX_IMAGE_FORMAT_BMP,///<BMP
}GX_IMAGE_FORMAT_TYPE;
```

Name	Description
GX_IMAGE_FORMAT_JPEG	JPEG format.
GX_IMAGE_FORMAT_PNG	PNG format.
GX_IMAGE_FORMAT_TIFF	TIFF format.
GX_IMAGE_FORMAT_RAW	RAW format.
GX_IMAGE_FORMAT_BMP	BMP format.

7.2.15. GX_VIDEO_FORMAT_TYPE

```
typedef enum GX_VIDEO_FORMAT_TYPE
{
GX_VIDEO_FORMAT_H264_AVI = 0, ///<AVI
GX_VIDEO_FORMAT_H264_MP4 = 1, ///<MP4
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    GX_VIDEO_FORMAT_ORIGINAL_AVI = 2,
}GX_VIDEO_FORMAT_TYPE;
```

Name	Description
GX_VIDEO_FORMAT_H264_AVI	H264_AVI format.
GX_VIDEO_FORMAT_H264_MP4	H264_MP4 format.
GX_VIDEO_FORMAT_ORIGINAL_AVI	RAW AVI format.

7.3. Structure

7.3.1. GX_OPEN_PARAM

Related interface: GXOpenDevice()

This structure is designed for the open device interface.

```
typedef struct GX_OPEN_PARAM
{
    char *pszContent;
    GX_OPEN_MODE_CMD      openMode;
    GX_ACCESS_MODE_CMD    accessMode;
} GX_OPEN_PARAM;
```

Name	Description
pszContent	Standard C string that is decided by openMode. It could be an IP address, a camera serial number, and so on
openMode	Device open mode. The device can be open via the SN, IP, MAC, etc. Please refer to GX_OPEN_MODE
accessMode	Device access mode, such as read-only, control, exclusive, etc. Please refer to GX_ACCESS_MODE

7.3.2. GX_FRAME_DATA_EX

Related interface:GXGetImageEx()

```
typedef struct GX_FRAME_DATA_EX
{
    GX_FRAME_STATUS    nStatus;
    int32_t            nWidth;
    int32_t            nHeight;
    int32_t            nOffsetX;
    int32_t            nOffsetY;
    int32_t            nPixelFormat;
    int32_t            nImgSize;
    pvoid64            pImgBuf;
    uint64_t           nFrameID;
    uint64_t           nTimestamp;
    uint64_t           nBufID;
    pvoid64            pUserParam;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    pvoid64             pChunkDataHandle;
    int32_t             i32Reserved[128];
}GX_FRAME_DATA;
```

Name	Description
nStatus	The state of the acquired image. Please refer to GX_FRAME_STATUS
nWidth	Image width
nHeight	Image height
nOffsetX	X-direction offset of the image
nOffsetY	Y-direction offset of the image
nPixelFormat	The pixel format of the image
nImgSize	Image sizer (when the frame information function is enabled, the nImgSize contains both image size and frame information size)
pImgBuf	Image data pointer (when the frame information function is enabled, the pImgBuf contains both image data and frame information data)
nFrameID	Frame identification of image
nTimestamp	Timestamp of image
nBufID	The bufID of the image
pUserParam	User parameters
pChunkDataHandle	Frame information handle. It is used to obtain the frame information data.
i32Reserved	Reserved

7.3.3. GX_REGISTER_STACK_ENTRY

Related interface: GXReadRemoteDevicePortStacked(), GXWriteRemoteDevicePortStacked().

The interface describes the register address and value information corresponding to the attribute value.

```
typedef struct GX_REGISTER_STACK_ENTRY
{
    uint64_t    nAddress;
    void*       pBuffer;
    size_t      nSize;
} GX_REGISTER_STACK_ENTRY;
```

Name	Description
nAddress	Register address
pBuffer	Memory of register value
nSize	Length of register value

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

7.3.4. GX_INTERFACE_INFO

Related interface: GXGetInterfaceInfo()
INTERFACE information.

```
typedef struct GX_INTERFACE_INFO
{
    GX_TL_TYPE                  emTLayerType;
    unsigned int                nReserved[4];
    union
    {
        GX_CXP_INTERFACE_INFO   stCXPIFInfo;
        GX_GEV_INTERFACE_INFO   stGEVIFInfo;
        GX_U3V_INTERFACE_INFO   stU3VIFInfo;
        GX_USB_INTERFACE_INFO   stUSBIFInfo;
        unsigned int            nReserved[64];
    }IFInfo;
}GX_INTERFACE_INFO;
```

Name	Description
emTLayerType	Device transport layer protocol type
nReserved	Reserved field
stCXPIFInfo	CXP frame grabber information
stGEVIFInfo	GEV frame grabber information
stU3VIFInfo	U3V frame grabber information
stUSBIFInfo	USB frame grabber information
nReserved	Reserved field

7.3.4.1. GX_CXP_INTERFACE_INFO

```
typedef struct GX_CXP_INTERFACE_INFO
{
    unsigned char       chInterfaceID[GX_INFO_LENGTH_64_BYTE];
    unsigned char       chDisplayName[GX_INFO_LENGTH_64_BYTE];
    unsigned char       chSerialNumber[GX_INFO_LENGTH_64_BYTE];
    unsigned int        ui32InitFlag;
    unsigned int        nPCIEInfo;
    unsigned int        nReserved[64];
}GX_CXP_INTERFACE_INFO;
```

Name	Description
chInterfaceID	Interface ID
chDisplayName	Display name
chSerialNumber	Serial number
ui32InitFlag	Initial flag

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

nPCIEInfo	PCIE slot information of frame grabber
nReserved	Reserved field

7.3.4.2. GX_GEV_INTERFACE_INFO

```
typedef struct GX_GEV_INTERFACE_INFO
{
    unsigned char       chInterfaceID[GX_INFO_LENGTH_64_BYTE];
    unsigned char       chDisplayName[GX_INFO_LENGTH_64_BYTE];
    unsigned char       chSerialNumber[GX_INFO_LENGTH_64_BYTE];
    char                szDescription[GX_INFO_LENGTH_256_BYTE];
    unsigned int        nReserved[64];
}GX_GEV_INTERFACE_INFO;
```

Name	Description
chInterfaceID	Interface ID
chDisplayName	Display name
chSerialNumber	Serial number
szDescription	Description
nReserved	Reserved field

7.3.4.3. GX_U3V_INTERFACE_INFO

```
typedef struct GX_U3V_INTERFACE_INFO
{
    unsigned char       chInterfaceID[GX_INFO_LENGTH_64_BYTE];
    unsigned char       chDisplayName[GX_INFO_LENGTH_64_BYTE];
    unsigned char       chSerialNumber[GX_INFO_LENGTH_64_BYTE];
    char                szDescription[GX_INFO_LENGTH_256_BYTE];
    unsigned int        nReserved[64];
}GX_U3V_INTERFACE_INFO;
```

Name	Description
chInterfaceID	Interface ID
chDisplayName	Display name
chSerialNumber	Serial number
szDescription	Description
nReserved	Reserved field

7.3.4.4. GX_USB_INTERFACE_INFO

```
typedef struct GX_USB_INTERFACE_INFO
{
    unsigned char       chInterfaceID[GX_INFO_LENGTH_64_BYTE];
    unsigned char       chDisplayName[GX_INFO_LENGTH_64_BYTE];
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    unsigned char       chSerialNumber[GX_INFO_LENGTH_64_BYTE];
    char                szDescription[GX_INFO_LENGTH_256_BYTE];
    unsigned int        nReserved[64];
}GX_USB_INTERFACE_INFO;
```

Name	Description
chInterfaceID	Interface ID
chDisplayName	Display name
chSerialNumber	Serial number
szDescription	Description
nReserved	Reserved field

7.3.5. GX_DEVICE_INFO

Related interface: GXGetDeviceInfo()
Description of device information.

```
typedef struct GX_DEVICE_INFO
{
    GX_DEVICE_CLASS             emDevType;
    unsigned int                nReserved[4];
    {
        GX_CXP_DEVICE_INFO      stCXPDevInfo;
        GX_GEV_DEVICE_INFO      stGEVDevInfo;
        GX_U3V_DEVICE_INFO      stU3VDevInfo;
        GX_USB_DEVICE_INFO      stUSBDevInfo;
        unsigned int            nReserved[256];
    }DevInfo;
}GX_DEVICE_INFO;
```

Name	Description
emDevType	Device type
nReserved	Reserved field
stCXPDevInfo	CXP device information
stGEVDevInfo	GEV device information
stU3VDevInfo	U3V device information
stUSBDevInfo	USB device information
nReserved	Reserved field

7.3.5.1. GX_CXP_DEVICE_INFO

```
typedef struct GX_CXP_DEVICE_INFO
{
    unsigned char       chVendorName[GX_INFO_LENGTH_64_BYTE];
    unsigned char       chModelName[GX_INFO_LENGTH_64_BYTE];
    unsigned char       chManufacturerInfo[GX_INFO_LENGTH_64_BYTE];
```

```
    unsigned char       chDeviceVersion[GX_INFO_LENGTH_64_BYTE];
    unsigned char       chSerialNumber[GX_INFO_LENGTH_64_BYTE];
    unsigned char       chUserDefinedName[GX_INFO_LENGTH_64_BYTE];
    unsigned int        nReserved[64];
}GX_CXP_DEVICE_INFO;
```

Name	Description
chVendorName	Vendor name
chModelName	Model name
chManufacturerInfo	Manufacturer information
chDeviceVersion	Device version
chSerialNumber	Serial number
chUserDefinedName	User defined name
nReserved	Reserved field

7.3.5.2. GX_GEV_DEVICE_INFO

```
typedef struct GX_GEV_DEVICE_INFO
{
    unsigned int        nIpCfgOption;
    unsigned int        nIpCfgCurrent;
    unsigned int        nCurrentIp;
    unsigned int        nCurrentSubNetMask;
    unsigned int        nDefultGateWay;
    unsigned int        nNetExport;
    uint64_t            nMacAddress;
    unsigned char       chVendorName[GX_INFO_LENGTH_64_BYTE];
    unsigned char       chModelName[GX_INFO_LENGTH_64_BYTE];
    unsigned char       chManufacturerInfo[GX_INFO_LENGTH_64_BYTE];
    unsigned char       chDeviceVersion[GX_INFO_LENGTH_64_BYTE];
    unsigned char       chSerialNumber[GX_INFO_LENGTH_64_BYTE];
    unsigned char       chUserDefinedName[GX_INFO_LENGTH_64_BYTE];
    unsigned int        nReserved[64];
}GX_GEV_DEVICE_INFO;
```

Name	Description
nIpCfgOption	IP configuration option
nIpCfgCurrent	Current IP configuration, please refer to the IP configuration option
nCurrentIp	Current IP address
nCurrentSubNetMask	Current subnet mask
nDefaultGateWay	Default gateway
nNetExport	Network card IP address

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

nMacAddress	MAC address
chVendorName	Vendor name
chModelName	Model name
chManufacturerInfo	Manufacturer information
chDeviceVersion	Device version
chSerialNumber	Serial number
chUserDefinedName	User defined name
nReserved	Reserved field

7.3.5.3. GX_U3V_DEVICE_INFO

```
typedef struct GX_U3V_DEVICE_INFO
{
    unsigned char    chVendorName[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chModelName[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chManufacturerInfo[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chDeviceVersion[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chSerialNumber[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chUserDefinedName[GX_INFO_LENGTH_64_BYTE];
    unsigned int     nReserved[64];
}GX_U3V_DEVICE_INFO;
```

Name	Description
chVendorName	Vendor name
chModelName	Model name
chManufacturerInfo	Manufacturer information
chDeviceVersion	Device version
chSerialNumber	Serial number
chUserDefinedName	User defined name
nReserved	Reserved field

7.3.5.4. GX_USB_DEVICE_INFO

```
typedef struct GX_USB_DEVICE_INFO
{
    unsigned char    chVendorName[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chModelName[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chManufacturerInfo[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chDeviceVersion[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chSerialNumber[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chUserDefinedName[GX_INFO_LENGTH_64_BYTE];
    unsigned int     nReserved[64];
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
}GX_USB_DEVICE_INFO;
```

Name	Description
chVendorName	Vendor name
chModelName	Model name
chManufacturerInfo	Manufacturer information
chDeviceVersion	Device version
chSerialNumber	Serial number
chUserDefinedName	User defined name
nReserved	Reserved field

7.3.6. GX_INT_VALUE

Related interface: GXGetIntValue()
Attribute value information.

```
typedef struct GX_INT_VALUE
{
    int64_t nCurValue;
    int64_t nMin;
    int64_t nMax;
    int64_t nInc;
    int32_t reserved[16];
}GX_INT_VALUE;
```

Name	Description
nCurValue	Current value
nMin	Minimum value
nMax	Maximum value
nInc	Step size
reserved	Reserved field

7.3.7. GX_FLOAT_VALUE

Related interface: GXGetFloatValue()
Attribute value information.

```
typedef struct GX_FLOAT_VALUE
{
    double  dCurValue;
    double  dMin;
    double  dMax;
    double  dInc;
    bool    bIncIsValid;
    char    szUnit[GX_INFO_LENGTH_8_BYTE];
    int32_t reserved[16];
}GX_FLOAT_VALUE;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Name	Description
dCurValue	Current value
dMin	Minimum value
dMax	Maximum value
dInc	Step size
szUnit	Unit
reserved	Reserved field

7.3.8. GX_STRING_VALUE

Related interface: GXGetStringValue()
Attribute value information.

```
typedef struct GX_STRING_VALUE
{
    char    strCurValue[GX_INFO_LENGTH_256_BYTE];
    int64_t nMaxLength;
    int32_t reserved[4];
}GX_STRING_VALUE;
```

Name	Description
strCurValue	Current value
nMaxLength	Maximum length
reserved	Reserved field

7.3.9. GX_ENUM_VALUE

Related interface: GXGetEnumValue()
Attribute value information.

```
typedef struct GX_ENUM_VALUE
{
    GX_ENUM_VALUE_DES       stCurValue;
    uint32_t                nSupportedNum;
    GX_ENUM_VALUE_DES       nArrySupportedValue[128];
    int32_t                 reserved[16];
}GX_ENUM_VALUE;
```

Name	Description
stCurValue	Current value
nSupportedNum	Number of supported subitems
nArrySupportedValue	Value of supported subitems
reserved	Reserved field

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

7.3.9.1. GX_ENUM_VALUE_DES

```
typedef struct GX_ENUM_VALUE_DES
{
    int64_t nCurValue;
    char    strCurSymbolic[GX_INFO_LENGTH_128_BYTE];
    int32_t reserved[4];
}GX_ENUM_VALUE_DES;
```

Name	Description
nCurValue	Current value
strCurSymbolic	Symbolic name of the current value
reserved	Reserved field

7.3.10. GX_GIGE_ACTION_COMMAND_RESULT

Described the status information returned by the Action command.

```
typedef struct GX_GIGE_ACTION_COMMAND_RESULT
{
    char strDeviceAddress[16];
    int32_t nStatus;
}GX_GIGE_ACTION_COMMAND_RESULT;
```

Definition	Explanation
strDeviceAddress	The current device IP that returns ack (Dotted Decimal Notation IPv4)
nStatus	The ACTION status returned by the device, the error originates from the GigaVision protocol **0:** Command sent successfully. **0x8013:** The device is not time synchronized with the master clock. Before executing this interface, “PtpEnable” must be enabled and “PtpStatus” must be set to “Master” or “Slave” (indicating synchronization with the master clock). **0x8015:** The device queue or packet data has overflowed. When the device corresponding to strDeviceAddress is executing the previous GXGigEIssueScheduledActionCommand request and receives a new GXGigEIssueScheduledActionCommand request, this error will be returned. **0x8016:** The Scheduled Action Command issued by GXGigEIssueScheduledActionCommand is outdated.

7.3.11. GX_SAVE_IMAGE_INFO

Described the parameter information of the saved images.

```
typedef struct GX_SAVE_IMAGE_INFO
{
unsigned char*          pImageBuffer;
uint32_t                nWidth;
uint32_t                nHeight;
GX_PIXEL_FORMAT_ENTRY   emSrcFormat;
GX_CFA_METHOD           emCfaMethod;
GX_IMAGE_FORMAT_TYPE    emImgFormat;
char*                   pImgPath;
uint32_t                nImgQuality;
uint32_t                nReserved[64];
}GX_SAVE_IMAGE_INFO;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Definition	Explanation
pImageBuffer	The buffer of captured images. It can be collected through function GXDQBufEx or function GXGetImageEx.
nWidth	The width of the saved image buffer.
nHeight	The height of the saved image buffer.
emSrcFormat	The pixel format of the saved image buffer.
emCfaMethod	The interpolation method of the saved image.
emImgFormat	The format of saved images[bmp/jpg/png/raw/tiff]
pImgPath	The path of saved images.
nImgQuality	Encoding quality, (0-100], applicable to JPEG only.
nReserved	Reserved.

7.3.12. GX_RECORD_PARAM

Described the configuration information for saved videos.

```
typedef struct GX_RECORD_PARAM
{
GX_PIXEL_FORMAT_ENTRY      emPixelFormat;    ///<Pixel format.
GX_VIDEO_FORMAT_TYPE       emVideoFormat;    ///<Video format.
                                               [h264_avi/h264_mp4/original_avi]
uint32_t                   nWidth;           ///<The width of the image.
uint32_t                   nHeight;          ///< The height of the image.
uint32_t                   nFrameRate;       ///<Frame rate (FPS).
uint32_t                   nBitRate;         ///< Bit rate (kbps).
char*                      pPathName;        ///< The name of the saving path.
uint32_t                   nReserved[64];
}GX_RECORD_PARAM;
```

Definition	Explanation
emPixelFormat	The pixel format of the saved image buffer.
emVideoFormat	Video format [bmp/jpg/png/raw/tiff]
nWidth	The width of the saved image buffer.
nHeight	The height of the saved image buffer.
nFrameRate	Frame rate (FPS).
nBitRate	Bit rate (kbps).
pPathName	The name of the saving path.
nReserved	Reserved.

7.4. Interfaces

7.4.1. Get SDK Version Information

7.4.1.1. GXGetLibVersion (Linux only)

Declarations:

```
GX_EXTC const char* GX_STDC GXGetLibVersion()
```

Descriptions:

Get the interface library version.

Parameters:

Not have.

Returns:

Library version of string type.

Sample Code:

```
#include "GxIAPI.h"

int main(int argc, char* argv[])
{
   //Get the library version.
   const char* pszLibVersion = GXGetLibVersion();
   return 0;
}
```

7.4.2. Initialization and Cancellation

7.4.2.1. GXInitLib

Declarations:

GX_API GXInitLib()

Descriptions:

Initialize the device library for some resource application operations. This interface must be called before using the GxIAPI to interact with the camera, and the GXCloseLib() must be called to release all the resources when the GxIAPI is stopped for all control of the device.

Parameters:

Not have.

Returns:

GX_STATUS_SUCCESS	The operation is successful, no error occurs.
GX_STATUS_NOT_FOUND_TL	Can not found the library.

The errors that are not covered above please reference GX_STATUS_LIST.

Precautions:

Before calling the other interfaces (except GXCloseLib() / GXGetLastError()), you must to call the GXInitLib() interface for initialization first, otherwise the error GX_STATUS_NOT_INIT_API will return.

Sample Code:

```
#include "GxIAPI.h"

int main(int argc, char* argv[])
{
   GX_STATUS emStatus = GX_STATUS_SUCCESS;

   //Call GXInitLib() at the start location to initialize and
   //apply for resources.
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    emStatus = GXInitLib();
    if (emStatus != GX_STATUS_SUCCESS)
    {
        return 0;
    }
    //Use GxIAPI.
    //...

    //Call GXCLoseLib() at the end of the program to release the
    //resource.
    emStatus = GXCloseLib();
    return 0;
}
```

7.4.2.2. GXCloseLib

Declarations:

GX_API GXCloseLib()

Descriptions:

Close the device library to release resources. You must to call this interface to release resources when the GxIAPI stopped all the controls of the device. Corresponding to the GXInitLib().

Parameters:

Not have.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
#include "GxIAPI.h"
int main(int argc, char* argv[])
{
    GX_STATUS emStatus = GX_STATUS_SUCCESS;

    //Call GXInitLib() at the start location to initialize and apply
    //for resources.
    emStatus  = GXInitLib();
    if (emStatus  != GX_STATUS_SUCCESS)
    {
        return 0;
    }
    //Use GxIAPI.
    //...

    //Call GXCLoseLib() at the end of the program to release the resource.
    emStatus  = GXCloseLib();
    return 0;
}
```

7.4.2.3. GXSetLogType

Declarations:

GX_API GXSetLogType(const uint32_t ui32LogType)

Descriptions:

Set the types of logs that can be generated.

Note:

Before calling this interface, initialization must be performed first.

Parameters:

[in]	ui32LogType	Log type, refer to GX_LOG_TYPE_LIST

Returns:

GX_STATUS_SUCCESS	Operation successful with no errors
GX_STATUS_NOT_INIT_API	GXInitLib() initialization library not called
GX_STATUS_INVALID_ACCESS	Error accessing for configuration file

The errors that are not covered above please reference GX_STATUS_LIST。

Sample Code:

```
#include "GxIAPI.h"
int main(int argc, char* argv[])
{
   GX_STATUS emStatus = GX_STATUS_SUCCESS;
   // Call GXInitLib() at the starting position for initialization
   //and request resources
   emStatus = GXInitLib();
   if (emStatus != GX_STATUS_SUCCESS)
   {
      return 0;
   }

// Set to generate three types of logs: Error, WARN, and INFO
uint32_t ui32LogType = GX_LOG_TYPE_ERROR | GX_LOG_TYPE_WARN
                       |GX_LOG_TYPE_INFO;

emStatus = GXSetLogType(ui32LogType);
if (emStatus != GX_STATUS_SUCCESS)
{
   // Handling failure in setting log type
}

// Set not to generate any logs
uint32_t ui32LogType = GX_LOG_TYPE_OFF;
emStatus = GXSetLogType(ui32LogType);
if (emStatus != GX_STATUS_SUCCESS)
{
   // Handling failure in setting log type
}

// Call GXCloseLib() at the end to release resources
emStatus = GXCloseLib();
return 0;
}
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

7.4.2.4. GXGetLogType

Declarations:

GX_API GXGetLogType(uint32_t* pui32Value)

Descriptions:

Get the generated log type.

Note:

Before calling this interface, initialization must be performed first.

Parameters:

[out]	pui32Value	Types of logs that can be generated, refer to GX_LOG_TYPE_LIST

Returns:

GX_STATUS_SUCCESS	Operation successful with no errors
GX_STATUS_NOT_INIT_API	GXInitLib() initialization library not called
GX_STATUS_INVALID_ACCESS	Error accessing for configuration file
GX_STATUS_INVALID_PARAMETER	The pointer entered by the user is NULL

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
emStatus = GXInitLib();

uint32_t ui32LogType;
emStatus = GXGetLogType(&ui32LogType);
if (emStatus != GX_STATUS_SUCCESS)
{
   // Failed to obtain the types of logs that can be generated
}

if (ui32LogType & GX_LOG_TYPE_OFF)
{
   // Do not generate any logs
}
if (ui32LogType & GX_LOG_TYPE_ERROR)
{
   // Generate logs of type ERROR
}
...
emStatus = GXCloseLib();
```

7.4.2.5. GXGetLastError

Declarations:

```
GX_API GXGetLastError (GX_STATUS *pErrorCode,
                       char *pszErrText,
                       size_t *pnSize)
```

Descriptions:

To get the latest error descriptions information of the program.

Parameters:

[out] pErrorCode	Return the last error code. You could set the parameter to NULL if you don't need this value.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

[out] pszErrText Return the address of the buffer allocated for error information.
[in,out] pnSize The address size of the buffer allocated for error information. Unit: byte.
If pszErrText is NULL:
[out] pnSize Return the actual required buffer size.
If pszErrText is not NULL:
[in] pnSize It is the actual allocated buffer size.
[out] pnSize Return the actual allocated buffer size.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.
GX_STATUS_NEED_MORE_BUFFER	The buffer that the user filled is too small.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus  = GX_STATUS_SUCCESS;
GX_STATUS emerrCode = GX_STATUS_SUCCESS;
char* pszTemp = NULL;
size_t nSize = 0;
//First, the user passed the NULL pointer to get the actual size,
//followed by the request buffer to obtain the description information.
GXGetLastError(&emerrCode, NULL, &nSize);
pszTemp = new char[nSize];
emStatus = GXGetLastError(&emerrCode, pszTemp, &nSize);
//After finished
If (NULL != pszTemp)
{
   delete[] pszTemp;
   pszTemp = NULL;
}
```

7.4.2.6. GXUpdateAllDeviceList

Declarations:

GX_API GXUpdateAllDeviceList (uint32_t* punNumDevices, uint32_t nTimeOut)

Descriptions:

Enumerating currently all available devices in subnet and gets the number of devices.

Parameters:

[out] punNumDevices	The address pointer used to return the number of devices, and the pointer can not be NULL.
[in] nTimeOut	The timeout time of enumeration (unit: ms). If the device is successfully enumerated within the specified timeout time, the value returns immediately. If the device is not enumerated within the specified timeout time, then it waits until the specified timeout time is over and then it returns.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus  = GX_STATUS_SUCCESS;
uint32_t ui32DeviceNum = 0;
//Enumerates to get the number of devices. The timeout time is
//decided by the user's environment, and can be set by the user,
//1000ms is an example.
emStatus  = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
```

7.4.2.7. GXUpdateAllDeviceListEx

Declarations:

GX_API GXUpdateAllDeviceListEx (uint64_t nTLType,
uint32_t* punNumDevices, uint32_tnTimeOut)

Descriptions:

Enumerating currently all available devices in entire network and gets the number of devices.

Parameters:

[in] nTLType	Enumerate specific types of devices, refer to GX_TL_TYPE_LIST
[out] punNumDevices	The address pointer used to return the number of devices, and the pointer can not be NULL
[in] nTimeOut	The timeout time of enumeration (unit: ms). If the device is successfully enumerated within the specified timeout time, the value returns immediately. If the device is not enumerated within the specified timeout time, then it waits until the specified timeout time is over and then it returns

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus  = GX_STATUS_SUCCESS;
uint32_t ui32DeviceNum = 0;

//The number of enumerated devices. The timeout time is limited by
//the user's use environment, and can be set by the user, not limited
//to 1000ms.
emStatus = GXUpdateAllDeviceListEx(GX_TL_TYPE_GEV, &ui32DeviceNum, 1000);
```

7.4.2.8. GXGetDeviceInfo

Declarations:

GX_API GXGetDeviceInfo(uint32_t nIndex, GX_DEVICE_INFO* pstDeviceInfo)

Descriptions:

Obtain basic information of nIndex devices.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Parameters:

[in] *nIndex*	The serial number of the device.
[out] *pstDeviceInfo*	The structure pointer of the device information.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_PARAMETER	The index that the user input is cross the border.

The errors that are not covered above please reference GX_STATUS_LIST.

Precautions:

You should call the GxUpdateAllDeviceList(), GXUpdateAllDeviceListEx() interface for an enumeration before calling the function to get the device information.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32DeviceNum = 0;
emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
if (emStatus == GX_STATUS_SUCCESS && (ui32DeviceNum > 0))
{
    GX_DEVICE_INFO stInfo;

    //Obtain information about the first device
    emStatus = GXGetDeviceInfo(1, &stInfo);
    //Taking information of gigabit network devices as an example
    unsigned int nIp = stInfo.stGEVDevInfo.nCurrentIp;
    uint64_t nMacAddress = stInfo.stGEVDevInfo.nMacAddress;
}
```

7.4.2.9. GXOpenDevice

Declarations:

GX_API GXOpenDevice (GX_OPEN_PARAM* pOpenParam, GX_DEV_HANDLE* phDevice)

Descriptions:

Open the device by a specific unique identification, such as: SN, IP, MAC, Index etc.

Parameters:

[in] pOpenParam	The open device parameter which is configurated by user. Ref:GX_OPEN_PARAM.
[out] phDevice	The device handle returned by the interface.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.
GX_STATUS_NOT_FOUND_DEVICE	Not found the device that matches the specific information.
GX_STATUS_INVALID_ACCESS	The device can not be opened under the current access mode.

The errors that are not covered above please reference GX_STATUS_LIST.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Precautions:

It is recommended that you call the GxUpdateAllDeviceList() interface to make an enumeration before calling the function. To ensure that device list within the library is consistent with the current device.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_OPEN_PARAM stOpenParam;

//Access mode.
stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
//stOpenParam.accessMode = GX_ACCESS_READONLY;
//stOpenParam.accessMode = GX_ACCESS_CONTROL;

//Access via serial number.
stOpenParam.openMode = GX_OPEN_SN;
stOpenParam.pszContent = "EA00010002";

//Access via IP address.
//stOpenParam.openMode = GX_OPEN_IP;
//stOpenParam.pszContent = "192.168.40.35";

//Access via MAC address.
//stOpenParam.openMode = GX_OPEN_MAC;
//stOpenParam.pszContent = "54-04-A6-C2-7C-2F";

//Access via enumeration number 1, 2, 3...
//stOpenParam.openMode = GX_OPEN_INDEX;
//stOpenParam.pszContent = "1";
GX_DEV_HANDLE hDevice = NULL;
emStatus = GXOpenDevice(&stOpenParam, &hDevice);
```

7.4.2.10. GXCloseDevice

Declarations:

GX_API GXCloseDevice (GX_DEV_HANDLE hDevice)

Descriptions:

Specify the device handle to close the device.

Parameters:

[in] hDevice	The device handle that the user specified to close. The *hDevice* can be get by GXOpenDevice() interface.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The illegal handle that the user introduces, or reclose the device.

The errors that are not covered above please reference GX_STATUS_LIST.

Precautions:

Close the device handle that has been closed, return the GX_STATUS_INVALID_HANDLE error.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_DEV_HANDLE hDevice = NULL;
GX_OPEN_PARAM stOpenParam;
uint32_t ui32DeviceNum = 0;
//Enumerates to get the number of devices.
//The timeout time is decided by the user's environment, and can be
//set by the user, 1000ms is an example.
emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
if (emStatus == GX_STATUS_SUCCESS && ui32DeviceNum > 0)
{
    //Sets the parameters of the opening device structures.
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode   = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";

    GX_DEV_HANDLE hDevice = NULL;
    emStatus = GXOpenDevice(&stOpenParam, &hDevice);
    if (emStatus == GX_STATUS_SUCCESS)
    {
        //Operates device: control, acquisition.
        //…
        //Closes the device.
        emStatus = GXCloseDevice(hDevice);
    }
}
```

7.4.2.11. GXGetInterfaceNum

Declarations:

GX_API GXGetInterfaceNum(uint32_t* punNumInterfaces);

Descriptions:

Obtain the number of interface.

Precautions：

Before calling this interface, please enumerate first.

Parameters:

[in,out] punNumInterfaces	Save the number of interface pointers

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs
GX_STATUS_INVALID_PARAMETER	Invalid parameter
GX_STATUS_NOT_INIT_API	GXInitLib() initialization library not called

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32InterFaces;
emStatus = GXGetInterfaceNum(&ui32InterFaces);
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

7.4.2.12. GXGetInterfaceInfo

Declarations:

GX_API GXGetInterfaceInfo(uint32_t nIndex, GX_INTERFACE_INFO* pstInterfaceInfo);

Descriptions:

Get the Interface information with the specified index.

Precautions：

The purpose of this interface is to obtain the enumerated Interface Info, please call GXUpdateAllDeviceList(), GXUpdateAllDeviceListEx() interface before calling.

Parameters:

[in] nIndex	Interface order, starting from 1
[in,out] pstInterfaceInfo	Pointer of the storage interface information structure

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs
GX_STATUS_INVALID_PARAMETER	Invalid parameter
GX_STATUS_NOT_INIT_API	GXInitLib() initialization library not called

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

GX_INTERFACE_INFO stInfo;
emStatus = GXGetInterfaceInfo(1, &stInfo);
```

7.4.2.13. GXGetInterfaceHandle

Declarations:

GX_API GXGetInterfaceHandle(uint32_t nIndex, GX_IF_HANDLE* phIF);

Descriptions:

Get the Interface handle with the specify index.

Precautions：

The purpose of this interface is to obtain the enumerated Handle, please call GXUpdateAllDeviceList(), GXUpdateAllDeviceListEx() interface before calling.

Parameters:

[in] nIndex	Interface order, starting from 1
[in,out] phIF	Pointer to Interface handle

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs
GX_STATUS_INVALID_PARAMETER	Invalid parameter
GX_STATUS_NOT_INIT_API	GXInitLib() initialization library not called

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

GX_IF_HANDLE hIF;
emStatus = GXGetInterfaceHandle(1, &hIF);
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

7.4.2.14. GXGetParentInterfaceFromDev

Declarations:

GX_API GXGetParentInterfaceFromDev(GX_DEV_HANDLE hDevice, GX_IF_HANDLE* phIF);

Descriptions:

Specify the device handle to obtain the interface handle to which it belongs.

Parameters:

[in] hDevice	Device handle
[in,out] phIF	Interface Handle

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs
GX_STATUS_INVALID_PARAMETER	Invalid parameter
GX_STATUS_INVALID_HANDLE	Invalid handle passed-in by users

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_IF_HANDLE hIF = NULL;
emStatus = GXGetParentInterfaceFromDev (hDevice, &hIF);
```

7.4.2.15. GXGetLocalDeviceHandleFromDev

Declarations:

GX_API GXGetLocalDeviceHandleFromDev(GX_DEV_HANDLE hDevice,
GX_LOCAL_DEV_HANDLE* phLocalDev);

Descriptions:

Get local device handle.

Parameters:

[in] hDevice	Device handle
[in,out] phLocalDev	Pointer to local device handle

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs
GX_STATUS_INVALID_PARAMETER	Invalid parameter
GX_STATUS_INVALID_HANDLE	Invalid handle passed-in by users

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_LOCAL_DEV_HANDLE hLocalDev = NULL;
emStatus = GXGetLocalDeviceHandleFromDev (hDevice, &hLocalDev);
```

7.4.2.16. GXGetDataStreamNumFromDev

Declarations:

GX_API GXGetDataStreamNumFromDev(GX_DEV_HANDLE hDevice, uint32_t* pnDSNum);

Descriptions:

Obtain the number of stream channels supported by the device.

Parameters:

[in] *hDevice*	Device handle

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

[in,out] *pnDSNum* Pointer to the number of flow channels

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs
GX_STATUS_INVALID_PARAMETER	Invalid parameter
GX_STATUS_INVALID_HANDLE	Invalid handle passed-in by users

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32DSNum = 0;
emStatus = GXGetDataStreamNumFromDev (hDevice, &ui32DSNum);
```

7.4.2.17. GXGetDataStreamHandleFromDev

Declarations:

```
GX_API GXGetDataStreamHandleFromDev(GX_DEV_HANDLE hDevice,
                                    uint32_t nDSIndex,
                                    GX_DS_HANDLE* phDS);
```

Descriptions:

Retrieve the stream channel ID handle for the specified device handle.

Parameters:

[in] hDevice	Device handle
[in] nDSIndex	Data stream order, starting from 1
[in,out] phDS	Data stream handle

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs
GX_STATUS_INVALID_PARAMETER	Invalid parameter
GX_STATUS_INVALID_HANDLE	Invalid handle passed-in by users

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_DS_HANDLE phDS = 0;
```

7.4.3. emStatus = GXGetDataStreamHandleFromDev (hDevice, 1, &hDS);Device Parameter Settings

7.4.3.1. Universal Read and Write Interface

7.4.3.1.1. GXGetIntValue

Declarations:

```
GX_API GXGetIntValue (GX_PORT_HANDLE hPort,
                      const char* strName,
                      GX_INT_VALUE* pstIntValue);
```

Descriptions:

Get the node value of int type.

Parameters:

[in] hPort Handle (Interface handle, remote device handle, local device handle,

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

	DataStream handle)
[in] strName	XML node name.
[out] pstIntValue	Pointer to the integer value information structure Refer to the definition of GX_INT_VALUE structure.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.
GX_STATUS_INVALID_ACCESS	Currently inaccessible, can not read.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_INT_VALUE stIntValue;
emStatus = GXGetIntValue(hDevice, "Width", &stIntValue);
int64_t i64Width = stIntValue.nCurValue;
```

7.4.3.1.2. GXSetIntValue

Declarations:

GX_API GXSetIntValue(GX_PORT_HANDLE hPort, const char* strName, int64_t i64Value);

Descriptions:

Set the node value of int type.

Parameters:

[in] hPort	Handle (Interface handle, remote device handle, local device handle, DataStream handle)
[in] strName	XML node name.
[in] i64Value	The value that the user will set.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_OUT_OF_RANGE	The value that the user introduces is across the border, smaller than the minimum, or larger than the maximum, or is not an integer multiple of the step.
GX_STATUS_INVALID_ACCESS	Currently inaccessible, can not write.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
int64_t i64Value = 200;
emStatus = GXSetIntValue(hDevice, “Width”, i64Value);
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

7.4.3.1.3. GXGetFloatValue

Declarations:

```
GX_API GXGetFloatValue(GX_PORT_HANDLE hPort,
                       const char* strName,
                       GX_FLOAT_VALUE* pstFloatValue);
```

Descriptions:

Get the node value of float type.

Parameters:

[in] hPort	Handle (Interface handle, remote device handle, local device handle, DataStream handle)
[in] strName	XML node name.
[out] pstFloatValue	Pointer to floating-point value information structure Refer to the definition of GX_FLOAT_Value structure.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.
GX_STATUS_INVALID_ACCESS	Currently inaccessible, can not read.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_FLOAT_VALUE stFloatValue;

emStatus = GXGetFloatValue(hDevice, "ExposureTime", &stFloatValue);

double dExposureTime = stFloatValue.dCurValue;
```

7.4.3.1.4. GXSetFloatValue

Declarations:

```
GX_API GXSetFloatValue (GX_PORT_HANDLE hPort,
                        const char* strName, double dValue);
```

Descriptions:

Set the node value of float type.

Parameters:

[in] hPort	Handle (Interface handle, remote device handle, local device handle, DataStream handle)
[in] strName	XML node name.
[in] dValue	User set value.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_OUT_OF_RANGE	The value that the user introduces is across the border, smaller than the minimum, or larger than the maximum.
GX_STATUS_INVALID_ACCESS	Currently inaccessible, can not write.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
double dValue = 3000;

emStatus = GXSetFloatValue(hDevice, "ExposureTime", dValue);
```

7.4.3.1.5. GXGetEnumValue

Declarations:

```
GX_API GXGetEnumValue(GX_PORT_HANDLE hPort,
                      const char* strName,
                      GX_ENUM_VALUE* pstEnumValue);
```

Descriptions:

To get the current node value of enumeration type.

Parameters:

[in] *hPort*	Handle (Interface handle, remote device handle, local device handle, DataStream handle)
[in] *strName*	XML node name.
[out] *pstEnumValue*	Pointer to the enumeration type value information structure Refer to the definition of GX_ENUM_VALUE structure.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.
GX_STATUS_INVALID_ACCESS	Currently inaccessible, can not read.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_ENUM_VALUE stEnumValue;
emStatus = GXGetEnumValue(hDevice, "GainAuto", &stEnumValue);
int64_t i64GainAuto = stEnumValue.stCurValue.nCurValue;
GXSetEnumValue
```

7.4.3.1.6. GXSetEnumValue

Declarations:

```
GX_API GXSetEnumValue(GX_PORT_HANDLE hPort,
                      const char* strName,
```

int64_t i64Value);

Descriptions:

Set the node value of enumeration type.

Parameters:

[in] hPort	Handle (Interface handle, remote device handle, local device handle, DataStream handle)
[in] strName	XML node name.
[in] i64Value	User set value.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_OUT_OF_RANGE	The value that the user introduces is cross the border.
GX_STATUS_INVALID_ACCESS	Currently inaccessible, can not write.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_ENUM_VALUE stEnumValue;
emStatus = GXGetEnumValue(hDevice, "GainAuto", &stEnumValue);
int64_t i64GainAuto = stEnumValue.stCurValue.nCurValue;
```

7.4.3.1.7. GXSetEnumValueByString

Declarations:

GX_API GXSetEnumValueByString(GX_PORT_HANDLE hPort,
const char* strName,
const char* strValue);

Descriptions:

Set value of the enumeration type.

Parameters:

[in] hPort	Handle (Interface handle, remote device handle, local device handle, DataStream handle)
[in] strName	XML node name.
[in] strValue	Pointer to the value that the user will set

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_OUT_OF_RANGE	The value that the user introduces is cross the border.
GX_STATUS_INVALID_ACCESS	Currently inaccessible, can not write.

The errors that are not covered above please reference GX_STATUS_LIST.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
emStatus = GXSetEnumValueByString (hDevice, "GainAuto",
                                   "Continuous");
```

7.4.3.1.8. GXGetBoolValue

Declarations:

```
GX_API GXGetBoolValue(GX_PORT_HANDLE hPort,
                      const char* strName,
                      bool* pbValue);
```

Descriptions:

Get the node value of bool type.

Parameters:

[in] hPort	Handle (Interface handle, remote device handle, local device handle, DataStream handle)
[in] strName	XML node name.
[out] pbValue	Pointer to the returned Boolean value.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.
GX_STATUS_INVALID_ACCESS	Currently inaccessible, can not read.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
bool bTemp = false;
emStatus = GXGetBoolValue(hDevice, "ReverseX", &bTemp);
```

7.4.3.1.9. GXSetBoolValue

Declarations:

```
GX_API GXSetBoolValue(GX_PORT_HANDLE hPort,
                      const char* strName,
                      bool bValue);
```

Descriptions:

Set the node value of bool type.

Parameters:

[in] hPort	Handle (Interface handle, remote device handle, local device handle, DataStream handle)
[in] strName	XML node name.
[in] bValue	The bool value set by the users.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_INVALID_ACCESS	Currently inaccessible, can not write.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
bool bTemp = true;
emStatus = GXSetBoolValue(hDevice, "ReverseX", bTemp);
```

7.4.3.1.10. GXGetStringValue

Declarations:

```
GX_API GXGetStringValue(GX_PORT_HANDLE hPort,
                        const char* strName,
                        GX_STRING_VALUE* pstStringValue);
```

Descriptions:

Get the node info of string type.

Parameters:

[in] hPort	Handle (Interface handle, remote device handle, local device handle, DataStream handle)
[in] strName	XML node name.
[out] pstStringValue	Pointer to a string type value information structure Refer to the definition of GX_STRING_VALUE structure.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_STRING_VALUE stVendorNameValue;
const char* pszVendorName = NULL;
emStatus = GXGetStringValue(hDevice,"DeviceVendorName",
                            &stVendorNameValue);
pszVendorName = stVendorNameValue.strCurValue;
```

7.4.3.1.11. GXSetStringValue

Declarations:

```
GX_API GXSetStringValue(GX_PORT_HANDLE hPort, const char* strName, const char* strValue);
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Descriptions:

Set the content of the string type node value.

Parameters:

[in] hPort	Handle (Interface handle, remote device handle, local device handle, DataStream handle)
[in] strName	XML node name.
[in] strValue	Pointer to the value that the user will set.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_INVALID_PARAMETER	The pointer that the user introduces is NULL.
GX_STATUS_OUT_OF_RANGE	The content written by the users exceeds the maximum length of the string.
GX_STATUS_INVALID_ACCESS	Currently inaccessible, unable to write.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus  = GX_STATUS_SUCCESS;
char *pszText = "test";
emStatus = GXSetStringValue(hDevice, "DeviceModelName", pszText);
```

7.4.3.1.12. GXSetCommandValue

Declarations:

GX_API GXSetCommandValue(GX_PORT_HANDLE hPort, const char* strName);

Descriptions:

Send the command.

Parameters:

[in] hPort	Handle (Interface handle, remote device handle, local device handle, DataStream handle)
[in] strName	XML node name.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_INVALID_ACCESS	Currently inaccessible, can not send command.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
emStatus = GXSetCommandValue(hDevice, "TriggerSoftware");
```

7.4.3.1.13. GXGetNodeAccessMode

Declarations:

GX_API GXGetNodeAccessMode(GX_PORT_HANDLE hPort, const char* strName,
GX_NODE_ACCESS_MODE* pemAccessMode);

Descriptions:

Get the read and write attributes of the node.

Parameters:

[in] hPort	Handle (Interface handle, remote device handle, local device handle, DataStream handle)
[in] strName	XML node name.
[in,out] pemAccessMode	Read and write attributes of the node. Refer to the structure definition of GX_NODE_ACCESS_MODE

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_INVALID_PARAMETER	The input parameter that the user introduces is invalid.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_NODE_ACCESS_MODE emAccessMode;
emStatus = GXGetNodeAccessMode(hDevice, "Width", &emAccessMode);
```

7.4.3.1.14. GXGetRegisterLength

Declarations:

GX_API GXGetRegisterLength(GX_PORT_HANDLE hPort,
const char* strName,
size_t* pnSize);

Descriptions:

Get the length of register type node data.

Parameters:

[in] hPort	Handle (Interface handle, remote device handle, local device handle, DataStream handle)
[in] strName	XML node name.
[in,out] pnSize	Save the pointer of data length.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_INVALID_PARAMETER	The input parameter that the user introduces is invalid.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
size_t nDataFieldValue;
emStatus =GXGetRegisterLength(hDevice, "DataFieldValue",
                              &nDataFieldValue);
```

7.4.3.1.15. GXGetRegisterValue

Declarations:

GX_API GXGetRegisterValue(GX_PORT_HANDLE hPort,
const char* strName,
uint8_t* pBuffer, size_t* pnSize);

Descriptions:

Get register type node values.

Parameters:

[in] hPort	Handle (Interface handle, remote device handle, local device handle, DataStream handle)
[in] strName	XML node name.
[out] pBuffer	Pointer to the memory address of the data requested by the user.
[in,out] pnSize	The length of the buffer address entered by the user.
If pBuffer is NULL:	
[out] pnSize	The actual required buffer size.
If pBuffer isnot NULL:	
[in] pnSize	User allocated buffer size.
[out] pnSize	Returns the actual fill buffer size.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_INVALID_PARAMETER	The input parameter that the user introduces is invalid.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
size_t nLength = 0;
emStatus = GXGetRegisterLength(hDevice, "DataFieldValue", &nLength);
uint8_t *pui8Buffer = new uint8_t[nLength];
emStatus = GXGetRegisterValue(hDevice, "DataFieldValue",pui8Buffer,
                              &nLength);

//After operation
if(NULL != pui8Buffer)
{
   delete[] pui8Buffer;
   pui8Buffer = NULL;
}
```

7.4.3.1.16. GXSetRegisterValue

Declarations:

GX_API GXSetRegisterValue(GX_PORT_HANDLE hPort,
const char* strName,
uint8_t* pBuffer,

size_t nSize);

Descriptions:

Set register type node values.

Parameters:

[in] hPort	Handle (Interface handle, remote device handle, local device handle, DataStream handle)
[in] strName	XML node name.
[in] pBuffer	Pointer to the memory address of the data requested by the user.
[in] pnSize	The length of the buffer address entered by the user.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_INVALID_PARAMETER	The input parameter that the user introduces is invalid.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
size_t nLength = 0;
emStatus = GXGetRegisterLength(hDevice, "DataFieldValue", &nLength);

uint8_t* pui8Buffer = new uint8_t[nLength];
emStatus = GXSetRegisterValue(hDevice, "DataFieldValue",
                              pui8Buffer, nLength);
//After operation
If (NULL != pui8Buffer)
{
   delete[] pui8Buffer;
   pui8Buffer = NULL;
}
```

7.4.3.2. Register Read and Write Interface

7.4.3.2.1. GXReadPort

Declarations:

GX_API GXReadPort(GX_PORT_HANDLE hPort, uint64_t ui64Address,
void* pBuffer, size_t* piSize);

Descriptions:

Read memory through register address.

Precautions:

Gigabit/10G devices read big endian, while other devices read little endian.

Parameters:

[in] hPort	Handle (Interface handle, remote device handle, local device handle, DataStream handle)
[in] ui64Address	Register address of camera attributes.
[in,out] pBuffer	Pointer address to the memory value read.
[in] piSize	The length of memory to be read

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_INVALID_PARAMETER	The input parameter that the user introduces is invalid.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
//Address corresponding to the camera attribute values
//The following addresse is only for reference, please consult technical
//support for specific addresses
uint64_t ui64Address = 0xFFFF;

//Apply memories for storing the camera's attribute value
size_t nSize = 4;
void *pBuffer = new int;

//Get the camera attribute value in batch
emStatus = GXReadPort(hDevice, ui64Address, pBuffer,& nSize);

//After finished
if (NULL != pBuffer)
{
   delete pBuffer;
   pBuffer = NULL;
}
```

7.4.3.2.2. GXWritePort

Declarations:

```
GX_API GXWritePort(GX_PORT_HANDLE hPort,
                   uint64_t ui64Address,
                   void* pBuffer, size_t* piSize);
```

Descriptions:

Write memory through register address.

Precautions:

1) The attribute values set for GigE cameras need to be converted to big endian first.
2) After calling the current interface, the node values obtained by using GXGetEnumValue(), GXGetIntValue(), and GXGetBoolValue() are still the values before modification. The latest property values can be obtained by using GXReadPort() or GXReadPortStacked().

Parameters:

[in] hPort	Handle (Interface handle, remote device handle, local device handle, DataStream handle)
[in] ui64Address	Register address of camera attributes.
[in] pBuffer	Pointer address to the written memory value.
[in] piSize	The length of memory to be read

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_INVALID_PARAMETER	The input parameter that the user introduces is invalid.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
//Address corresponding to the camera attribute values
//The following addresse is only for reference, please consult technical
//support for specific addresses
uint64_t ui64Address = 0xFFFF;
//Apply memories for storing the camera's attribute value
int32_t i32Value = 1;
size_t nSize = sizeof(i32Value);

//Set camera attribute values
emStatus = GXWritePort(hDevice, ui64Address, &i32Value , &nSize);

//After operation
if (NULL != pBuffer)
{
   delete[] pBuffer;
   pBuffer = NULL;
}
```

7.4.3.2.3. GXReadPortStacked

Declarations:

```
GX_API GXReadPortStacked(GX_PORT_HANDLE hPort,
                         GX_REGISTER_STACK_ENTRY* pstEntries,
                         size_t *piSize);
```

Descriptions:

Batch read multiple memory addresses specified by the user.

Parameters:

[in] hPort	Handle (Interface handle, remote device handle, local device handle, DataStream handle)
[in,out] pstEntries	Point to the address structure memory that needs to obtain camera attributes, and return the value of the attribute. Refer to the structure definition of GX_REGISTER_STACK_ENTRY
[in,out] piSize	Return the number of successfully obtained camera attribute values.

Precautions:

1) Batch read memory only supports 4 bytes per unit.
2) Gigabit/10G devices read large segments, while other devices read small segments.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_INVALID_PARAMETER	The input parameter that the user introduces is invalid.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
//The following example is to obtain 3 attribute values
```

```
//which can be modified according to your needs
size_t nStackEntry = 3;
GX_REGISTER_STACK_ENTRY pstStackEntry[3];
char chBufferWidth[256] = {0};
char chBufferHeight[256] = {0};
char chBufferOffsetX[256] = {0};

//Property address (currently only supports integer types)
//The following addresse is only for reference, please consult technical
//support for specific addresses
pstackEntry[0].nAddress = 0xFFFF;

//Attribute value length (currently only supports integer types)
pstackEntry[0].nSize = 4;

//Apply memories for storing the camera's attribute value
pstStackEntry[0].pBuffer = chBufferWidth;

pstStackEntry[1].nAddress = 0xFFFF;
pstStackEntry[1].nSize = 4;
pstStackEntry[1].pBuffer = chBufferHeight;

pstStackEntry[2].nAddress = 0xFFFF;
pstStackEntry[2].nSize = 4;
pstStackEntry[2].pBuffer = chBufferOffsetX;

//Batch obtaining multiple camera attribute values
emStatus = GXReadPortStacked(hDevice,pstackEntry,&nStackEntry);
```

7.4.3.2.4. GXWritePortStacked

Declarations:

GX_API GXWritePortStacked(GX_PORT_HANDLE hPort,
const GX_REGISTER_STACK_ENTRY* pstEntries, size_t *piSize);

Descriptions:

Batch write multiple memory addresses specified by the user.

Parameters:

[in] hPort	Handle (Interface handle, remote device handle, local device handle, DataStream handle)
[in] pstEntries	Point to the address structure memory that needs to obtain camera attributes, and return the value of the attribute. Refer to the structure definition of GX_REGISTER_STACK_ENTRY
[in,out] piSize	Return the number of successfully obtained camera attribute values.

Precautions:

1) The attribute values set for GigE cameras need to be converted to big endian first.
2) After calling the current interface, the node values obtained by using GXGetEnumValue(), GXGetIntValue(), and GXGetBoolValue() are still the values before modification. The latest property values can be obtained by using GXReadPort() or GXReadPortStacked().
3) Batch read memory only supports 4 bytes per unit.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

GX_STATUS_INVALID_PARAMETER	The input parameter that the user introduces is invalid.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//The following example is to obtain 3 attribute values
//which can be modified according to your needs
size_t nStackEntry = 3;
GX_REGISTER_STACK_ENTRY pstStackEntry[3];
char chBufferWidth[256] = {0};
char chBufferHeight[256] = {0};
char chBufferOffsetX[256] = {0};

//Property address (currently only supports integer types)
//The following addresse is only for reference, please consult technical
//support for specific addresses
pstStackEntry[0].nAddress = 0xFFFF;

//Attribute value length (currently only supports integer types)
pstStackEntry[0].nSize = 4;

//Apply memories for storing the camera's attribute value
pstStackEntry[0].pBuffer = new int32_t;

//The following values can be set according to your need
pstStackEntry[1].nAddress = 0xFFFF;
pstStackEntry[1].nSize = 4;
pstStackEntry[1].pBuffer =  chBufferHeight;

pstStackEntry[2].nAddress = 0xFFFF;
pstStackEntry[2].nSize = 4;
pstStackEntry[2].pBuffer =  chBufferOffsetX;

//Batch setting multiple camera attribute values
emStatus = GXWritePortStacked(hDevice, pstStackEntry, &nStackEntry);
```

7.4.4. Device Functional Interface

7.4.4.1. GXGetDevicePersistentIpAddress

Declarations:

```
GX_API GXGetDevicePersistentIpAddress(GX_DEV_HANDLE hDevice,
                                      char*         pszIP,
                                      size_t*       pnIPLength,
                                      char*         pszSubNetMask,
                                      size_t*       pnSubNetMaskLength,
                                      char*         pszDefaultGateWay,
                                      size_t*       pnDefaultGateWayLength);
```

Descriptions:

Obtain the permanent IP information of the device.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Parameters:

[in] hDevice	The handle of the device.
[in] pszIP	Device permanent IP string address
[in, out] pnIPLength	The length of the device's permanent IP address string (in bytes).
[in] pnIPLength:	User buffer size
[out] pnIPLength:	Actual fill size
[in] pszSubNetMask	Device persistent subnet mask string address
[in, out] pnSubNetMaskLength	Device persistent subnet mask string length
[in] pnSubNetMaskLength:	User buffer size
[out] pnSubNetMaskLength:	Actual fill size
[in] pszDefaultGateWay	Device permanent gateway string address
[in, out] pnDefaultGateWayLength	Device permanent gateway string length
[in] pnDefaultGateWayLength:	User buffer size
[out] pnDefaultGateWayLength:	Actual fill size

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_PARAMETER	The input parameter that the user introduces is invalid.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
char chArrIP[32] = {0};
size_t nIPLen = 32;
char chArrSubNetMask[32] = {0};
size_t nSubNetMaskLen = 32;
 char chArrDefaultGateWay[32] = {0};
size_t nDefaultGateWayLen = 32;
emStatus = GXGetDevicePersistentIpAddress(hDevice,
                                          chArrIP,
                                          &nIPLen,
                                          chArrSubNetMask,
                                          &nSubNetMaskLen,
                                          chArrDefaultGateWay,
                                          &nDefaultGateWayLen);
```

7.4.4.2. GXSetDevicePersistentIpAddress

Declarations:

```
GX_API GXSetDevicePersistentIpAddress(GX_DEV_HANDLE hDevice,
                                      const char* pszIP,
                                      const char* pszSubNetMask,
                                      const char* pszDefaultGateWay);
```

Descriptions:

Set the permanent IP information of the device.

Parameters:

[in] hDevice	The handle of the device.
[in] pszIP	Device permanent IP string address, end with '\0'.
[in] pszSubNetMask	Device persistent subnet mask string address, end with '\0'.
[in] pszDefaultGateWay	Device permanent gateway string address, end with '\0'.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
//The IP address information in the code is only an example
//and the actual value can be set by the users
emStatus = GXSetDevicePersistentIpAddress(hDevice,
                                          "192.168.1.2",
                                          "255.255.255.0",
                                          "192.168.1.1");
```

7.4.4.3. GXGigEIpConfiguration

Declarations:

```
GX_API GXGigEIpConfiguration(const char* pszDeviceMacAddress,
                             GX_IP_CONFIGURE_MODE emIpConfigMode,
                             const char* pszIpAddress,
                             const char* pszSubnetMask,
                             const char* pszDefaultGateway,
                             const char* pszUserID);
```

Descriptions:

Configure the camera's static IP address.

Parameters:

[in] pszDeviceMacAddress	MAC address
[in] emIpConfigMode	IP configuration mode
[in] pszIpAddress	IP address to be set
[in] pszSubnetMask	Subnet mask to be set
[in] pszDefaultGateway	Default gateway to be set
[in] pszUserID	User defined name to be set

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_PARAMETER	Invalid parameter.
GX_STATUS_NOT_FOUND_DEVICE	Found no device.
GX_STATUS_ERROR	Error operation.
GX_STATUS_INVALID_ACCESS	Access denied.
GX_STATUS_TIMEOUT	Operation timeout.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
char chMAC[] = "00-21-49-00-00-00";
char chIpAddress[] = "192.168.10.10";
char chSubnetMask[] = "255.255.255.0";
char chDefaultGateway[] = "192.168.10.2";
char chUserID[] = "Daheng Imaging";
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
GX_IP_CONFIGURE_MODE emIpConfigureMode = GX_IP_CONFIGURE_STATIC_IP;
emStatus = GXGigEIpConfiguration(chMAC,
                                 emIpConfigureMode,
                                 chIpAddress,
                                 chSubnetMask,
                                 chDefaultGateway,
                                 chUserID);
```

7.4.4.4. GXGigEForceIp

Declarations:

GX_API GXGigEForceIp(const char* pszDeviceMacAddress,
const char* pszIpAddress,
const char* pszSubnetMask,
const char* pszDefaultGateway);

Descriptions:

Execute Force IP operation.

Parameters:

[in] pszDeviceMacAddress	MAC address
[in] pszIpAddress	IP address to be set
[in] pszSubnetMask	Subnet mask to be set
[in] pszDefaultGateway	Default gateway to be set

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_PARAMETER	Invalid parameter.
GX_STATUS_NOT_FOUND_DEVICE	Found no device.
GX_STATUS_ERROR	Error operation.
GX_STATUS_INVALID_ACCESS	Access denied.
GX_STATUS_TIMEOUT	Operation timeout.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
char chMAC[] = "00-21-49-00-00-00";
char chIpAddress[] = "192.168.10.10";
char chSubnetMask[] = "255.255.255.0";
char chDefaultGateway[] = "192.168.10.2";
emStatus = GXGigEForceIp(chMAC, chIpAddress, chSubnetMask,
                         chDefaultGateway);
```

7.4.4.5. GXGigEResetDevice

Declarations:

GX_API GXGigEResetDevice(const char* pszDeviceMacAddress,
GX_RESET_DEVICE_MODE ui32FeatureInfo);

Descriptions:

Execute device reset operations. Please refer to section 2.2.10.5 for details.
Device reset is usually applied to GigE camera's debugging process. When the program of the opened device is abnormal, and there is an error occurs after reopen the device immediately

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

(because the debugging heartbeat is 5 minutes and the device is still turned on). At this time, execute reset device operation to make the device in unopnened status, then reopen the device to connect successfully.
Device reset is usually applied when the camera is in an abnormal state, in this situation, device reset function cannot take effect, and unable to power up the device again, this moment, you can try using the device reset function to power off and then power on the device. After resetting the device, it is necessary to perform the enumeration and device opening operations again.

Precautions:

1) Please call the enumeration interface after at least 1s, because the device reset time takes about 1s.
2) If the device is in acquisition mode, it is prohibited to use the device reset functions, otherwise it may cause the device wire dropping.

Parameters:

[in] pszDeviceMacAddress	MAC address
[in] ui32FeatureInfo	Reset the device mode.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_PARAMETER	Invalid parameter.
GX_STATUS_NOT_FOUND_DEVICE	Found no device.
GX_STATUS_ERROR	Error operation.
GX_STATUS_INVALID_ACCESS	Access denied.
GX_STATUS_TIMEOUT	Operation timeout.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code: Device Reconnection

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
char chMAC[] = "00-21-49-00-00-00";

//Reset device
emStatus = GXGigEResetDevice(chMAC, GX_MANUFACTURER_SPECIFIC_RESET);
//Wait 1s to enumerate the device again
uint32_tnDeviceNum = 0;
emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
```

7.4.4.6. GXFeatureSave

Declarations:

GX_API GXFeatureSave(GX_PORT_HANDLE hPort, const char* strFileName);

Descriptions:

Save the current frame grabber or camera configuration parameters to a text file.

Parameters:

[in] hPort	Handle (Interface handle, remote device handle, local device handle, DataStream handle)
[in] strFileName	The path of the configuration files to be generated.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_INVALID_PARAMETER	The unsupported event ID or the callback function is illegal.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
const char* pszFilePath = "feature_save.txt";
emStatus = GXFeatureSave(hDevice, pszFilePath);
```

7.4.4.7. GXFeatureSaveW

Declarations:

GX_API GXFeatureSaveW(GX_PORT_HANDLE hPort, const wchar_t* strFileName);

Descriptions:

Save the current frame grabber or camera configuration parameters to a text file.

Parameters:

[in] hPort	Handle (Interface handle, remote device handle, local device handle, DataStream handle)
[in] strFileName	The path of the configuration files to be generated (wide byte).

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_INVALID_PARAMETER	The unsupported event ID or the callback function is illegal.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code: Get the event of remote device

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
const wchar_t* pszFilePath = L"feature_save.txt";
emStatus = GXFeatureSaveW(hDevice, pszFilePath);
```

7.4.4.8. GXFeatureLoad

Declarations:

GX_API GXFeatureLoad(GX_PORT_HANDLE hPort, const char* strFileName, bool bVerify);

Descriptions:

Loading parameters from the configuration file to the frame grabber or camera.

Parameters:

[in] hPort	Handle (Interface handle, remote device handle, local device handle, DataStream handle)
[in] strFileName	The path of the configuration files.
[in] bVerify	If the value is true, All imported values will be read out for consistency verification

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_INVALID_PARAMETER	The unsupported event ID or the callback function is illegal.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

const char* pszFilePath = "feature_save.txt";
emStatus = GXFeatureLoad(hDevice,  pszFilePath, true);
```

7.4.4.9. GXFeatureLoadW

Declarations:

GX_API GXFeatureLoadW(GX_PORT_HANDLE hPort, const char* strFileName, bool bVerify);

Descriptions:

Loading parameters from the configuration file to the frame grabber or camera.

Parameters:

[in] hPort	Handle (Interface handle, remote device handle, local device handle, DataStream handle)
[in] strFileName	The path of the configuration files (wide byte).
[in] bVerify	If the value is true, All imported values will be read out for consistency verification

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_INVALID_PARAMETER	The unsupported event ID or the callback function is illegal.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

const wchar_t* pszFilePath = L"feature_save.txt";
emStatus = GXFeatureLoadW(hDevice, pszFilePath, true);
```

7.4.4.10. GXExportConfigFile

Declarations:

GX_API GXExportConfigFile (GX_DEV_HANDLE hDevice, const char * pszFilePath);

Descriptions:

1) Export the current camera configuration to a text file.

2) To be used in conjunction with the import device configuration function of GalaxyView.

Parameters:

[in] hDevice	Device handle
[in] pszFilePath	The path to the configuration file to be generated

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Returns:

GX_STATUS_SUCCESS	Operation successful with no errors
GX_STATUS_NOT_INIT_API	GXInitLib() initialization library not called
GX_STATUS_INVALID_HANDLE	User passed in an illegal handle

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

emStatus = GXExportConfigFile(hDevice, "feature_save.txt");
```

7.4.4.11. GXImportConfigFile

Declarations:

GX_API GXImportConfigFile(GX_DEV_HANDLE hDevice,
const char * pszFilePath, bool bVerify = false);

Descriptions:

1) Import parameters from the configuration file into the camera.

2) To be used in conjunction with the import device configuration function of the GalaxyView.

Parameters:

[in] hDevice	Device handle
[in] pszFilePath	The path to the configuration file to be generated
[in] bVerify	If this value is true, all imported values will be read out for verification of consistency

Returns:

GX_STATUS_SUCCESS	Operation successful with no errors
GX_STATUS_NOT_INIT_API	GXInitLib() initialization library not called
GX_STATUS_INVALID_HANDLE	User passed in an illegal handle

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
emStatus = GXImportConfigFile(hDevice, "feature_save.txt");
```

7.4.4.12. GXExportConfigFileW

Declarations:

GX_API GXExportConfigFileW (GX_DEV_HANDLE hDevice, const wchar_t * pszFilePath) ;

Descriptions:

1) Export the current camera configuration to a text file (UNICODE interface).

2) To be used in conjunction with the import device configuration feature of GalaxyView.

Parameters:

[in] hDevice	Device handle
[in] pszFilePath	The path to the configuration file to be generated

Returns:

GX_STATUS_SUCCESS	Operation successful with no errors

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

GX_STATUS_NOT_INIT_API	GXInitLib() initialization library not called
GX_STATUS_INVALID_HANDLE	User passed in an illegal handle

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
emStatus = GXExportConfigFileW(hDevice, L"feature_save.txt");
```

7.4.4.13. GXImportConfigFileW

Declarations:

```
GX_API GXImportConfigFileW(GX_DEV_HANDLE hDevice,
                           const wchar_t * pszFilePath, bool bVerify = false);
```

Descriptions:

1) Import parameters from the configuration file into the camera (UNICODE interface).
2) To be used in conjunction with the import device configuration function of the demo.

Parameters:

[in] hDevice	Device handle
[in] pszFilePath	The path to the configuration file to be generated
[in] bVerify	If this value is true, all imported values will be read out for verification of consistency

Returns:

GX_STATUS_SUCCESS	Operation successful with no errors
GX_STATUS_NOT_INIT_API	GXInitLib() initialization library not called
GX_STATUS_INVALID_HANDLE	User passed in an illegal handle

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
emStatus = GXImportConfigFileW(hDevice, L"feature_save.txt");
```

7.4.4.14. GXGigEIssueActionCommand

Declarations:

```
GX_API GXGigEIssueActionCommand(uint32_t nDeviceKey, uint32_t nGroupKey,
                                uint32_t nGroupMask,
                                const char* strBroadcastAddress,
                                const char* strSpecialAddress,
                                uint32_t nTimeoutMs, uint32_t* pnNumResults,
                                GX_GIGE_ACTION_COMMAND_RESULT* pstResults );
```

Descriptions:

All cameras on the network execute “action” simultaneously by sending ActionCommond.

Parameters:

[in] *nDeviceKey*	Device secret key, corresponding to the “ActionDeviceKey” attribute of the device
[in] *nGroupKey*	Group secret key, corresponding to the “ActionGroupKey” attribute of the device

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

[in] *nGroupMask*	Group mask, corresponding to the “ActionGroupMask” attribute of the device, can not be 0
[in] *strBroadcastAddress*	The destination IP for sending the “action” command can be broadcast, subnet broadcast, or unicast.
[in] *strSpecialAddress*	The source IP address for sending the “action” command is used to clearly indicate which network adapter is sending the command from. If not specified, each IP of all network adapters will send the current “action” command. Please fill in an empty string without specifying the network port.
[in] *nTimeoutMs*	0 means there is no need to wait for ack to return; Non zero indicates the maximum waiting time for ack return, and after the time is up, the parameter pNumResults returns the actual number of ack received.
[in][out] *pNumResults*	Indicate the expected number of ack returns. This value represents the expected number of ACKs that the user expects to receive in feedback from the device executing the “action” command. If nTimeoutMs is 0, ignore this parameter. Therefore, if nTimeoutMs is 0, this parameter can be NULL
[in][out] *pResults*	An array containing *pNumResults elements, used to store the state of operation command results. The buffer is filled from scratch. If the received results are less than the available result items, the remaining results will not be changed. If nTimeoutMs is 0, ignore this parameter. Therefore, if nTimeoutMs is 0, this parameter can be NULL. Please refer to GX_GIGE_ACTION_COMMAND_RESULT.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

7.4.4.15. GXGigEIssueScheduledActionCommand

Declarations:

```
GX_API GXGigEIssueScheduledActionCommand (uint32_t nDeviceKey, uint32_t nGroupKey,
                                          uint32_t nGroupMask,
                                          uint64_t nActiontimeNs,
                                          const char* strBroadcastAddress,
                                          const char* strSpecialAddress,
                                          uint32_t nTimeoutMs,
                                          uint32_t* pnNumResults,
                                          GX_GIGE_ACTION_COMMAND_RESULT* pstResults);
```

Descriptions:

By sending ActionCommond at an absolute point in time, all cameras on the network can simultaneously execute “action”.

Parameters:

[in] *nDeviceKey*	Device secret key, corresponding to the “ActionDeviceKey” attribute of the device
[in] *nGroupKey*	Group secret key, corresponding to the “ActionGroupKey” attribute of the device

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

[in] *nGroupMask*	Group mask, corresponding to the “ActionGroupMask” attribute of the device, can not be 0
[in] *nActiontimeNs*	Execution time (in nanoseconds). The actual value can be the master clock value used plus the expected delay time. The master clock value of a set of synchronized camera devices can be obtained by locking the timestamp value GXSetCommandValue (hDevice, "TimestampMatch") from a set of camera devices and reading the timestamp value GXGetIntValue (hDevice, "TimestampLatchValue", &pstIntValue)
[in] *strBroadcastAddress*	The destination IP for sending the “action” command can be broadcast, subnet broadcast, or unicast.
[in] *strSpecialAddress*	The source IP address for sending the “action” command is used to clearly indicate which network adapter is sending the command from. If not specified, each IP of all network adapters will send the current “action” command. Please fill in an empty string without specifying the network port.
[in] *nTimeoutMs*	0 means there is no need to wait for ack to return; Non zero indicates the maximum waiting time for ack return, and after the time is up, the parameter pNumResults returns the actual number of ack received.
[in][out] *pNumResults*	Indicate the expected number of ack returns. This value represents the expected number of ACKs that the user expects to receive in feedback from the device executing the “action” command. If nTimeoutMs is 0, ignore this parameter. Therefore, if nTimeoutMs is 0, this parameter can be NULL
[in][out] *pResults*	An array containing *pNumResults elements, used to store the state of operation command results. The buffer is filled from scratch. If the received results are less than the available result items, the remaining results will not be changed. If nTimeoutMs is 0, ignore this parameter. Therefore, if nTimeoutMs is 0, this parameter can be NULL. Please refer to GX_GIGE_ACTION_COMMAND_RESULT.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

7.4.4.16. GXCreateWnd

Declarations:

```
GX_API    GXCreateWnd ( GX_DEV_HANDLE hDevice,
                        GX_WINDOWS_ID emID,
                        GX_WIND_HANDLE hParentWnd,
                        GX_OPERATE_HANDLE* hWnd);
```

Descriptions:

Create the handle of window operation corresponding to device.

Parameters:

[in] *hDevice*	The handle of the device..
[in] *emID*	Window type. For details, please refer to GX_WINDOWS_ID.

[in] *hParentWnd*	The handle of parent window. Only when NULL is passed will a window pop up. (This parameter is reserved for future support of embedded display)
[out] *hWnd*	The window operation handle successfully created.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

7.4.4.17. GXDestroyWnd

Declarations:

```
GX_API    GXDestroyWnd ( GX_OPERATE_HANDLE hWnd);
```

Descriptions:

Destroy the related windows.

Parameters:

[in] *hWnd*	The handle of window operation.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

7.4.4.18. GXSetShowPosition

Declarations:

```
GX_API    GXSetShowPosition ( GX_OPERATE_HANDLE hWnd,
                              int32_t nPosX,
                              int32_t nPosY,
                              int32_t nWidth,
                              int32_t nHeight);
```

Descriptions:

Set display position and size of the window.

Parameters:

[in] *hWnd*	The handle of the window..
[in] *nPosX*	The horizontal position of the window relative to the screen.
[in] *nPosY*	The vertical position of the window relative to the screen.
[in] *nWidth*	The width of the window, minimum width 450.
[in] *nPosY*	The height of the window, minimum height 600.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

The errors that are not covered above please reference GX_STATUS_LIST.

7.4.4.19. GXSetShowMode

Declarations:

```
GX_API    GXSetShowMode ( GX_OPERATE_HANDLE hWnd,
                          GX_WINDOWS_SHOW_MODE emMode);
```

Descriptions:

Set the display mode of the window.

Notes

When set to NON_BLOCK_SHOW_MODE, subsequent operations will not be blocked. when set it to BLOCK_SHOW_MODE, subsequent operations will be blocked. If your programme launches in console mode, this interface must be called to set the mode to BLOCK_SHOW_MODE before displaying the window. If your programme is a GUI application (Qt, MFC, Winforms, etc.), do not call this interface to set the BLOCK_SHOW_MODE mode, as undefined behaviour may occur.

Parameters:

[in] *hWnd*	The handle of the window.
[in] *emMode*	Display mode. For details, please refer to GX_WINDOWS_SHOW_MODE.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

7.4.4.20. GXSetWndTitle

Declarations:

```
GX_API    GXSetWndTitle( GX_OPERATE_HANDLE hWnd, const char*    strTitle);
```

Descriptions:

Set a title of the window.

Parameters:

[in] *hWnd*	The handle of the window.
[in] *strTitle*	Title string pointer, must not be NULL.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

7.4.4.21. GXShowWnd

Declarations:

```
GX_API    GXShowWnd ( GX_OPERATE_HANDLE hWnd, bool bVisible);
```

Descriptions:

Set the window to display or hide.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Parameters:

[in] *hWnd*	The handle of the window.
[in] *bVisible*	Ture: display, False: hide.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

7.4.5. Device Acquisition Interface

7.4.5.1. GXSetAcqusitionBufferNumber

Declarations:

```
GX_API GXSetAcqusitionBufferNumber(GX_DEV_HANDLE hDevice,
                                   uint64_t nBufferNum);
```

Descriptions:

Set the number of the acquisition buffers.

Parameters:

[in] hDevice	The handle of the device.
[in] nBufferNum	The number of the acquisition buffers that the user sets.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_INVALID_PARAMETER	The input parameter that the user introduces is invalid.
GX_STATUS_INVALID_CALL	After sending the start acquisition command, the user can not set the number of the acquisition buffers.

The errors that are not covered above please reference GX_STATUS_LIST.

Precautions:

1) This interface is an optional interface, not a necessary part of the acquisition process.
2) When the acquisition command is sent, it will be not allowed to set up the number of buffers, otherwise, it will return GX_STATUS_INVALID_CALL.
3) The number of the acquisition buffers that the user set must greater than 0, and if the number of buffers is 0, it will return GX_STATUS_INVALID_PARAMETER.
4) The user should consider the reasonableness when setting the number of acquisition buffers, if the number is set too many, the memory that is occupied in the acquisition process is large; if the number is set too small, the memory is small when in the acquisition process, but some frames will be lost because of the lack of the buffer.
5) Once the user has set the number of the buffers and has captured successfully, then the number of buffers will remain valid until the device is turned off.

Sample Code:

```
#include"GxIAPI.h"
#include"GxIAPI.h"

//Image callback processing function
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
static void GX_STDC OnFrameCallbackFun(GX_FRAME_DATA_EX* pFrame)
{
    if (pFrame->status == GX_FRAME_STATUS_SUCCESS)
    {
        //Performing certain operations on images
    }
    return;
}

int main(int argc, char* argv[])
{
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    GX_OPEN_PARAM stOpenParam;
    uint32_t ui32DeviceNum = 0;

    //Initialize Library
    emStatus = GXInitLib();
    if (emStatus != GX_STATUS_SUCCESS)
    {
        return 0;
    }

    //Enumerate device list
    emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
    if ((emStatus != GX_STATUS_SUCCESS)||(ui32DeviceNum <= 0))
    {
        return 0;
    }

    //Open device
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emStatus = GXOpenDevice(&stOpenParam, &hDevice);
    if (emStatus == GX_STATUS_SUCCESS)
    {
        // Set the flow channel packet length attribute of the camera
        //to improve the collection performance of network cameras
        bool bImplementPacketSize = false;
        uint32_t ui32PacketSize = 0;

        //Determine whether the device supports streaming channel
        //packet function
        GX_NODE_ACCESS_MODE emAccessMode;
         emStatus = GXGetNodeAccessMode(hDevice,"GevSCPSPacketSize",
                                        &emAccessMode);
        bImplementPacketSize = (emAccessMode == GX_NODE_ACCESS_MODE_RW)
                               ? true : false;

        if (bImplementPacketSize)
        {
            //Obtain the optimal packet length for the current network
```

```
            //environment
            emStatus = GXGetOptimalPacketSize (hDevice, &ui32PacketSize);

            //Set the optimal packet length to the current device's
            //stream channel packet length value
            emStatus = GXSetIntValue(hDevice, "GevSCPSPacketSize",
                                     ui32PacketSize);
        }
         //Set the number of collection buffers
         emStatus = GXSetAcquisitionBufferNumber(hDevice, 10);

         //Register image processing callback function
         emStatus = GXRegisterCaptureCallbackEx(hDevice, NULL,
                                                OnFrameCallbackFun);
         //Start acquisition
         emStatus = GXSetCommandValue(hDevice, "AcquisitionStart");

         //----------------------
         //
         //In this interval, the image will be returned to the user
         //through the OnFrameCallbackFun interface
         //
         //----------------------

         //Stop acquisition
         emStatus = GXSetCommandValue (hDevice, "AcquisitionStop");

        //Cancel collection callback
        emStatus = GXUnregisterCaptureCallback(hDevice);
    }
    emStatus = GXCloseDevice(hDevice);
    emStatus = GXCloseLib();
    return 0;
}
```

7.4.5.2. GXStreamOn (Linux only)

Declarations:

GX_API GXStreamOn(GX_DEV_HANDLE hDevice)

Descriptions:

Start acquisition, including stream acquisition and device acquisition.

Parameters:

[in] hDevice	The handle of the device.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_INVALID_ACCESS	Device access mode error.
GX_STATUS_ERROR	Unspecified internal errors that are not expected to occur

The errors that are not covered above please reference GX_STATUS_LIST.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Precautions:

Start acquisition for all stream channels.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
emStatus = GXStreamOn(hDevice);
```

7.4.5.3. GXStreamOff (Linux only)

Declarations:

GX_API GXStreamOff(GX_DEV_HANDLE hDevice)

Descriptions:

Stop acquisition, including stop stream acquisition and stop device acquisition

Parameters:

[in] hDevice The handle of the device.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_INVALID_ACCESS	Device access mode error.
GX_STATUS_INVALID_CALL	Invalid interface call.
GX_STATUS_ERROR	Unspecified internal errors that are not expected to occur.

The errors that are not covered above please reference GX_STATUS_LIST.

Precautions:

GXStreamOff() will close all stream channels.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
emStatus = GXStreamOff(hDevice);
```

7.4.5.4. GXDQBufEx

Declarations:

GX_API GXDQBufEx (GX_DEV_HANDLE hDevice,
PGX_FRAME_DATA_EX* ppFrameBuffer,
uint32_t nTimeOut)

Descriptions:

Zero copy method is used to capture a single frame. After obtaining the image, the GXQBufEx() interface needs to be called to return the image to the image acquisition system.

Parameters:

[in] hDevice	The handle of the device.
[out] ppFrameBuffer	Address pointer of image data output by the interface.
[in] nTimeOut	Take timeout time (unit: ms).

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_INVALID_PARAMETER	The pointer that the user introduces is NULL.
GX_STATUS_INVALID_CALL	Acquisition is not started or the callback is registered, this interface is not allowed to be called.
GX_STATUS_TIMEOUT	Acquire image timeout error.
GX_STATUS_ERROR	Unspecified internal errors that are not expected to occur.

The errors that are not covered above please reference GX_STATUS_LIST.

Precautions:

1) The GXDQBufEx() interface is not allowed to be called until the acquisition is started. If called, it will return GX_STATUS_INVALID_CALL error;
2) After registering the capture callback, the GXDQBufEx() interface is not allowed to be called. If called, it will return GX_STATUS_INVALID_CALL error;
3) The GXDQBufEx() interface needs to be used with the GXQBufEx() interface, otherwise the image cannot be continuously acquired.

Sample Code:

```
//------------------------------
//The GXDQBufEx interface acquires one frame of image at a time. This
   sample
//code demonstrates how to use this interface to get a frame of image.
//------------------------------
#include "GxIAPI.h"

int main(int argc, char* argv[])
{
GX_STATUS     emStatus = GX_STATUS_SUCCESS;
GX_DEV_HANDLE hDevice = NULL;
uint32_t      ui32DeviceNum = 0;

//Initializes the library.
emStatus = GXInitLib();
if (emStatus != GX_STATUS_SUCCESS)
{
return 0;
}

//Updates the enumeration list for the devices.
emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
if ((emStatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
{
return 0;
}

//Opens the device.
GX_OPEN_PARAM stOpenParam;
stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
stOpenParam.openMode = GX_OPEN_INDEX;
stOpenParam.pszContent = "1";
emStatus = GXOpenDevice(&stOpenParam, &hDevice);
if (emStatus == GX_STATUS_SUCCESS)
{
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    //Defines the incoming parameters of GXDQBufEx.
    PGX_FRAME_DATA_EX pFrameBuffer;

    //Start acquisition.
    #ifdef __linux__
    emStatus = GXStreamOn(hDevice);
    #else
    emStatus = GXSetCommandValue(hDevice, "AcquisitionStart");
    #endif
    if (emStatus == GX_STATUS_SUCCESS)
    {
        //Calls GXDQBufEx to get a frame of image.
        emStatus = GXDQBufEx(hDevice, &pFrameBuffer, 1000);
            if (emStatus == GX_STATUS_SUCCESS)
            {
                if (pFrameBuffer->nStatus == GX_FRAME_STATUS_SUCCESS)
                {
                    //Successfully acquired images.
                    //Image processing...
                }

                    //Calls GXQBuf to put the image buf back into the library
                    //and continue to acquire images.
                    emStatus = GXQBufEx(hDevice, pFrameBuffer);
                }
            }
            //Sends a stop acquisition command.
            #ifdef __linux__
            emStatus = GXStreamOff(hDevice);
            #else
            emStatus = GXSetCommandValue(hDevice, "AcquisitionStop");
            #endif
        }
        emStatus = GXCloseDevice(hDevice);
        emStatus = GXCloseLib();
        return 0;
    }
```

7.4.5.5. GXQBufEx

Declarations:

GX_API GXQBuf(GX_DEV_HANDLE hDevice,
PGX_FRAME_DATA_EX pFrameBuffer)

Descriptions:

Return the image (zero copy) obtained by GXDQBufEx() to the image acquisition system.

Parameters:

[in] hDevice	The handle of the device.
[in] pFrameBuffer	Image data buffer pointer to be placed back into the GxIAPI library.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_INVALID_PARAMETER	The user incoming pointer is NULL. / Incoming unreasonable pointer.
GX_STATUS_INVALID_CALL	Acquisition is not started or the callback is registered. It is not allowed to call the interface.

The errors that are not covered above please reference GX_STATUS_LIST.

Precautions:

1) It is not allowed to call the GXQBufEx() interface before starting the acquisition. If it is called, it will return the GX_STATUS_INVALID_CALL error;
2) After registering the capture callback, the GXQBufEx() interface is not allowed to be called. If it is called, it will return GX_STATUS_INVALID_CALL error;
3) The GXDQBufEx() interface needs to be used with the GXQBufEx() interface. Otherwise, the image cannot be continuously acquired;
4) After the GXQBufEx() interface puts the image buffer back into the GxIAPI library, the image buffer pointer can no longer be accessed.

Sample Code:

See the GXDQBufEx() sample program.

7.4.5.6. GXDQAllBufsEx (Linux only)

Declarations:

```
GX_API GXDQAllBufsEx(GX_DEV_HANDLE hDevice,
                     PGX_FRAME_DATA_EX *ppFrameBufferArray,
                     uint32_t nFrameBufferArraySize,
                     uint32_t *pnFrameCount,
                     uint32_t nTimeOut)
```

Descriptions:

After starting the acquisition, return all image buffers (zero copy) obtained by GXDQBufEx() to the image acquisition system. The order of the stored images in the image data array is from old to new, that is, ppFrameBufferArray[0] stores the oldest image, and ppFrameBufferArray[nFrameCount – 1] stores the latest image.

Parameters:

[in] hDevice	The handle of the device.
[out] ppFrameBufferArray	Array of image data pointers.
[in] nFrameBufferArraySize	The number of applications for image arrays.
[out] pnFrameCount	Returns the number of actual filled images.
[in] nTimeOut	Take timeout time (unit: ms).

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_INVALID_PARAMETER	The pointer that the user introduces is NULL.
GX_STATUS_INVALID_CALL	Acquisition is not started or the callback is registered. It is not allowed to call the interface.
GX_STATUS_NEED_MORE_BUFFER	Insufficient buffer requested by the user: When reading, the user input buffersize is smaller than the actual need.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

GX_STATUS_TIMEOUT	Acquire image timeout error.
GX_STATUS_ERROR	Unspecified internal errors that are not expected to occur.

The errors that are not covered above please reference GX_STATUS_LIST.

Precautions:

1) It is not allowed to call the GXDQAllBufsEx() interface before starting the acquisition. If it is called, it will return the GX_STATUS_INVALID_CALL error;
2) After registering the capture callback, the GXDQAllBufsEx() interface is not allowed to be called. If it is called, it will return GX_STATUS_INVALID_CALL error;
3) The GXDQAllBufsEx() interface needs to be used with the GXQAllBufsEx() interface. Otherwise, the image cannot be continuously acquired;
4) The array size of the image data pointer should be greater than or equal to the number of image acquire buffers (default is 5). Otherwise, GX_STATUS_NEED_MORE_BUFFER error will be returned.

Sample Code:

```
#include "GxIAPI.h"
#include <stdint.h>

int main(int argc, char* argv[])
{
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    uint32_t ui32DeviceNum = 0;

    //Initializes the library.
    emStatus = GXInitLib();
    if (emStatus != GX_STATUS_SUCCESS)
    {
        return 0;
    }

    //Updates the enumeration list for the devices.
    emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
    if ((emStatus != GX_STATUS_SUCCESS)||(emStatus <= 0))
    {
        return 0;
    }

    //Opens the device.
    GX_OPEN_PARAM stOpenParam;
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emStatus = GXOpenDevice(&stOpenParam, &hDevice);
    if (emStatus == GX_STATUS_SUCCESS)
    {
        //Defines the array of received images.
        //The number of image acquire buffers is 5 by default, and the image
        //array size should be greater than or equal to the number of image
        //acquire buffers.
        PGX_FRAME_DATA_Ex pFrameBuffer[5];
```

```
        //Defines the actual number of filled images.
        uint32_t ui32FrameCount = 0;

        //Stream On.
        emStatus = GXStreamOn(hDevice);
        if (emStatus == GX_STATUS_SUCCESS)
        {
            //Calls GXDQAllBufsEx to get all the images in the queue.
           emStatus = GXDQAllBufsEx(hDevice, pFrameBuffer, 5, &nFrameCount,
                                    1000);
            if (emStatus == GX_STATUS_SUCCESS)
            {
                for (int i = 0; i < ui32FrameCount; i++)
                {
                   if (pFrameBuffer[i] != NULL && pFrameBuffer[i]->nStatus
                       == GX_FRAME_EMSTATUS_SUCCESS)
                    {
                       //The image i was successfully acquired.
                       //Image processing...
                    }
                }

           //Calls GXQAllBufsEx to put all the acquired image buffers back
           //into the library and continue to acquire images.
           emStatus = GXQAllBufsEx(hDevice);
           }
       }
        //Sends a stop acquisition command.
        emStatus = GXStreamOff(hDevice);
    }
    emStatus = GXCloseDevice(hDevice);
    emStatus = GXCloseLib();

    return 0;
}
```

7.4.5.7. GXQAllBufs (Linux only)

Declarations:

GX_API GXQAllBufs(GX_DEV_HANDLE hDevice)

Descriptions:

After the acquisition is started, all the acquired image data buffers can be put back into the GxIAPI library through this interface, and continue to be used for acquisition.

Parameters:

[in] hDevice	The handle of the device.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

GX_STATUS_INVALID_CALL	Acquisition is not started or the callback is registered. It is not allowed to call the interface.

The errors that are not covered above please reference GX_STATUS_LIST.

Precautions:

1) It is not allowed to call the GXQAllBufs() interface before starting the acquisition. If it is called, it will return the GX_STATUS_INVALID_CALL error;
2) After registering the capture callback, the GXQAllBufs() interface is not allowed to be called. If it is called, it will return GX_STATUS_INVALID_CALL error;
3) The GXQAllBufs() interface needs to be used with the GXDQAllBufs() interface. Otherwise, the image cannot be continuously acquired;
4) The GXQAllBufs() interface puts all the image buffers obtained by GXDQAllBufs() back into the GxIAPI library, and these image buffer pointers can no longer be accessed.

Sample Code:

See the sample code of GXDQAllBufs().

7.4.5.8. GXRegisterCaptureCallbackEx

Declarations:

```
GX_API GXRegisterCaptureCallbackEx(GX_DEV_HANDLE hDevice, void *pUserParam,
                                   GXCaptureCallBackEx callBackFun);
```

Descriptions:

Register the capture callback function, corresponding to GXUnregisterCaptureCallback().

Parameters:

[in] *hDevice*	The handle of the device.
[in] *pUserParam*	The private data pointer that the user will use in the callback function, which can be obtained by GX_FRAME_DATA_EX.pUserParam
[in] *callBackFun*	The callback function that the user will register, for the function type, see GXCaptureCallBack().

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_INVALID_PARAMETER	The pointer that the user introduces is NULL.
GX_STATUS_INVALID_CALL	After sending the start acquisition command, the user can not register the capture callback function.

The errors that are not covered above please reference GX_STATUS_LIST.

Precautions:

After sending the start acquisition command, the user can not register the callback function. Otherwise, it will return GX_STATUS_INVALID_CALL.

Sample Code:

```
#include "GxIAPI.h"

//Image processing callback function
```

```
Static void GX_STDCOnFrameCallbackFun(GX_FRAME_DATA_EX* pFrame)
{
   if (pFrame->status == GX_FRAME_STATUS_ SUCCESS)
   {
      //Do some image processing operations
   }
   return;
}

int main(int argc, char* argv[])
{
   GX_STATUS emStatus = GX_STATUS_SUCCESS;
   GX_DEV_HANDLE hDevice = NULL;
   GX_OPEN_PARAM stOpenParam;
   uint32_t ui32DeviceNum = 0;

   //Initializes the library
   emStatus = GXInitLib();
   if (emStatus!= GX_STATUS_SUCCESS)
   {
      return 0;
   }

   //Updates the enumeration list for the devices
   emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
   if ((emStatus!= GX_STATUS_SUCCESS)||(ui32DeviceNum<= 0))
   {
      return 0;
   }

   //Open the device
   stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
   stOpenParam.openMode = GX_OPEN_INDEX;
   stOpenParam.pszContent = "1";
   emStatus = GXOpenDevice(&stOpenParam, &hDevice);
   if (emStatus == GX_STATUS_SUCCESS)
   {
      //Setting device's property of GevSCPSPacketSize to improve
      //the acquisition performance of the network camera
      bool bImplementPacketSize = false;
      uint32_t ui32PacketSize = 0;

      //Determine whether the device supports GevSCPSPacketSize
      GX_NODE_ACCESS_MODE emAccessMode;
      EmStatus=GXGetNodeAccessMode(hDevice, "GevSCPSPacketSize",
                                   &emAccessMode);

      bImplementPacketSize=(emAccessMode==GX_NODE_ACCESS_MODE_RW)
                           ? true : false;

      if (bImplementPacketSize)
      {
         //Get Optimal PacketSize
```

```
            emStatus = GXGetOptimalPacketSize (hDevice,
                                               &ui32PacketSize);

            //Set the Optimal PacketSize to GevSCPSPacketSize
            emStatus = GXSetIntValue(hDevice, "GevSCPSPacketSize",
                                     ui32PacketSize);
        }
        //Registers image processing callback function
        emStatus = GXRegisterCaptureCallbackEx(hDevice, NULL,
                                               OnFrameCallbackFun);

        //Send a start acquisition command
        emStatus = GXSetCommandValue(hDevice, "AcquisitionStart");
        //---------------------
        //
        //In this interval, the image will be returned to the user
        //via the OnFrameCallbackFun interface
        //
        //---------------------

        //Send a stop acquisition command
        emStatus = GXSetCommandValue (hDevice, "AcquisitionStop");
        //Unregisters image processing callback function
        emStatus = GXUnregisterCaptureCallback(hDevice);
    }
    emStatus = GXCloseDevice(hDevice);
    emStatus = GXCloseLib();
    return 0;
}
```

7.4.5.9. GXUnregisterCaptureCallback

Declarations:

GX_API GXUnregisterCaptureCallback(GX_DEV_HANDLE hDevice);

Descriptions:

Unregister the capture callback function, corresponding to GXRegisterCaptureCallbackEx().

Parameters:

[in] hDevice	The handle of the device.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_INVALID_CALL	After sending the stop acquisition command, the user can not unregister the capture callback function.

The errors that are not covered above please reference GX_STATUS_LIST.

Precautions:

Before sending the stop acquisition command, the user can not unregister the callback function. Otherwise, it will return GX_STATUS_INVALID_CALL.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Sample Code:

```
#include "GxIAPI.h"

//Image processing callback function.
static void GX_STDC OnFrameCallbackFun(GX_FRAME_DATA_EX * pFrame)
{
    if (pFrame->Status == GX_FRAME_STATUS_SUCCESS)
    {
        //Do some image processing operations.
    }
    return;
}

int main(int argc, char* argv[])
{
    GX_STATUS     emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    GX_OPEN_PARAM stOpenParam;
    uint32_t      ui32DeviceNum = 0;

    //Initializes the library.
    emStatus = GXInitLib();
    if (emStatus != GX_STATUS_SUCCESS)
    {
        return 0;
    }

    //Updates the enumeration list for the devices.
    emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
    if ((emStatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
    {
        return 0;
    }

    //Opens the device.
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode   = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emStatus = GXOpenDevice(&stOpenParam, &hDevice);
    if (emStatus == GX_STATUS_SUCCESS)
    {
        //Setting device's property of GevSCPSPacketSize to improve
        //the acquisition performance of the network camera.
        bool   bImplementPacketSize = false;
        uint32_t ui32PacketSize     = 0;

        //Determine whether the device supports GevSCPSPacketSize.
       GX_NODE_ACCESS_MODE pemAccessMode;
        emStatus = GXGetNodeAccessMode(hDevice,"GevSCPSPacketSize",
                                       &emAccessMode);
        bImplementPacketSize=(emAccessMode==GX_NODE_ACCESS_MODE_RW)
                             ? true : false;
        if (bImplementPacketSize)
```

```
        {
         //Get Optimal PacketSize.
         emStatus = GXGetOptimalPacketSize (hDevice, &ui32PacketSize);

         //Set the Optimal PacketSize to GevSCPSPacketSize.
         emStatus=GXSetIntValue(hDevice, "GevSCPSPacketSize", ui32PacketSize);
        }

        //Registers image processing callback function.
        emStatus = GXRegisterCaptureCallback(hDevice, NULL,
                                             OnFrameCallbackFun);

        //Send a start acquisition command.
        emStatus = GXSetCommandValue(hDevice, "AcquisitionStart");

        //----------------------
        //
        //In this interval, the image will be returned to the user via the
        //OnFrameCallbackFun interface.
        //
        //----------------------

        //Sends a stop acquisition command.
        emStatus = GXSetCommandValue(hDevice, "AcquisitionStop");


        //Unregisters image processing callback function.
        emStatus = GXUnregisterCaptureCallback(hDevice);
    }
    emStatus = GXCloseDevice(hDevice);
    emStatus = GXCloseLib();

    return 0;
}
```

7.4.5.10. GXGetImage

Declarations:

GX_API GXGetImage(GX_DEV_HANDLE hDevice, GX_FRAME_DATA *pFrameData,
int32_t nTimeout);

Descriptions:

After starting acquisition, you can call this function to get images directly. Noting that the interface can not be mixed with the callback capture mode.

Parameters:

[in] hDevice	The handle of the device.
[in,out] pFrameData	The pointer to the address that the user introduced to receive the image data.
[in] nTimeout	The timeout time of capture image (unit: ms).

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_INVALID_CALL	After registering the capture callback function, the user calls the GXGetImage() to get image.
GX_STATUS_INVALID_PARAMETER	User incoming image address pointer is NULL.

The errors that are not covered above please reference GX_STATUS_LIST.

Precautions:

The GXGetImage() interface is not allowed to be called after the capture callback function is registered, and the call will return GX_STATUS_INVALID_CALL error. When using high resolution cameras for high-speed acquisition, because there is a buffer copy within the GXGetImage() interface, it will affect the transport performance. It is recommended that users use the capture callback mode in this case.

Sample Code:

```
//--------------------------------
//The GXGetImage interface can take one image at a time. The implementation
//process is shown below.
//--------------------------------
#include "GxIAPI.h"

int main(int argc, char* argv[])
{
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    GX_OPEN_PARAM stOpenParam;
    uint32_t ui32DeviceNum = 0;

    //Initializes the library.
    emStatus = GXInitLib();
    if (emStatus != GX_STATUS_SUCCESS)
    {
        return 0;
    }
    //Updates the enumeration list for the devices.
    emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
    if ((emStatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
    {
        return 0;
    }

    //Opens the device.
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode   = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emStatus = GXOpenDevice(&stOpenParam, &hDevice);
    if (emStatus == GX_STATUS_SUCCESS)
    {
        //Setting device's property of GevSCPSPacketSize to improve
        //the acquisition performance of the network camera.
        bool bImplementPacketSize = false;
        uint32_t ui32PacketSize     = 0;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
        //Determine whether the device supports GevSCPSPacketSize.
        GX_NODE_ACCESS_MODE emAccessMode;
        emStatus=GXGetNodeAccessMode(hDevice,"GevSCPSPacketSize",
                                     &emAccessMode);
        bImplementPacketSize=(emAccessMode==GX_NODE_ACCESS_MODE_RW) ?
                              true : false;

        if (bImplementPacketSize)
        {
        //Get Optimal PacketSize.
        emStatus = GXGetOptimalPacketSize (hDevice, &ui32PacketSize);

        //Set the Optimal PacketSize to GevSCPSPacketSize.
        emStatus = GXSetIntValue(hDevice, "GevSCPSPacketSize", ui32PacketSize);
        }

        GX_INT_VALUE stPayLoadSize = 0;
        //Gets the image buffer size, and then apply for dynamic memory.
        emStatus = GXGetIntValue(hDevice, "PayloadSize", &stPayLoadSize);

        if (emStatus == GX_STATUS_SUCCESS&&stPayLoadSize.nCurValue > 0)
        {
            //Defines the incoming parameters of the GXGetImage interface.
            GX_FRAME_DATA stFrameData;

            //Applies for buffer according to the acquired image buffer size
            //m_nPayLoadSize.
            stFrameData.pImgBuf = malloc((size_t)stPayLoadSize.nCurValue);

            //Sends a start acquisition command.
            emStatus = GXSetCommandValue(hDevice, "AcquisitionStart");
            if (emStatus == GX_STATUS_SUCCESS)
            {
                //Calls GXGetImage to get an image.
                while(GXGetImage(hDevice, &stFrameData, 100) != GX_STATUS_SUCCESS)
                {
                    Sleep(10);
                }
                 if (stFrameData.nStatus == GX_FRAME_STATUS_SUCCESS)
                {
                    //Acquiring image is successful.
                    //Image processing...
                }
            }
            //Sends a stop acquisition command.
            emStatus = GXSetCommandValue (hDevice, "AcquisitionStop");

            //Frees image buffer.
            free(stFrameData.pImgBuf);
        }
    }
    emStatus = GXCloseDevice(hDevice);
    emStatus = GXCloseLib();
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
        return 0;
}
```

7.4.5.11. GXFlushQueue

Declarations:

GX_API GXFlushQueue(GX_DEV_HANDLE hDevice)

Descriptions:

Empty the cache image in the image output queue.

Parameters:

[in] hDevice	The handle of the device.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

Precautions:

If the user processes the images too slow, the image of last acquisition may be remained in the queue. Especially in the trigger mode, after the user send the trigger signal, and get the old image (last image). If you want to get the current image that corresponding to trigger signal, you should call the GXFlushQueue() interface before sending the trigger signal to empty the image output queue.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
emStatus = GXFlushQueue(hDevice);
```

7.4.5.12. GXGetOptimalPacketSize (Windows only)

Declarations:

GX_API GXGetOptimalPacketSize (GX_DEV_HANDLE hDevice,uint32_t* punPacketSize);

Descriptions:

Obtain the optimal packet size for the current network environment.
The optimal packet size is the maximum allowable GevSCPSPacketSize of the network camera in the current network environment. It is suggested that after opening the network camera, users obtain the optimal packet size value through the interface, and set the GevSCPSPacketSize attribute of the network camera to improve the acquisition performance of the network camera.

Parameters:

[in] hDevice	The handle of the device.
[out] punPacketSize	Used to return the optimal packet size obtained.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_INVALID_PARAMETER	The unsupported event ID or the callback function is illegal.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	This function is not currently supported.

The errors that are not covered above please reference GX_STATUS_LIST.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32PacketSize = 0;

//Get Optimal PacketSize
S emStatus = GXGetOptimalPacketSize (hDevice, &ui32PacketSize);

//Set the Optimal PacketSize to GevSCPSPacketSize
emStatus = GXSetIntValue(hDevice, "GevSCPSPacketSize", ui32PacketSize);
```

7.4.5.13. GXGetPayLoadSize

Declarations:

GX_API GXGetPayLoadSize(GX_DS_HANDLE hDStream, uint32_t* punPacketSize);

Descriptions:

Get device stream PayLoadSize.

Parameters:

[in] hDStream	Device stream handle.
[out] punPacketSize	PayLoadSize pointer.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_INVALID_PARAMETER	The unsupported event ID or the callback function is illegal.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32PacketSize = 0;
emStatus = GXGetPayLoadSize(hDStream, &ui32PacketSize);
```

7.4.5.14. GXRegisterBuffer

Declarations:

GX_API GXRegisterBuffer(GX_DEV_HANDLE hDevice,
void* pBuffer,
size_t pBufferSize,
void* pUserParam);

Descriptions:

Registering the buffer requested by user is used for image acquisition, and users can independently manage the buffer memory.

Note:

1) The buffer size requested by the user cannot be calculated by itself and needs to be obtained from the GXGetPayLoadSize() interface, otherwise it may cause exceptions. Products such as POLS data frame grabber require alignment of memory size and first address for data acquisition based on performance considerations. Therefore, user needs to align the starting address and size according to the alignment byte requirements specified by the StreamBufferAlignment attribute when using this interface.
2) When the number of registered buffers is small, the acquisition driver completes a single acquisition (buffer is in the consumer queue), and no buffer available in the driver may result in

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

frame loss. Therefore, it is recommended to register at least 3 buffers and complete the use of the image as soon as possible. In addition, hardware such as POLS requires a minimum AnnounceBuffer quantity of 3 or more. If the number of buffers registered by the user is insufficient, it will result in acquisition failure.

3) After calling the GXRegisterBuffer() interface, if the user modifies the parameters of the camera or frame grabber, resulting in a change of PayloadSize, the registered buffer needs to be deregistered, and memory needs to be reapplied and registered based on the new length.
4) GXRegisterBuffer() is an optional interface. If this interface is not called to register the buffer, the API library will automatically allocate buffers and start acquisition during image acquisition , without affecting the acquisition process.

Parameters:

[in] *hDevice*	Device handle
[in] *pBuffer*	The buffer pointer requested by the user
[in] *nSize*	The buffer length requested by the user
[in] *pUserParam*	User defined parameters, which can obtained from GX_FRAME_DATA.pUserParam after setting

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
//Obtain the buffer length and address alignment value registered
//by the user
uint64_t u64BufferSize = 0;
size_t  uBufAddrAlignment = 0;
emStatus = GXGetPayLoadSize(hDStream, &u64BufferSize);
if(GX_STATUS_SUCCESS != emStatus)
{
   //Exception handling...
}

GX_INT_VALUE stIntValue;
emStatus = GXGetIntValue(hStreamFeatureControl,
                         "StreamBufferAlignment", &stIntValue);
if(GX_STATUS_SUCCESS != emStatus)
{
   //Exception handling...
}
uBufAddrAlignment = stIntValue.nCurValue;

//Apply for the buffer length obtained based on the address
//alignment value
char* pBuffer = _aligned_malloc(u64BufferSize, uBufAddrAlignments);
if(NULL == pBuffer)
{
   //Exception handling...
}
//User defined data, NULL for no value
void* pUserParam = NULL;

//Register buffer
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    emStatus = GXRegisterBuffer(hDevice, pBuffer, u64BufferSize
                                pUserParam);
    if(GX_STATUS_SUCCESS != emStatus)
    {
       //Exception handling...
    }
    //Start acquisition...
    //Image processing...
    //Stop acquisition...

    //Anti registration buffer...

    //Free memory
    if(NULL != pBuffer)
    {
       _aligned_free(pBuffer);
       pBuffer = NULL;
    }
```

7.4.5.15. GXUnRegisterBuffer

Declarations:

GX_API GXUnRegisterBuffer(GX_DEV_HANDLE hDevice, void* pBuffer)

Descriptions:

Unregister the buffer registered by the user throughGXRegisterBuffer() interface.

Note：

This interface needs to be called after the acquisition is stopped. Calling it during the acquisition process will affect the normal process.

Parameters:

[in] *hDevice*	Device handle
[in] *pBuffer*	The buffer pointer requested by the user

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
    GX_STATUS emStatus = GX_STATUS_SUCCESS;

    //Obtain the buffer length and address alignment value registered
    //by the user
    uint64_t u64BufferSize = 0;
    size_t  uBufAddrAlignment = 0;
    emStatus = GXGetPayLoadSize(hDStream, &u64BufferSize);
    if(GX_STATUS_SUCCESS != emStatus)
    {
       //Exception handling...
    }

    GX_INT_VALUE stIntValue;
    emStatus = GXGetIntValue(hStreamFeatureControl,
                             "StreamBufferAlignment", &stIntValue);
    if(GX_STATUS_SUCCESS != emStatus)
```

```
    {
        //Exception handling...
    }
    uBufAddrAlignment = stIntValue.nCurValue;

    //Apply for the buffer length obtained based on the address
    //alignment value
    char* pBuffer = _aligned_malloc(u64BufferSize, uBufAddrAlignment);
    if(NULL == pBuffer)
    {
        //Exception handling...
    }

    //User defined data, NULL for no value
    void* pUserParam = NULL;

    //Register buffer
    emStatus = GXRegisterBuffer(hDevice, pBuffer, u64BufferSize ,
                                pUserParam);
    if(GX_STATUS_SUCCESS != emStatus)
    {
        //Exception handling...
    }

    //Start acquisition...
    //Image processing...
    //Stop acquisition...

    //Anti registration buffer...
    emStatus = GXUnRegisterBuffer(hDevice, pBuffer);
    if(GX_STATUS_SUCCESS != emStatus)
    {
        //Exception handling...
    }

    //Free memory
    if(NULL != pBuffer)
    {
        _aligned_free(pBuffer);
        pBuffer = NULL;
    }
```

7.4.6. Device Event Interface

7.4.6.1. GXRegisterDeviceOfflineCallback

Declarations:

```
GX_API GXRegisterDeviceOfflineCallback(GX_DEV_HANDLE hDevice,
                                       void* pUserParam,
                                       GXDeviceOfflineCallBack callBackFun,
                                       GX_EVENT_CALLBACK_HANDLE   *pHCallBack)
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Descriptions:

At present, the Mercury Gigabit camera provides the device offline notification event mechanism, the user can call this interface to register the event handle callback function.

Parameters:

[in] hDevice	The handle of the device.
[in] pUserParam	User private parameter.
[in] callBackFun	The user event handle callback function, for the function type, see GXDeviceOfflineCallBack().
[out] pHCallBack	The handle of offline callback function, the handle is used for unregistering the callback function.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_INVALID_PARAMETER	The unsupported event ID or the callback function is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code: Get the offline event

```
#include "GxIAPI.h"

//Device offline callback function.
static void GX_STDC OnDeviceOfflineCallbackFun(void* pUserParam)
{
    //After receiving the offline notification, the user needs to notify
    //the main thread actively to stop acquiring or closing the device.
    return;
}

int main(int argc, char* argv[])
{
    GXInitLib();
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    GX_OPEN_PARAM stOpenParam;
    uint32_t ui32DeviceNum = 0;
    emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
    if ((emStatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
    {
        return 0;
    }
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode   = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emStatus = GXOpenDevice(&stOpenParam, &hDevice);
    if (emStatus == GX_STATUS_SUCCESS)
    {
        //Defines the handle to the offline callback function.
        GX_EVENT_CALLBACK_HANDLE hCB;

        //Registers device offline callback function.
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
        GXRegisterDeviceOfflineCallback(hDevice,NULL,
                                        OnDeviceOfflineCallbackFun, &hCB);

        //----------------------
        //
        //The event notification will be returned to the user via the
        //OnDeviceOfflineCallbackFun interface.
        //
        //----------------------

        //Unregisters device offline callback function.
        GXUnregisterDeviceOfflineCallback(hDevice,hCB);
    }
    emStatus = GXCloseDevice(hDevice);
    GXCloseLib();
    return 0;
}
```

7.4.6.2. GXUnregisterDeviceOfflineCallback

Declarations:

GX_API GXUnregisterDeviceOfflineCallback(GX_DEV_HANDLE hDevice,
GX_EVENT_CALLBACK_HANDLE hCallBack)

Descriptions:

Unregister event handle callback function.

Parameters:

[in] hDevice	The handle of the device.
[in] hCallBack	The handle of device offline callback function.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code: Get device offline event

```
#include "GxIAPI.h"

//Device offline callback function.
static void GX_STDC OnDeviceOfflineCallbackFun(void* pUserParam)
{
    //After receiving the offline notification, the user needs to notify
    //the main thread actively to stop acquiring or close the device.
    return;
}

int main(int argc, char* argv[])
{
    GXInitLib();
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    GX_OPEN_PARAM stOpenParam;
    uint32_t ui32DeviceNum = 0;
    emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
    if ((emStatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
    {
        return 0;
    }
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode   = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emStatus = GXOpenDevice(&stOpenParam, &hDevice);
    if (emStatus == GX_STATUS_SUCCESS)
    {
        //Defines the handle to the offline callback function.
        GX_EVENT_CALLBACK_HANDLE hCB;

       //Registers device offline callback function.
       GXRegisterDeviceOfflineCallback(hDevice, NULL,
                                       OnDeviceOfflineCallbackFun, &hCB);

        //----------------------
        //
        //The event notification will be returned to the user via the
        //OnDeviceOfflineCallbackFun interface.
        //
        //----------------------

        //Unregisters device offline callback function.
        GXUnregisterDeviceOfflineCallback(hDevice, hCB);
    }
    emStatus = GXCloseDevice(hDevice);
    GXCloseLib();
    return 0;
}
```

7.4.6.3. GXRegisterDeviceReconnectCallback

Declarations:

GX_API GXRegisterDeviceReconnectCallback(GX_DEV_HANDLE hDevice, void * pUserParam, GXDeviceReconnectCallBack callBackFun);

Descriptions:

The user can call this interface to register the event handle callback function.(Only supports GigE vision camera and USB3.0 vision camera)

Parameters:

[in] hDevice	Device handle
[in] pUserParam	Private parameters
[in] callBackFun	Device reconnection callback function, for the function type, see GXDeviceReconnectCallBack.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_INVALID_PARAMETER	The pointer that the user introduces is NULL.

The errors that are not covered above please reference GX_STATUS_LIST.

Code Sample:

```
#include "GxIAPI.h"

//Device reconnection callback function.
static void GX_STDC OnDeviceReconnectCallbackFun(void* pUserParam)
{
    //Upon receiving a reconnection notification, users perform the
    //necessary operations…
    return;
}

int main(int argc, char* argv[])
{
    GXInitLib();
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    GX_OPEN_PARAM stOpenParam;
   uint32_t ui32DeviceNum = 0;
    emStatus = GXUpdateDeviceList(&ui32DeviceNum, 1000);
    if ((emStatus != GX_STATUS_SUCCESS)||(ui32DeviceNum<= 0))
    {
        return 0;
    }
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emStatus = GXOpenDevice(&stOpenParam, &hDevice);
    if (emStatus == GX_STATUS_SUCCESS)
    {
       //Enable device reconnection mechanism.
        GX_LOCAL_DEV_HANDLE hLocalDevice;
        GXGetLocalDeviceHandleFromDev(hDevice, &hLocalDevice);
       emStatus =
       GXSetBoolValue(hLocalDevice, "EnableAutoConnection", true);
        if (emStatus != GX_STATUS_SUCCESS)
        {
            return 0;
        }


       //Register device reconnection callback function.
       emStatus = GXRegisterDeviceReconnectCallback(hDevice,NULL,
       OnDeviceReconnectCallbackFun);
        if (emStatus != GX_STATUS_SUCCESS)
        {
            return 0;
        }
```

```
        //----------------------
        //
        //Reconnection event notifications are returned to the user via
        //the OnDeviceReconnectCallbackFun interface.
        //
        //----------------------
        //Unregister device reconnection event callback.
        GXUnregisterDeviceReconnectCallback(hDevice);
    }
    emStatus = GXCloseDevice(hDevice);
    GXCloseLib();
    return 0;
}
```

7.4.6.4. GXUnregisterDeviceReconnectCallback

Declarations:

GX_API GXUnregisterDeviceReconnectCallback(GX_DEV_HANDLE hDevice);

Descriptions:

Unregister the reconnection event callback function of the device reconnection mechanism. (Only supports GigE vision camera and USB3.0 vision camera)

Parameters:

[in] hDevice	Device handle

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_INVALID_PARAMETER	The pointer that the user introduces is NULL.

The errors that are not covered above please reference GX_STATUS_LIST.

Code Sample:

```
#include "GxIAPI.h"

//Device reconnection callback function
static void GX_STDC OnDeviceReconnectCallbackFun(void* pUserParam)
{
    //Upon receiving a reconnection notification, users perform the
    //necessary operations…
    return;
}

int main(int argc, char* argv[])
{
    GXInitLib();
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    GX_OPEN_PARAM stOpenParam;
   uint32_t ui32DeviceNum = 0;
    emStatus = GXUpdateDeviceList(&ui32DeviceNum, 1000);
    if ((emStatus != GX_STATUS_SUCCESS)||(ui32DeviceNum<= 0))
```

```
        {
            return 0;
        }
        stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
        stOpenParam.openMode = GX_OPEN_INDEX;
        stOpenParam.pszContent = "1";
        emStatus = GXOpenDevice(&stOpenParam, &hDevice);
        if (emStatus == GX_STATUS_SUCCESS)
        {
           // Enable device reconnection mechanism.
            GX_LOCAL_DEV_HANDLE hLocalDevice;
            GXGetLocalDeviceHandleFromDev(hDevice, &hLocalDevice);
           emStatus =
           GXSetBoolValue(hLocalDevice, "EnableAutoConnection", true);
           if (emStatus != GX_STATUS_SUCCESS)
            {
            return 0;
            }


           // Register device reconnection callback function.
           emStatus = GXRegisterDeviceReconnectCallback(hDevice,NULL,
           OnDeviceReconnectCallbackFun);
            if (emStatus != GX_STATUS_SUCCESS)
            {
                return 0;
            }

           //----------------------
           //
           // Reconnection event notifications are returned to the user via
           //the OnDeviceReconnectCallbackFun interface.
           //
           //----------------------
           // Unregister device reconnection event callback.
           emStatus = GXUnregisterDeviceReconnectCallback(hDevice);
            if (emStatus != GX_STATUS_SUCCESS)
            {
                return 0;
            }

        }
        emStatus = GXCloseDevice(hDevice);
        GXCloseLib();
        return 0;
    }
```

7.4.6.5. GXRegisterDeviceDisconnectCallback

Declarations:

```
GX_API GXRegisterDeviceDisconnectCallback(GX_DEV_HANDLE hDevice,
                                          void * pUserParam,
                                          GXDeviceDisconnectCallBack callBackFun);
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Descriptions:

Register the disconnection event callback function of the device reconnection mechanism. (Only supports GigE vision camera and USB3.0 vision camera)

Parameters:

[in] hDevice	Device handle
[in] pUserParam	Private parameters
[in] callBackFun	Device disconnection callback function, for the function type, see GXDeviceReconnectCallBack.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_INVALID_PARAMETER	The pointer that the user introduces is NULL.

The errors that are not covered above please reference GX_STATUS_LIST.

Code Sample:

```
#include "GxIAPI.h"

//Device disconnection callback function.
static void GX_STDC OnDeviceDisconnectCallbackFun(void* pUserParam)
{
    // Upon receiving a disconnection notification, users perform the
    //necessary operations…

    return;
}

int main(int argc, char* argv[])
{
    GXInitLib();
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    GX_OPEN_PARAM stOpenParam;
   uint32_t ui32DeviceNum = 0;
    emStatus = GXUpdateDeviceList(&ui32DeviceNum, 1000);
    if ((emStatus != GX_STATUS_SUCCESS)||(ui32DeviceNum<= 0))
    {
        return 0;
    }
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emStatus = GXOpenDevice(&stOpenParam, &hDevice);
    if (emStatus == GX_STATUS_SUCCESS)
    {
      //Enable auto reconnection.
       GX_LOCAL_DEV_HANDLE hLocalDevice;
       GXGetLocalDeviceHandleFromDev(hDevice, &hLocalDevice);
      emStatus =
      GXSetBoolValue(hLocalDevice, "EnableAutoConnection", true);
```

```
        if (emStatus != GX_STATUS_SUCCESS)
        {
            return 0;
        }

        // Register device disconnection callback function.
        emStatus = GXRegisterDeviceDisconnectCallback(hDevice,NULL,
        OnDeviceDisconnectCallbackFun);
        if (emStatus != GX_STATUS_SUCCESS)
        {
            return 0;
        }

        //----------------------
        //
        //Notifications are returned to the use
        //via the OnDeviceReconnectCallbackFun interface.
        //
        //----------------------
        //Unregister device disconnection event callback.
        emStatus =       GXUnregisterDeviceDisconnectCallback(hDevice);
        if (emStatus != GX_STATUS_SUCCESS)
        {
            return 0;
        }

    }
    emStatus = GXCloseDevice(hDevice);
    GXCloseLib();
    return 0;
}
```

7.4.6.6. GXUnregisterDeviceDisconnectCallback

Declarations:

GX_API GXUnregisterDeviceDisconnectCallback(GX_DEV_HANDLE hDevice);

Descriptions:

Unregister the disconnection event callback function of the device reconnection mechanism. (Only supports GigE vision camera and USB3.0 vision camera)

Parameters:

[in] hDevice	Device handle

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_INVALID_PARAMETER	The pointer that the user introduces is NULL.

The errors that are not covered above please reference GX_STATUS_LIST.

Code Sample:

```
#include "GxIAPI.h"
```

```
// Device disconnection callback function.
static void GX_STDC OnDeviceDisconnectCallbackFun(void* pUserParam)
{
    // Upon receiving a disconnection notification, users perform the
    //necessary operations…
    return;
}

int main(int argc, char* argv[])
{
    GXInitLib();
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    GX_OPEN_PARAM stOpenParam;
   uint32_t ui32DeviceNum = 0;
    emStatus = GXUpdateDeviceList(&ui32DeviceNum, 1000);
    if ((emStatus != GX_STATUS_SUCCESS)||(ui32DeviceNum<= 0))
    {
        return 0;
    }
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emStatus = GXOpenDevice(&stOpenParam, &hDevice);
    if (emStatus == GX_STATUS_SUCCESS)
    {
       // Enable device reconnection mechanism.
        GX_LOCAL_DEV_HANDLE hLocalDevice;
        GXGetLocalDeviceHandleFromDev(hDevice, &hLocalDevice);
       emStatus =
       GXSetBoolValue(hLocalDevice, "EnableAutoConnection", true);
        if (emStatus != GX_STATUS_SUCCESS)
        {
            return 0;
        }


       // Register device disconnection callback function.
       emStatus = GXRegisterDeviceDisconnectCallback(hDevice,NULL,
       OnDeviceDisconnectCallbackFun);
        if (emStatus != GX_STATUS_SUCCESS)
        {
               return 0;
        }

       //----------------------
       //
       // Notifications are returned to the use
      //via the OnDeviceReconnectCallbackFun interface.
       //
       //----------------------
       // Unregister device disconnection event callback.
```

```
        emStatus = GXUnregisterDeviceDisconnectCallback(hDevice);
        if (emStatus != GX_STATUS_SUCCESS)
        {
                return 0;
        }
    }
    emStatus = GXCloseDevice(hDevice);
    GXCloseLib();
    return 0;
}
```

7.4.6.7. GXRegisterFeatureCallbackByString

Declarations:

```
GX_API GXRegisterFeatureCallbackByString(GX_PORT_HANDLE hPort,
                                         void* pUserParam,
                                         GXFeatureCallBackByString callBackFun,
                                         const char* strfeatureName,
                                         GX_FEATURE_CALLBACK_BY_STRING_HANDLE *pHCallBack);
```

Descriptions:

Register property update callback function.

Parameters:

[in] hPort	The handle of the device
[in] pUserParam	User Private Parameters
[in] callBackFun	User event processing callback function, please reference to GXFeatureCallBackByString()
[in] strfeatureName	*Node name*
[out] pHCallBack	Property update callback function handle, which is used to unregister the callback function

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_INVALID_PARAMETER	The parameter is invalid.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code: Get remote device events

```
#include"GxIAPI.h"
GX_DEV_HANDLE hDevice = NULL;

//Remote device event callback processing function
static void GX_STDC OnFeatureCallbackFun(const char* strFeatureName,
                                         void* pUserParam)
{
if (strFeatureName == "EventExposureEnd")
    {
        //Obtain event data such as timestamp and frame ID
        //corresponding to the end of exposure event for this frame
        GX_INT_VALUE stIntValue;
        GXGetIntValue(hDevice, "EventExposureEndFrameID", &stIntValue);
```

```
    }

    return;
}

int main(int argc, char* argv[])
{
    GXInitLib();
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_OPEN_PARAM stOpenParam;
    uint32_t ui32DeviceNum = 0;
    emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
    if ((emStatus!= GX_STATUS_SUCCESS)||(ui32DeviceNum <= 0))
    {
        return 0;
    }
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emStatus= GXOpenDevice(&stOpenParam, &hDevice);
    if (emStatus == GX_STATUS_SUCCESS)
    {
        //Select exposure end event
        GXSetEnumValueByString(hDevice, "EventSelector",
                               "ExposureEnd");

        //Start exposure end event
        GXSetEnumValueByString(hDevice, "EventNotification", "On");

        //Define property update callback function handle
        GX_FEATURE_CALLBACK_HANDLE hCB;

        //Registering the callback function for the end of
        //exposure event
        GXRegisterFeatureCallbackByString(hDevice,NULL,
                                          OnFeatureCallbackFun,
                                          "EventExposureEnd",
                                          &hCB);

      //----------------------
      //
      //Event notifications will be returned to users
      //through the OnFeatureCallbackFun interface
      //
      //----------------------

      //Cancellation of exposure end event callback
      GXUnregisterFeatureCallbackByString(hDevice, "EventExposureEnd",
                                          hCB);
    }
    emStatus= GXCloseDevice(hDevice);
    GXCloseLib();
    return 0;

}
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

7.4.6.8. GXUnregisterFeatureCallbackByString

Declarations:

GX_API GXUnregisterFeatureCallbackByString(GX_PORT_HANDLE hPort,
const char* strfeatureName,
GX_FEATURE_CALLBACK_BY_STRING_HANDLE hCallBack);

Descriptions:

Unregister device attribute update callback function.

Precautions:

Used in conjunction with GXRegisterFeatureCallback(), each registration must have a corresponding cancellation corresponding to it.

Parameters:

[in] hPort	The handle of the device
[in] strfeatureName	Node name
[in] hCallBack	Property update callback function handle

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_INVALID_PARAMETER	The parameter is invalid.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code: Get remote device events

```
#include"GxIAPI.h"
GX_DEV_HANDLE hDevice = NULL;

//Remote device event callback processing function

static void GX_STDC OnFeatureCallbackFun(const char* strFeatureName,
                                         void* pUserParam)
{
if (strFeatureName == "EventExposureEnd")
    {
        //Obtain event data such as timestamp and frame ID
        //corresponding to the end of exposure event for this frame
        GX_INT_VALUE stIntValue;
        GXGetIntValue(hDevice, "EventExposureEndFrameID", &stIntValue);
    }

    return;
}

int main(int argc, char* argv[])
{
    GXInitLib();
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_OPEN_PARAM stOpenParam;
    uint32_t ui32DeviceNum = 0;
    emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
    if ((emStatus!= GX_STATUS_SUCCESS)||(ui32DeviceNum<= 0))
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
        {
            return 0;
        }
        stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
        stOpenParam.openMode = GX_OPEN_INDEX;
        stOpenParam.pszContent = "1";
        emStatus = GXOpenDevice(&stOpenParam, &hDevice);
        if (emStatus == GX_STATUS_SUCCESS)
        {
            //Select exposure end event
            GXSetEnumValueByString(hDevice, "EventSelector",
                                   "ExposureEnd");

            //Start exposure end event
            GXSetEnumValueByString(hDevice, "EventNotification", "On");

            // Define property update callback function handle
            GX_FEATURE_CALLBACK_HANDLE hCB;

            //Registering the callback function for the end of
            //exposure event
            GXRegisterFeatureCallbackByString(hDevice,
                                              NULL,
                                              OnFeatureCallbackFun,
                                              "EventExposureEnd",
                                              &hCB);
          //----------------------
          //
          //Event notifications will be returned to users
          //through the OnFeatureCallbackFun interface

          //
          //----------------------

          //Cancellation of exposure end event callback
          GXUnregisterFeatureCallbackByString(hDevice, "EventExposureEnd",
                                              hCB);
        }
        emStatus = GXCloseDevice(hDevice);
        GXCloseLib();
        return 0;
    }
```

7.4.6.9. GXFlushEvent

Declarations:

GX_API GXFlushEvent (GX_DEV_HANDLE hDevice)

Descriptions:

Empty the device event, such as the frame exposure to end the event data queue.

Parameters:

[in] hDevice	The handle of the device.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

Precautions:

The library internal event data receiving and processing using caching mechanism, if the user receiving, processing event speed is slower than the event generates, then the event data will be accumulated in the library, it will affect the the user to get real-time event data. If you want to get the real-time event data, you need to call the GXFlushEvent() interface to clear the event cache data. This interface empties all the event data at once.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
emStatus = GXFlushEvent(hDevice);
```

7.4.6.10. GXGetEventNumInQueue

Declarations:

```
GX_API GXGetEventNumInQueue (GX_DEV_HANDLE hDevice,
                             uint32_t *pnEventNum);
```

Descriptions:

Get the number of the events in the current remote device event queue cache.

Parameters:

[in] hDevice	The handle of the device.
[in] pnEventNum	The pointer of event number.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32EventNum = 0;
emStatus = GXGetEventNumInQueue(hDevice, &ui32EventNum);
```

7.4.7. Image and Video Saving Interface

7.4.7.1. GXSaveImage

Declarations:

```
GX_API GXSaveImage(GX_SAVE_IMAGE_INFO* stSaveImageInfo);
```

Descriptions:

Save the capturing images.

Parameters:

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

[in] *stSaveImageInfo* Image information.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_INVALID_PARAMETER	The unsupported event ID or the callback function is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code: Save the captured images locally.

```
int main()
{
GX_STATUS emStatus = GX_STATUS_SUCCESS;

// Initialise library.
emStatus = GXInitLib();
if (emStatus != GX_STATUS_SUCCESS)
{
       return 0;
}

// Enumerate cameras.
uint32_t ui32DeviceNum = 0;
emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
if (emStatus != GX_STATUS_SUCCESS)
{
       return 0;
}

if (ui32DeviceNum <= 0)
{
    cout << "No device!" << endl;
    return 0;
}

// Turn on the camera by index.
GX_DEV_HANDLE hDevice = NULL;
emStatus = GXOpenDeviceByIndex(1, &hDevice);
if (emStatus != GX_STATUS_SUCCESS)
{
       return 0;
}

emStatus = GXSetCommandValue(hDevice, "AcquisitionStart");
if (emStatus != GX_STATUS_SUCCESS)
{
       return 0;
}

PGX_FRAME_DATA_EX pFrameBuffer = NULL;
GXDQBufEx(hDevice, &pFrameBuffer, 1000);
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    GX_SAVE_IMAGE_INFO stSaveImageInfo;
    stSaveImageInfo.emCfaMethod = GX_CFA_METHOD_BALANCE;
    stSaveImageInfo.emImgFormat = IMAGE_FORMAT_BMP;
    stSaveImageInfo.emSrcFormat =
        (GX_PIXEL_FORMAT_ENTRY)pFrameBuffer->nPixelFormat;
    stSaveImageInfo.nHeight = pFrameBuffer->nHeight;
    stSaveImageInfo.nWidth = pFrameBuffer->nWidth;
    stSaveImageInfo.nImgQuality = 99;
    stSaveImageInfo.pImageBuffer =
        (unsigned char*)(pFrameBuffer->pImgBuf);
    stSaveImageInfo.pImgPath = "SaveImageTest.bmp";
    // Save images.
    GXSaveImage(&stSaveImageInfo);

    GXQBuf(hDevice, pFrameBuffer);

    GXSetCommandValue(hDevice, "AcquisitionStop");

    // Turn off the camera.
    GXCloseDevice(hDevice);

    // Close the library.
    GXCloseLib();
    return 0;
    }
```

7.4.7.2. GXCreateRecorder

Declarations:

GX_API GXCreateRecorder(GX_RECORD_PARAM * stRecordParam,
GX_RECORDER_HANDLE * phRecorder);

Descriptions:

Save frame images locally. Create the video handle and set the parameter of saved video.

Parameters:

[in] *stRecordParam*	Video parameter information.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_INVALID_PARAMETER	The unsupported event ID or the callback function is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample code: Save frame images locally.

```
    int main()
    {
    GX_STATUS emStatus = GX_STATUS_SUCCESS;

    // Initialise library.
    emStatus = GXInitLib();
    if (emStatus != GX_STATUS_SUCCESS)
    {
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
        return 0;
}

// Enumerate cameras.
uint32_t ui32DeviceNum = 0;
emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
if (emStatus != GX_STATUS_SUCCESS)
{
        return 0;
}

if (ui32DeviceNum <= 0)
{
    cout << "No device!" << endl;
    return 0;
}

// Turn on the camera by index.
GX_DEV_HANDLE hDevice = NULL;
emStatus = GXOpenDeviceByIndex(1, &hDevice);
if (emStatus != GX_STATUS_SUCCESS)
{
        return 0;
}

GX_INT_VALUE stHeight;
emStatus = GXGetIntValue(hDevice, "Height", &stHeight);
GX_INT_VALUE stWidth;
emStatus = GXGetIntValue(hDevice, "Width", &stWidth);
GX_ENUM_VALUE stEnumValue;
emStatus = GXGetEnumValue(hDevice, "PixelFormat", &stEnumValue);

GX_RECORDER_HANDLE phRecorder = NULL;
GX_RECORD_PARAM stRecordParam;
stRecordParam.emPixelFormat = stEnumValue.stCurValue.nCurValue;
stRecordParam.emVideoFormat = VIDEO_FORMAT_H264_MP4;
stRecordParam.nBitRate = 2000;
stRecordParam.nFrameRate = 1;
stRecordParam.nWidth = stWidth.nCurValue;
stRecordParam.nHeight = stHeight. nCurValue;
stRecordParam.pPathName = "testVideo.avi";
// Recording configuration.
emStatus = GXCreateRecorder(&stRecordParam, &phRecorder);
if (emStatus != GX_STATUS_SUCCESS)
{
    GXCloseDevice(hDevice);
        return 0;
}

emStatus = GXSetCommandValue(hDevice, "AcquisitionStart");
for (int i = 0; i < 100; ++i)
{
        PGX_FRAME_DATA_EX pFrameBuffer = NULL;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
        GXDQBufEx(hDevice, &pFrameBuffer, 1000);
        // Add frame images.
        GXAddFrame(phRecorder,
             (unsigned char*)pFrameBuffer->pImgBuf);
        GXQBuf(hDevice, pFrameBuffer);
}

GXSetCommandValue(hDevice, "AcquisitionStop");

// Stop recording.
GXDestroyRecorder(phRecorder);
// Turn off the camera.
GXCloseDevice(hDevice);

// Close the library.
GXCloseLib();
return 0;
}
```

7.4.7.3. GXAddFrame

Declarations:

GX_API GXAddFrame(GX_RECORDER_HANDLE phRecorder,
unsigned char * pImageBuffer);

Descriptions:

Add frame images to the video handle.

Parameters:

[in] *phRecorder*	The video handle.
[in] *pImageBuffer*	Image data.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_INVALID_PARAMETER	The unsupported event ID or the callback function is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample code: Similar to GXCreateRecorder

7.4.7.4. GXDestroyRecorder

Declarations:

GX_API GXDestroyRecorder(GX_RECORDER_HANDLE phRecorder);

Descriptions:

Stop recording and delete the video handle.

Parameters:

[in] *phRecorder*	The video handle.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_INVALID_PARAMETER	The unsupported event ID or the callback function is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample code: Similar to GXCreateRecorder

7.5. Not recommended Definitions

7.5.1. Type

7.5.2. Constants

7.5.2.1. GX_ACCESS_STATUS

```
typedef enum GX_ACCESS_STATUS
{
    GX_ACCESS_STATUS_UNKNOWN      = 0,
    GX_ACCESS_STATUS_READWRITE    = 1,
    GX_ACCESS_STATUS_READONLY     = 2,
    GX_ACCESS_STATUS_NOACCESS     = 3,
} GX_ACCESS_STATUS;
typedef int32_t GX_ACCESS_STATUS_CMD;
```

Name	Description
GX_ACCESS_STATUS_UNKNOWN	The device's current status is unknown
GX_ACCESS_STATUS_READWRITE	The device currently supports reading and writing
GX_ACCESS_STATUS_READONLY	The device currently only supports reading
GX_ACCESS_STATUS_NOACCESS	The device currently does neither support reading nor support writing

7.5.2.2. GX_FEATURE_LEVEL

```
typedef enum GX_FEATURE_LEVEL
{
    GX_FEATURE_LEVEL_REMOTE_DEV     =0x00000000,
    GX_FEATURE_LEVEL _TL            =0X01000000,
    GX_FEATURE_LEVEL _IF            =0x02000000,
    GX_FEATURE_LEVEL _DEV           =0x03000000,
    GX_FEATURE_LEVEL _DS            =0x04000000,
} GX_FEATURE_LEVEL;
```

Name	Description
GX_FEATURE_LEVEL_REMOTE_DEV	Remote device layer
GX_FEATURE_LEVEL_TL	System layer
GX_FEATURE_LEVEL_IF	Interface layer

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

GX_FEATURE_LEVEL_DEV	Device layer
GX_FEATURE_LEVEL_DS	DataStream layer

7.5.2.3. GX_FEATURE_TYPE

```
typedef enum GX_FEATURE_TYPE
{
    GX_FEATURE_INT          =0x10000000,
    GX_FEATURE_FLOAT        =0X20000000,
    GX_FEATURE_ENUM         =0x30000000,
    GX_FEATURE_BOOL         =0x40000000,
    GX_FEATURE_STRING       =0x50000000,
    GX_FEATURE_BUFFER       =0x60000000,
    GX_FEATURE_COMMAND      =0x70000000,
} GX_FEATURE_TYPE;
```

Name	Description
GX_FEATURE_INT	Integer type
GX_FEATURE_FLOAT	Floating point type
GX_FEATURE_ENUM	Enum type
GX_FEATURE_BOOL	Boolean type
GX_FEATURE_STRING	String type
GX_FEATURE_BUFFER	Block data type
GX_FEATURE_COMMAND	Command type

7.5.3. Structure

7.5.3.1. GX_INT_RANGE

Related interface:GXGetIntRange()

The interface describes the maximum value, minimum value and step length of the integer type.

```
typedef struct GX_INT_RANGE
{
    int64_t nMin;
    int64_t nMax;
    int64_t nInc;
    int32_t reserved[8];
}GX_INT_RANGE;
```

Name	Description
nMin	Minimum value

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

nMax	Maximum value
nInc	Step size
reserved	32 bytes, reserved

7.5.3.2. GX_FLOAT_RANGE

Related interface:GXGetFloatRange()

The interface describes the maximum value, minimum value, step length and unit of the float-point type.

```
typedef struct GX_FLOAT_RANGE
{
    double   dMin;
    double   dMax;
    double   dInc;
    char     szUnit[GX_INFO_LENGTH_8_BYTE];
    bool     bIncIsValid;
    int8_t   reserved[31];
}GX_FLOAT_RANGE;
```

Name	Description
dMin	Minimum value
dMax	Maximum value
dInc	Step size
szUnit	Unit. 8 bytes
bIncIsValid	1 byte, indicates whether the step size is supported
reserved	31 bytes, reserved

7.5.3.3. GX_ENUM_DESCRIPTION

Related interface:GXGetEnumDescription()

The interface describes the value and description information of all enumerated items.

```
typedef struct GX_ENUM_DESCRIPTION
{
    int64_t   nValue;
    char      szSymbolic[GX_INFO_LENGTH_64_BYTE];
    int32_t   reserved[8];
}GX_ENUM_DESCRIPTION;
```

Name	Description
nValue	The value of the enumeration item
szSymbolic	64 bytes, the character description information of the enumeration item
reserved	32 bytes, reserved

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

7.5.3.4. GX_DEVICE_IP_INFO

Related interface: GXGetDeviceIPInfo()

This structure represents some of the GigE cameras property descriptions.

```
typedef struct GX_DEVICE_IP_INFO
{
    char szDeviceID[GX_INFO_LENGTH_64_BYTE + 4];
    char szMAC[GX_INFO_LENGTH_32_BYTE];
    char szIP[GX_INFO_LENGTH_32_BYTE];
    char szSubNetMask[GX_INFO_LENGTH_32_BYTE];
    char szGateWay[GX_INFO_LENGTH_32_BYTE];
    char szNICMAC[GX_INFO_LENGTH_32_BYTE];
    char szNICIP[GX_INFO_LENGTH_32_BYTE];
    char szNICSubNetMask[GX_INFO_LENGTH_32_BYTE];
    char szNICGateWay[GX_INFO_LENGTH_32_BYTE];
    char szNICDescription[GX_INFO_LENGTH_128_BYTE + 4];
    char reserved[512];
} GX_DEVICE_IP_INFO;
```

Name	Description
szDeviceID	64+4 bytes, the unique identifier of the device
szMAC	32 bytes, MAC address
szIP	32 bytes, IP address
szSubNetMask	32 bytes, subnet mask
szGateWay	32 bytes, gateway
szNICMAC	32 bytes, the MAC address of the corresponding NIC (Network Interface Card)
szNICIP	32 bytes, the IP address of the corresponding NIC
szNICSubNetMask	32 bytes, the subnet mask of the corresponding NIC
szNICGateWay	32 bytes, the gateway of the corresponding NIC
szNICDescription	128+4 bytes, the description of the corresponding NIC
reserved	512 bytes, reserved

7.5.3.5. GX_DEVICE_BASE_INFO

Related interface:GXGetAllDeviceBaseInfo()

This structure represents the basic information of the device, whether it is a USB camera or a GigE camera.

```
typedef struct GX_DEVICE_BASE_INFO
{
    char szVendorName[GX_INFO_LENGTH_32_BYTE];
    char szModelName[GX_INFO_LENGTH_32_BYTE];
    char szSN[GX_INFO_LENGTH_32_BYTE];
    char szDisplayName[GX_INFO_LENGTH_128_BYTE + 4];
    char szDeviceID[GX_INFO_LENGTH_64_BYTE + 4];
    char szUserID[GX_INFO_LENGTH_64_BYTE + 4];
    GX_ACCESS_STATUS_CMD   accessStatus;
    GX_DEVICE_CLASS      deviceClass;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    char reserved[300];
} GX_DEVICE_BASE_INFO;
```

Name	Description
szVendorName	32 bytes, vendor name
szModelName	32 bytes, model name
szSN	32 bytes, device serial number
szDisplayName	128+4 bytes, device display name
szUserID	64+4 bytes, user-defined name
szDeviceID	64+4 bytes, the unique identifier of the device
accessStatus	4 bytes, access status that is currently supported by the device. Refer to GX_ACCESS_STATUS
deviceClass	4 bytes, device type, such as USB2.0, GEV
reserved	300 bytes, reserved

7.5.3.6. GX_FRAME_BUFFER

Related interface:GXDQBuf()、GXQBuf()、GXDQAllBufs()

windows:

```
typedef struct GX_FRAME_BUFFER
{
uint64_t                nFrameID;
uint64_t                nTimestamp;
uint64_t                nBufID;
void_64                 pImgBuf;
GX_FRAME_STATUS         nStatus;
int32_t                 nWidth;
int32_t                 nHeight;
int32_t                 nPixelFormat;
int32_t                 nImgSize;
int32_t                 nOffsetX;
int32_t                 nOffsetY;
void_64                 pChunkDataHandle;
int32_t                 reserved[21];
}GX_FRAME_BUFFER;
typedef GX_FRAME_BUFFER* PGX_FRAME_BUFFER;
```

Linux :

```
typedef struct GX_FRAME_BUFFER
{
    GX_FRAME_STATUS      nStatus;
    void*                pImgBuf;
    int32_t              nWidth;
    int32_t              nHeight;
    int32_t              nPixelFormat;
    int32_t              nImgSize;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    uint64_t        nFrameID;
    uint64_t        nTimestamp;
    uint64_t        nBufID;
    int32_t         nOffsetX;
    int32_t         nOffsetY;
    int32_t         reserved[16];
}GX_FRAME_BUFFER;
typedef GX_FRAME_BUFFER* PGX_FRAME_BUFFER;
```

Name	Description
nStatus	The state of the acquired image. Please refer to GX_FRAME_STATUS
pImgBuf	The image data pointer (After the frame information is enabled, the pImgBuf contains image data and frame information data)
nWidth	Image width
nHeight	Image height
nPixelFormat	Pixel format of image
nImgSize	Data size, in bytes (After the frame information is enabled, nImgsize is the sum of the size of the image data and the size of the frame information)
nFrameID	Frame identification of image
nTimestamp	Timestamp of image
nBufID	BufID
nOffsetX	X-direction offset of the image
nOffsetY	Y-direction offset of the image
pChunkDataHandle	Frame information handle. It is used to obtain the frame information data.
reserved	64 bytes, reserved

7.5.3.7. GX_FRAME_CALLBACK_PARAM

Related interface:GXRegisterCaptureCallback()

This is the formal parameter type of the callback function, and the formal parameter type of the callback function that the user writes must be of this type.

```
typedef struct GX_FRAME_CALLBACK_PARAM
{
    void*               pUserParam;
    GX_FRAME_STATUS     status;
    const   void*       pImgBuf;
    int32_t             nImgSize;
    int32_t             nWidth;
    int32_t             nHeight;
    int32_t             nPixelFormat;
    uint64_t            nFrameID;
    uint64_t            nTimestamp;
```

```
    void_64             pChunkDataHandle;
    int32_t             reserved[64];
} GX_FRAME_CALLBACK_PARAM;
```

Name	Description
pUserParam	User's private data pointer
status	The image state returned by the callback function. Please refer to GX_FRAME_STATUS
pImgBuf	The image data address (After the frame information is enabled, the pImgBuf contains image data and frame information data)
nImgSize	Data size, in bytes (After the frame information is enabled, nImgsize is the sum of the size of the image data and the size of the frame information)
nWidth	Image width
nHeight	Image height
nPixelFormat	PixelFormat of image
nFrameID	Frame identification of image
nTimestamp	Timestamp of image
pChunkDataHandle	Frame information handle. It is used to obtain the frame information data.
reserved	4 bytes, reserved

7.5.3.8. GX_FRAME_DATA

Related interface:GXGetImage()

```
typedef struct GX_FRAME_DATA
{
    GX_FRAME_STATUS     nStatus;
    void*               pImgBuf;
    int32_t             nWidth;
    int32_t             nHeight;
    int32_t             nPixelFormat;
    int32_t             nImgSize;
    uint64_t            nFrameID;
    uint64_t            nTimestamp;
    uint64_t*           pUserParam;
    void_64             pChunkDataHandle;
}GX_FRAME_DATA;
```

Name	Description
nStatus	The state of the acquired image. Please refer to GX_FRAME_STATUS
pImgBuf	The image data address (After the frame information is enabled, the pImgBuf contains image data and frame information data)
nWidth	Image width

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

nHeight	Image height
nPixelFormat	Pixel format of image
nImgSize	Data size (After the frame information is enabled, nImgsize is the sum of the size of the image data and the size of the frame information)
nFrameID	Frame identification of image
nTimestamp	Timestamp of image
nOffsetX	X-direction offset of the image
nOffsetY	Y-direction offset of the image
pChunkDataHandle	Frame information handle. It is used to obtain the frame information data.

7.5.4. Interface

7.5.4.1. GXReadRemoteDevicePortStacked (Support only by Windows)

Declarations:

```
GX_API GXReadRemoteDevicePortStacked(GX_DEV_HANDLE hDevice,
                                     GX_REGISTER_STACK_ENTRY* pstEntries,
                                     size_t *piSize);
```

Descriptions:

Read user specified cameras attribute value in batch (Support Int currently, 4 bytes).

Note:

The acquired attribute value of GigE camera is BigEndian.

Parameters:

[in] hDevice	The handle of the device.
[in,out] pstEntries	It is used to point to the address structure memory where camera attributes need to be obtained and return the attribute value.
[in,out] piSize	Used to return the number of obtained camera attribute values.

Returns:

GX_STATUS_SUCCESS	Successfully operation without error.
GX_STATUS_INVALID_PARAMETER	Invalid parameter.
GX_STATUS_INVALID_HANDLE	Invalid handle warning.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//The following example is getting 3 attribute values (Modifiable for your
//need).
size_t sStackEntry = 3;
GX_REGISTER_STACK_ENTRY* pstStackEntry[3];
char chBufferWidth[256] = {0};
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
char chBufferHeight[256] = {0};
char chBufferOffsetX[256] = {0};

//Attribute address(Support for Int only).
//The following addresses are only for reference. Please consult our
//technical support for specific addresses.
pstStackEntry[0].nAddress = 0xFFFF;

//Length of attribute value(Support for Int only).
pstStackEntry[0].nSize = 4;

//Apply for camera store attribute value memory.
pstStackEntry[0].pBuffer = new int32_t;

pstStackEntry[1].nAddress = 0xFFFF;
pstStackEntry[1].nSize = 4;
pstStackEntry[1].pBuffer = new int32_t;

pstStackEntry[2].nAddress = 0xFFFF;
pstStackEntry[2].nSize = 4;
pstStackEntry[2].pBuffer = new int32_t;

//Get attribute values of multiple cameras in batch.
emStatus = GXReadRemoteDevicePortStacked(hDevice,
                                         pstStackEntry,&sStackEntry);
```

7.5.4.2. GXWriteRemoteDevicePortStacked (Support only by Windows)

Declarations:

```
GX_API GXWriteRemoteDevicePortStacked(GX_DEV_HANDLE hDevice,
                                      const GX_REGISTER_STACK_ENTRY* pstEntries,
                                      size_t *piSize);
```

Descriptions:

Set user specified cameras attribute value in batch (Support Int currently, 4 bytes).

Note:

1) Set GigE camera attribute values under BigEndian mode.
2) After calling the current interface, the node values obtained by using GXGetEnum(), GXGetInt(), GXGetBool() and other interfaces are still the values before modification. You can get the latest attribute values via GXReadRemoteDevicePort() or GXReadRemoteDevicePortStacked().

Parameters:

[in] hDevice	The handle of the device.
[in,out] pstEntries	It is used to point to the address structure memory where camera attributes need to be obtained and return the attribute value.
[in,out] piSize	Used to return the number of obtained camera attribute values.

Returns:

GX_STATUS_SUCCESS	Successfully operation without error.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

GX_STATUS_INVALID_PARAMETER	Invalid parameter.
GX_STATUS_INVALID_HANDLE	Invalid handle warning.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//The following example is to get 3 U3 cameras' attribute values
//(Modifiable for your need).
size_t sStackEntry = 3;
GX_REGISTER_STACK_ENTRY pstStackEntry[3];
char chBufferWidth[256] = {0};
char chBufferHeight[256] = {0};
char chBufferOffsetX[256] = {0};

//Attribute address(Support for Int only).
//The following addresses are only for reference. Please consult our
//technical support for specific addresses.
pstStackEntry[0].nAddress = 0xFFFF;

//Length of attribute value(Support for Int only).
pstStackEntry[0].nSize = 4;

//Apply for camera store attribute value memory.
pstStackEntry[0].pBuffer = chBufferWidth ;

//The following values are modifiable for your need.

pstStackEntry[1].nAddress = 0xFFFF;
pstStackEntry[1].nSize = 4;
pstStackEntry[1].pBuffer = chBufferHeight ;

pstStackEntry[2].nAddress = 0xFFFF;
pstStackEntry[2].nSize = 4;
pstStackEntry[2].pBuffer = chBufferOffsetX;

//Get attribute values of multiple cameras in batch.
emStatus = GXWriteRemoteDevicePortStacked (hDevice, pstStackEntry,&sStackEntry);
```

7.5.4.3. GXReadRemoteDevicePort

Declarations:

```
GX_API GXReadRemoteDevicePort (GX_DEV_HANDLE hDevice, uint64_t ui64Address,
                               void *pBuffer, size_t *piSize);
```

Descriptions:

Read the value from the user specified register.

Note:

The acquired attribute value of GigE camera is BigEndian.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Parameters:

[in] hDevice	The handle of the device.
[in] ui64Address	The register address of the camera attribute.
[out] pBuffer	The memory of the camera attribute, not NULL.
[in, out] piSize	The memory size of the camera attribute, not NULL.

Returns:

GX_STATUS_SUCCESS	Successfully operation without error.
GX_STATUS_INVALID_PARAMETER	Invalid parameter.
GX_STATUS_INVALID_HANDLE	Invalid handle warning.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//Address corresponding to the camera attribute value
//The following addresses are only for reference. Please consult our
//technical support for specific addresses
uint64_t ui64Address = 0xFFFF;

//Apply for camera store attribute value memory.
size_t nSize = 4;
void *pBuffer = new int;
//Get camera attribute values in batch (U3 camera)
emStatus = GXReadRemoteDevicePort ( hDevice, ui64Address, pBuffer, &nSize);

//After finished
if(NULL != pBuffer)
{
   delete pBuffer;
   pBuffer = NULL;
}
```

7.5.4.4. GXWriteRemoteDevicePort

Declarations:

```
GX_API GXWriteRemoteDevicePort(GX_DEV_HANDLE hDevice,
                               uint64_t ui64Address,
                               const void *pBuffer,
                               size_t *piSize);
```

Descriptions:

Set the user-specified register value.

Note:

1) Set GigE camera attribute values under BigEndian mode.
2) After calling the current interface, the node values obtained by using GXGetEnum(), GXGetInt(), GXGetBool() and other interfaces are still the values before modification. You can get the latest attribute values via GXReadRemoteDevicePort() or GXReadRemoteDevicePortStacked().

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Parameters:

[in] hDevice	The handle of the device.
[in] ui64Address	The register address of the camera attribute.
[in] pBuffer	The memory of the camera attribute, not NULL.
[in, out] piSize	The memory size of the camera attribute, not NULL.

Returns:

GX_STATUS_SUCCESS	Successfully operation without error.
GX_STATUS_INVALID_PARAMETER	Invalid parameter.
GX_STATUS_INVALID_HANDLE	Invalid handle warning.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//Address corresponding to the camera attribute value
//The following addresses are only for reference. Please consult our
//technical support for specific addresses
uint64_t ui64Address = 0xFFFF;

//Apply for camera store attribute value memory.
int32_t i32Value = 1;
size_t nSize = sizeof(i32Value);

//Set camera attribute values
 emStatus = GXWriteRemoteDevicePort(hDevice, ui64Address,
                                    &i32Value,& nSize);
 //After finished
If (NULL != pBuffer)
 {
    delete pBuffer;
    pBuffer = NULL;
}
```

7.5.4.5. GXUnregisterFeatureCallback

Declarations:

```
GX_API GXUnregisterFeatureCallback(GX_DEV_HANDLE    hDevice,
                                   GX_FEATURE_ID_CMD featureID,
                                   GX_FEATURE_CALLBACK_HANDLE    hCallBack)
```

Descriptions:

Unregister device attribute update callback function.

Parameters:

[in] hDevice	The handle of the device.
[in] featureID	The feature code ID.
[in] hCallBack	The attribute update callback function handle.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_INVALID_PARAMETER	The unsupported event ID.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code: Get the event of remote device

```
#include "GxIAPI.h"
GX_DEV_HANDLE hDevice = NULL;

//The event callback processing function of the remote device.
static void GX_STDC OnFeatureCallbackFun(GX_FEATURE_ID_CMD featureID,
                                         void* pUserParam)
{
    if (featureID == GX_INT_EVENT_EXPOSUREEND)
    {
        //Gets the event data such as timestamp and frame ID for the frame
        //exposure end event.
        int64_t i64FrameID=0;
        GXGetInt(hDevice, GX_INT_EVENT_EXPOSUREEND_FRAMEID, &i64FrameID);
    }

    return;
}

int main(int argc, char* argv[])
{
    GXInitLib();
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    uint32_t ui32DeviceNum = 0;
    emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
    if ((emStatus != GX_STATUS_SUCCESS)||(ui32DeviceNum <= 0))
    {
        return 0;
    }

    GX_OPEN_PARAM stOpenParam;
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emStatus = GXOpenDevice(&stOpenParam, &hDevice);
    if (emStatus == GX_STATUS_SUCCESS)
    {
        //Selects the exposure end event.
        GXSetEnum(hDevice,GX_ENUM_EVENT_SELECTOR,
                  GX_ENUM_EVENT_SELECTOR_EXPOSUREEND);
        //Enables the exposure end event.
        GXSetEnum(hDevice,GX_ENUM_EVENT_NOTIFICATION,
                  GX_ENUM_EVENT_NOTIFICATION_ON);

        //Declares the attribute update callback handle.
        GX_FEATURE_CALLBACK_HANDLE hCB;

       //Registers the callback function for the exposure end event.
```

```
        GXRegisterFeatureCallback(hDevice, NULL, OnFeatureCallbackFun,
                                  GX_INT_EVENT_EXPOSUREEND, &hCB);
        //----------------------
        //
        //The event notification will be returned to the user via the
        //OnFeatureCallbackFun interface.
        //
        //----------------------

        //Unregisters the callback function for the exposure end event.
        GXUnregisterFeatureCallback(hDevice, GX_INT_EVENT_EXPOSUREEND,
                                    hCB);
    }
    emStatus = GXCloseDevice(hDevice);
    GXCloseLib();
    return 0;
}
```

7.5.4.6. GXRegisterFeatureCallback

Declarations:

GX_API GXRegisterFeatureCallback(GX_DEV_HANDLE hDevice, void* pUserParam,
GXFeatureCallBack callBackFun,
GX_FEATURE_ID_CMD featureID,
GX_FEATURE_CALLBACK_HANDLE *pHCallBack)

Descriptions:

Register device attribute update callback function. When the current value of the device property has updated, or the accessible property is changed, call this callback function.

Parameters:

[in] hDevice	The handle of the device.
[in] pUserParam	User private parameter.
[in] callBackFun	The user event handle callback function, for function type, see GXFeatureCallBack().
[in] featureID	The feature code ID.
[out] pHCallBack	The handle of property update callback function, to unregister the callback function.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_INVALID_PARAMETER	The unsupported event ID or the callback function is illegal.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code: Get the event of remote device

```
#include "GxIAPI.h"
GX_DEV_HANDLE hDevice = NULL;

//The event callback processing function of the remote device.
```

```
static void GX_STDC OnFeatureCallbackFun(GX_FEATURE_ID_CMD featureID,
                                         void* pUserParam)
{
if (featureID == GX_INT_EVENT_EXPOSUREEND)
    {
        //Gets the event data such as timestamp and frame ID for the
        //frame exposure end event.
        int64_t i64FrameID=0;
        GXGetInt(hDevice, GX_INT_EVENT_EXPOSUREEND_FRAMEID, &i64FrameID);
    }

    return;
}
int main(int argc, char* argv[])
{
    GXInitLib();
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_OPEN_PARAM stOpenParam;
    uint32_t ui32DeviceNum = 0;
    emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
    if ((emStatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
    {
        return 0;
    }
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode   = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emStatus = GXOpenDevice(&stOpenParam, &hDevice);
    if (emStatus == GX_STATUS_SUCCESS)
    {
        //Selects the exposure end event.
        GXSetEnum(hDevice,GX_ENUM_EVENT_SELECTOR,
                  GX_ENUM_EVENT_SELECTOR_EXPOSUREEND);

        //Enables the exposure end event.
        GXSetEnum(hDevice,GX_ENUM_EVENT_NOTIFICATION,
                  GX_ENUM_EVENT_NOTIFICATION_ON);

        //Declares the attribute update callback function handle.
        GX_FEATURE_CALLBACK_HANDLE hCB;

        //Registers the callback function for the exposure end event.
        GXRegisterFeatureCallback(hDevice, NULL, OnFeatureCallbackFun,
                                  GX_INT_EVENT_EXPOSUREEND, &hCB);
         //----------------------
        //
        //The event notification will be returned to the user via the
        //OnFeatureCallbackFun interface.
        //
        //----------------------

        //Unregisters the callback function for the exposure end event.
        GXUnregisterFeatureCallback(hDevice, GX_INT_EVENT_EXPOSUREEND,
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
                                        hCB);
    }
    emStatus = GXCloseDevice(hDevice);
    GXCloseLib();
    return 0;
}
```

7.5.4.7. GXSendCommand

Declarations:

GX_API GXSendCommand (GX_DEV_HANDLE hDevice,
GX_FEATURE_ID featureID)

Descriptions:

Send the command.

Parameters:

[in] hDevice	The handle of the device.
[in] featureID	The feature code ID.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_INVALID_ACCESS	Currently inaccessible, can not send command.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
emStatus = GXSendCommand(hDevice, GX_COMMAND_TRIGGER_SOFTWARE);
```

7.5.4.8. GXGetBufferLength

Declarations:

GX_API GXGetBufferLength (GX_DEV_HANDLE hDevice,
GX_FEATURE_ID featureID,
size_t* pnSize);

Descriptions:

Get the data length of buffer type value and the unit is byte, the user can apply the buffer based on the length obtained, and then call the GXGetBuffer() to get the buffer data.

Parameters:

[in] hDevice	The handle of the device.
[in] featureID	The feature code ID.
[in] bValue	The pointer that points to the length value returned. Unit: byte.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
size_t nLength = 0;
emStatus = GXGetBufferLength(hDevice, GX_BUFFER_FRAME_INFORMATION,
                             &nLength);
```

7.5.4.9. GXGetBuffer

Declarations:

```
GX_API GXGetBuffer(GX_DEV_HANDLE hDevice,
                   GX_FEATURE_ID featureID,
                   uint8_t* pBuffer, size_t* pnSize);
```

Descriptions:

Get the Buffer data.

Parameters:

[in] hDevice	The handle of the device.
[in] featureID	The feature code ID.
[out] pBuffer	The pointer that point to the buffer data memory address that the user applied.
[in,out] pnSize	The length of the buffer address that the user inputs.
If pBuffer is NULL:	
[out] pnSize	Return the actual size of the buffer needed.
If pBuffer is not NULL:	
[in] pnSize	The size of the buffer that the user allocated.
[out] pnSize	Return the actual filled buffer size.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.
GX_STATUS_INVALID_ACCESS	Currently inaccessible, can not read.
GX_STATUS_NEED_MORE_BUFFER	The buffer that the user allocates is too small.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//Takes the read of the look-up table as an example.
//Reads the look-up table length.
size_t nLutLength = 0;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
emStatus = GXGetBufferLength(hDevice, GX_BUFFER_LUT_VALUEALL,
                             &nLutLength);

//Applies buffer.
uint8_t *pui8GetLutBuffer = new uint8_t[nLutLength];

//Reads the look-up table contents.
emStatus = GXGetBuffer(hDevice, GX_BUFFER_LUT_VALUEALL,
                       pui8GetLutBuffer, &nLutLength);

//After finished
If (NULL != pui8GetLutBuffer)
{
   delete[] pui8GetLutBuffer;
   pui8GetLutBuffer = NULL;
}
```

7.5.4.10. GXSetBuffer

Declarations:

GX_API GXSetBuffer (GX_DEV_HANDLE hDevice,
GX_FEATURE_ID featureID,
uint8_t* pBuffer, size_t nSize);

Descriptions:

Set the Buffer data.

Parameters:

[in] hDevice	The handle of the device.
[in] featureID	The feature code ID.
[in] pBuffer	The pointer that point to the Buffer data memory address that the user will set.
[in] nSize	The length of the buffer address that the user inputs.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_INVALID_PARAMETER	The pointer that the user introduces is NULL.
GX_STATUS_OUT_OF_RANGE	The maximum length that the content the user writes exceeds the string size.
GX_STATUS_INVALID_ACCESS	Currently inaccessible, can not write.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//To set the look-up table as an example.
//Reads the look-up table length.
size_t nLutLength = 0;
emStatus = GXGetBufferLength(hDevice, GX_BUFFER_LUT_VALUEALL,
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
                                    &nLutLength);

    //Applies buffer.
    uint8_t *pui8SetLutBuffer = new uint8_t[nLutLength];
    //Sets the look-up table contents (For example, you can read a look-
    //up table from the local file).
    //There does not explain how to set the contents of the look-up
    //table buffer. Please install the settings you need to find the
    //look-up table contents.
    //Sets the look-up table.
    emStatus = GXSetBuffer(hDevice, GX_BUFFER_LUT_VALUEALL, pui8SetLutBuffer,
                           nLutLength);


    //After finished
    If (NULL != pui8SetLutBuffer)
    {
       delete[] pui8SetLutBuffer;
       pui8SetLutBuffer = NULL;
    }
```

7.5.4.11. GXGetStringMaxLength

Declarations:

GX_API GXGetStringMaxLength (GX_DEV_HANDLE hDevice,
GX_FEATURE_ID featureID, size_t* pnSize)

Descriptions:

Get the maximum length of the string type value. Unit: byte. The user allocates buffer according to the length information obtained, then call the GXGetString() to get the string information. This interface can get the maximum possible length of the string (including the terminator '\ 0'), but the actual length of the string might not be that long, if the user wants to allocate buffer according to the actual string length, the user can call the *GXGetStringLength* interface to get the actual string length.

Parameters:

[in] hDevice	The handle of the device.
[in] featureID	The feature code ID.
[out] pnSize	The pointer that point to the length value returned. The length value is end with '\0', unit: byte.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
size_t nSize = 0;
emStatus = GXGetStringMaxLength(hDevice, GX_STRING_DEVICE_VENDOR_NAME, &nSize)
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

7.5.4.12. GXGetString

Declarations:

```
GX_API GXGetString (GX_DEV_HANDLE hDevice,
                    GX_FEATURE_ID featureID,
                    char* pszContent,
                    size_t* pnSize);
```

Descriptions:

Get the content of the string type value.

Parameters:

[in] hDevice	The handle of the device.
[in] featureID	The feature code ID.
[out] pszContent	Point to the string buffer address that the user allocated.
[in,out] pnSize	The length of the string buffer address that the user inputs.
If pszContent is NULL:	
[out] pnSize	Return the actual size of the buffer needed.
If pszContent is not NULL:	
[in] pnSize	The size of the buffer that the user allocated.
[out] pnSize	Return the actual filled buffer size.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.
GX_STATUS_INVALID_ACCESS	Currently inaccessible, can not read.
GX_STATUS_NEED_MORE_BUFFER	The buffer that the user allocates is too small.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
size_t nSize = 0;
//Gets string length.
emStatus = GXGetStringLength(hDevice, GX_STRING_DEVICE_VENDOR_NAME, &nSize);

//Applies memory based on the length.
char *pszText = new char[nSize];

emStatus = GXGetString(hDevice, GX_STRING_DEVICE_VENDOR_NAME, pszText, &nSize);

//After finished
if(NULL != pszText)
{
   delete[] pszText;
   pszText = NULL;
}
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

7.5.4.13. GXSetString

Declarations:

```
GX_API GXSetString(GX_DEV_HANDLE hDevice,
                   GX_FEATURE_ID featureID,
                   char* pszContent)
```

Descriptions:

Set the content of the string value.

Parameters:

[in] hDevice	The handle of the device.
[in] featureID	The feature code ID.
[in] pszContent	The string address that the user will set. The string is end with '\0'.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_INVALID_PARAMETER	The pointer that the user introduces is NULL.
GX_STATUS_OUT_OF_RANGE	The maximum length that the content the user writes exceeds the string size.
GX_STATUS_INVALID_ACCESS	Currently inaccessible, can not write.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
char* pszText = "test";
emStatus = GXSetString(hDevice, GX_STRING_USER_PASSWORD, pszText);
```

7.5.4.14. GXGetStringLength

Declarations:

```
GX_API GXGetStringLength(GX_DEV_HANDLE hDevice,
                         GX_FEATURE_ID featureID,
                         size_t* pnSize)
```

Descriptions:

Get the current value length of the character string type. Unit: byte. The user can allocate the buffer size according to the length information that is get from the function, and then call the GXGetString() to get the character string information.

Parameters:

[in] hDevice	The handle of the device.
[in] featureID	The feature code ID.
[out] pnSize	The pointer that point to the length value returned. The length value is end with '\0', unit: byte.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
size_t nSize = 0;
emStatus = GXGetStringLength(hDevice, GX_STRING_DEVICE_VENDOR_NAME, &nSize);
```

7.5.4.15. GXSetBool

Declarations:

```
GX_API GXSetBool(GX_DEV_HANDLE  hDevice,
                 GX_FEATURE_ID  featureID,
                 bool  bValue)
```

Descriptions:

Set the value of bool type.

Parameters:

[in] hDevice	The handle of the device.
[in] featureID	The feature code ID.
[in] bValue	The bool value that the user will set.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_INVALID_ACCESS	Currently inaccessible, can not write.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
bool bTemp = true;
emStatus = GXSetBool(hDevice, GX_BOOL_REVERSE_X, bTemp);
```

7.5.4.16. GXGetBool

Declarations:

```
GX_API GXGetBool (GX_DEV_HANDLE hDevice,
                  GX_FEATURE_ID featureID, bool* pbValue)
```

Descriptions:

Get the value of bool type.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Parameters:

[in] hDevice	The handle of the device.
[in] featureID	The feature code ID.
[out] pbValue	The pointer that point to the bool value returned.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.
GX_STATUS_INVALID_ACCESS	Currently inaccessible, can not read.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
bool bTemp = false;
emStatus = GXGetBool(hDevice, GX_BOOL_REVERSE_X, &bTemp);
```

7.5.4.17. GXSetEnum

Declarations:

GX_API GXSetEnum (GX_DEV_HANDLE hDevice,
GX_FEATURE_ID featureID, int64_t nValue)

Descriptions:

Set the enumeration value.

Parameters:

[in] hDevice	The handle of the device.
[in] featureID	The feature code ID.
[in] nValue	The enumeration values that the user will set. The value range can be got by the *nValue* of the GX_ENUM_DESCRIPTION.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_OUT_OF_RANGE	The value that the user introduces is cross the border.
GX_STATUS_INVALID_ACCESS	Currently inaccessible, can not write.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
int64_t i64Value = GX_GAIN_AUTO_CONTINUOUS;
emStatus = GXSetEnum(hDevice, GX_ENUM_GAIN_AUTO, i64Value);
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

7.5.4.18. GXGetEnumEntryNums

Declarations:

GX_API GXGetEnumEntryNums (GX_DEV_HANDLE hDevice,
GX_FEATURE_ID featureID,
uint32_t* pnEntryNums)

Descriptions:

Get the number of the options for the enumeration item.

Parameters:

[in] hDevice	The handle of the device.
[in] featureID	The feature code ID.
[out] pnEntryNums	The pointer that point to the number returned.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32EntryNums = 0;
emStatus = GXGetEnumEntryNums(hDevice, GX_ENUM_GAIN_AUTO, &ui32EntryNums);
```

7.5.4.19. GXGetEnumDescription

Declarations:

GX_API GXGetEnumDescription(GX_DEV_HANDLE hDevice,
GX_FEATURE_ID featureID,
GX_ENUM_DESCRIPTION* pEnumDescription,
size_t* pnBufferSize)

Descriptions:

To get the description information of the enumerated type values: the number of enumerated items and the value and descriptions of each item, please reference GX_ENUM_DESCRIPTION.

Parameters:

[in] hDevice	The handle of the device.
[in] featureID	The feature code ID.
[out] pEnumDescription	The array pointer, used for the enumeration description information returned.
[in,out] pnBufferSize	The size of the GX_ENUM_DESCRIPTION array that the user introduces, unit: byte.
If pEnumDescription is NULL:	
[out] pnBufferSize	The actual size of the buffer needed.
If pEnumDescription is not NULL:	

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

[in] pnBufferSize	The size of the buffer that the user allocated.
[out] pnBufferSize	Return the actual filled buffer size.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.
GX_STATUS_NEED_MORE_BUFFER	The buffer that the user allocates is too small.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32EntryNums = 0;

//Gets the number of this types supported by the device.
emStatus = GXGetEnumEntryNums(hDevice, GX_ENUM_GAIN_AUTO,
                              &ui32EntryNums);

//Applies memory according to the number.
GX_ENUM_DESCRIPTION* pstEnumDescrption = new GX_ENUM_DESCRIPTION[ui32EntryNums];

size_t nSize = ui32EntryNums * sizeof(GX_ENUM_DESCRIPTION);
emStatus = GXGetEnumDescription(hDevice, GX_ENUM_GAIN_AUTO, pstEnumDescrption,
                              &nSize);

//After finished
If (NULL != pstEnumDescrption)
{
   delete[] pstEnumDescrption;
   pstEnumDescrption = NULL;
}
```

7.5.4.20. GXGetEnum

Declarations:

```
GX_API GXGetEnum (GX_DEV_HANDLE hDevice,
                  GX_FEATURE_ID featureID,
                  int64_t* pnValue)
```

Descriptions:

To get the current enumeration value.

Parameters:

[in] hDevice	The handle of the device.
[in] featureID	The feature code ID.
[out] pnValue	The pointer that point to the return values.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.
GX_STATUS_INVALID_ACCESS	Currently inaccessible, can not read.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
int64_t i64Value = 0;
emStatus = GXGetEnum(hDevice, GX_ENUM_GAIN_AUTO, &i64Value);
```

7.5.4.21. GXSetFloat

Declarations:

```
GX_API GXSetFloat (GX_DEV_HANDLE hDevice,
                   GX_FEATURE_ID featureID,
                   double dValue)
```

Descriptions:

Set the value of float type.

Parameters:

[in] hDevice	The handle of the device.
[in] featureID	The feature code ID.
[in] dValue	The float value that the user will set.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_OUT_OF_RANGE	The value that the user introduces is across the border, smaller than the minimum, or larger than the maximum.
GX_STATUS_INVALID_ACCESS	Currently inaccessible, can not write.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
double dValue = 3000;
emStatus = GXSetFloat(hDevice, GX_FLOAT_EXPOSURE_TIME, dValue);
```

7.5.4.22. GXGetFloatRange

Declarations:

```
GX_API GXGetFloatRange (GX_DEV_HANDLE hDevice,
                        GX_FEATURE_ID featureID,
                        GX_FLOAT_RANGE* pFloatRange)
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Descriptions:

To get the minimum value, maximum value, steps and unit of the float type.

Parameters:

[in] hDevice	The handle of the device.
[in] featureID	The feature code ID.
[out] pFloatRange	The description structure pointer of float type. Reference the GX_FLOAT_RANGE.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.
GX_STATUS_INVALID_ACCESS	Currently inaccessible, can not read the range of the float type.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_FLOAT_RANGE stFloatRange;
emStatus = GXGetFloatRange(hDevice, GX_FLOAT_EXPOSURE_TIME,
                           &stFloatRange);
```

7.5.4.23. GXGetFloat

Declarations:

```
GX_API GXGetFloat (GX_DEV_HANDLE hDevice,
                   GX_FEATURE_ID featureID,
                   double* pdValue)
```

Descriptions:

Get the value of float type.

Parameters:

[in] hDevice	The handle of the device.
[in] featureID	The feature code ID.
[out] pdValue	Point to the pointer of the float value returned.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.
GX_STATUS_INVALID_ACCESS	Currently inaccessible, can not read.

The errors that are not covered above please reference GX_STATUS_LIST.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
double dValue = 0;
emStatus = GXGetFloat(hDevice, GX_FLOAT_EXPOSURE_TIME, &dValue);
```

7.5.4.24. GXSetInt

Declarations:

GX_API GXSetInt(GX_DEV_HANDLE hDevice, GX_FEATURE_ID featureID, int64_t nValue)

Descriptions:

Set the value of int type.

Parameters:

[in] hDevice	The handle of the device.
[in] featureID	The feature code ID.
[in] nValue	The value that the user will set.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_OUT_OF_RANGE	The value that the user introduces is across the border, smaller than the minimum, or larger than the maximum, or is not an integer multiple of the step.
GX_STATUS_INVALID_ACCESS	Currently inaccessible, can not write.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
int64_t i64Value = 200;
emStatus = GXSetInt(hDevice, GX_INT_WIDTH, i64Value);
```

7.5.4.25. GXGetFeatureName

Declarations:

GX_API GXGetFeatureName (GX_DEV_HANDLE hDevice, GX_FEATURE_ID featureID,
char* pszName, size_t* pnSize)

Descriptions:

Get the string description for the feature code.

Parameters:

[in] hDevice	The handle of the device.
[in] featureID	The feature code ID.
[out] pszName	The character string buffer address that the user inputs. The character string length includes the end terminator '\0'.
[in,out] pnSize	The length of the character string buffer address that the user inputs. Unit: byte.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

If pszName is NULL:

[out] pnSize	Return the actual size of the character string.

If pszName is not NULL:

[in] pnSize	The size of the buffer that the user allocated.
[out] pnSize	Return the actual filled buffer size.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.
GX_STATUS_NEED_MORE_BUFFER	The buffer that the user allocated is too small.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

char* pszName = NULL;
size_t nSize = 0;
//First, the user passed the NULL pointer to get the actual size, followed
//by the request buffer to obtain the description information.
GXGetFeatureName(hDevice, GX_FLOAT_GAIN, NULL, &nSize);
pszName = new char[nSize];
emStatus = GXGetFeatureName(hDevice, GX_FLOAT_GAIN, pszName, &nSize);

//After finished
if(NULL != pszName)
{
   delete[] pszName;
   pszName = NULL;
}
```

7.5.4.26. GXIsImplemented

Declarations:

GX_API GXIsImplemented (GX_DEV_HANDLE hDevice, GX_FEATURE_ID featureID,
bool* pbIsImplemented)

Descriptions:

Inquire the current camera whether support a special feature. Usually the camera doesn't support a feature means that:

1) By inquiring the camera register, the current camera really does not support this feature.
2) There is no description of this feature in the current camera description file.

Parameters:

[in] hDevice	The handle of the device.
[in] featureID	The feature code ID.
[out] pbIsImplemented	To return the result whether is support this feature. If support, then returns **true**, if not support, **false** will return.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
bool bIsImplemented = false;
//The parameter hDevice has been obtained via GXOpenDevice, and is
//no longer described later.
emStatus = GXIsImplemented(hDevice, GX_FLOAT_GAIN, &bIsImplemented);
```

7.5.4.27. GXIsReadable

Declarations:

GX_API GXIsReadable(GX_DEV_HANDLE hDevice, GX_FEATURE_ID featureID,
bool* pbIsReadable)

Descriptions:

Inquire if a feature code is currently readable.

Parameters:

[in] hDevice	The handle of the device.
[in] featureID	The feature code ID.
[out] pbIsReadable	To return the result whether the feature code ID is readable. If readable, then will return **true**, if not readable, **false** will return.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
bool bIsReadable = false;
emStatus = GXIsReadable(hDevice, GX_FLOAT_GAIN, &bIsReadable);
```

7.5.4.28. GXIsWritable

Declarations:

GX_API GXIsWritable(GX_DEV_HANDLE hDevice, GX_FEATURE_ID featureID, bool* pbIsWritable)

Descriptions:

Inquire if a feature code is currently writable.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Parameters:

[in] hDevice	The handle of the device.
[in] featureID	The feature code ID.
[out] pbIsWritable	To return the result whether the feature code ID is writable. If writable, then will return **true**, if not writable, **false** will return.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
bool bIsWritable = false;
emStatus = GXIsWritable(hDevice, GX_FLOAT_GAIN, &bIsWritable);
```

7.5.4.29. GXGetIntRange

Declarations:

GX_API GXGetIntRange (GX_DEV_HANDLE hDevice, GX_FEATURE_ID featureID,
GX_INT_RANGE* pIntRange)

Descriptions:

To get the minimum value, maximum value and steps of the int type.

Parameters:

[in] hDevice	The handle of the device.
[in] featureID	The feature code ID.
[out] pIntRange	The structure of range description. Reference GX_INT_RANGE.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.
GX_STATUS_INVALID_ACCESS	Currently inaccessible, can not read the int range.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_INT_RANGE stIntRange;
emStatus = GXGetIntRange(hDevice, GX_INT_WIDTH, &stIntRange);
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

7.5.4.30. GXGetInt

Declarations:

GX_API GXGetInt (GX_DEV_HANDLE hDevice,
GX_FEATURE_ID featureID,
int64_t* pnValue)

Descriptions:

Get the current value of the int type.

Parameters:

[in] hDevice	The handle of the device.
[in] featureID	The feature code ID.
[out] pnValue	Point to the pointer of the current value returned.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_HANDLE	The handle that the user introduces is illegal.
GX_STATUS_NOT_IMPLEMENTED	The feature that is not support currently.
GX_STATUS_ERROR_TYPE	The featureID type that the user introduces is error.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.
GX_STATUS_INVALID_ACCESS	Currently inaccessible, can not read.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
int64_t i64Value = 0;
emStatus = GXGetInt(hDevice, GX_INT_WIDTH, &i64Value);
```

7.5.4.31. GXGetDeviceIPInfo

Declarations:

GX_API GXGetDeviceIPInfo(uint32_t nIndex, GX_DEVICE_IP_INFO* pstDeviceIPInfo)

Descriptions:

Specify the device index to obtain the network information of the device.

Parameters:

[in] nIndex	The serial number of the device.
[out] pstDeviceIPInfo	The structure pointer of the device information.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_PARAMETER	The index that the user input is cross the border.

The errors that are not covered above please reference GX_STATUS_LIST.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Precautions:

You should call the GxUpdateAllDeviceList() interface for an enumeration before calling the function to get the device information. Otherwise, the device information that the user gets is inconsistent with the device that is currently connected.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32DeviceNum = 0;
emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
if (emStatus == GX_STATUS_SUCCESS &&(ui32DeviceNum > 0))
{
   GX_DEVICE_IP_INFO stIPInfo;

   //Gets the network information of the first device.
   emStatus = GXGetDeviceIPInfo(1, &stIPInfo);
}
```

7.5.4.32. GXGetAllDeviceBaseInfo

Declarations:

GX_API GXGetAllDeviceBaseInfo (GX_DEVICE_BASE_INFO* pDeviceInfo, size_t* pnBufferSize)

Descriptions:

To get the basic information of all devices.

Parameters:

[out] pDeviceInfo	The structure pointer of the device information.
[in,out] pnBufferSize	The buffer size of device information structure, unit: byte.
If pDeviceInfo is NULL:	
[out] pnBufferSize	Return the actual size of the device information.
If pDeviceInfo is not NULL:	
[in] pnBufferSize	The size of the buffer that the user allocated.
[out] pnBufferSize	Return the actual allocated buffer size.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.

The errors that are not covered above please reference GX_STATUS_LIST.

Precautions:

You should call the GxUpdateAllDeviceList() interface for an enumeration before calling the function to get the device information. Otherwise, the device information that the user gets is inconsistent with the device that is currently connected.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32DeviceNum = 0;
emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
if (emStatus == GX_STATUS_SUCCESS && (ui32DeviceNum > 0))
{
   GX_DEVICE_BASE_INFO *pstBaseinfo =
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
                                        new GX_DEVICE_BASE_INFO[ui32DeviceNum];
    size_t nSize = ui32DeviceNum * sizeof(GX_DEVICE_BASE_INFO);

    //Gets the basic information of all devices.
    emStatus = GXGetAllDeviceBaseInfo(pstBaseinfo, &nSize);

    //After finished
    If (NULL != pstBaseinfo)
    {
        delete[]pstBaseinfo;
        pstBaseinfo = NULL;
    }
}
```

7.5.4.33. GXOpenDeviceByIndex

Declarations:

GX_API GXOpenDeviceByIndex (uint32_t nDeviceIndex, GX_DEV_HANDLE* phDevice)

Descriptions:

Open the device by index, starting from 1.

Parameters:

[in] nDeviceIndex	The index of the device starts from 1, for example: 1, 2, 3, 4...
[out] phDevice	Device handle returned by the interface.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.
GX_STATUS_OUT_OF_RANGE	The index of the user input is bigger than the available devices number.

The errors that are not covered above please reference GX_STATUS_LIST.

Precautions:

The index of the device starts from 1.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

GX_DEV_HANDLE hDevice = NULL;

//Opens the first device.
emStatus = GXOpenDeviceByIndex(1, &hDevice);
```

7.5.4.34. GXUpdateDeviceList

Declarations:

GX_API GXUpdateDeviceList (uint32_t* punNumDevices,
uint32_t nTimeOut)

Descriptions:

Enumerating currently all available devices in subnet and gets the number of devices.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Parameters:

[out] punNumDevices	The address pointer used to return the number of devices, and the pointer can not be NULL.
[in] nTimeOut	The timeout time of enumeration (unit: ms). If the device is successfully enumerated within the specified timeout time, the value returns immediately. If the device is not enumerated within the specified timeout time, then it waits until the specified timeout time is over and then it returns.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL.

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32DeviceNum = 0;

//Enumerates to get the number of devices. The timeout time is
//decided by the user's environment, and can be set by the user,
//1000ms is an example.
emStatus = GXUpdateDeviceList(&ui32DeviceNum, 1000);
```

7.5.4.35. GXGetFeatureNameList

Declarations:

```
GX_API GXGetFeatureNameList (GX_PORT_HANDLE      hPort,
                             GX_FEATURE_NAME *   pFeatureNameList,
                             uint32_t *          pCount)
```

Descriptions:

Get all node names.

Parameters:

[in] *hPort*	Handle
[in] *pFeatureNameList*	Node name storage location
[in, out] *pCount*	Given the capacity of the pFeatureNameList array, return the actual number of writes

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs.
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called.
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL
GX_STATUS_INVALID_HANDLE	The user enters an illegal handle or closes a device that has already been shut down

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_FEATURE_NAME stFeatureNameList;
uint32_t ui32Count = 0;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
emStatus = GXGetFeatureNameList(hDevice, &stFeatureNameList,
                                &ui32Count);
```

7.5.4.36. GXGetFeatureName

Declarations:

GX_API GXGetFeatureName (GX_DEV_HANDLE hDevice,
GX_FEATURE_ID_CMD featureID,
char* pszName,
size_t* pnSize)

Descriptions:

Obtain the string description corresponding to the function control code.

Parameters:

[in] *hDevice*	Device handle
[in] *featureID*	Feature ID
[in] *pszName*	The string buffer address entered by the user, the string length including the terminator '\0'
[in, out] pnSize	The length of the string buffer address entered by the user, in bytes If the user inputs a null pszName: [out] pnSize returns the actual length required If the user inputs a non NULL pszName: [in] pnSize is the buffer size allocated to users [out] pnSize returns the actual size of the filled buffer

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL
GX_STATUS_INVALID_HANDLE	The user passed in an illegal handle
GX_STATUS_NEED_MORE_BUFFER	The buffer allocated by the user is too small

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_ENUM_VALUE stEnumValue;
GX_FEATURE_ID_CMD emFeatureID = GX_INT_TIMESTAMP_TICK_FREQUENCY;
size_t nSize =0;

emStatus = GXGetFeatureName(hDevice, emFeatureID, NULL, &nSize);

char* pFeatureName = new char[nSize];
emStatus = GXGetFeatureName(hDevice, emFeatureID, pFeatureName,
                            &nSize);

//......

delete[] pFeatureName;
```

7.5.4.37. GXGetFeatureType

Declarations:

GX_API GXGetFeatureType(GX_PORT_HANDLE hPort,
const char * strName,
GX_FEATURE_TYPE * type)

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Descriptions:

Get the type of node.

Parameters:

[in] *hPort*	Handle
[in] *strName*	Node name
[out] *type*	Node type

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called
GX_STATUS_INVALID_HANDLE	The user passed in an illegal handle
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL

The errors that are not covered above please reference GX_STATUS_LIST.

Sample Code:

```
GX_ENUM_VALUE     stEnumValue;
GX_FEATURE_ID_CMD emFeatureID = GX_INT_TIMESTAMP_TICK_FREQUENCY;
size_t             nSize =0;

emStatus = GXGetFeatureName(hDevice, emFeatureID, NULL, &nSize);

char* pFeatureName = new char[nSize];
emStatus = GXGetFeatureName(hDevice, emFeatureID, pFeatureName,
                            &nSize);

GX_FEATURE_TYPE emType;
emStatus = GXGetFeatureType(hDevice, pFeatureName, &emType);
if( NULL != pFeatureName)
    {
       delete[] pFeatureName;
       pFeatureName = NULL;
    }
```

7.5.4.38. GXRegisterCaptureCallback

Declarations:

GX_API GXRegisterCaptureCallback(GX_DEV_HANDLE hDevice, void *pUserParam,
GXCaptureCallBack callBackFun);

Descriptions:

Register capture callback function which corresponds to the GXUnregisterCaptureCallback() interface.

Parameters:

[in] *hDevice*	Device handle
[in] *pUserParam*	Points to the private data pointer that the user will use in the callback function. It can be obtained via the GX_FRAME_DATA_EX pUserParam.
[in, out] callBackFun	The callback function to be registered by the user. For the function type, please see GXCaptureCallBack()

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

GX_STATUS_INVALID_HANDLE	The user passed in an illegal handle
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL
GX_STATUS_INVALID_CALL	After sending the acquisition command, the capture callback function cannot be registered

The errors that are not covered above please reference GX_STATUS_LIST.

Precautions:

Callback functions cannot be registered after sending the acquisition command. If registration occurs after data capture has begun, the callback returns GX_STATUS_INVALID_CALL.

Sample Code:

```
#include "GxIAPI.h"

//Image processing callback function.
static void GX_STDC OnFrameCallbackFun(GX_FRAME_CALLBACK_PARAM* pFrame)
{
    if (pFrame-> status == GX_FRAME_STATUS_SUCCESS)
    {
              //Process images.
    }
    return;
}

int main(int argc, char* argv[])
{
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    GX_OPEN_PARAM stOpenParam;
    uint32_t ui32DeviceNum = 0;

    //Initialisation the library.
    emStatus = GXInitLib();
    if (emStatus != GX_STATUS_SUCCESS)
    {
        return 0;
    }

    // Enumerate the list of devices.
emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
if ((emStatus != GX_STATUS_SUCCESS)||(ui32DeviceNum<= 0))
{
    return 0;
}

// Turn on devices.
stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
stOpenParam.openMode = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emStatus = GXOpenDevice(&stOpenParam, &hDevice);
    if (emStatus == GX_STATUS_SUCCESS)
    {
        // Configure the packet size of the camera's stream channel to
        // improve the network camera's capture performance
```

```
            bool    bImplementPacketSize = false;
            uint32_t ui32PacketSize       = 0;

            // Determine whether the device supports the stream channel packet
            // feature
            GX_NODE_ACCESS_MODE emAccessMode;
            emStatus = GXGetNodeAccessMode(hDevice, "GevSCPSPacketSize",
                                           &emAccessMode);
            bImplementPacketSize = (emAccessMode ==
                                    GX_NODE_ACCESS_MODE_RW)? true : false;

            if (bImplementPacketSize)
            {
                // Obtain the optimal packet size for the current network
                // network environment
                emStatus = GXGetOptimalPacketSize (hDevice,
                                                   &ui32PacketSize);

                // Set the Optimal PacketSize to GevSCPSPacketSize
                emStatus = GXSetIntValue(hDevice, "GevSCPSPacketSize",
                                         ui32PacketSize);
            }
            // Register the image capture callback function
            emStatus = GXRegisterCaptureCallback(hDevice, NULL,
                                                 OnFrameCallbackFun);

            // Sending the AcquisitionStart command.
            emStatus = GXSetCommandValue(hDevice, "AcquisitionStart");
            //----------------------
            //
            // During this interval, the image is returned to the user via the
            // OnFrameCallbackFun interface
            //
            //----------------------
            //Send the AcquisitionStop command.
            emStatus = GXSetCommandValue (hDevice, "AcquisitionStop");

            // Unregister the capture callback function
            emStatus = GXUnregisterCaptureCallback(hDevice);
        }
        emStatus = GXCloseDevice(hDevice);
        emStatus = GXCloseLib();

        return 0;
    }
```

7.5.4.39. GXGetImage

Declarations:

GX_API GXGetImage(GX_DEV_HANDLE hDevice,
GX_FRAME_DATA *pFrameData, int32_t nTimeout);

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Descriptions:

Once capture has begun, images can be retrieved directly through this interface. Please note that this interface cannot be used in conjunction with the callback capture method.

Parameters:

[in] *hDevice*	Device handle
[in,out] *pFrameData*	The address pointer provided by the user to receive image
[in] *nTimeout*	Image retrieval timeout (ms)

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called
GX_STATUS_INVALID_HANDLE	The user passed in an illegal handle
GX_STATUS_INVALID_CALL	The user calls the GXGetImage() interface to retrieve the image after the capture callback function has been registered
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL

The errors that are not covered above please reference GX_STATUS_LIST.

Precautions:

After the capture callback function has been registered, the GXGetImage() interface must not be called, doing so will result in a GX_STATUS_INVALID_CALL error. When performing high-speed image acquisition with a high-resolution camera, the buffer copying performed internally by the GXGetImage() interface may affect transmission performance. Therefore, in such circumstances, the callback acquisition method is recommended.

Sample Code:

```
//------------------------------
// The GXGetImage interface retrieves a single frame at a time. This sample
// code demonstrates how to use this interface to retrieve a single image.
//------------------------------
#include "GxIAPI.h"

int main(int argc, char* argv[])
{
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    GX_OPEN_PARAM stOpenParam;
    uint32_t ui32DeviceNum = 0;

    // Initialisation the library.
    emStatus = GXInitLib();
    if (emStatus != GX_STATUS_SUCCESS)
    {
        return 0;
    }

    // Enumerate the list of devices.
emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
if ((emStatus != GX_STATUS_SUCCESS)||(ui32DeviceNum<= 0))
{
return 0;
}
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
// Turn on devices.
stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
stOpenParam.openMode = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emStatus = GXOpenDevice(&stOpenParam, &hDevice);
    if (emStatus == GX_STATUS_SUCCESS)
    {
        // Configure the packet size of the camera's stream channel to
        // improve the network camera's capture performance
        Bool bImplementPacketSize = false;
        uint32_t ui32PacketSize     = 0;

        // Determine whether the device supports the stream channel packet
        // feature
       GX_NODE_ACCESS_MODE emAccessMode;
        emStatus = GXGetNodeAccessMode(hDevice, "GevSCPSPacketSize",
                                       &emAccessMode);
       bImplementPacketSize = (emAccessMode == GX_NODE_ACCESS_MODE_RW)
                             ? true : false;

        if (bImplementPacketSize)
        {
            // Obtain the optimal packet size for the current network
            // network environment.
            emStatus = GXGetOptimalPacketSize (hDevice, &ui32PacketSize);

            // Set the Optimal PacketSize to GevSCPSPacketSize.
            emStatus = GXSetIntValue(hDevice, "GevSCPSPacketSize",
                                     ui32PacketSize);
        }

        GX_INT_VALUE stPayLoadSize;
        // Retrieve the image buffer size and dynamically allocate memory
        emStatus = GXGetIntValue(hDevice, "PayloadSize", &stPayLoadSize);

        if(emStatus==GX_STATUS_SUCCESS&&stPayLoadSize.nCurValue>0)
        {
            // Define the input parameters for GXGetImage.
            GX_FRAME_DATA stFrameData;

            // Allocate a buffer based on the image buffer size
            // (m_nPayLoadSize).
            stFrameData.pImgBuf= malloc((size_t)stPayLoadSize.nCurValue);

            // Sending the AcquisitionStart command.
            emStatus = GXSetCommandValue(hDevice, "AcquisitionStart");
            if (emStatus == GX_STATUS_SUCCESS)
            {
                // obtain a single image by calling the GXGetImage interface.
                while(GXGetImage(hDevice, &stFrameData, 100) != GX_STATUS_SUCCESS)
                {
                    Sleep(10);
                }
                if (stFrameData.nStatus == GX_FRAME_STATUS_SUCCESS)
                {
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
                    //Obtain the image successfully.
                    //process the image.
                }
            }

            // Sending the AcquisitionStop command.
            emStatus = GXSetCommandValue (hDevice, "AcquisitionStop");

            //Release the image buffer.
            free(stFrameData.pImgBuf);
        }
    }
    emStatus = GXCloseDevice(hDevice);
    emStatus = GXCloseLib();

    return 0;
}
```

7.5.4.40. GXDQBuf

Declarations:

GX_API GXDQBuf(GX_DEV_HANDLE hDevice,
GX_FRAME_BUFFER* ppFrameBuffer,
uint32_t nTimeOut);

Descriptions:

When capturing a single frame using the zero-copy method, once the image has been obtained, the GXQBuf() interface must be called to return the image to the capture system.

Parameters:

[in] *hDevice*	Device handle
[out] p*pFrameBuffer*	The address pointer for the image output by the interface.
[in] *nTimeout*	Image retrieval timeout (ms)

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called
GX_STATUS_INVALID_HANDLE	The user passed in an illegal handle
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL
GX_STATUS_INVALID_CALL	This interface must not be called unless a callback function has been registered or capture has been initiated.
GX_STATUS_TIMEOUT	Image retrieval timeout error.
GX_STATUS_ERROR	An unspecified internal error that was not expected to occur.

The errors that are not covered above please reference GX_STATUS_LIST.

Precautions:

1) The GXDQBuf() interface must not be called prior to capture. If called, it will return the GX_STATUS_INVALID_CALL error.
2) After the capture callback function has been registered, the GXDQBuf() interface must not be called, doing so will return the GX_STATUS_INVALID_CALL error.
3) The GXDQBuf() interface must be used in conjunction with the GXQBuf() interface. Otherwise, continuous image capture will not be possible.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Sample Code:

```
//--------------------------------
// The GXGetImage interface retrieves a single image at a time. This sample
// code demonstrates how to use this interface to retrieve a single image.
//--------------------------------
#include "GxIAPI.h"

int main(int argc, char* argv[])
{
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    uint32_t ui32DeviceNum = 0;

    // Initialisation the library.
    emStatus = GXInitLib();
    if (emStatus != GX_STATUS_SUCCESS)
    {
              return 0;
    }

    // Enumerate the list of devices.
        emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
        if ((emStatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
        {
              return 0;
        }

        // Turn on devices.
    GX_OPEN_PARAM stOpenParam;
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode = GX_OPEN_INDEX;
   stOpenParam.pszContent = "1";
   emStatus = GXOpenDevice(&stOpenParam, &hDevice);
   if (emStatus == GX_STATUS_SUCCESS)
   {
      //Define the input parameters of GXDQBuf.
      GX_FRAME_BUFFER pFrameBuffer;

      // Acquisition start.
      #ifdef __linux__
      emStatus = GXStreamOn(hDevice);
      #else
      emStatus = GXSetCommandValue(hDevice, "AcquisitionStart");
      #endif
      if (emStatus == GX_STATUS_SUCCESS)
      {
          //Get one single frame through GXDQBuf.
          emStatus = GXDQBuf(hDevice, &pFrameBuffer, 1000);
          if (emStatus == GX_STATUS_SUCCESS)
            {
                 if(pFrameBuffer-> nStatus==GX_FRAME_STATUS_SUCCESS)
                 {
                   // Obtain images successfully.
                   // Process images.
                 }

                // Call GXQBuf to return the image buffer to the library
                // so that image capture can continue.
                emStatus = GXQBuf(hDevice, pFrameBuffer);
            }
      }
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
        // Acquisition stop.
        #ifdef __linux__
          emStatus = GXStreamOff(hDevice);
        #else
          emStatus = GXSetCommandValue(hDevice, "AcquisitionStop");
        #endif
      }
      emStatus = GXCloseDevice(hDevice);
      emStatus = GXCloseLib();
      return 0;
  }
```

7.5.4.41. GXQBuf

Declarations:

GX_API GXQBuf(GX_DEV_HANDLE hDevice,
GX_FRAME_BUFFER pFrameBuffer);

Descriptions:

Return the image that obtained through the GXDQBuf() interface (zero-copy) to capture system.

Parameters:

[in] *hDevice*	Device handle.
[in] *pFrameBuffer*	The buffer pointer for the image to be returned to the GxIAPI library.

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called
GX_STATUS_INVALID_HANDLE	The user passed in an illegal handle
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL or an invalid pointer
GX_STATUS_INVALID_CALL	This interface may not be called unless a callback has been registered or acquisition has commenced.

The errors that are not covered above please reference GX_STATUS_LIST.

Precautions:

1) The GXDQBuf() interface must not be called prior to capture. If called, it will return the GX_STATUS_INVALID_CALL error.
2) After the capture callback function has been registered, the GXDQBuf() interface must not be called, doing so will return the GX_STATUS_INVALID_CALL error.
3) The GXDQBuf() interface must be used in conjunction with the GXQBuf() interface. Otherwise, continuous image capture will not be possible.
4) Once the GXQBuf() interface has returned the image buffer to the GxIAPI library, the image buffer pointer can no longer be accessed.

Sample Code:

Please see the sample code for GXDQBuf().

7.5.4.42. GXDQAllBufs (only Linux)

Declarations:

GX_API GXDQAllBufs(GX_DEV_HANDLE hDevice,
GX_FRAME_BUFFER *ppFrameBufferArray,
uint32_t nFrameBufferArraySize,
uint32_t *pnFrameCount,

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

uint32_t nTimeOut);

Descriptions:

Once acquisition has begun, return the buffers for all images obtained using the GXDQBuf() interface (zero-copy) to the acquisition system. The images are stored in the acquired image data array in chronological order from oldest to newest. That is, ppFrameBufferArray[0] contains the oldest image, and ppFrameBufferArray[nFrameCount - 1] contains the most recent image.

Parameters:

[in] *hDevice*	Device handle.
[out] *ppFrameBufferArray*	The array for image pointers.
[in] *nFrameBufferArraySize*	Number of image arrays requested.
[out] *pnFrameCount*	Returns the actual number of images filled.
[in] nTimeOut	Image retrieval timeout (ms)

Returns:

GX_STATUS_SUCCESS	The operation is successful and no error occurs
GX_STATUS_NOT_INIT_API	The GXInitLib() initialization library is not called
GX_STATUS_INVALID_HANDLE	The user passed in an illegal handle
GX_STATUS_INVALID_PARAMETER	The pointer that the user input is NULL
GX_STATUS_INVALID_CALL	This interface may not be called unless a callback has been registered or acquisition has commenced.
GX_STATUS_NEED_MORE_BUFFER	Insufficient buffer requested by the user. The buffer size specified by the user is smaller than the actual requirement.
GX_STATUS_TIMEOUT	Image retrieval timeout error.
GX_STATUS_ERROR	An unspecified internal error that was not expected to occur.

The errors that are not covered above please reference GX_STATUS_LIST.

Precautions:

1) The GXDQBuf() interface must not be called prior to capture. If called, it will return the GX_STATUS_INVALID_CALL error.
2) After the capture callback function has been registered, the GXDQBuf() interface must not be called, doing so will return the GX_STATUS_INVALID_CALL error.
3) The GXDQBuf() interface must be used in conjunction with the GXQBuf() interface. Otherwise, continuous image capture will not be possible.
4) The size of the array of image pointers must be greater than or equal to the number of image acquisition buffers (the default value is 5). Otherwise, a GX_STATUS_NEED_MORE_BUFFER error will be returned.

Sample Code:

```
#include "GxIAPI.h"
#include <stdint.h>

int main(int argc, char* argv[])
{
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    uint32_t ui32DeviceNum = 0;

    // Initialisation the library.
    emStatus = GXInitLib();
    if (status != GX_STATUS_SUCCESS)
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
        {
            return 0;
        }
        // Enumerate the list of devices.
        emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
        if ((emStatus != GX_STATUS_SUCCESS) || (emStatus <= 0))
        {
            return 0;
        }

        // Turn on devices.
        GX_OPEN_PARAM stOpenParam;
stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
stOpenParam.openMode = GX_OPEN_INDEX;
        stOpenParam.pszContent = "1";
        emStatus = GXOpenDevice(&stOpenParam, &hDevice);
        if (emStatus == GX_STATUS_SUCCESS)
        {
            // Define an array to receive images.
            // The default number of image acquisition buffers is 5.The size of the image
            // array must be greater than or equal to the number of image acquisition
            // buffers.
            GX_FRAME_BUFFER pFrameBuffer[5];

           //Define the actual number of images filled.
           uint32_t ui32FrameCount = 0;
           // Acquisition start.
           emStatus = GXStreamOn(hDevice);
           if (emStatus == GX_STATUS_SUCCESS)
           {
              // Get all images in the image array by calling the GXDQAllBufs interface.
               emStatus = GXDQAllBufs(hDevice, pFrameBuffer,5,&nFrameCount ,
                                      1000);
               if (emStatus == GX_STATUS_SUCCESS)
               {
                  for (int i = 0; i < ui32FrameCount; i++)
                  {
                     if (pFrameBuffer[i] != NULL &&
                         pFrameBuffer[i]-> nStatus == GX_FRAME_STATUS_SUCCESS)
                     {
                           // Successfully get the image i.
                           // Process the image.
                     }
                  }
                   // Calling GXQAllBufs interface returns all the image buffers
                   // obtained to the library so that image capture can continue.
                   emStatus = GXQAllBufs(hDevice);
                   }
           }

            // Acquisition stop.
            emStatus = GXStreamOff(hDevice);
        }
```

```
        emStatus = GXCloseDevice(hDevice);
        emStatus = GXCloseLib();

        return 0;
    }
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

8. Image Processing Interface Description

The image processing interface mainly includes image pixel format conversion interface and color quality enhancement interface. The prototype of the image processing interface function is declared in the DxImageProc.h file.

8.1. Type

8.1.1. Data Type

Name	Description
VxInt8	8-bit signed integer
VxInt16	16-bit signed integer
VxInt32	32-bit signed integer
VxInt64	64-bit signed integer
VxUint8	8-bit unsigned integer
VxUint16	16-bit unsigned integer
VxUint32	32-bit unsigned integer

8.2. Constant

8.2.1. Status code

```
typedef enum    tagDX_STATUS
{
    DX_OK                                = 0,
    DX_PARAMETER_INVALID                 = -101,
    DX_PARAMETER_OUT_OF_BOUND            = -102,
    DX_NOT_ENOUGH_SYSTEM_MEMORY          = -103,
    DX_NOT_FIND_DEVICE                   = -104,
    DX_STATUS_NOT_SUPPORTED              = -105,
    DX_CPU_NOT_SUPPORT_ACCELERATE        = -106
} DX_STATUS;
```

Name	Description
DX_OK	Successful operation
DX_PARAMETER_INVALID	Invalid input parameter
DX_PARAMETER_OUT_OF_BOUND	The parameter is out of bound
DX_NOT_ENOUGH_SYSTEM_MEMORY	The system does not have enough memory
DX_NOT_FIND_DEVICE	No device found
DX_STATUS_NOT_SUPPORTED	The format is not supported.
DX_CPU_NOT_SUPPORT_ACCELERATE	The CPU does not support acceleration.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

DX_PARAMETER_NOT_INITIALIZED	Parameter not initialized
DX_FILE_OPEN_FAIL	File opening failed
DX_FILE_READ_FAIL	File reading failed
DX_FILE_VERSION_NOSUPPORT	File version number error
DX_FILE_CONTENT_NOT_CALIB	The file content contains parameters that have not been calibrated
DX_FILE_CONTENT_NOT_ENOUGH	File content missing
DX_FILE_CONTENT_MODEL_ERR	Calibration model error
DX_FILE_CONTENT_NOT_LOAD	No calibration file loaded
DX_FILE_CONTENT_ERR	File content error
DX_CALC_ERR	Calculation error

8.2.2. Pixel Bayer format

```
typedef enum tagDX_PIXEL_COLOR_FILTER
{
    NONE    = 0,
    BAYERRG = 1,
    BAYERGB = 2,
    BAYERGR = 3,
    BAYERBG = 4
} DX_PIXEL_COLOR_FILTER;
```

Name	Description
NONE	Non-Bayer format
BAYERRG	The first line of the Raw image starts with RG
BAYERGB	The first line of the Raw image starts with GB
BAYERGR	The first line of the Raw image starts with GR
BAYERBG	The first line of the Raw image starts with BG

8.2.3. Bayer conversion type

```
typedef enum tagDX_BAYER_CONVERT_TYPE
{
    RAW2RGB_NEIGHBOUR = 0,
    RAW2RGB_ADAPTIVE = 1,
    RAW2RGB_NEIGHBOUR3 = 2,
    RAW2RGB_WEIGHT = 3
} DX_BAYER_CONVERT_TYPE;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Name	Description
RAW2RGB_NEIGHBOUR	Neighborhood average interpolation algorithm
RAW2RGB_ADAPTIVE	Edge adaptive interpolation algorithm
RAW2RGB_NEIGHBOUR3	Neighborhood average interpolation algorithm for larger regions
RAW2RGB_WEIGHT	Weight interpolation algorithm

8.2.4. Valid data bit

```
typedef enum tagDX_VALID_BIT
{
    DX_BIT_0_7      = 0,
    DX_BIT_1_8      = 1,
    DX_BIT_2_9      = 2,
    DX_BIT_3_10         = 3,
    DX_BIT_4_11         = 4,
} DX_VALID_BIT;
```

If the original Raw image is 8 bits (0~7), you can only select the DX_BIT_0_7 algorithm; If the original image is 10 bits (0~9), when you conversion it to 8 bits, you can select the DX_BIT_0_7, DX_BIT_1_8 and DX_BIT_2_9 algorithms; If the original image is 12 bits (0~11), when you conversion it to 8 bits, you can select the DX_BIT_0_7, DX_BIT_1_8, DX_BIT_2_9, DX_BIT_3_10 and DX_BIT_4_11 all the five algorithms.

Name	Description
DX_BIT_0_7	Takes the 0~7bit
DX_BIT_1_8	Takes the 1~8bit
DX_BIT_2_9	Takes the 2~9bit
DX_BIT_3_10	Takes the 3~10bit
DX_BIT_4_11	Takes the 4~11bit

8.2.5. The actual image bit depth

```
typedef enum tagDX_ACTUAL_BITS
{
    DX_ACTUAL_BITS_8    = 8,
    DX_ACTUAL_BITS_10 = 10,
    DX_ACTUAL_BITS_12 = 12,
    DX_ACTUAL_BITS_14 = 14,
    DX_ACTUAL_BITS_16 = 16,
} DX_ACTUAL_BITS;
```

The actual bit depth of the image data, which is the number of bytes occupied by per pixel.

Name	Description
DX_ACTUAL_BITS_8	8 bits
DX_ACTUAL_BITS_10	10 bits
DX_ACTUAL_BITS_12	12 bits

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

DX_ACTUAL_BITS_14	14 bits
DX_ACTUAL_BITS_16	16 bits

8.2.6. The image mirror and flip type

```
typedef enum DX_IMAGE_MIRROR_MODE
{
    HORIZONTAL_MIRROR = 0,
    VERTICAL_MIRROR   = 1,
}DX_IMAGE_MIRROR_MODE;
```

Name	Description
HORIZONTAL_MIRROR	Horizontal mirror
VERTICAL_MIRROR	Vertical mirror

8.2.7. Color image RGB order

```
typedef enum DX_RGB_CHANNEL_ORDER
{
    DX_ORDER_RGB = 0,
    DX_ORDER_BGR = 1
}DX_RGB_CHANNEL_ORDER;
```

Name	Description
DX_ORDER_RGB	The color image sequence is RGB
DX_ORDER_BGR	The color image sequence is BGR

8.2.8. Image Format Conversion Handle

```
#define IMAGE_CONVERT_DECLARE_HANDLE(name) \
struct name##_; typedef struct name##_ *name
IMAGE_CONVERT_DECLARE_HANDLE(DX_IMAGE_FORMAT_CONVERT_HANDLE);
```

8.2.9. IMAGE_CONVERT_DECLARE_HANDLE(DX_IMAGE_FORMAT_CONVERT_HANDLE);System Parameter

```
typedef enum DX_SYSTEM_PARAM
{
    THREAD_NUM = 0,
    SSSE3_ENABLE = 1,
    NEON_ENABLE = 2
} DX_SYSTEM_PARAM;
```

Name	Description
THREAD_NUM	Number of threads
SSSE3_ENABLE	SSSE3 Enable
NEON_ENABLE	NEON Enable

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

8.2.10. Flat Field Correction Handle

```
#define FLAT_FIELD_CORRECTION_DECLARE_HANDLE(name) \
struct name##_; typedef struct name##_ *name
FLAT_FIELD_CORRECTION_DECLARE_HANDLE(DX_FLAT_FIELD_CORRECTION_HANDLE);
```

8.2.11. Decompression Handle

```
#define DECOMPRESSION_DECLARE_HANDLE(name) \
struct name##_; typedef struct name##_ *name
DECOMPRESSION_DECLARE_HANDLE(DX_DECOMPRESSION_HANDLE);
```

8.3. Structure

8.3.1. Monochrome image process function set structure

```
typedef struct MONO_IMG_PROCESS
{
    bool        bDefectivePixelCorrect;
    bool        bSharpness;
    bool        bAccelerate;
    float fSharpFactor;
    VxUint8     *pProLut;
    VxUint16    nLutLength;
    VxUint8     arrReserved[32];
} MONO_IMG_PROCESS;
```

Input the parameters by the DxMono8ImgProcess interface.

Name	Description
bDefectivePixelCorrect	Dead pixel correction switch
bSharpness	Sharpening switch
bAccelerate	The enabling state of an accelerated function
fSharpFactor	Sharpening intensity factor
pProLut	Look-up table buffer pointer
nLutLength	The length of the look-up table buffer
arrReserved[32]	Reserved 32 bytes

8.3.2. Color image process function set structure

```
typedef struct COLOR_IMG_PROCESS
{
    bool        bDefectivePixelCorrect;
    bool        bDenoise;
    bool        bSharpness;
    bool        bAccelerate;
    VxInt16     *parrCC;
    VxUint8     nCCBufLength;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    Float                   fSharpFactor;
    VxUint8                 *pProLut;
    VxUint16                nLutLength;
    DX_BAYER_CONVERT_TYPEcvType;
    DX_PIXEL_COLOR_FILTER   emLayOut;
    bool                    bFlip;
    VxUint8                 arrReserved[32];
} COLOR_IMG_PROCESS;
```

Input the parameters by the DxRaw8ImgProcess interface.

Name	Description
bDefectivePixelCorrect	Dead pixel correction switch
bDenoise	Noise reduction switch
bSharpness	Sharpening switch
bAccelerate	The enabling state of an accelerated function
*parrCC	Color processing parameter array address
nCCBufLength	The length of parrCC (sizeof (VxInt16) *9)
fSharpFactor	Sharpening intensity factor
pProLut	Look-up table buffer pointer
nLutLength	The length of the look-up table buffer
cvType	Interpolation method
emLayOut	BAYER format
bFlip	Flip sign
arrReserved[32]	Reserved 32 bytes

8.3.3. Flat field correction process function set structure

```
typedef  struct  FLAT_FIELD_CORRECTION_PROCESS
{
        void                        *pBrightBuf;
        void                        *pDarkBuf;
        VxUint32                    nImgWid;
        VxUint32                    nImgHei;
        DX_ACTUAL_BITS              nActualBits;
        DX_PIXEL_COLOR_FILTER       emBayerType;
} FLAT_FIELD_CORRECTION_PROCESS;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Input the parameters by the DxGetFFCCoefficients interface.

Name	Description
pBrightBuf	Bright image buffer
pDarkBuf	Dark image buffer
nImgWid	Image width
nImgHei	Image height
nActualBits	Image actual bits
emBayerType	BAYER format

8.3.4. Color correction UserSet process function set struct

```
typedef  struct  COLOR_TRANSFORM_FACTOR
{
    float fGain00;
    float fGain01;
    float fGain02;

    float fGain10;
    float fGain11;
    float fGain12;

    float fGain20;
    float fGain21;
    float fGain22;
} COLOR_TRANSFORM_FACTOR;
```

Input the parameters by the DxCalcUserSetCCParam interface.

Name	Description
fGain00	Red gain applying to red Pixel
fGain01	Red gain applying to green Pixel
fGain02	Red gain applying to blue Pixel
fGain10	Green gain applying to red Pixel
fGain11	Green gain applying to green Pixel
fGain12	Green gain applying to blue Pixel
fGain20	Blue gain applying to red Pixel
fGain21	Blue gain applying to green Pixel
fGain22	Blue gain applying to blue Pixel

8.3.5. Static defect correction function set structure

```
typedef  struct  STATIC_DEFECT_CORRECTION
{
    VxUint32                nImgWidth;
    VxUint32                nImgHeight;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    VxUint32                    nImgOffsetX;
    VxUint32                    nImgOffsetY;
    VxUint32                    nImgWidthMax;
    DX_PIXEL_COLOR_FILTER       nBayerType;
    DX_ACTUAL_BITS              emActualBits;
} STATIC_DEFECT_CORRECTION;
```

Input the parameters by the DxStaticDefectCorrection interface.

Name	Description
nImgWidth	Image width
nImgHeight	Image height
nImgOffsetX	Image X-offset
nImgOffsetY	Image Y-offset
nImgWidthMax	Maximum image width supported by the device
nBayerType	Bayer format
emActualBits	Image actual bits

8.3.6. Flat field correction function set structure

```
typedef  struct  FLAT_FIELD_CORRECTION_PARAMETER
{
        void                        *pBrightBuf;
        void                        *pDarkBuf;
        GX_PIXEL_FORMAT_ENTRY       emPixelFormat;
        VxInt32                     nImgWid;
        VxInt32                     nImgHei;
        VxInt32                     nFFCBlockSize;
        VxInt32                     nFFCExpectedGray;
        VxInt32                     arrReserved[64];
} FLAT_FIELD_CORRECTION_PARAMETER;
```

Input the parameters by the DxGetFFCCoefficients interface.

Name	Description
pBrightBuf	Bright-field image Buffer
pDarkBuf	Dark-field image Buffer
emPixelFormat	Image pixel format
nImgWid	Image width
nImgHei	Image height
nFFCBlockSize	The size of the FFC block
nFFCExpectedGray	The expected gray value after FFC
arrReserved[64]	Reserved 256 bytes

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

8.4. Interfaces

8.4.1. DxRaw12PackedToRaw16

Declarations:

VxInt32 DHDECL DxRaw12PackedToRaw16 (void* pInputBuffer, void* pOutputBuffer, VxUint32 nWidth, VxUint32 nHeight)

Descriptions:

This function is used to convert the Raw12 Packed format data to Raw16 format.

Parameters:

pInputBuffer	Point to the 12 bits Packed data buffer of the original image.
pOutputBuffer	Point to the data buffer of the target image, it needs to be new, not to use the original data buffer, and the size is image width * image height * 2.
nWidth	Image width.
nHeight	Image height.

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
    int32_t i32Width = 600;
    int32_t i32Height = 400;
   BYTE *pRaw16Buf = new BYTE[i32Width * i32Height * 2];
   // Open output buffer
   if(pRaw16Buf == NULL)
   {
       return ;
   }
   else
   {

      //Initializes the buffer.
       memset(pRaw16Buf, 0, i32Width * i32Height * 2 * sizeof(BYTE));
   }

   //pPacketRaw12Buf is the original image data
   VxInt32 DxStatus = DxRaw12PackedToRaw16(pPackedRaw12Buf, pRaw16Buf,
                                           i32Width, i32Height);
   if (DxStatus != DX_OK)
   {
       if (pRaw16Buf != NULL)
       {
           delete []pRaw16Buf;
           pRaw16Buf = NULL;
       }
       return ;
   }

   //Processing the data of the 16 bits raw image.
   //.....................
```

```
//Processing is complete.
if(pRaw16Buf != NULL)
{
    delete []pRaw16Buf;
    pRaw16Buf = NULL;
}
```

8.4.2. DxRaw10PackedToRaw16

Declarations:

VxInt32 DHDECL DxRaw10PackedToRaw16 (void* pInputBuffer, void* pOutputBuffer, VxUint32 nWidth, VxUint32 nHeight)

Descriptions:

This function is used to convert the Raw10 Packed format data to Raw16 format data.

Parameters:

pInputBuffer	Point to the 10 bits Packed data buffer of the original image.
pOutputBuffer	Point to the data buffer of the target image, it needs to be new, not to use the original data buffer, and the size is image width * image height * 2.
nWidth	Image width.
nHeight	Image height.

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
int32_t i32Width = 600;
int32_t i32Height = 400;
//Applies for the output buffer.
BYTE *pRaw16Buf = new BYTE[i32Width * i32Height * 2];
if(pRaw16Buf == NULL)
{
    return ;
}
else
{
    //Initializes the buffer.
    memset(pRaw16Buf, 0, i32Width * i32Height * 2 * sizeof(BYTE));
}

//pPacketRaw10Buf is the original image data
VxInt32 DxStatus = DxRaw10PackedToRaw16(pPackedRaw10Buf, pRaw16Buf,
                                        i32Width, i32Height);
if (DxStatus != DX_OK)
{
    if (pRaw16Buf != NULL)
    {
        delete []pRaw16Buf;
        pRaw16Buf = NULL;
    }

    return ;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    }

    //Processing the data of the 16 bits raw image.
    //..................

    //Processing is complete.
    if(pRaw16Buf != NULL)
    {
        delete []pRaw16Buf;
        pRaw16Buf = NULL;
    }
```

8.4.3. DxRaw8toRGB24

Declarations:

VxInt32 DHDECL DxRaw8toRGB24 (void *pInputBuffer, void *pOutputBuffer, VxUint32 nWidth, VxUint32 nHeight, DX_BAYER_CONVERT_TYPE cvtype, DX_PIXEL_COLOR_FILTER nBayerType, bool bFlip)

Descriptions:

This function is used to convert the Bayer image to RGB image.

Parameters:

pInputBuffer	Point to the 8 bits data buffer of the original image.
pOutputBuffer	Point to the data buffer (RGB data) of the target image, and the size is image width * image height * 3.
nWidth	Image width.
nHeight	Image height.
cvtype	The type of conversion algorithm.
nBayerType	The type of Bayer image format.
bFlip	If flip, the value is **true**, and the image will vertical flip; otherwise, the value is **false**, not flip.

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
    int32_t i32Width = 600;
    int32_t i32Height = 400;
    //Applies for the RGB data of the output image.
    BYTE *pRGB24Buf = new BYTE[i32Width * i32Height * 3];

    if (pRGB24Buf == NULL)
    {
        return ;
    }
    else
    {
        //Initializes the buffer.
        memset(pRGB24Buf, 0, i32Width * i32Height * 3 * sizeof(BYTE));
    }

    //Selects the interpolation algorithm.
```

```
    DX_BAYER_CONVERT_TYPE emType    = RAW2RGB_NEIGHBOUR;

    //Selects the image Bayer format.
    DX_PIXEL_COLOR_FILTER emBayerType = BAYERRG;
    bool bFlip = true;

    //PRaw8Buf is the original image data
    VxInt32 DxStatus = DxRaw8toRGB24(pRaw8Buf, pRGB24Buf, i32Width,
                                     i32Height, emType, emBayerType, bFlip);
    if (DxStatus != DX_OK)
    {
        if (pRGB24Buf != NULL)
        {
            delete []pRGB24Buf;
            pRGB24Buf = NULL;
        }

        return ;
    }

    //Processing the data of the 24 bits RGB image...................

    if (pRGB24Buf != NULL)
    {
        delete []pRGB24Buf;
        pRGB24Buf = NULL;
    }
```

8.4.4. DxRaw8toRGB24Ex

Declarations:

VxInt32 DHDECL DxRaw8toRGB24Ex (void *pInputBuffer, void *pOutputBuffer, VxUint32 nWidth, VxUint32 nHeight, DX_BAYER_CONVERT_TYPE cvtype, DX_PIXEL_COLOR_FILTER nBayerType, bool bFlip, DX_RGB_CHANNEL_ORDER emChannelOrder)

Descriptions:

This function is used to convert the Bayer image to RGB image, whose channel order can be selected as RGB or BGR.

Parameters:

pInputBuffer	Point to the 8 bits data buffer of the original image.
pOutputBuffer	Point to the data buffer of the target image, and the size is image width * image height * 3.
nWidth	Image width.
nHeight	Image height.
cvtype	The type of conversion algorithm.
nBayerType	The type of Bayer image format.
bFlip	If flip, the value is **true**, and the image will vertical flip; otherwise, the value is **false**, not flip.
emChannelOrder	RGB channel order of target image.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
int32_t i32Width = 600;
int32_t i32Height = 400;
//Applies for the BGR data of the output image.
BYTE *pBGR24Buf = new BYTE[i32Width * i32Height * 3];

if (pBGR24Buf == NULL)
{
    return ;
}
else
{
    //Initializes the buffer.
    memset(pBGR24Buf, 0, i32Width * i32Height * 3 * sizeof(BYTE));
}

//Selects the interpolation algorithm.
DX_BAYER_CONVERT_TYPE emType    = RAW2RGB_NEIGHBOUR;

//Selects the image Bayer format.
DX_PIXEL_COLOR_FILTER emBayerType = BAYERRG;
bool bFlip = true;
DX_RGB_CHANNEL_ORDER emChannelOrder = DX_ORDER_BGR;

//pRaw8Buf is the original image data
VxInt32 DxStatus = DxRaw8toRGB24Ex (pRaw8Buf, pBGR24Buf, i32Width,
                                    i32Height,emType, emBayerType,
                                   bFlip, emChannelOrder);
if (DxStatus != DX_OK)
{
    if (pBGR24Buf != NULL)
    {
        delete []pBGR24Buf;
        pBGR24Buf = NULL;
    }

    return ;
}

//Process the data of the 24 bits BGR image...................

if (pBGR24Buf != NULL)
{
    delete []pBGR24Buf;
    pBGR24Buf = NULL;
}
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

8.4.5. DxRotate90CW8B

Declarations:

VxInt32 DHDECL DxRotate90CW8B (void* pInputBuffer, void* pOutputBuffer, VxUint32 nWidth, VxUint32 nHeight)

Descriptions:

The function will rotate the 8 bits image 90 degrees clockwise. For Bayer image, the Bayer format will be changed after the rotation. For example, rotate the raw image of BAYER_RG type clockwise, it becomes a BAYER_GR type. After rotation, the width and height of the image are equal to those of the original one.

Parameters:

pInputBuffer	Point to the data buffer of the original image.
pOutputBuffer	Point to the data buffer of the target image.
nWidth	The width of original image.
nHeight	The height of original image.

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
//Rotates the Raw image in BAYER_RG format with a width of 1600 and a height
//of 1234.
int32_t i32Width = 1600;
int32_t i32Height = 1234;

BYTE* pTemp = new BYTE[i32Width * i32Height];
if(pTemp == NULL)
{
    return;
}
else
{
    memset(pTemp, 0, i32Width * i32Height * sizeof(BYTE));
}

//Rotates 90 degrees clockwise.
//pRaw8Buf is the original image data
VxInt32 DxStatus = DxRotate90CW8B(pRaw8Buf, pTemp, i32Width, i32Height);
if (DxStatus != DX_OK)
{
    if(pTemp != NULL)
    {
        delete []pTemp;
        pTemp = NULL;
    }

    return ;
}
else
{
   //Copies to the Raw image buffer.
```

```
    memcpy(pRaw8Buf,pTemp, i32Width * i32Height * sizeof(BYTE));
}

if(pTemp != NULL)
{
    delete []pTemp;
    pTemp = NULL;
}
//At this point, the bayer type in pRaw8Buf is BAYER_GR. The width
//of the image is 1234 and the height is 1600.
```

8.4.6. DxRotate90CCW8B

Declarations:

VxInt32 DHDECL DxRotate90CCW8B (void* pInputBuffer, void* pOutputBuffer, VxUint32 nWidth, VxUint32 nHeight)

Descriptions:

The function will rotate the 8 bits gray image 90 degrees counter clockwise. For Bayer image, the Bayer format will be changed after the rotation. For example, rotate the raw image of BAYER_RG type counter clockwise, it becomes a BAYER_GB type. After rotation, the width and height of the image are equal to those of the original one.

Parameters:

pInputBuffer	Point to the data buffer of the original image.
pOutputBuffer	Point to the data buffer of the target image.
nWidth	The width of original image.
nHeight	The height of original image.

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
//Rotates the Raw image in BAYER_RG format with a width of 1600 and a height
//of 1234.
int32_t i32Width = 1600;
int32_t i32Height = 1234;

BYTE* pTemp = new BYTE[i32Width * i32Height];
if(pTemp == NULL)
{
    return;
}
else
{
    memset(pTemp, 0, i32Width * i32Height * sizeof(BYTE));
}

//Rotates 90 degrees counter clockwise.
//pRaw8Buf is the original image data
VxInt32 DxStatus = DxRotate90CCW8B(pRaw8Buf, pTemp, i32Width, i32Height);
if (DxStatus != DX_OK)
{
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
        if(pTemp != NULL)
        {
            delete []pTemp;
            pTemp = NULL;
        }
        return ;
    }
    else
    {
   //Copies to the Raw image buffer.
    memcpy(pRaw8Buf, pTemp, i32Width * i32Height * sizeof(BYTE));
    }
    if(pTemp != NULL)
    {
        delete []pTemp;
        pTemp = NULL;
    }
   //At this point, the bayer type in pRaw8Buf is BAYER_GR. The width of the
   //image is 1234 and the height is 1600.
```

8.4.7. DxRotate90CW16B

Declarations:

VxInt32 DHDECL DxRotate90CW16B (void* pInputBuffer, void* pOutputBuffer,
VxUint32 nWidth, VxUint32 nHeight)

Descriptions:

This function performs a ClockWise rotation of 90 degrees on a 16 bit image. For Bayer images, rotation will change the Bayer format. For example, a Raw graph of BAYER_RG type becomes BAYER_GR type when rotated clockwise. The width of the rotated image is equal to the height of the original image, and the height is equal to the width of the original image.

Parameters:

pInputBuffer	Point to the data buffer of the original image.
pOutputBuffer	Point to the data buffer of the target image.
nWidth	The width of original image.
nHeight	The height of original image.

Returns:
If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
    //Rotates the Raw image in BAYER_RG format with a width of 1600 and a height
    //of 1234.
    int32_t i32Width = 1600;
    int32_t i32Height = 1234;

    unsigned * pTemp = new BYTE[i32Width * i32Height * 2];
    if(pTemp == NULL)
    {
        return;
    }
    else
    {
```

```
        memset(pTemp, 0, i32Width * i32Height * sizeof(BYTE) * 2);
    }

    //Rotate clockwise 90 degrees
    //pRaw16Buf is the original image data
    VxInt32 DxStatus = DxRotate90CW16B(pRaw16Buf, pTemp, i32Width,
                                       i32Height);

    if (DxStatus != DX_OK)
    {
        if (pTemp != NULL)
        {
            delete []pTemp;
            pTemp = NULL;
        }

        return ;
    }
    else
    {
        //Copy the original image buffer
        memcpy(pRaw16Buf,pTemp,i32Width * i32Height * sizeof(BYTE)* 2);
    }
    if(pTemp != NULL)
    {
        delete []pTemp;
        pTemp = NULL;
    }
    //At this point, the bayer type in pRaw8Buf is BAYER_GR. The width of the
    //image is 1234 and the height is 1600.
```

8.4.8. DxRotate90CCW16B

Declarations:

VxInt32 DHDECL DxRotate90CCW16B (void* pInputBuffer, void* pOutputBuffer, VxUint32 nWidth, VxUint32 nHeight)

Descriptions:

This function performs a Counter ClockWise rotation of 90 degrees on a 16 bit grayscale image. For Bayer images, rotation will change the Bayer format. For example, a Raw graph of BAYER_RG type becomes BAYER_GB type when rotated counterclockwise. The width of the rotated image is equal to the height of the original image, and the height is equal to the width of the original image.

Parameters:

pInputBuffer	Point to the data buffer of the original image.
pOutputBuffer	Point to the data buffer of the target image.
nWidth	The width of original image.
nHeight	The height of original image.

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Sample Code:

```
//Rotates the Raw image in BAYER_RG format with a width of 1600 and a height
//of 1234.
int32_t i32Width = 1600;
int32_t i32Height = 1234;

BYTE* pTemp = new BYTE[i32Width * i32Height * 2];
if(pTemp == NULL)
{
   return;
}
else
{
   memset(pTemp, 0, i32Width * i32Height * sizeof(BYTE) * 2);
}
// Rotate 90°CCW
//pRaw16Buf is the original image data
VxInt32 DxStatus = DxRotate90CCW16B(pRaw16Buf, pTemp, i32Width,
                                    i32Height);
if (DxStatus != DX_OK)
{
   if (pTemp != NULL)
   {
      delete []pTemp;
      pTemp = NULL;
   }
   return ;
}
else
{
memcpy(pRaw16Buf, pTemp, i32Width * i32Height * sizeof(BYTE) * 2);
//Copy the original image buffer
}
if (pTemp != NULL)
{
   delete []pTemp;
   pTemp = NULL;
}
//At this point, the bayer type in pRaw16Buf is BAYER_GR. The width of the
//image is 1234 and the height is 1600.
```

8.4.9. DxBrightness

Declarations:

VxInt32 DHDECL DxBrightness (void* pInputBuffer, void* pOutputBuffer, VxUint32 nImagesize,
VxInt32 nFactor)

Descriptions:

The function will adjust the brightness of the input images, and the input images are 24 bits RGB or 8 bits gray image.

Parameters:

pInputBuffer	Point to the data buffer of the original image.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

POutputBuffer	Point to the data buffer of the target image.
nImagesize	The buffer length of input images, unit: byte. (For RGB images, the size equal the image width * image height* 3).
nFactor	The factor of brightness adjustment, range of value: -150~150. 0: The brightness has not changed. > 0: Increase the brightness. < 0: Reduces the brightness.

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
//Adjusts the brightness of the 24bitRGB image.
int32_t i32Width = 1600;
int32_t i32Height = 1234;

BYTE* pRGBOutputBuf = new BYTE[i32Width * i32Height];
if(pRGBOutputBuf == NULL)
{
    return;
}
else
{
    memset(pRGBOutputBuf, 0, i32Width * i32Height * sizeof(BYTE));
}

//pRGBInputBuf is the original image data
VxInt32 DxStatus = DxBrightness((BYTE*)pRGBBuf,pRGBBuf,nWidth * nHeight * 3,50);
if (DxStatus != DX_OK)
{
    return ;
}

//Adjusts the brightness of the 8 bits gray image.
VxInt32 DxStatus = DxBrightness((BYTE*)pRGBInputBuf, pRGBOutputBuf,
                                i32Width*i32Height*3, 50);
if (DxStatus != DX_OK)
{
    return ;
}

//Brightness adjustment of 8 bit grayscale images
BYTE* pYOutputbuf = new BYTE[i32Width * i32Height];
if (pYOutputbuf == NULL)
{
    return;
}
else
{
    memset(pYOutputbuf, 0, i32Width * i32Height * sizeof(BYTE));
}
```

```
VxInt32 DxStatus = DxBrightness((BYTE*)pYInputbuf, pYOutputbuf,
                                 i32Width*i32Height, 50);
if (DxStatus != DX_OK)
{
    return ;
}
```

Effect images:

Figure 8-1 shows an image with no brightness adjustment. Figure 8-2 shows the image after brightness adjustment.

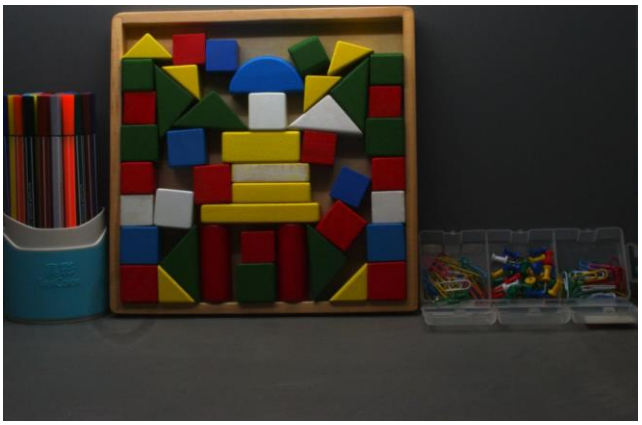


Figure 8-1 Before brightness adjustment



Figure 8-2 After brightness adjustment

8.4.10. DxContrast

Declarations:

VxInt32 DHDECL DxContrast (void* pInputBuffer, void* pOutputBuffer, VxUint32 nImagesize, VxInt32 nFactor)

Descriptions:

The function will adjust the contrast of the input images, and the input images are 24 bits RGB or 8 bits gray image.

Parameters:

pInputBuffer	Point to the data buffer of the original image.
pOutputBuffer	Point to the data buffer of the target image.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

nImagesize	The buffer length of input images, unit: byte. (For RGB images, the size equal the image width * image height * 3).
nFactor	The factor of contrast adjustment, range of value: -50~100. 0: The contrast has not changed. > 0: Increase the contrast. < 0: Reduces the contrast.

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
//Adjusts the contrast of the 24bitRGB image.
int32_t i32Width = 1600;
int32_t i32Height = 1234;

BYTE* pRGBOutputBuf = new BYTE[i32Width * i32Height];
if(pRGBOutputBuf == NULL)
{
   return;
}
else
{
   memset(pRGBOutputBuf, 0, i32Width * i32Height * sizeof(BYTE));
}

//pRGBInputBuf is the original image data
VxInt32 DxStatus = DxContrast ((BYTE*)pRGBInputBuf, pRGBOutputBuf,
                               i32Width*i32Height*3, 50);
if (DxStatus != DX_OK)
{
   return ;
}
//Adjusts the contrast of the 8 bits gray image.
BYTE* pYOutputbuf = new BYTE[i32Width * i32Height];
if(pYOutputbuf == NULL)
{
   return;
}
else
{
   memset(pYOutputbuf, 0, i32Width * i32Height * sizeof(BYTE));
}
//pYInputBuf is the original image data
VxInt32 DxStatus = DxContrast ((BYTE*)pYInputbuf, pYOutputbuf,
                               i32Width*i32Height, 50);
if (DxStatus != DX_OK)
{
   return ;
}
```

Effect images:

Figure 8-3 shows an image with no contrast adjustment. Figure 8-4 shows the image after contrast adjustment.



Figure 8-3 Before contrast adjustment



Figure 8-4 After contrast adjustment

8.4.11. DxSharpen24B

Declarations:

VxInt32 DHDECL DxSharpen24B (void* pInputBuffer, void* pOutputBuffer, VxUint32 nWidth, VxUint32 nHeight, float fFactor)

Descriptions:

The function will sharpen the input images, and the input images are 24bitRGB images.

Parameters:

pInputBuffer	Point to the data buffer of the original image.
pOutputBuffer	The buffer of the RGB image that after being sharpened.
nWidth	The width of the image.
nHeight	The height of the image.
fFactor	The factor of sharpness adjustment, range: 0.1~5.0.

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Sample Code:

```
//Sharpens the 24bitRGB image.
int32_t i32Width = 1600;
int32_t i32Height = 1234;

BYTE* pRGBOutputBuf = new BYTE [i32Width * i32Height * 3];
if(pRGBOutputBuf == NULL)
{
    return;
}
else
{
    memset(pRGBOutputBuf, 0, i32Width * i32Height * sizeof(BYTE) * 3);
}

//pRGBInputBufis the original image data
VxInt32 DxStatus = DxSharpen24B ((BYTE*)pRGBInputBuf, pRGBOutputBuf,
                                 i32Width,i32Height, 2.0);
if (DxStatus != DX_OK)
{
    return ;
}
```

Effect images:

Figure 8-5 shows an image with no sharpen adjustment. Figure 8-6 shows the image after sharpen adjustment.



Figure 8-5: Original image



Figure 8-6: After sharpen adjustment

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

8.4.12. DxSharpenMono8

Declarations:

VxInt32 DHDECL DxSharpenMono8 (void* pInputBuffer, void* pOutputBuffer,
VxUint32 nWidth, VxUint32 nHeight, float fFactor)

Descriptions:

This function sharpens the input image, which is an 8-bit RGB image.

Parameters:

pInputBuffer	Point to the data buffer of the original image.
pOutputBuffer	The buffer of the RGB image that after being sharpened.
nWidth	The width of the image.
nHeight	The height of the image.
fFactor	The factor of sharpness adjustment, range: 0.1~5.0.

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
//Sharpen 8-bit images
int32_t i32Width = 1600;
int32_t i32Height = 1234;
BYTE* pOutputBuf = new BYTE [i32Width * i32Height];
if(pOutputBuf == NULL)
{
    return;
}
else
{
    memset(pOutputBuf, 0, i32Width * i32Height * sizeof(BYTE));
}
//pInputBuf is the original image data
VxInt32 DxStatus = DxSharpenMono8 ((BYTE*)pInputBuf, pOutputBuf,
                                   i32Width,i32Height, 2.0);
if (DxStatus != DX_OK)
{
    return ;
}
```

8.4.13. DxSaturation

Declarations:

VxInt32 DHDECL DxSaturation (void* pInputBuffer, void* pOutputBuffer, VxUint32 nImagesize,
VxInt32 nFactor)

Descriptions:

The function is used to adjust the saturation of the input image, and the input image is 24bitRGB image.

Parameters:

pInputBuffer	Point to the data buffer of the original image.
pOutputBuffer	The buffer of the image that after saturation adjustment.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

nImagesize	The buffer length of the image, unit: byte. (For RGB image, the buffer length is equal to the image width * image height * 3)
nFactor	The factor of saturation adjustment, range: 0~128. 64: There's no change in saturation. > 64: Increase the saturation. < 64: Reduce the saturation. 128: The saturation is twice as the current value. 0: It is a monochrome image.

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
//Adjusts the saturation of the 24bitRGB image.
int32_t i32Width = 1600;
int32_t i32Height = 1234;

BYTE* pRGBOutputBuf = new BYTE[i32Width * i32Height];
if(pRGBOutputBuf == NULL)
{
    return;
}
else
{
    memset(pRGBOutputBuf, 0, i32Width * i32Height * sizeof(BYTE));
}
//pRGBInputBufis is the original image data
VxInt32 DxStatus = DxSaturation((BYTE*)pRGBInputBuf, pRGBOutputBuf,
                                i32Width*i32Height, 90);
if (DxStatus != DX_OK)
{
    return ;
}
```

Effect images:

Figure 8-7 shows an image with no saturation adjustment. Figure 8-8 shows the image after saturation adjustment.



Figure 8-7: Before saturation adjustment



Figure 8-8:After saturation adjustment

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

8.4.14. DxGetWhiteBalanceRatio

Declarations:

VxInt32 DHDECL DxGetWhiteBalanceRatio (void *pInputBuffer, VxUint32 nWidth, VxUint32 nHeight, double* dRatioR, double* dRatioG, double* dRatioB)

Descriptions:

This function is used to get the white balance ratio. The input is 24 bits RGB image and output is white balance ratio. In order to calculate accurately, the input image should be objective "white" area.

Parameters:

pInputBuffer	"White" image data buffer.
nWidth	The image width.
nHeight	The image height.
dRatioR	The white balance ratio of the red component.
dRatioG	The white balance ratio of the green component.
dRatioB	The white balance ratio of the blue component.

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
double dRatioR = 1.0;
double dRatioG = 1.0;
double dRatioB = 1.0;
int32_t i32Width = 1600;
int32_t i32Height = 1234;

//Calculates the white balance coefficient.
//pRGBInputBuf is the original image data
VxInt32 DxStatus =DxGetWhiteBalanceRatio((BYTE*)pRGBInputBuf,
                                         i32Width,i32Height,
                                         &dRatioR, &dRatioG,
                                         &dRatioB);
if (DxStatus != DX_OK)
{
   return ;
}
```

Effect images:

Figure 8-9 shows an RGB image with no white balance adjustment:



Figure 8-9: An RGB image with no white balance adjustment

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

1) If use this function to get the white balance ratio, to be accurate, you should give an objective "white" region sub-image to the function in the RGB image. As shown in the frame area in Figure 8-10. (The actual object in the area is the white building block). The image after white balance adjustment is shown in Figure 8-11.
2) The use method: (pRGBBuf is the buffer of the sub-image in the frame area in Figure 8-10, nWidth is the sub-image width, nHeight is the sub-image height), it needs to create a new buffer for the sub-image and then pass the new buffer to DxGetWhiteBalanceRatio.



Figure 8-10 The "white" region selected

After calculating the white balance ratio of the three components of R/G/B, the white balance operation method is as follows:

```
//Adds macro definitions to the header file.

//<Determines the range of bit data.
#define CLIP8(a)   (((a) & 0xFFFFFF00) ? (((a) < 0) ? 0 : 255 ) : (a))

int i    = 0;
int j    = 0;
int nPos = 0;

//Whites balance look-up table.
BYTE  arrRLut[256];
BYTE  arrGlut[256];
BYTE  arrBLut[256];

//Defines R, G, and B pixel.
int  R = 0;
int  G = 0;
int  B = 0;
double dRatioR = 1.0;
double dRatioG = 1.0;
double dRatioB = 1.0;
int32_t i32Width = 1600;
int32_t i32Height = 1234;

//Calculates the look-up table for white balance coefficients.
for(i = 0; i < 256; i++)
{
   //Calculates the pixel values after white balance processing.
   R = i * dRatioR;
```

```
        G = i * dRatioG;
        B = i * dRatioB;

        arrRLut[i] = CLIP8(R);
        arrGlut[i] = CLIP8(G);
        arrBLut[i] = CLIP8(B);
    }

    //White balance processing.
     for (i = 0; i < i32Height; i++)
    {
       for(j = 0; j < i32Width; j++)
      {
          nPos = 3 * i * i32Width + 3 * j;
          pRGBBuf[nPos + 0] = arrRLut[pRGBBuf[nPos + 0]];
          pRGBBuf[nPos + 1] = arrGLut[pRGBBuf[nPos + 1]];
          pRGBBuf[nPos + 2] = arrBLut[pRGBBuf[nPos + 2]];
       }
    }
```



Figure 8-11 An image after white balance adjustment

8.4.15. DxAutoRawDefectivePixelCorrect

Declarations:

VxInt32 DHDECL DxAutoRawDefectivePixelCorrect (void* pRawImgBuf, VxUint32 nWidth, VxUint32 nHeight, VxInt32 nBitNum)

Descriptions:

The function automatically detects and corrects the bad pixel of the raw image in real time. The input image can be 8 bits raw image or 16bits raw image. This function supports raw images in Bayer format. The nBitNum is the actual bit depth of the data, if the image is 8 bits raw, the value is 8, and if the image is 16bits raw, then the value is the actual bit depth of the data. For example, a raw image of a 12 bits data, it takes two bytes (16bits), but the actual bit depth is 12, then the value is 12. This function does not support the raw image of Packet format. For the Packet format, you should use the DxRaw10PackedToRaw16 function or DxRaw10PackedToRaw16 function to convert it into raw16 format first. Due to the real-time detection and correction, every image must be checked and corrected when this function is turned on.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Parameters:

pRawImgBuf	The input buffer of Raw image (8 bits or 16 bits Raw image).
nWidth	The width of the image.
nHeight	The height of the image.
nBitNum	The actual bit depth of the data.

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
int32_t i32Width = 1600;
int32_t i32Height = 1234;

//Determines whether to open the automatic dead pixel correction.
//pPaw16Buf is the 16 bits raw image.
if(bAutoCorrect)
{
   //pPaw16Bufis is the original image data
   VxInt32 DxStatus= DxAutoRawDefectivePixelCorrect(pPaw16Buf,
                                                    i32Width,
                                                    i32Height,12);
   if (DxStatus != DX_OK)
   {
      if (pRaw16Buf != NULL)
      {
         delete []pRaw16Buf;
         pRaw16Buf = NULL;
      }

      return;
   }
}

//Processes Raw images.
//..................
```

8.4.16. DxRaw16toRaw8

Declarations:

VxInt32 DHDECL DxRaw16toRaw8 (void *pInputBuffer, void *pOutputBuffer, VxUint32 nWidth, VxUint32 nHeight, DX_VALID_BIT nValidBits);

Descriptions:

The function converts raw16 images (the actual bit depth is 16, effective bit depth is 10 or 12) to raw8 images (the actual bit depth and valid bit depth are 8 bits).

Parameters:

pInputBuffer	The data buffer of original image.
pOutputBuffer	The data buffer of target image.
nWidth	The width of the image.
nHeight	The height of the image.
nValidBits	The valid bit of data.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
int32_t i32Width = 1600;
int32_t i32Height = 1234;

//Applies for the output buffer.
BYTE *pRaw8Buf = new BYTE[i32Width * i32Height];
if(pRaw8Buf == NULL)
{
    return;
}
else
{
    //Initializes the buffer.
    memset(pRaw8Buf, 0, i32Width * i32Height * sizeof(BYTE));
}

 //pPaw16Buf is the input image data
VxInt32 DxStatus = DxRaw16toRaw8(pRaw16Buf, pRaw8Buf,
                                 i32Width, i32Height, DX_BIT_4_11);
if (DxStatus != DX_OK)
{
    if (pRaw8Buf != NULL)
    {
        delete []pRaw8Buf;
        pRaw8Buf = NULL;
    }

    return;
}

//Processing the data of the 8 bits raw image.
//.................

//Processing is complete.
if(pRaw8Buf != NULL)
{
delete []pRaw8Buf;
pRaw8Buf = NULL;
}
```

8.4.17. DxRGB48toRGB24

Declarations:

VxInt32 DHDECL DxRGB48toRGB24 (void *pInputBuffer, void *pOutputBuffer, VxUint32 nWidth, VxUint32 nHeight, DX_VALID_BIT nValidBits);

Descriptions:

The function converts RGB 48 bits color image data to RGB 24 bits image data, because the three channels process the image at the same time, each channel will convert the 16 bits (the valid bit depth may be 12 bits, 10 bits, etc.) data to 8 bits data.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Parameters:

pInputBuffer	The data buffer of original image (the buffer size is nWidth * nHeight * 3 * 2 BYTE).
pOutputBuffer	The data buffer of target image (the buffer size is nWidth * nHeight * 3 * BYTE).
nWidth	The width of the image.
nHeight	The height of the image.
nValidBits	The valid bit of data.

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
int32_t i32Width = 1600;
int32_t i32Height = 1234;

//Applies for the output buffer.
BYTE *pRGB24Buf = new BYTE[i32Width * i32Height * 3];
if(pRGB24Buf == NULL)
{
    return;
}
else
{
    //Buffer initialization
    memset(pRGB24Buf, 0, i32Width * i32Height * 3 * sizeof(BYTE));
}

//pPaw48Buf is input image data
VxInt32 DxStatus = DxRGB48toRGB24(pRGB48Buf, pRGB24Buf, i32Width,
                                  i32Height, DX_BIT_4_11);
if (DxStatus != DX_OK)
{
    if (pRGB24Buf != NULL)
    {
        delete []pRGB24Buf;
        pRGB24Buf = NULL;
    }

    return ;
}

//Processing the data of the 8 bits raw image.
//.................

//Processing is complete.
if(pRGB24Buf != NULL)
{
    delete []pRGB24Buf;
    pRGB24Buf = NULL;
}
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

8.4.18. DxRaw16toRGB48

Declarations:

VxInt32 DHDECL DxRaw16toRGB48 (void *pInputBuffer, void *pOutputBuffer, VxUint32 nWidth, VxUint32 nHeight, DX_ACTUAL_BITS nActualBits, DX_BAYER_CONVERT_TYPE cvtype, DX_PIXEL_COLOR_FILTER nBayerType, bool bFlip)

Descriptions:

The function converts raw 16bits image (each pixel is 16 bits) to RGB 48 bits image data (each RGB component is 16 bits), and you can also use the function DxRGB48toRGB24 to convert RGB 48 bits image to RGB 24 bits image for display.

Parameters:

pInputBuffer	The data buffer of original image.
pOutputBuffer	The data buffer of target image.
nWidth	The width of the image.
nHeight	The height of the image.
nActualBits	The valid bit of data.
cvtype	The conversion algorithm type.
nBayerType	The type of Bayer image format.
bFlip	If flip, the value is **true**, and the image will vertical flip; otherwise, the value is **false**, not flip.

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
int32_t i32Width = 1600;
int32_t i32Height = 1234;

//Outputs RGB image data.
BYTE *pRGB48Buf = new BYTE[i32Width * i32Height * 3 * 2];
if (pRGB48Buf == NULL)
{
    return ;
}
else
{
    //Initializes the buffer.
    memset(pRGB48Buf, 0, i32Width * i32Height * 3 * 2 * sizeof(BYTE));
}

//Selects the interpolation algorithm.
DX_BAYER_CONVERT_TYPE emType     = RAW2RGB_NEIGHBOUR;
//Selects the image Bayer format.
DX_PIXEL_COLOR_FILTER emBayerType   = BAYERRG;
//The actual bit width.
DX_ACTUAL_BITS    emActualBits  = DX_ACTUAL_BITS_16;
bool bFlip = true;

//pPaw16Buf is the input image data
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
VxInt32 DxStatus = DxRaw16toRGB48(pRaw16Buf, pRGB48Buf, i32Width,
                                  i32Height, emActualBits,
                                  emType, emBayerType, bFlip);
if (DxStatus != DX_OK)
{
    if (pRGB48Buf != NULL)
    {
        delete []pRGB48Buf;
        pRGB48Buf = NULL;
    }

    return ;
}

//Processing RGB48 data.
//...................

if (pRGB48Buf != NULL)
{
    delete []pRGB48Buf;
    pRGB48Buf = NULL;
}
```

8.4.19. DxRaw8toARGB32

Declarations:

VxInt32 DHDECL DxRaw8toARGB32 (void *pInputBuffer, void *pOutputBuffer, VxUint32 nWidth, VxUint32 nHeight, int nStride, DX_BAYER_CONVERT_TYPE cvtype, DX_PIXEL_COLOR_FILTER nBayerType, bool bFlip, VxUint8 nAlpha)

Descriptions:

The function is used to convert raw 8 bits image to ARGB 32 bits image data.

Parameters:

pInputBuffer	The data buffer of original image.
pOutputBuffer	The data buffer of target image, and the size is image width * image height * 4.
nWidth	The width of the image.
nHeight	The height of the image.
nStride	The scan width.
cvtype	The conversion algorithm type.
nBayerType	The type of Bayer image format.
bFlip	If flip, the value is **true**, and the image will vertical flip; otherwise, the value is **false**, not flip.
nAlpha	The value of channal Alpha, which ranges from 0 to 255

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
int32_t i32Width = 1600;
int32_t i32Height = 1234;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
//Outputs ARGB image data.
BYTE *pARGB32Buf = new BYTE [i32Width * i32Height * 4];
if (pARGB32Buf == NULL)
{
    return ;
}
else
{
    //Initializes the buffer.
    memset(pARGB32Buf, 0, i32Width * i32Height * sizeof(BYTE));
}

//Selects the interpolation algorithm.
DX_BAYER_CONVERT_TYPE emType= RAW2RGB_NEIGHBOUR;

//Selects the image Bayer format.
DX_PIXEL_COLOR_FILTER emBayerType = BAYERRG;
int      i32Stride = i32Width;    // The scan width.
bool     bFlip = true;
VxUint8 nAlpha = 127;

//pPaw8Buf is the input image data
VxInt32 DxStatus = DxRaw8toARGB32 (pRaw8Buf, pARGB32Buf, i32Width,
                                   i32Height, i32Stride, emType,
                                   emBayerType, bFlip, nAlpha);
if (DxStatus != DX_OK)
{
    if (pARGB32Buf != NULL)
    {
        delete []pARGB32Buf;
        pARGB32Buf = NULL;
    }

    return ;
}

//Process ARGB32 data.
//.................

if (pARGB32Buf!= NULL)
{
    delete []pARGB32Buf;
    pARGB32Buf = NULL;
}
```

8.4.20. DxGetContrastLut

Declarations:

VxInt32 DHDECL DxGetContrastLut (int nContrastParam, void *pContrastLut, int *pLutLength);

Descriptions:

This function is used to calculate the contrast look-up table for the input of image quality promotion function, and only 24 bits RGB images are supported.

Parameters:

nContrastParam	The parameter of contrast adjustment. You can get it by the GX_INT_CONTRAST_PARAM code of the GxIAPI library.
pContrastLut	Contrast look-up table.
pLutLength	The length of contrast look-up table, unit: byte;

If the length value inputted by the user less than the current actual length value, it will return errors, and the *pLutLength* returns the actual length value.

If the *pContrastLut* inputted by the user is NULL, it returns success, the *pLutLength* returns the actual length value.

If the operation succeeds, the *pLutLength* returns the actual length value.

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
int64_t i64ContrastParam = 0;

//Determines whether the current camera supports contrast acquisition.
bool bIsImplemented = false;
GX_STATUS emStatus = GXIsImplemented(hDevice, GX_INT_CONTRAST_PARAM,
                                     &bIsImplemented);
if (emStatus != GX_STATUS_SUCCESS)
{
    return;
}

if (bIsImplemented)
{
   //Gets the contrast adjustment parameter.
   emStatus = GXGetInt (hDevice, GX_INT_CONTRAST_PARAM, &i64ContrastParam);
   if (emStatus != GX_STATUS_SUCCESS)
    {
      return;
    }
}
else
{
    i64ContrastParam = 0;
}

//Gets the length of the contrast look-up table.
int32_t i32LutLength = 0;
VxInt32 DxStatus= DxGetContrastLut((int)i64ContrastParam, NULL,
                            &i32LutLength);
if(DxStatus != DX_OK)
{
    return;
}

//Applies memory for the contrast look-up table.
BYTE *pContrastLut = new BYTE[i32LutLength];
if (pContrastLut == NULL)
{
   return;
}

//Calculates the contrast look-up table.
```

```
DxStatus = DxGetContrastLut((int)i64ContrastParam, pContrastLut,
                            &i32LutLength);
if (DxStatus != DX_OK)
{
    if (pContrastLut!= NULL)
    {
       delete []pContrastLut;
       pContrastLut = NULL;
    }
    return;
}

//Image processing.
//……

if (pContrastLut!= NULL)
{
   delete []pContrastLut;
   pContrastLut= NULL;
}
```

8.4.21. DxGetGammatLut

Declarations:

VxInt32 DHDECL DxGetGammatLut (double dGammaParam, void *pGammaLut, int *pLutLength);

Descriptions:

This function is used to calculate the Gamma look-up table for the input of image quality promotion function, and only 24 bits RGB images are supported.

Parameters:

dGammaParam	The parameter of Gamma adjustment. You can get it by the GX_FLOAT_GAMMA_PARAM code of the GxIAPI library.
pGammaLut	Gamma look-up table.
pLutLength	The length of Gamma look-up table, unit: byte;

If the length value inputted by the user less than the current actual length value, it will return errors, and the *pLutLength* returns the actual length value.
If the pGammaLut inputted by the user is NULL, it returns success, the *pLutLength* returns the actual length value.
If the operation succeeds, the *pLutLength* returns the actual length value.

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
int32_t i32LutLength = 0;
double dGammaParam = 0.0;

//Determines whether the current camera supports Gamma capture.
bool bIsImplemented = false;
GX_STATUS emStatus = GXIsImplemented(hDevice, GX_FLOAT_GAMMA_PARAM,
                                     &bIsImplemented);
if (emStatus != GX_STATUS_SUCCESS)
{
    return;
}
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    if (bIsImplemented)
    {
        //Gets the Gamma adjustment parameter.
        emStatus = GXGetFloat (hDevice, GX_FLOAT_GAMMA_PARAM, &dGammaParam);
      if (emStatus != GX_STATUS_SUCCESS)
        {
            return;
        }
    }
    else
    {
        dGammaParam = 1;
    }

   //Gets the length of the Gamma look-up table.
    VxInt32 DxStatus= DxGetGammaLut(dGammaParam, NULL, &i32LutLength);
    if (DxStatus != DX_OK)
    {
        return;
    }
    //Applies memory for the Gamma look-up table.
    BYTE* pGammaLut= new BYTE[i32LutLength];
    if (pGammaLut== NULL)
    {
        return;
    }

    //Calculates the Gamma look-up table.
    DxStatus = DxGetGammaLut(dGammaParam, pGammaLut, &i32LutLength);
    if (DxStatus != DX_OK)
    {
        if (pGammaLut!= NULL)
        {
            delete []pGammaLut;
            pGammaLut= NULL;
        }

        return;
    }

    //Image processing
    // ……
    if (pGammaLut!= NULL)
    {
        delete []pGammaLut;
        pGammaLut= NULL;
    }
```

8.4.22. DxImageImprovment

Declarations:

VxInt32 DHDECL DxImageImprovment (void *pInputBuffer, void *pOutputBuffer, VxUint32 nWidth, VxUint32 nHeight, VxInt64 nColorCorrectionParam, void *pContrastLut, void *pGammaLut);

Descriptions:

This function is used for the image quality promotion of the input images, and only 24bitRGB images are supported.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Parameters:

pInputBuffer	The data buffer of original image.
pOutputBuffer	The data buffer of target image.
nWidth	The width of the image.
nHeight	The height of the image.
nColorCorrectionParam	The color correction value and can be get by the GX_INT_COLOR_CORRECTION_PARAM code in the GxIAPI library, and also it can be set to 0.
pContrastLut	The contrast look-up table, which can be calculated by DxGetContrastLut function, is only calculated once, and can be set to NULL.
pGammaLut	The Gamma look-up table, which can be calculated by DxGetGammatLut function, is only calculated once, and can be set to NULL.

nColorCorrectionParam, pContrastLut, pGammaLut, different combinations can be carried out to achieve different effects. The combination is shown in Table 8-1:

NO.	nColorCorrectionParam	pContrastLut	pGammaLut	Effect
1	≠ 0	≠ NULL	≠ NULL	Color correction, contrast, Gamma adjustment (**At this point, the image quality is the best**)
2	≠ 0	NULL	≠ NULL	Color correction, Gamma adjustment
3	≠ 0	≠ NULL	NULL	Color correction, contrast adjustment
4	0	≠ NULL	≠ NULL	Contrast, Gamma adjustment
5	≠ 0	NULL	NULL	Color correction
6	0	≠ NULL	NULL	Contrast adjustment
7	0	NULL	≠ NULL	Gamma adjustment

Table 8-1

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
//Gets the contrast adjustment parameter value.
VxInt32 DxStatus = DX_OK;
int32_t i32Width = 1600;
int32_t i32Height = 1234;
int64_t i64ContrastParam = 0;
int64_t i64ColorCorrectionParam = 0;
int32_t  i32LutLength = 0;
double dGammaParam = 0.0;
BYTE* pGammaLut;
BYTE* pContrastLut;
GX_STATUS emStatus = GXGetInt (hDevice, GX_INT_CONTRAST_PARAM,
                               & i64ContrastParam);
if (emStatus != GX_STATUS_SUCCESS)
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    {
        return;
    }

    //Gets the adjustment parameter value of the color correction.
    emStatus  = GXGetInt (hDevice, GX_INT_COLOR_CORRECTION_PARAM,
                          &i64ColorCorrectionParam);
    if (emStatus != GX_STATUS_SUCCESS)
    {
        return;
    }

    //Gets the Gamma adjustment parameter.
    emStatus = GXGetFloat (hDevice, GX_FLOAT_GAMMA_PARAM, &dGammaParam);
    if (emStatus != GX_STATUS_SUCCESS)
    {
        return;
    }

    do
    {
       //Gets the length of the Gamma look-up table.
       DxStatus= DxGetGammaLut(dGammaParam, NULL, &i32LutLength);
       if (DxStatus != DX_OK)
       {
           break;
       }

       //Applies memory for the Gamma look-up table.
       pGammaLut = new BYTE[i32LutLength];
       if (pGammaLut == NULL)
       {
        DxStatus = DX_NOT_ENOUGH_SYSTEM_MEMORY;
        break;
       }

       //Calculates the Gamma look-up table.
       DxStatus = DxGetGammaLut(dGammaParam, pGammaLut, &i32LutLength);
       if (DxStatus != DX_OK)
       {
           break;
       }

       //Gets the length of the contrast look-up table.
       DxStatus = DxGetContrastLut((int)i64ContrastParam, NULL, &i32LutLength);
       if (DxStatus != DX_OK)
       {
           break;
       }

       //Applies memory for the contrast look-up table.
       pContrastLut = new BYTE[i32LutLength];
       if (pContrastLut == NULL)
```

```
    {
        DxStatus = DX_NOT_ENOUGH_SYSTEM_MEMORY;
        break;
    }

    //Calculates the contrast look-up table.
    DxStatus = DxGetContrastLut((int)i64ContrastParam, pContrastLut,
                                &i32LutLength);
    if (DxStatus != DX_OK)
    {
       break;
    }
}while(0);

//Sets look-up table failed, and then release the resource.
if (DXStatus != DX_OK)
{
    if (pGammaLut != NULL)
    {
        delete[] m_pGammaLut;
        pGammaLut = NULL;
    }
    if (pContrastLut != NULL)
    {
       delete[] pContrastLut;
       pContrastLut = NULL;
    }
    return;
}

 //Output image ARGB data
 BYTE * pOutputBuffer = new BYTE[i32Width * i32Height];
 if (pOutputBuffer == NULL)
 {
    return;
 }
 else
 {
    //Buffer initialization
    memset(pOutputBuffer, 0, i32Width * i32Height * sizeof(BYTE));
 }

 //Improves the quality of the image.
 //pInputBuffer is the input image data
DxStatus = DxImageImprovment(pInputBuffer,pOutputBuffer,i32Width,i32Height,
                             i64ColorCorrectionParam, pContrastLut,
                             pGammaLut);

 if (pGammaLut!= NULL)
 {
    delete []pGammaLut;
    pGammaLut = NULL;
 }
```

```
if (pContrastLut!= NULL)
{
    delete []pContrastLut;
    pContrastLut = NULL;
}
```

Effect images:

Figure 8-12 shows an image with no quality promotion. Figure 8-13 shows the image after quality promotion.



Figure 8-12: An image with no quality promotion



Figure 8-13: An image after quality promotion

8.4.23. DxARGBImageImprovment

Declarations:

VxInt32 DHDECL DxARGBImageImprovment (void *pInputBuffer, void *pOutputBuffer,
VxUint32 nWidth, VxUint32 nHeight,
VxInt64 nColorCorrectionParam, void *pContrastLut,
void *pGammaLut);

Descriptions:

This function is used for the image quality promotion of the input images, and only ARGB images are supported.

Parameters:

pInputBuffer	The data buffer of original image.
pOutputBuffer	The data buffer of target image.
nWidth	The width of the image.
nHeight	The height of the image.
nColorCorrectionParam	The color correction value and can be get by the GX_INT_COLOR_CORRECTION_PARAM code in the GxIAPI library, and also it can be set to 0.
pContrastLut	The contrast look-up table, which can be calculated by DxGetContrastLut function, is only calculated once, and can be set to NULL.
pGammaLut	The Gamma look-up table, which can be calculated by DxGetGammatLut function, is only calculated once, and can be set to NULL.

In addition, nColorCorrectionParam, pContrastLut, pGammaLut, different combinations can be carried out to achieve different effects. The combination is shown in Figure 8-2:

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

NO.	nColorCorrectionParam	pContrastLut	pGammaLut	Effect
1	≠ 0	≠ NULL	≠ NULL	Color correction, contrast, Gamma adjustment (**At this point, the image quality is the best**)
2	≠ 0	NULL	≠ NULL	Color correction, Gamma adjustment
3	≠ 0	≠ NULL	NULL	Color correction, contrast adjustment
4	0	≠ NULL	≠ NULL	Contrast, Gamma adjustment
5	≠ 0	NULL	NULL	Color correction
6	0	≠ NULL	NULL	Contrast adjustment
7	0	NULL	≠ NULL	Gamma adjustment

Table 8-2

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
    VxInt32 DxStatus = DX_OK;
    int32_t i32Width = 1600;
    int32_t i32Height = 1234;
    int64_t i64ContrastParam = 0;
    int64_t i64ColorCorrectionParam = 0;
    int32_t  i32LutLength = 0;
    double dGammaParam = 0.0;
    BYTE* pGammaLut;
    BYTE* pContrastLut;

    // Get the contrast adjustment parameter value.
    GX_STATUS emStatus = GXGetInt (hDevice, GX_INT_CONTRAST_PARAM,
                                   &i64ContrastParam);
    if (emStatus != GX_STATUS_SUCCESS)
    {
        return;
    }

    //Get the adjustment parameter value of the color correction.
    emStatus = GXGetInt (hDevice, GX_INT_COLOR_CORRECTION_PARAM,
                         &i64ColorCorrectionParam);
    if (emStatus != GX_STATUS_SUCCESS)
    {
        return;
    }

    //Get the Gamma adjustment parameter.
    emStatus = GXGetFloat (hDevice, GX_FLOAT_GAMMA_PARAM, &dGammaParam);
    if (emStatus != GX_STATUS_SUCCESS)
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
{
    return;
}

do
{
    //Get the length of the Gamma look-up table.
    VxInt32 DxStatus = DxGetGammaLut (dGammaParam, NULL, &i32LutLength);
    if (DxStatus != DX_OK)
    {
        break;
    }

    //Apply memory for the Gamma look-up table.
    pGammaLut = new BYTE[i32LutLength];
    if (pGammaLut == NULL)
    {
        DxStatus = DX_NOT_ENOUGH_SYSTEM_MEMORY;
        break;
    }

    //Calculate the Gamma look-up table.
    DxStatus = DxGetGammaLut (dGammaParam, pGammaLut, &i32LutLength);
    if (DxStatus != DX_OK)
    {
        break;
    }

    //Get the length of the contrast look-up table.
    DxStatus = DxGetContrastLut ((int)i64ContrastParam, NULL,
                                 &i32LutLength);
    if (DxStatus != DX_OK)
    {
        break;
    }

    //Apply memory for the contrast look-up table.
    pContrastLut = new BYTE[i32LutLength];
    if (pContrastLut == NULL)
    {
        DxStatus = DX_NOT_ENOUGH_SYSTEM_MEMORY;
        break;
    }

    //Calculate the contrast look-up table.
   DxStatus = DxGetContrastLut ((int)i64ContrastParam, pContrastLut,
                                &i32LutLength);
    if (DxStatus != DX_OK)
    {
        break;
    }
}while(0);
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
//If the setting of look-up table is failed, release the resource.
if (DxStatus != DX_OK)
{
    if (pGammaLut != NULL)
    {
        delete[] pGammaLut;
        pGammaLut = NULL;
    }
    if (pContrastLut != NULL)
    {
        delete[] pContrastLut;
        pContrastLut = NULL;
    }
    return;
}

//Output image ARGB data
BYTE * pOutputBuffer = new BYTE[i32Width * i32Height];/
if (pOutputBuffer == NULL)
{
    return;
}
else
{
    //Buffer initialization
    memset(pOutputBuffer, 0, i32Width * i32Height * sizeof(BYTE));
}
//Improve the quality of the image.
//pInputBuffer is input image data
DxStatus = DxARGBImageImprovment(pInputBuffer,pOutputBuffer,
                                 i32Width,i32Height,
                                 i64ColorCorrectionParam,
                                  pContrastLut, pGammaLut);
if (pGammaLut!= NULL)
{
    delete []pGammaLut;
    pGammaLut = NULL;
}

if (pContrastLut!= NULL)
{
    delete []pContrastLut;
    pContrastLut = NULL;
}
```

8.4.24. DxImageImprovmentEx

Declarations:

VxInt32 DHDECL DxImageImprovmentEx (void *pInputBuffer, void *pOutputBuffer, VxUint32 nWidth,
VxUint32 nHeight, VxInt64 nColorCorrectionParam,
void *pContrastLut, void *pGammaLut,
DX_RGB_CHANNEL_ORDER emChannelOrder);

Descriptions:

This function is used for the image quality promotion of the input images, and only 24bitRGB images are supported.

Parameters:

pInputBuffer	The data buffer of original image.
pOutputBuffer	The data buffer of target image.
nWidth	The width of the image.
nHeight	The height of the image.
nColorCorrectionParam	The color correction value and can be get by the GX_INT_COLOR_CORRECTION_PARAM code in the GxIAPI library, and also it can be set to 0.
pContrastLut	The contrast look-up table, which can be calculated by DxGetContrastLut function, is only calculated once, and can be set to NULL.
pGammaLut	The Gamma look-up table, which can be calculated by DxGetGammatLut function, is only calculated once, and can be set to NULL.
emChannelOrder	RGB channel order of output image.

In addition, nColorCorrectionParam, pContrastLut, pGammaLut, different combinations can be carried out to achieve different effects. The combination is shown in Table 8-3:

NO.	nColorCorrectionParam	pContrastLut	pGammaLut	Effect
1	≠ 0	≠ NULL	≠ NULL	Color correction, contrast, Gamma adjustment (**At this point, the image quality is the best**)
2	≠ 0	NULL	≠ NULL	Color correction, Gamma adjustment
3	≠ 0	≠ NULL	NULL	Color correction, contrast adjustment
4	0	≠ NULL	≠ NULL	Contrast, Gamma adjustment
5	≠ 0	NULL	NULL	Color correction
6	0	≠ NULL	NULL	Contrast adjustment
7	0	NULL	≠ NULL	Gamma adjustment

Table 8-3

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
VxInt32 DxStatus = DX_OK;
int32_t i32Width = 1600;
int32_t i32Height = 1234;
int64_t i64ContrastParam = 0;
int64_t i64ColorCorrectionParam = 0;
int32_t  i32LutLength = 0;
double dGammaParam = 0.0;
BYTE* pGammaLut;
BYTE* pContrastLut;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
//Get the contrast adjustment parameter value.
GX_STATUS emStatus = GXGetInt (hDevice, GX_INT_CONTRAST_PARAM,
                               &i64ContrastParam);
if (emStatus != GX_STATUS_SUCCESS)
{
    return;
}

//Get the adjustment parameter value of the color correction.
emStatus = GXGetInt (hDevice, GX_INT_COLOR_CORRECTION_PARAM,
                     &i64ColorCorrectionParam);
if (emStatus != GX_STATUS_SUCCESS)
{
    return;
}

//Get the Gamma adjustment parameter.
emStatus = GXGetFloat (hDevice, GX_FLOAT_GAMMA_PARAM, &dGammaParam);
if (emStatus != GX_STATUS_SUCCESS)
{
    return;
}

do
{
    //Get the length of the Gamma look-up table.
    DxStatus= DxGetGammaLut(dGammaParam, NULL, &i32LutLength);
    if (DxStatus != DX_OK)
    {
        break;
    }

    //Apply memory for the Gamma look-up table.
    pGammaLut = new BYTE[nLutLength];
    if (pGammaLut == NULL)
    {
        DxStatus = DX_NOT_ENOUGH_SYSTEM_MEMORY;
        break;
    }

    //Calculate the Gamma look-up table.
    DxStatus = DxGetGammaLut (dGammaParam, pGammaLut, &i32LutLength);
    if (DxStatus != DX_OK)
    {
        break;
    }

    //Get the length of the contrast look-up table.
    DxStatus = DxGetContrastLut ((int)i64ContrastParam, NULL,
                                 &i32LutLength);
    if (DxStatus != DX_OK)
    {
        break;
```

```
        }

        //Apply memory for the contrast look-up table.
        pContrastLut = new BYTE[nLutLength];
        if (pContrastLut == NULL)
        {
            DxStatus = DX_NOT_ENOUGH_SYSTEM_MEMORY;
            break;
        }

        //Calculate the contrast look-up table.
        DxStatus = DxGetContrastLut ((int)i64ContrastParam, pContrastLut,
                                     &i32LutLength);
        if (DxStatus != DX_OK)
        {
            break;
        }
    }while(0);

    //If the setting of look-up table is failed, release the resource.
    if (nStatus != DX_OK)
    {
        if (pGammaLut != NULL)
        {
            delete []pGammaLut;
            pGammaLut = NULL;
        }
        if (pContrastLut != NULL)
        {
            delete[] pContrastLut;
            pContrastLut = NULL;
        }
        return;
    }
    DX_RGB_CHANNEL_ORDER emChannelOrder = DX_ORDER_RGB;

    //Output image ARGB data
    BYTE * pOutputBuffer = new BYTE[i32Width * i32Height];/
    if (pOutputBuffer == NULL)
    {
        return;
    }
    else
    {
        //Buffer initialization
        memset(pOutputBuffer, 0, i32Width * i32Height * sizeof(BYTE));
    }

    //Improve the quality of the image.
    //pInputBuffer is input image data
    DxStatus = DxImageImprovmentEx(pInputBuffer,pOutputBuffer,i32Width,
                                   i32Height, i64ColorCorrectionParam,
                                   pContrastLut, pGammaLut, emChannelOrder);
```

```
if (pGammaLut!= NULL)
{
    delete []pGammaLut;
    pGammaLut = NULL;
}

if (pContrastLut!= NULL)
{
    delete []pContrastLut;
    pContrastLut = NULL;
}
```

8.4.25. DxImageMirror

Declarations:

VxInt32 DHDECLDxImageMirror (void *pInputBuffer, void *pOutputBuffer,
VxUint32 nWidth, VxUint32 nHeight,
DX_IMAGE_MIRROR_MODE emMirrorMode)

Descriptions:

This function is used to generate a mirror image of the original image in the horizontal or vertical direction, and the input image is 8 bits raw image or 8 bits monochrome image.

Parameters:

pInputBuffer	The data buffer of original image.
pOutputBuffer	The data buffer of target image.
nWidth	The width of the image.
nHeight	The height of the image.
emMirrorMode	The mode of image mirror, HORIZONTAL_MIRROR and VERTICAL_MIRROR, that is horizontal mirror flip and vertical mirror flip.

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
int32_t i32Width = 1600;
int32_t i32Height = 1234;

//Output image ARGB data
BYTE * pOut8BitBuf = new BYTE[i32Width * i32Height];/
if (pOut8BitBuf == NULL)
{
    return;
}
else
{
    //Buffer initialization
    memset(pOut8BitBuf, 0, i32Width * i32Height * sizeof(BYTE));
}
//Horizontal mirror an image. (Inputs and outputs can not be the same
//buffer).
VxInt32 DxStatus=DxImageMirror((BYTE*)pIn8BitBuf,
                               (BYTE*)pOut8BitBuf,
```

```
                                    i32Width, i32Height,
                                    HORIZONTAL_MIRROR);
if (DxStatus != DX_OK)
{
   return ;
}
//Vertical mirror an image. (Inputs and outputs can not be the same
//buffer).
VxInt32 DxStatus=DxImageMirror((BYTE*)pIn8BitBuf,
                               (BYTE*)pOut8BitBuf, i32Width,
                                i32Height, VERTICAL_MIRROR);
if (DxStatus != DX_OK)
{
   return ;
}
```

Effect images:

Figure 8-14 shows an 8 bits monochrome image. Figure 8-15 shows the image after horizontal flip. Figure 8-16 shows the image after vertical flip.

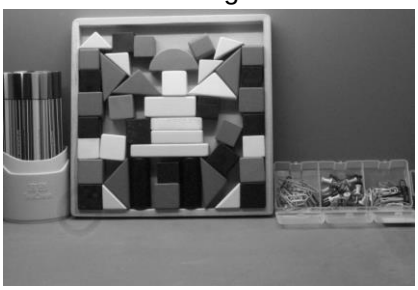


Figure 8-14: Original image

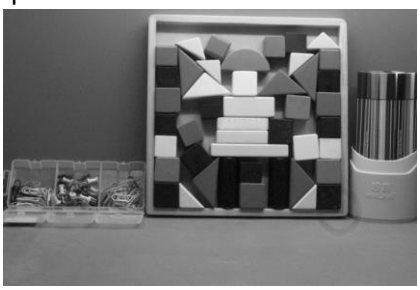


Figure 8-15: The image after horizontal flip

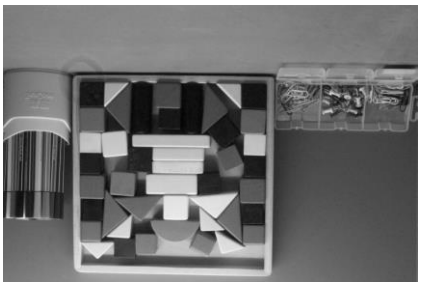


Figure 8-16: The image after vertical flip

8.4.26. DxImageMirror16B

Declarations:

VxInt32 DHDECL DxImageMirror16B (void *pInputBuffer, void *pOutputBuffer,
VxUint32 nWidth, VxUint32 nHeight,
DX_IMAGE_MIRROR_MODE emMirrorMode)

Descriptions:

This function generates a mirror image that is horizontally or vertically symmetrical to the original image, with the input image being a 16 bit raw image or a 16 bit mono image.

Parameters:

pInputBuffer	The data buffer of original image.
pOutputBuffer	The data buffer of target image.
nWidth	The width of the image.
nHeight	The height of the image.
emMirrorMode	The mode of image mirror, HORIZONTAL_MIRROR and VERTICAL_MIRROR, that is horizontal mirror flip and vertical mirror flip.

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Sample Code:

```
    int32_t i32Width = 1600;
    int32_t i32Height = 1234;

    //Output image ARGB data
    BYTE * pOut8BitBuf = new BYTE[i32Width * i32Height * 2];
    if (pOut8BitBuf == NULL)
    {
        return;
    }
    else
    {
        //Buffer initialization
        memset(pOut8BitBuf, 0, i32Width * i32Height * sizeof(BYTE) * 2);
    }
    //Image horizontal mirror flipping (input and output cannot be the
    //same buffer)
    VxInt32 DxStatus = DxImageMirror16B((BYTE*)pIn16BitBuf, (BYTE*)pOut16BitBuf,
                                        i32Width, i32Height, HORIZONTAL_MIRROR);
    if (DxStatus != DX_OK)
    {
        return ;
    }
    //Image vertical mirror flipping (input and output cannot be the
    //same buffer)
    //pIn16BitBuf is input image data
    DxStatus = DxImageMirror16B((BYTE*)pIn16BitBuf, (BYTE*)pOut16BitBuf,
                                i32Width, i32Height, VERTICAL_MIRROR);
    if (DxStatus != DX_OK)
    {
        return ;
    }
```

8.4.27. DxGetLut

Declarations:

VxInt32 DHDECL DxGetLut (VxInt32 nContrastParam, double dGamma, VxInt32 nLightness, VxUint8 *pLut, VxUint16 *pLutLength);

Descriptions:

The function is used to calculate the 8bit look-up table of image processing.

Parameters:

nContrastParam	The parameter of contrast adjustment. Range: -50~100
nGamma	The parameter of Gamma adjustment. Range: 0.1~10
nLightness	The parameter of lightness adjustment. Range: -150~150
pLut	Look-up table. If the contrast, Gamma or lightness parameters change, it needs to be recalculated.
pLutLength	The length of look-up table, unit: byte.

If the length value inputted by the user not equal to the current actual length value, it will return errors, and the *pLutLength* returns the actual length value.

If the *pLut* inputted by the user is NULL, it returns success, the *pLutLength* returns the actual length value.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

If the operation succeeds, the pLutLength returns the actual length value.

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
VxInt32 i32ContrastParam = 0;
VxUint16  ui16LutLength = 0;
int32_t  i32Lightness = 0;
double dGamma = 0.0;

//Gets the length of the look-up table.

VxInt32 DxStatus = DxGetLut (i32ContrastParam, dGamma, i32Lightness,
                             NULL, &ui16LutLength);
 if (DxStatus != DX_OK)
 {
    return;
 }

 //Applies memory for the look-up table.
VxUint8* pLut = new VxUint8[ui16LutLength];
if (pLut == NULL)
{
   return;
}

 //Calculates and gets the look-up table.
DxStatus = DxGetLut(i32ContrastParam, dGamma, i32Lightness, pLut,
                    &ui16LutLength);
 if (DxStatus != DX_OK)
 {
    if (pLut!= NULL)
    {
       delete []pLut;
       pLut = NULL;
    }
    return;
 }

//Image processing.
//……

if (pLut!= NULL)
{
   delete []pLut;
   pLut = NULL;
 }
```

8.4.28. DxCalcCCParam

Declarations:

VxInt32 DHDECL DxCalcCCParam (VxInt64 nColorCorrectionParam, VxInt16nSaturation,
VxInt16 *parrCC, VxUint8nLength);

Descriptions:

The function is used to calculate the array of image color correction.

Parameters:

nColorCorrectionParam	The color correction value, you can get it by the GX_INT_COLOR_CORRECTION_PARAM code of the GxIAPI library, and you can also set it to 0.
nSaturation	The parameter of saturation adjustment, range: 0~128
parrCC	The address of the array.
nLength	The length of array (sizeof (VxInt16 * 9)).

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
//Applies memory for the image color adjustment array.
VxInt64 nColorCorrectionParam = 0;
VxInt16 nSaturation = 0;
VxInt16* parrCC = new VxInt16[sizeof (VxInt16) * 9];
if (parrCC== NULL)
{
   return;
}

//Gets the color correction parameter.
emStatus = GXGetInt (hDevice, GX_INT_COLOR_CORRECTION_PARAM,
                     &nColorCorrectionParam);

//Calculates the color adjustment array.
DxStatus = DxCalcCCParam(nColorCorrectionParam, nSaturation, parrCC,
                         sizeof (VxInt16) * 9);
if (DxStatus != DX_OK)
{
    if (parrCC!= NULL)
   {
      delete []parrCC;
      parrCC= NULL;
   }
    return;
}

//Image processing.
//.....................

if (parrCC!= NULL)
{
   delete []parrCC;
   parrCC= NULL;
}
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

8.4.29. DxRaw8ImgProcess

Declarations:

VxInt32 DHDECL DxRaw8ImgProcess (void *pRaw8Buf, void *pRgbBuf, VxInt32 nWidth,
VxInt32 nHeight,
COLOR_IMG_PROCESS *pstClrImageProc);

Descriptions:

The function is used to process the raw 8bit image.

Parameters:

pRaw8Buf	Point to the original image 8-bit data buffer.
pRgbBuf	Point to the target image data buffer (RGB data), the size is image width *image height*3.
nWidth	The width of the image.
nHeight	The height of the image.

Note: If the current COLOR_IMG_PROCESS:: bAccelerate is set to true, to speed up, the *nHeight* must be the integer multiple of 4.

pstClrImageProc The structure pointer of color image processing method, and the define as follows:

```
typedef struct    COLOR_IMG_PROCESS
{
    bool                    bDefectivePixelCorrect;    /// The switch of bad pixel correction.
    bool                    bDenoise;                  /// The switch of denoise.
    bool                    bSharpness;                /// The switch of sharpness.
    bool                    bAccelerate;               /// The switch of acceleration.
    VxInt16                 *parrCC;                   /// The array address of color processing
                                                           parameter.
    VxUint8                 nCCBufLength;              /// The length of parrCC (sizeof (VInt16)*9).
    float                   fSharpFactor;              /// The factor of sharpness.
    VxUint8                 *pProLut;                  /// The buffer of look-up table.
    VxUint16                nLutLength;                /// The length of look-up table.
    DX_BAYER_CONVERT_TYPE   cvType;                    /// The interpolation method.
    DX_PIXEL_COLOR_FILTER   emLayOut;                  /// The BAYER format.
    bool                    bFlip;                     /// The flip sign.
    VxUint8                 arrReserved[32];           /// Reserve 32byte.
} COLOR_IMG_PROCESS;
```

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
VxInt32 i32ContrastParam = 0;
VxUint16  ui16LutLength = 0;
int32_t  i32Lightness = 0;
double dGamma = 0.0;
float fSharpen = 0.0;
int32_t i32Width = 1600;
int32_t i32Height = 1234;
int64_t i64ContrastParam = 0;
int64_t i64ColorCorrectionParam = 0;
BYTE* pContrastLut;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
int32_t nPixelColorFilter = 0;
VxInt16 nSaturation = 0;

//Initializes the structure parameters of the color image processing
//function settings.
COLOR_IMG_PROCESS stClrImageProc;

stClrImageProc.bAccelerate= false;
stClrImageProc.bDefectivePixelCorrect= false;
stClrImageProc.bDenoise= false;
stClrImageProc.bFlip= true;
stClrImageProc.bSharpness = false;
stClrImageProc.fSharpFactor= fSharpen;
stClrImageProc.cvType= RAW2RGB_NEIGHBOUR;
stClrImageProc.emLayOut= (DX_PIXEL_COLOR_FILTER)nPixelColorFilter;

//Gets the look-up table length.
VxInt32DxStatus = DxGetLut(i32ContrastParam,dGamma,i32Lightness,
                           NULL, &stClrImageProc.nLutLength);
if (DxStatus != DX_OK)
{
    return;
}

//Applies memory for the look-up table.
stClrImageProc.pProLut = new VxUint8[stClrImageProc.nLutLength];
if (pContrastLut == NULL)
{
   return;
}

//Calculates and gets the look-up table.
DxStatus = DxGetLut(i32ContrastParam, dGamma,i32Lightness,
                    stClrImageProc.pProLut,
                    &stClrImageProc.nLutLength);
if (DxStatus != DX_OK)
{
    if (stClrImageProc.pProLut!= NULL)
   {
      delete []stClrImageProc.pProLut;
      stClrImageProc.pProLut= NULL;
   }
    return;
}

//Applies memory for the image color adjustment array.
stClrImageProc.nCCBufLength = sizeof (VxInt16)*9;
stClrImageProc.parrCC = new VxInt16[stClrImageProc.nCCBufLength];
if (stClrImageProc.parrCC == NULL)
{
    if (stClrImageProc.pProLut!= NULL)
   {
      delete []stClrImageProc.pProLut;
      stClrImageProc.pProLut = NULL;
```

```
    }

    return;
}

//Gets the color correction parameter.
emStatus = GXGetInt (hDevice, GX_INT_COLOR_CORRECTION_PARAM,
                     &i64ColorCorrectionParam);

//Calculates the color adjustment array.
DxStatus = DxCalcCCParam(i64ColorCorrectionParam, nSaturation,
                         stClrImageProc.parrCC,
                         stClrImageProc.nCCBufLength);
if (DxStatus != DX_OK)
{
    if (stClrImageProc.pProLut!= NULL)
    {
        delete []stClrImageProc.pProLut;
        stClrImageProc.pProLut = NULL;
    }

    if (stClrImageProc.parrCC!= NULL)
    {
        delete []stClrImageProc.parrCC;
        stClrImageProc.parrCC= NULL;
    }
    return;
}

//Output image ARGB data
BYTE * pRgbBuf = new BYTE[i32Width * i32Height];
if (pRgbBuf == NULL)
{
    return;
}
else
{
    //Buffer initialization
    memset(pRgbBuf, 0, i32Width * i32Height * sizeof(BYTE));
}

//Processes the 8 bits Raw images
// pRaw8Buf is the input image data
emStatus = DxRaw8ImgProcess(pRaw8Buf,pRgbBuf, i32Width, i32Height,
                            &stClrImageProc);

if (stClrImageProc.pProLut!= NULL)
{
    delete []stClrImageProc.pProLut;
    stClrImageProc.pProLut = NULL;
}
if (stClrImageProc.parrCC!= NULL)
{
```

```
    delete []stClrImageProc.parrCC;
    stClrImageProc.parrCC= NULL;
}
```

8.4.30. DxMono8ImgProcess

Declarations:

VxInt32 DHDECL DxMono8ImgProcess(void *pInputBuf, void *pOutputBuf,
VxUint32 nWidth, VxUint32 nHeight,
MONO_IMG_PROCESS *pstGrayImgProc);

Descriptions:

The function is used to process the mono 8bit image.

Parameters:

pInputBuf	Point to the 8-bit data buffer of original image.
pOutputBuf	Point to the data buffer of target image, the size is image width * image height
nWidth	The width of the image.
nHeight	The height of the image.

Note: If the current MONO_IMG_PROCESS:: bAccelerate is set to true, to speed up, the *nHeight* must be the integer multiple of 4.

pstGrayImgProc The structure pointer of mono image processing method, and the define as follows:

```
Typedef struct MONO_IMG_PROCESS
{
    bool      bDefectivePixelCorrect;   /// The switch of bad pixel correction.
    bool      bSharpness;               /// The switch of sharpness.
    bool      bAccelerate;              /// The switch of acceleration.
    float     fSharpFactor;             /// The factor of sharpness.
    VxUint8   *pProLut;                 /// The buffer of look-up table.
    VxUint16  nLutLength;               /// The length of look-up table.
    VxUint8   arrReserved[32];          /// Reserve 32byte.
} MONO_IMG_PROCESS;
```

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
VxInt32 i32ContrastParam = 0;
int32_t  i32Lightness = 0;
double dGamma = 0.0;
float fSharpen = 0.0;
int32_t i32Width = 1600;
int32_t i32Height = 1234;
BYTE* pContrastLut;

//Initializes the structure parameters of the Mono8 image processing
//function settings.
MONO_IMG_PROCESS stGrayImageProc;

stGrayImageProc.bAccelerate= false;
stGrayImageProc.bDefectivePixelCorrect= false;
stGrayImageProc.bSharpness           = false;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
stGrayImageProc.fSharpFactor= fSharpen;

//Gets the length of look-up table.
VxInt32 DxStatus = DxGetLut (i32ContrastParam,dGamma,i32Lightness,
                             NULL, &stGrayImageProc.nLutLength);
if (DxStatus != DX_OK)
{
    return;
}

//Applies memory for the look-up table.
stGrayImageProc.pProLut = new VxUint8[stGrayImageProc.nLutLength];
if (pContrastLut == NULL)
{
   return;
}

//Calculates and gets the look-up table.
DxStatus = DxGetLut(i32ContrastParam, dGamma,i32Lightness,
                    stGrayImageProc.pProLut,
                    &stGrayImageProc.nLutLength);
if (DxStatus != DX_OK)
{
    if (stGrayImageProc.pProLut!= NULL)
   {
      delete []stGrayImageProc.pProLut;
       stGrayImageProc.pProLut = NULL;
   }
    return;
}

//Output image ARGB data
BYTE * pOutputBuf = new BYTE[i32Width * i32Height];
if (pOutputBuf == NULL)
{
   return;
}
else
{
   //Buffer initialization
   memset(pOutputBuf, 0, i32Width * i32Height * sizeof(BYTE));
}

//Processes the 8 bits mono images.
//pInputBuf is input image data
emStatus = DxMono8ImgProcess(pInputBuf,pOutputBuf, i32Width,
                             i32Height, &stGrayImageProc);

if (stGrayImageProc.pProLut!= NULL)
{
   delete []stGrayImageProc.pProLut;
   stGrayImageProc.pProLut = NULL;
}
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

8.4.31. DxGetFFCCoefficients

Declarations:

```
VxInt32 DHDECL DxGetFFCCoefficients
                    (FLAT_FIELD_CORRECTION_PROCESS stFlatFieldCorrection,
                    void *pFFCCoefficients, int *pnLength,
                    int *pnTargetValue = NULL);
```

Descriptions:

This function is used to calculate the flat field correction coefficients, and only the Raw images between the bits of 8 and 12 are supported.

Parameters:

nContrastParam	The flat field correction process structure.
pFFCCoefficients	The flat field correction coefficients.
pnlength	The length of flat field correction coefficients, and its unit is byte.
pnTargetValue	The target value of flat field correction processing, and the default is NULL.

If the length value inputted by the user less than the current actual length value, it will return errors, and the pnLength returns the actual length value.
If the pFFCCoefficients inputted by the user is NULL, it returns success, the pnLength returns the actual length value.
If the operation succeeds, the pnLength returns the actual length value.
The definition of the flat field correction process structure is as follows:

```
typedef   struct   FLAT_FIELD_CORRECTION_PROCESS
{
    void                    *pBrightBuf;      ///< Bright image buffer
    void                    *pDarkBuf;        ///< Dark image buffer
    VxUint32                 nImgWid;         ///< Image width
    VxUint32                 nImgHei;        ///< Image height
    DX_ACTUAL_BITS           nActualBits;     ///< Image actual bits
    DX_PIXEL_COLOR_FILTER    emBayerType;     ///< BAYER format
} FLAT_FIELD_CORRECTION_PROCESS;
```

Note: When calculating the flat field correction coefficient, the dark field image may not be acquired, then the pointer to the dark field image should be set to NULL.

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
int32_t i32Width = 1600;
int32_t i32Height = 1234;
int32_t i32FFClength = 0;

//Initializes the structure of the flat field correction process
//structure.
//pImgBrightBuf is the bright field image data
//pImgDarkBuf is the dark field image data
FLAT_FIELD_CORRECTION_PROCESS stFlatFieldCorrection;
stFlatFieldCorrection.pBrightBuf = pImgBrightBuf;

//If the dark field image is not acquired, set it to NULL.
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
stFlatFieldCorrection.pDarkBuf    = pImgDarkBuf;
stFlatFieldCorrection.nImgWid     = i32Width;
stFlatFieldCorrection.nImgHei     = i32Height;
stFlatFieldCorrection.nActualBits = DX_ACTUAL_BITS_8;
stFlatFieldCorrection.emBayerType = BAYERRG;

//Get the length of flat field correction coefficients.
VxInt32 DxStatus= DxGetFFCCoefficients (stFlatFieldCorrection, NULL,
                                        &i32FFClength);
if(DxStatus != DX_OK)
{
    return;
}

//Apply memory for the flat field correction coefficients.
BYTE* pFFCCoefficients = new BYTE[i32FFClength];
if (pFFCCoefficients == NULL)
{
    return;
}

//Calculate the flat field correction coefficients.
DxStatus = DxGetFFCCoefficients (stFlatFieldCorrection, pFFCCoefficients,
                                 &i32FFClength);
if (DxStatus != DX_OK)
{
    if (pFFCCoefficients!= NULL)
    {
        delete []pFFCCoefficients;
        pFFCCoefficients = NULL;
    }
    return;
}
//Image processing
//……
if (pFFCCoefficients!= NULL)
{
    delete []pFFCCoefficients;
    pFFCCoefficients = NULL;
}
```

8.4.32. DxFlatFieldCorrection

Declarations:

VxInt32 DHDECL DxFlatFieldCorrection (void *pImgBuf, DX_ACTUAL_BITS nActualBits,
VxUint32 nImgWidth, VxUint32 nImgHeight,
void *pFFCCoefficients, int *pnLength);

Descriptions:

This function performs the flat field correction operation on the images whose bits are between 8 and 12. Besides, the function supports raw images in Bayer format, and does not support the raw image of Packet format.

Parameters:

pInputBuffer	Point to the data buffer of the original image.
pOutputBuffer	Point to the data buffer of the target image.
nActualBits	The image actual bits.
nImgWidth	The image width.
nImgHeight	The image height.
pFFCCoefficients	The flat field correction coefficients.
pnLength	The length of flat field correction coefficients, and its unit is byte.

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Sample Code:

```
int32_t i32Width = 1600;
int32_t i32Height = 1234;
int32_t i32FFClength = 0;
DX_ACTUAL_BITS nActualBits = DX_ACTUAL_BITS_8;

//Output image ARGB data
BYTE * pOutputBuffer = new BYTE[i32Width * i32Height];
if (pOutputBuffer == NULL)
{
   return;
}
else
{
   //Buffer initialization
   memset(pOutputBuffer, 0, i32Width * i32Height * sizeof(BYTE));
}

//Performing flat field correction on images
//pInputBuffer is the input image data
//pFFCCoefficients is the flat field correction coefficient data
VxInt32 DxStatus = DxFlatFieldCorrection (pInputBuffer, pOutputBuffer,
                                          nActualBits, i32Width,
                                          i32ImgHeight, pFFCCoefficients,
                                   &i32FFClength);
if (DxStatus != DX_OK)
{
   if (pFFCCoefficients!= NULL)
       {
          delete []pFFCCoefficients;
          pFFCCoefficients = NULL;
       }
   return;
}
//Processes Raw images.
//.................
```

8.4.33. DxCalcUserSetCCParam

Declaration:

VxInt32 DHDECL DxCalcUserSetCCParam(COLOR_TRANSFORM_FACTOR

*pstColorTransformFactor, VxInt16 nSaturation,
VxInt16 *parrCC, VxUint8 nLength);

Descriptions:

The function is used to calculate the array of image color correction. (UserSet mode)

Formal parameter:

pstColorTransformFactor	The color correction value by user set, and you can also set it to NULL.
nSaturation	The parameter of saturation adjustment, range: 0~128
parrCC	The address of the array.
nLength	The length of array (sizeof (VxInt16 * 9)).

Note: The recommended range of color correction structure parameters is -4 ~ 4. If the parameter setting is less than -4, the correction effect is consistent with -4. If the parameter setting is greater than 4, the correction effect is consistent with 4.

Return value:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Code Sample:

```
//Apply memory for the image color adjustment array.
VxInt16* parrCC = new VxInt16[sizeof (VxInt16) * 9];
if (parrCC== NULL)
{
    return;
}

//Set color transform factor.
COLOR_TRANSFORM_FACTOR stColorTransformFactor;

stColorTransformFactor.fGain00 = 1.463680;
stColorTransformFactor.fGain01 = -0.545809;
stColorTransformFactor.fGain02 = 0.082128;
stColorTransformFactor.fGain10 = -0.229388;
stColorTransformFactor.fGain11 = 1.354321;
stColorTransformFactor.fGain12 = -0.124932;
stColorTransformFactor.fGain20 = 0.097042;
stColorTransformFactor.fGain21 = -0.627857;
stColorTransformFactor.fGain22 = 1.530815;

//Set Saturation factor
VxInt16 nSaturation = 64;
VxInt32 DxStatus = DxCalcUserSetCCParam (&stColorTransformFactor,
                                         nSaturation, parrCC,
                                         sizeof (VxInt16) * 9);
if (DxStatus != DX_OK)
{
    if (parrCC!= NULL)
    {
        delete []parrCC;
        parrCC= NULL;
    }
    return;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    }

    //Image processing.
    //.....................


    if (parrCC!= NULL)
    {
        delete []parrCC;
        parrCC= NULL;
    }
```

8.4.34. DxImageFormatConvert

Declaration:

VxInt32 DHDECL DxImageFormatConvert (DX_IMAGE_FORMAT_CONVERT_HANDLE handle,
void *pInputBuffer, int nInBufferSize,
void *pOutputBuffer, int nOutBufferSize,
GX_PIXEL_FORMAT_ENTRY emInPixelFormat,
VxUint32 nImgWidth, VxUint32 nImgHeight,
bool bFlip);

Descriptions:

The function is used to execute image format conversion.

Formal parameter:

handle	Image Format Conversion handle, it cannot be NULL
pInputBuffer	Input Buffer Supposed unsigned char type
nInBufferSize	Input Buffer Size（Calculated based on unsigned char type）
pOutputBuffer	Output Buffer Supposed unsigned char type
nOutBufferSize	Output Buffer Size（Calculated based on unsigned char type）
emInPixelFormat	Input Image Format
nImgWidth	Image Width
nImgHeight	Image Height
bFlip	If flip, the value is **true**, and the image will vertical flip; otherwise, the value is **false**, not flip

Supported inputs and outputs:

	Source image format	Destination image format
Bayer 8bit	GX_PIXEL_FORMAT_BAYER_GR8 GX_PIXEL_FORMAT_BAYER_RG8 GX_PIXEL_FORMAT_BAYER_GB8 GX_PIXEL_FORMAT_BAYER_BG8	GX_PIXEL_FORMAT_MONO8 GX_PIXEL_FORMAT_RGB8 GX_PIXEL_FORMAT_BGR8 GX_PIXEL_FORMAT_RGBA8 GX_PIXEL_FORMAT_BGRA8 GX_PIXEL_FORMAT_ARGB8 GX_PIXEL_FORMAT_ABGR8 GX_PIXEL_FORMAT_RGB8_PLANAR

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Bayer 16bit	GX_PIXEL_FORMAT_BAYER_GR10 GX_PIXEL_FORMAT_BAYER_RG10 GX_PIXEL_FORMAT_BAYER_GB10 GX_PIXEL_FORMAT_BAYER_BG10 GX_PIXEL_FORMAT_BAYER_GR12 GX_PIXEL_FORMAT_BAYER_RG12 GX_PIXEL_FORMAT_BAYER_GB12 GX_PIXEL_FORMAT_BAYER_BG12 GX_PIXEL_FORMAT_BAYER_GR16 GX_PIXEL_FORMAT_BAYER_RG16 GX_PIXEL_FORMAT_BAYER_GB16 GX_PIXEL_FORMAT_BAYER_BG16	GX_PIXEL_FORMAT_MONO16 GX_PIXEL_FORMAT_RGB16 GX_PIXEL_FORMAT_BGR16 GX_PIXEL_FORMAT_RGB8 GX_PIXEL_FORMAT_BGR8 GX_PIXEL_FORMAT_RGB16_PLANAR
RGB 8bit	GX_PIXEL_FORMAT_RGB8 GX_PIXEL_FORMAT_BGR8	GX_PIXEL_FORMAT_YUV444_8 GX_PIXEL_FORMAT_YUV422_8 GX_PIXEL_FORMAT_YUV411_8 GX_PIXEL_FORMAT_YUV420_8_PLANAR GX_PIXEL_FORMAT_YCBCR444_8 GX_PIXEL_FORMAT_YCBCR422_8 GX_PIXEL_FORMAT_YCBCR411_8 GX_PIXEL_FORMAT_MONO8 GX_PIXEL_FORMAT_RGB8
RGB 16bit	GX_PIXEL_FORMAT_RGB16 GX_PIXEL_FORMAT_BGR16	GX_PIXEL_FORMAT_MONO16
COORD3D_C16	GX_PIXEL_FORMAT_COORD3D_C16	GX_PIXEL_FORMAT_COORD3D_ABC32F GX_PIXEL_FORMAT_COORD3D_ABC32F_PLANAR
Mono Packed (GVSP)	GX_PIXEL_FORMAT_MONO10_PACKED GX_PIXEL_FORMAT_MONO12_PACKED GX_PIXEL_FORMAT_MONO10_P GX_PIXEL_FORMAT_MONO12_P GX_PIXEL_FORMAT_MONO14_P	GX_PIXEL_FORMAT_MONO8 GX_PIXEL_FORMAT_MONO10 GX_PIXEL_FORMAT_MONO12 GX_PIXEL_FORMAT_MONO14 GX_PIXEL_FORMAT_MONO16 GX_PIXEL_FORMAT_RGB8 GX_PIXEL_FORMAT_BGR8
Bayer Packed (GVSP)	GX_PIXEL_FORMAT_BAYER_GR10_PACKED GX_PIXEL_FORMAT_BAYER_RG10_PACKED GX_PIXEL_FORMAT_BAYER_GB10_PACKED GX_PIXEL_FORMAT_BAYER_BG10_PACKED GX_PIXEL_FORMAT_BAYER_GR12_PACKED GX_PIXEL_FORMAT_BAYER_RG12_PACKED GX_PIXEL_FORMAT_BAYER_GB12_PACKED GX_PIXEL_FORMAT_BAYER_BG12_PACKED GX_PIXEL_FORMAT_BAYER_GR10_P GX_PIXEL_FORMAT_BAYER_RG10_P GX_PIXEL_FORMAT_BAYER_GB10_P GX_PIXEL_FORMAT_BAYER_BG10_P GX_PIXEL_FORMAT_BAYER_GR12_P GX_PIXEL_FORMAT_BAYER_RG12_P GX_PIXEL_FORMAT_BAYER_GB12_P GX_PIXEL_FORMAT_BAYER_BG12_P GX_PIXEL_FORMAT_BAYER_GR14_P	GX_PIXEL_FORMAT_RGB8 GX_PIXEL_FORMAT_BGR8 GX_PIXEL_FORMAT_BAYER_GR8 GX_PIXEL_FORMAT_BAYER_RG8 GX_PIXEL_FORMAT_BAYER_GB8 GX_PIXEL_FORMAT_BAYER_BG8 GX_PIXEL_FORMAT_BAYER_GR10 GX_PIXEL_FORMAT_BAYER_RG10 GX_PIXEL_FORMAT_BAYER_GB10 GX_PIXEL_FORMAT_BAYER_BG10 GX_PIXEL_FORMAT_BAYER_GR12 GX_PIXEL_FORMAT_BAYER_RG12 GX_PIXEL_FORMAT_BAYER_GB12 GX_PIXEL_FORMAT_BAYER_BG12 GX_PIXEL_FORMAT_BAYER_GR14 GX_PIXEL_FORMAT_BAYER_RG14 GX_PIXEL_FORMAT_BAYER_GB14

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

	GX_PIXEL_FORMAT_BAYER_RG14_P GX_PIXEL_FORMAT_BAYER_GB14_P GX_PIXEL_FORMAT_BAYER_BG14_P	GX_PIXEL_FORMAT_BAYER_BG14 GX_PIXEL_FORMAT_BAYER_GR16 GX_PIXEL_FORMAT_BAYER_RG16 GX_PIXEL_FORMAT_BAYER_GB16 GX_PIXEL_FORMAT_BAYER_BG16

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Code Sample:

```
//Declaration handle for Image format conversion.
DX_IMAGE_FORMAT_CONVERT_HANDLE handle;
DX_STATUS emDxStatus = DX_OK;

//Create handle.
DxImageFormatConvertCreate(&handle);
if (handle == NULL)
{
    return;
    }

GX_PIXEL_FORMAT_ENTRY emOutPixelFormat = GX_PIXEL_FORMAT_RGBA8;
GX_PIXEL_FORMAT_ENTRY emInPixelFormat  = GX_PIXEL_FORMAT_BAYER_RG8;

VxUint8 nAlphaValue = 128;
DX_BAYER_CONVERT_TYPE cvtype = RAW2RGB_NEIGHBOUR;
int nBufferSizeIn = 0;
int nBufferSizeOut = 0;
int nWidth      = 720;
int nHeight     = 540;
unsigned char *pszBufferIn = NULL;
unsigned char *pszBufferOut = NULL;
bool bFlip = false;
//Set Output format.
emDxStatus = (DX_STATUS)DxImageFormatConvertSetOutputPixelFormat (
                                                    handle, emPixelFormat);
if (emDxStatus != DX_OK)
{
    DxImageFormatConvertDestroy(handle);
    handle = NULL;
    return;
}

//Set alpha value.
emDxStatus = (DX_STATUS)DxImageFormatConvertSetAlphaValue(
                                                        handle, nAlphaValue);
if (emDxStatus != DX_OK)
{
    DxImageFormatConvertDestroy(handle);
    handle = NULL;
    return;
}
```

```
 //Set interpolation type.
 emDxStatus = (DX_STATUS)DxImageFormatConvertSetInterpolationType(
                                                        handle, cvtype);
if (emDxStatus != DX_OK)
 {
     DxImageFormatConvertDestroy(handle);
    handle = NULL;
    return;
}

 //Get buffer size for output buffer.
 emDxStatus=(DX_STATUS)DxImageFormatConvertGetBufferSizeForConversion(
                                          handle, emOutPixelFormat,
                                          nWidth, nHeight, &nBufferSizeOut);
if (emDxStatus != DX_OK)
 {
     DxImageFormatConvertDestroy(handle);
    handle = NULL;
    return;
}

 //Get buffer size for input buffer.
 emDxStatus = (DX_STATUS)DxImageFormatConvertGetBufferSizeForConversion (
                                          handle, emInPixelFormat, nWidth,
                                          nHeight,  &nBufferSizeIn);
if (emDxStatus != DX_OK)
 {
     DxImageFormatConvertDestroy(handle);
    handle = NULL;
    return;
}

//Apply memory for buffer.
pszBufferIn = new unsigned char[nBufferSizeIn];
if (pszBufferIn == NULL)
 {
     DxImageFormatConvertDestroy(handle);
    handle = NULL;
    return;
}
//Apply memory for buffer.
pszBufferOut = new unsigned char[nBufferSizeOut];
if (pszBufferOut == NULL)
 {
     DxImageFormatConvertDestroy(handle);
    handle = NULL;

    delete pszBufferIn;
    pszBufferIn = NULL;
    return;
}
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
//Execute image format conversion.
emDxStatus = (DX_STATUS)DxImageFormatConvert(handle, pszBufferIn ,
                                             nBufferSizeIn , pszBufferOut,
                                             nBufferSizeOut, emInPixelFormat,
                                             nWidth,nHeight,bFlip);


 DxImageFormatConvertDestroy(handle);
 handle = NULL;
```

8.4.35. DxImageFormatConvertCreate

Declaration:

VxInt32 DHDECL DxImageFormatConvertCreate (DX_IMAGE_FORMAT_CONVERT_HANDLE *phandle);

Descriptions:

The function is used to create handle for image format conversion.

Formal parameter:

phandle	The handle for image format conversion

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Code Sample:

```
 //Declaration handle for Image format conversion.
DX_IMAGE_FORMAT_CONVERT_HANDLE handle;
DX_STATUS emDxStatus  = DX_OK;

 //Create handle.
 DxStatus = (DX_STATUS)DxImageFormatConvertCreate(&handle);
```

8.4.36. DxImageFormatConvertDestroy

Declaration:

VxInt32 DHDECL DxImageFormatConvertDestroy (DX_IMAGE_FORMAT_CONVERT_HANDLE handle);

Descriptions:

The function is used to destroy the handle.

Formal parameter:

handle	The handle for image format conversion

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Code Sample:

```
 //Declaration handle for Image format conversion.
DX_IMAGE_FORMAT_CONVERT_HANDLE handle;
DX_STATUS emDxStatus  = DX_OK;

 //Create handle.
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    emDxStatus = (DX_STATUS)DxImageFormatConvertCreate(&handle);
    if (handle == NULL)
    {
        return;
    }

    //Destroy handle.
    emDxStatus = (DX_STATUS)DxImageFormatConvertDestroy(handle);
```

8.4.37. DxImageFormatConvertSetOutputPixelFormat

Declaration:

VxInt32 DHDECL DxImageFormatConvertSetOutputPixelFormat(
DX_IMAGE_FORMAT_CONVERT_HANDLE handle
GX_PIXEL_FORMAT_ENTRY emPixelFormat);

Descriptions:

The function is used to set Output pixel format.

Formal parameter:

handle	The handle for image format conversion
emPixelFormat	Output format refered to GX_PIXEL_FORMAT_ENTRY

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Code Sample:

```
    //Declaration The handle for image format conversion.
    DX_IMAGE_FORMAT_CONVERT_HANDLE handle;
    DX_STATUS emDxStatus  = DX_OK;

    //Create handle.
    DxImageFormatConvertCreate(&handle);
    if (handle == NULL)
    {
        return;
        }

    GX_PIXEL_FORMAT_ENTRY emPixelFormat = GX_PIXEL_FORMAT_MONO8;

    //Set Output format.
    emDxStatus = (DX_STATUS)DxImageFormatConvertSetOutputPixelFormat (handle,
                                                                      emPixelFormat);
    if (emDxStatus != DX_OK)
    {
        DxImageFormatConvertDestroy(handle);
        handle = NULL;
        return;
    }

    //Other operation.
    //.....................
    DxImageFormatConvertDestroy(handle);
```

```
    handle = NULL;
```

8.4.38. DxImageFormatConvertSetAlphaValue

Declaration:

VxInt32 DHDECL DxImageFormatConvertSetAlphaValue (
DX_IMAGE_FORMAT_CONVERT_HANDLE handle
VxUint8 nAlphaValue);

Descriptions:

The function is used to set alpha value.

Formal parameter:

handle	The handle for image format conversion
nAlphaValue	The alpha value for pixel format with alpha channel,if not set, the value is 255.

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Code Sample:

```
    //Declaration the handle for image format conversion
    DX_IMAGE_FORMAT_CONVERT_HANDLE handle;
    DX_STATUS emDxStatus  = DX_OK;

    //Create handle.
    DxImageFormatConvertCreate(&handle);
    if (handle == NULL)
    {
        return;
        }

    VxUint8 nAlphaValue = 128;

    //Set alpha value.
    emDxStatus = (DX_STATUS)DxImageFormatConvertSetAlphaValue(handle,
                                                              nAlphaValue);
    if (emDxStatus != DX_OK)
    {
        DxImageFormatConvertDestroy(handle);
        handle = NULL;
        return;
    }

    //Other operation.
    //.....................


    DxImageFormatConvertDestroy(handle);
    handle = NULL;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

8.4.39. DxImageFormatConvertSetInterpolationType

Declaration:

VxInt32 DHDECL DxImageFormatConvertSetInterpolationType (
DX_IMAGE_FORMAT_CONVERT_HANDLE handle
DX_BAYER_CONVERT_TYPE emCvtType);

Descriptions:

The function is used to set interpolation type to make bayer to RGB type.

Formal parameter:

handle	The handle for image format conversion
emCvtType	The interpolation type to make bayer to RGB type, if not set, the value is RAW2RGB_NEIGHBOUR

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Code Sample:

```
//Declaration The handle for image format conversion.
DX_IMAGE_FORMAT_CONVERT_HANDLE handle;
DX_STATUS emDxStatus  = DX_OK;

//Create handle.
DxImageFormatConvertCreate(&handle);
if (handle == NULL)
{
    return;
    }

//Declaration interpolation type.
DX_BAYER_CONVERT_TYPE cvtype = RAW2RGB_NEIGHBOUR;

//Set interpolation type.
emDxStatus = (DX_STATUS)DxImageFormatConvertSetInterpolationType(handle,
                                                                 cvtype);
if (emDxStatus != DX_OK)
{
    DxImageFormatConvertDestroy(handle);
    handle = NULL;
    return;
}

//Other operation.
//.....
DxImageFormatConvertDestroy(handle);
handle = NULL;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

8.4.40. DxImageFormatConvertGetBufferSizeForConversion

Declaration:

VxInt32 DHDECL DxImageFormatConvertGetBufferSizeForConversion (
DX_IMAGE_FORMAT_CONVERT_HANDLE handle,
GX_PIXEL_FORMAT_ENTRY emPixelFormat,
VxUint32 nImgWidth, VxUint32 nImgHeight,
int *pBufferSize);

Descriptions:

The function is used to get buffer size according to image width, image height and pixel format.

Formal parameter:

handle	The handle for image format conversion
emPixelFormat	Image Format
nImgWidth	Image Width
nImgHeight	Image Height
pBufferSize	Output Buffer Size (Calculated based on unsigned char type)

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Code Sample:

```
//DeclarationThe handle for image format conversion.
DX_IMAGE_FORMAT_CONVERT_HANDLE handle;
DX_STATUS emDxStatus  = DX_OK;

//Create handle.
DxImageFormatConvertCreate(&handle);
if (handle == NULL)
{
    return;
    }
int nBufferSize = 0;
int nWidth      = 720;
int nHeight     = 540;
GX_PIXEL_FORMAT_ENTRY emPixelFormat = GX_PIXEL_FORMAT_MONO8;

//Get buffer size for output buffer.
emDxStatus=(DX_STATUS)DxImageFormatConvertGetBufferSizeForConversion (
                                        handle, emPixelFormat, nWidth
                                        nHeight, &nBufferSize);
if (emDxStatus != DX_OK)
{
    DxImageFormatConvertDestroy(handle);
    handle = NULL;
    return;
}
//Other operation.
//.....
DxImageFormatConvertDestroy(handle);
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
    handle = NULL;
```

8.4.41. DxImageFormatConvertGetOutputPixelFormat

Declaration:

VxInt32 DHDECL DxImageFormatConvertGetOutputPixelFormat (
DX_IMAGE_FORMAT_CONVERT_HANDLE handle,
GX_PIXEL_FORMAT_ENTRY *pemPixelFormat);

Descriptions:

The function is used to get Output pixel format.

Formal parameter:

handle	The handle for image format conversion
pemPixelFormat	Output format

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Code Sample:

```
    //DeclarationThe handle for image format conversion.
    DX_IMAGE_FORMAT_CONVERT_HANDLE handle;
    DX_STATUS emDxStatus  = DX_OK;

    //Create handle.
    DxImageFormatConvertCreate(&handle);
    if (handle == NULL)
    {
        return;
    }

    GX_PIXEL_FORMAT_ENTRY emPixelFormat = GX_PIXEL_FORMAT_UNDEFINED;

    //Get Output format.
    emDxStatus = (DX_STATUS)DxImageFormatConvertGetOutputPixelFormat (
                                                    handle, &emPixelFormat);
    if (emDxStatus != DX_OK)
    {
        DxImageFormatConvertDestroy(handle);
        handle = NULL;
        return;
    }

    //Other operation.
    //.....................
    DxImageFormatConvertDestroy(handle);
    handle = NULL;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

8.4.42. DxImageFormatConvertSetValidBits

Declaration:

VxInt32 DHDECL DxImageFormatConvertSetValidBits(
DX_IMAGE_FORMAT_CONVERT_HANDLE handle,
DX_VALID_BIT emValidBits);

Descriptions:

This function sets the number of significant digits for users when converting 10, 12, 14, and 16 bit depth data to 8 bit. The default is the low 8 bit.

Formal parameter:

handle	Image format conversion handle
emValidBits	Significant digits, low 8 bits by default.

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Code Sample:

```
//Declare image format conversion handle
DX_IMAGE_FORMAT_CONVERT_HANDLE handle;
DX_STATUS emDxStatus  = DX_OK;

//Create image format conversion handle
DxImageFormatConvertCreate(&handle);
if (handle == NULL)
{
   return;
}

DX_VALID_BIT emValidBits = DX_BIT_2_9;

//Set image alpha value
emDxStatus =(DX_STATUS)DxImageFormatConvertSetValidBits(handle,
                                                        emValidBits);
if (emDxStatus != DX_OK)
{
   DxImageFormatConvertDestroy(handle);
   handle = NULL;
   return;
}

//Other operations
//.....


DxImageFormatConvertDestroy(handle);
handle = NULL;
```

8.4.43. DxImageFormatConvertSet3DcalibParam

Declaration:

VxInt32 DHDECL DxImageFormatConvertSet3DCalibParam(
DX_IMAGE_FORMAT_CONVERT_HANDLE handle,
const void* pCalibParamBuffer, const int nBufferSize);

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Descriptions:

This function is used to load 3D calibration parameters to the conversion handle, in order to call the image format conversion interface to convert the height map into a point cloud map.

Formal parameter:

handle	Image format conversion handle
pCalibParamBuffer	3D calibration parameter memory
nBufferSize	3D calibration parameter memory length

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Code Sample:

```
#include <iostream>
#include <fstream>
//Define tool function: Read buffer from bin file
DX_STATUS LoadCalibFile(const char* strFileName,
                        char*&pBuffer, int& nBufferLen)
{
    std::ifstream file(strFileName, std::ios::binary);
    if (!file)
    {
        return DX_FILE_OPEN_FAIL;
    }
    //Get file length
    file.seekg(0, std::ios::end);
    std::streampos fileSize = file.tellg();
    file.seekg(0, std::ios::beg);
    nBufferLen = fileSize;

    //Create a buffer and read the file content
    pBuffer = new char[nBufferLen];
    file.read(pBuffer, nBufferLen);
    //Close file
    file.close();
    return DX_OK;
}


GX_PIXEL_FORMAT_ENTRY emOutPixelFormat =
                                    GX_PIXEL_FORMAT_COORD3D_ABC32F;
GX_PIXEL_FORMAT_ENTRY emInPixelFormat =
                                      GX_PIXEL_FORMAT_COORD3D_C16;
int nBufferSizeIn = 0;
int nBufferSizeOut = 0;
int nWidth      = 720;
int nHeight     = 540;
unsigned char* pszBufferIn = NULL;
unsigned char* pszBufferOut = NULL;
bool bFlip = false;
DX_STATUS emDxStatus  = DX_OK;

//Declare image format conversion handle
DX_IMAGE_FORMAT_CONVERT_HANDLE handle;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
//Create image format conversion handle
DxImageFormatConvertCreate(&handle);
if (handle == NULL)
{
   return;
}

DX_VALID_BIT emValidBits = DX_BIT_2_9;

//Read buffer from calibration parameter file
char* pBuffer = NULL;
int nBufferLen = 0;
emDxStatus  = LoadCalibFile("CalibParam.bin", pBuffer, nBufferLen);
if (emDxStatus  != DX_OK)
{
   if (NULL!=pBuffer)
   {
      delete[] pBuffer;
      pBuffer=NULL;
   }
   DxImageFormatConvertDestroy(handle);
   handle = NULL;
   return;
}

emDxStatus = (DX_STATUS)DxImageFormatConvertSet3DCalibParam(
                                                        handle,
                                                        pBuffer,
                                                        nBufferLen);
if(NULL!=pBuffer)
{
   delete[] pBuffer;
   pBuffer=NULL;
}
if (emDxStatus  != DX_OK)
{
    DxImageFormatConvertDestroy(handle);
    handle = NULL;
    return;
}

//Set Y-direction step size
emDxStatus  = (DX_STATUS)DxImageFormatConvertSetYStep(handle, 0.1);

//Get output image memory size
emDxStatus = (DX_STATUS)DxImageFormatConvertGetBufferSizeForConversion(
                                                      handle,
                                                      emOutPixelFormat,
                                                      nWidth, nHeight,
                                                      &nBufferSizeOut);
if (emDxStatus  != DX_OK)
{
    DxImageFormatConvertDestroy(handle);
```

```
        handle = NULL;
        return;
    }

    //Get input image memory size
    emDxStatus=(DX_STATUS)DxImageFormatConvertGetBufferSizeForConversion(
                                                        handle,
                                                        emInPixelFormat,
                                                        nWidth,
                                                        nHeight,
                                                        &nBufferSizeIn);
    if (emDxStatus != DX_OK)
    {
        DxImageFormatConvertDestroy(handle);
        handle = NULL;
        return;
    }

    //Apply memories
    pszBufferIn = new unsigned char[nBufferSizeIn];
    if (pszBufferIn == NULL)
    {
        DxImageFormatConvertDestroy(handle);
        handle = NULL;
        return;
    }

    pszBufferOut = new unsigned char[nBufferSizeOut];
    if (pszBufferOut == NULL)
    {
        DxImageFormatConvertDestroy(handle);
        handle = NULL;

        delete pszBufferIn;
        pszBufferIn = NULL;
        return;
    }
    //Perform image format conversion
    emDxStatus  = (DX_STATUS) DxImageFormatConvert(handle, pszBufferIn,
                                                   nBufferSizeIn,
                                                   pszBufferOut,
                                                   nBufferSizeOut,
                                                   emInPixelFormat,
                                                   nWidth,
                                                   nHeight,
                                                   bFlip);

    DxImageFormatConvertDestroy(handle);
    handle = NULL;
```

8.4.44. DxImageFormatConvertSetYStep

Declaration:

VxInt32 DHDECL DxImageFormatConvertSetYStep(
DX_IMAGE_FORMAT_CONVERT_HANDLE handle,
const double dY);

Descriptions:

This function sets the Y-direction step size for converting the height map to a point cloud.

Formal parameter:

handle	Image format conversion handle
dY	Step size in Y direction

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Code Sample:

See DxImageFormatConvertSet3DCalibParam section.

8.4.45. DxStaticDefectCorrection

Declaration:

VxInt32 DHDECL DxStaticDefectCorrection(void *pInputBuffer, void *pOutputBuffer,
STATIC_DEFECT_CORRECTION stDefectCorrection,
void* pDefectPosBuffer, VxUint32 nDefectPosBufferSize);

Descriptions:

The function is used to realize the static defect correction.

Formal parameter:

pInputBuffer	The data buffer of original image
pOutputBuffer	The data buffer of target image
stDefectCorrection	The image parameters for defect correction, refer to section 8.3.5
pDefectPosBuffer	The data buffer of defect position
nDefectPosBufferSize	The data buffer size of defect position

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Code Sample:

```
//Initializes the parameters of the defect correction data structure.
STATIC_DEFECT_CORRECTION stDefectCorrection;
stDefectCorrection.nImgWidth = nImgWidth;
stDefectCorrection.nImgHeight = nImgHeight;
stDefectCorrection.nImgOffsetX = nImgOffsetX;
stDefectCorrection.nImgOffsetY = nImgOffsetY;
stDefectCorrection.nImgWidthMax = nImgWidthMax;
stDefectCorrection.nBayerType = BAYERRG;
stDefectCorrection.emActualBits= DX_ACTUAL_BITS_8;

//Read defect files in binary mode.
//Get the parameters of pDefectPosBuffer and nDefectPosBufferSize.
//Save the defect files through defect correction plugin.
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
//Execute defect correction.
VxInt32 DxStatus= DxStaticDefectCorrection (pInputBuffer,
                    pOutputBuffer, stDefectCorrection,
                    pDefectPosBuffer,nDefectPosBufferSize);
if(DxStatus != DX_OK)
{
    return;
}
```

8.4.46. DxCalcCameraLutBuffer

Declaration:

VxInt32 DHDECL DxCalcCameraLutBuffer (VxInt32 nContrastParam, double dGamma,
VxInt32 nLightness，void *pLut,
VxUint32 *pnLutLength);

Descriptions:

This function is used to adjust the value of the camera lookup table by contrast, gamma, and brightness.

Formal parameter:

nContrastParam	The parameter of Contrast adjustment, range of value: -50~100.
nGamma	The parameter of Gamma adjustment, range of value: 0.1~10.
nLightness	The parameter of Lightness adjustment, range of value: -150~150.
pLut	Lookup table, calculate according to contrast, Gamma, or brightness parameters.
pLutLength	The length of camera lookup table, unit: byte.

The length of camera lookup table is obtained through the GXGetBufferLength interface. Users apply for memory based on the obtained length. If the lookup table pointer inputted by the user is NULL, an error is returned.

Returns:

If success, returns DX_OK, otherwise, see the DX_STATUS definition.

Code Sample:

```
VxInt32 DxStatus= DX_OK;
VxInt32 nContrastParam = -20;
double  dGamma = 1.0;
VxInt32 nLightness = -20;

//Read the look-up table length.
size_t nLutLength = 0;
DxStatus = GXGetBufferLength(hDevice,
                             GX_BUFFER_LUT_VALUEALL,
                             &nLutLength);
if(DxStatus != DX_OK)
{
    return;
}

//Applie for buffer.
uint8_t *pSetLutBuffer = new uint8_t[nLutLength];
if(NULL == pSetLutBuffer)
{
    return;
```

```
    }

    //Set the contrast, Gamma and lightness value to adjust the lookup table value.
    DxStatus = DxCalcCameraLutBuffer(nContrastParam,
                                     dGamma,
                                     nLightness,
                                     pSetLutBuffer,
                                     (VxUint32*)&nLutLength);
    if(DxStatus!=DX_OK)
    {
        if(NULL != pSetLutBuffer)
        {
            delete []pSetLutBuffer;
            pSetLutBuffer = NULL;
        }
        return;
    }

    //Set the lookup table.
    DxStatus = GXSetBuffer(hDevice,
                           GX_BUFFER_LUT_VALUEALL,
                           pSetLutBuffer,
                           nLutLength);
    if(DxStatus!=DX_OK)
    {
        if(NULL != pSetLutBuffer)
        {
            delete []pSetLutBuffer;
             pSetLutBuffer = NULL;
        }
        return;
    }
```

8.4.47. DxReadLutFile

Declaration:

VxInt32 DHDECL DxReadLutFile(const char *pchLutFilePath,
void *pLut,
VxUint32* pnLutLength);

Descriptions:

The function is to read the lookup table tool saved in the .lut file and parse the data into lookup table format that can be set to the corresponding camera based on the camera lookup table length.

Formal parameter:

pchLutFilePath	Lookup table file path.The lookup table file (xxx.lut) can get from the lookup table plugin, and the path is: GalaxyView->Menu Bar->Plugins->LUT Create Tool
pLut	Lookup table, the parsed data from .lut file. needs a prior memory request from the users, and the length is the requested pointer memory(nLutLength), which can get via GXGetBufferLength interface: GXGetBufferLength(hDevice, GX_BUFFER_LUT_VALUEALL, &nLutLength)
pnLutLength	Lookup table length in bytes, which can get via GXGetBufferLength: GXGetBufferLength(hDevice, GX_BUFFER_LUT_VALUEALL,

&nLutLength)

Returns:

Returns DX_OK if successful, otherwise see the DX_STATUS related definition.

Code Sample:

```
VxInt32 DxStatus= DX_OK;
const char* pszchLutFilePath = "D:\\a.lut".

//Read the lookup table length.
size_t nLutLength = 0;
DxStatus = GXGetBufferLength(hDevice,
                             GX_BUFFER_LUT_VALUEALL,
                             &nLutLength);
if(DxStatus != DX_OK)
{
    return;
}

//Apply for Buffer.
uint8_t* pSetLutBuffer = new uint8_t[nLutLength];
if(NULL == pSetLutBuffer)
{
   return;
}

//Set the contrast, gamma and brightness to adjust the lookup table value.
DxStatus = DxReadLutFile (pszLutFilePath,
                          pSetLutBuffer,
                          (VxUint32*)&nLutLength);
if(DxStatus!=DX_OK)
{
    if(NULL != pSetLutBuffer)
    {
        delete[] pSetLutBuffer;
        pSetLutBuffer = NULL;
    }
    return;
}

//Set the lookup table.
DxStatus = GXSetBuffer(hDevice,
                       GX_BUFFER_LUT_VALUEALL,
                       pSetLutBuffer,
                       nLutLength);
if(DxStatus!=DX_OK)
{
    if(NULL != pSetLutBuffer)
    {
        delete []pSetLutBuffer;
        pSetLutBuffer = NULL;
    }
    return;
}
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

8.4.48. DxSetSystem

Declaration:

VxInt32 DHDECL DxSetSystem(DX_SYSTEM_PARAM emSystemParam, int nValue)

Descriptions:

This function is used to set system parameters, such as the number of threads and whether SSSE3 is enabled.

Formal parameter:

emSystemParam	System parameter
nValue	System parameter values

Returns:

Returns DX_OK if successful, otherwise see the DX_STATUS related definition.

Code Sample:

```
DX_SYSTEM_PARAM emSystemParam = THREAD_NUM;
int nValue = 6;

//Set the number of threads
VxInt32 emStatus = DxSetSystem(emSystemParam, nValue);
if (emStatus != DX_OK)
{
   return emStatus;
}
```

8.4.49. DxGetSystem

Declaration:

VxInt32 DHDECL DxGetSystem(DX_SYSTEM_PARAM emSystemParam, int *pnValue)

Descriptions:

This function is used to obtain system parameters, such as the number of threads, SSSE3 enable status, and so on.

Formal parameter:

emSystemParam	System parameter
pnValue	System parameter values

Returns:

Returns DX_OK if successful, otherwise see the DX_STATUS related definition.

Code Sample:

```
DX_SYSTEM_PARAM emSystemParam = THREAD_NUM;
int nValue = 0;
//Get the number of threads
VxInt32 emStatus = DxGetSystem(emSystemParam, &nValue);
if (emStatus != DX_OK)
{
   return emStatus;
}
```

8.4.50. DxFFCCalculate

Declaration:

VxInt32 DHDECL DxFFCCalculate(DX_FLAT_FIELD_CORRECTION_HANDLE handle, const FLAT_FIELD_CORRECTION_PARAMETER *pstFFCParam, void *pFFCCoefficients,

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

VxInt32 *pnFFCCoefficientsSize);

Descriptions:

This function is used to calculate the FFC coefficients.

Formal parameter:

handle	FFC handle. Cannot be NULL
pstFFCParam	FFC parameters structure
pFFCCoefficients	FFC coefficients
pnFFCCoefficientsSize	The length of FFC coefficients, unit byte.

If the length value entered by the user is less than the current actual length value, return an error.

If the length value entered by the user is greater than the current actual length value, return success, and pnFFCCoefficientsSize returns the actual length value.

The FFC data structure is defined as follows:

```
{
    void                    *pBrightBuf;        ///<bright-field imageBuffer
    void                    *pDarkBuf;          ///<dark-field image Buffer
    GX_PIXEL_FORMAT_ENTRY   emPixelFormat;      ///<pixel format
    VxInt32                 nImgWid;            ///<image width
    VxInt32                 nImgHei;            ///<image height
    VxInt32                 nFFCBlockSize;      ///<the size of FFC block
    VxInt32                 nFFCExpectedGray;   ///<the expected gray value after FFC
    VxInt32                 arrReserved[64];    ///<reverse 64 byte
} FLAT_FIELD_CORRECTION_PROCESS;
```

The nFFCBlockSize value is used to distinguish whether the FFC coefficient is used on the software side or the camera side in the algorithm for calculating the FFC coefficient. If the FFC coefficients is used on the software side, the nFFCBlockSize value will be -1. If the FFC coefficients is used on the camera side, the actual value of nFFCBlockSize needs to be obtained from the camera. Different nFFCBlockSize values have the following requirements for FFC parameters:

nFFC Block Size	pBrightBuf	pDarkBuf	emPixelFormat	nFFCExpectedGray
-1	Capture bright-field images	If dark field images are not captured, it should be set to NULL.	GX_PIXEL_FORMAT_MONO8 GX_PIXEL_FORMAT_BAYER_GR8 GX_PIXEL_FORMAT_BAYER_RG8 GX_PIXEL_FORMAT_BAYER_GB8 GX_PIXEL_FORMAT_BAYER_BG8 GX_PIXEL_FORMAT_MONO10 GX_PIXEL_FORMAT_BAYER_GR10 GX_PIXEL_FORMAT_BAYER_RG10 GX_PIXEL_FORMAT_BAYER_GB10 GX_PIXEL_FORMAT_BAYER_BG10 GX_PIXEL_FORMAT_MONO12 GX_PIXEL_FORMAT_BAYER_GR12 GX_PIXEL_FORMAT_BAYER_RG1	-1: Mean value of the brightest block Others: Set value

			2 GX_PIXEL_FORMAT_BAYER_GB12 GX_PIXEL_FORMAT_BAYER_BG12	
0	Capture bright-field images	If dark field images are not captured, it should be set to NULL.	GX_PIXEL_FORMAT_BAYER_GR8 GX_PIXEL_FORMAT_BAYER_RG8 GX_PIXEL_FORMAT_BAYER_GB8 GX_PIXEL_FORMAT_BAYER_BG8	-1: Default 127 Others: Set value
1	Capture bright-field images	If dark field images are not captured, it should be set to NULL.	GX_PIXEL_FORMAT_MONO8	-1: Default 127 Others: Set value
2	Capture bright-field images	NULL	GX_PIXEL_FORMAT_MONO8 GX_PIXEL_FORMAT_MONO10 GX_PIXEL_FORMAT_MONO12	-1: Mean value of the brightest block Others: Set value
3	Capture bright-field images	NULL	GX_PIXEL_FORMAT_BAYER_GR8 GX_PIXEL_FORMAT_BAYER_RG8 GX_PIXEL_FORMAT_BAYER_GB8 GX_PIXEL_FORMAT_BAYER_BG8 GX_PIXEL_FORMAT_BAYER_GR10 GX_PIXEL_FORMAT_BAYER_RG10 GX_PIXEL_FORMAT_BAYER_GB10 GX_PIXEL_FORMAT_BAYER_BG10 GX_PIXEL_FORMAT_BAYER_GR12 GX_PIXEL_FORMAT_BAYER_RG12 GX_PIXEL_FORMAT_BAYER_GB12 GX_PIXEL_FORMAT_BAYER_BG12	-1: Mean value of the brightest block Others: Set value

Note: If dark field images are not captured, the pointer pointing to the dark field image should be set to NULL when calculating the FFC coefficients.

Returns:

Returns DX_OK if successful, otherwise see the DX_STATUS related definition.

Code Sample:

```
// Declaration of FFC handle.
DX_FLAT_FIELD_CORRECTION_HANDLE handle = NULL;
DX_STATUS emDxStatus = DX_OK;

//Create FFC handle.
DxFFCCreate(&handle);
if (handle == NULL)
{
        return;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
}


GX_PIXEL_FORMAT_ENTRY emPixelFormat = GX_PIXEL_FORMAT_BAYER_RG8;
int nWidth = 720;
int nHeight = 540;
int nFFCBlockSize = -1;
int nFFCExpectedGray = -1;
int nFFCCoefficientsSize = 0;
unsigned char *pFFCCoefficients = NULL;

FLAT_FIELD_CORRECTION_PARAMETER stFFCParam;
memset(&stFFCParam, 0, sizeof(FLAT_FIELD_CORRECTION_PARAMETER));

stFFCParam.pBrightBuf = pBrightBuf;
stFFCParam.pDarkBuf = NULL;
stFFCParam.emPixelFormat = emPixelFormat;
StFFCParam.nImgWid = nWidth;
StFFCParam.nImgHei = nHeight;
StFFCParam.nFFCBlockSize = nFFCBlockSize;
StFFCParam.nFFCExpectedGray = nFFCExpectedGray;

//Get the length of the FFC coefficients.
emDxStatus = DxFFCGetCoefficientsSize(handle, &stFFCParam,
                                      &nFFCCoefficientsSize);
if (emDxStatus != DX_OK)
{
   DxFFCDestroy(handle);
   handle = NULL;
   return;
}

//Apply for memory.
pFFCCoefficients = new unsigned char[nFFCCoefficientsSize];
if (pszBufferIn == NULL)
{
   DxFFCDestroy(handle);
   handle = NULL;
   return;
}

//If the bright field image is obtained by averaging multiple images, the
//number of frames to be captured must be set.
emDxStatus = DxFFCSetFrameCount(handle, 2);
if (emDxStatus != DX_OK)
{
   DxFFCDestroy(handle);
   handle = NULL;
   return;
}

//Calculate the FFC coefficients.
emDxStatus = DxFFCCalculate(handle, &stFFCParam, pFFCCoefficients,
                            &nFFCCoefficientsSize);
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
//If FFC coefficients are used in the software, call DxFlatFieldCorrection
//If the camera supports FFC, coefficients can be written to the camera,
//Code GXSetRegisterValue(hDevice,
//"FFCValueAll", pFFCCoefficients, nFFCCoefficientsSize);

DxFFCDestroy(handle);
handle = NULL;
```

8.4.51. DxFFCCreate

Declaration:

VxInt32 DHDECL DxFFCCreate(DX_FLAT_FIELD_CORRECTION_HANDLE *phandle);

Descriptions:

This function is used to create the FFC handle.

Formal parameter:

phandle	FFC handle

Returns:

Returns DX_OK if successful, otherwise see the DX_STATUS related definition.

Code Sample:

```
// Declaration of FFC handle.
DX_FLAT_FIELD_CORRECTION_HANDLE handle;
DX_STATUS emDxStatus  = DX_OK;

// Create FFC handle.
DxStatus = DxFFCCreate(&handle);
```

8.4.52. DxFFCDestroy

Declaration:

VxInt32 DHDECL DxFFCDestroy(DX_FLAT_FIELD_CORRECTION_HANDLE handle);

Descriptions:

This function is used to destroy the FFC handle.

Formal parameter:

handle	FFC handle

Returns:

Returns DX_OK if successful, otherwise see the DX_STATUS related definition.

Code Sample:

```
// Declaration of FFC handle.
DX_FLAT_FIELD_CORRECTION_HANDLE handle;
DX_STATUS emDxStatus  = DX_OK;

// Create FFC handle.
DxStatus = DxFFCCreate(&handle);
if (handle == NULL)
    {
        return;
    }
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
// Destroy of FFC handle.
emDxStatus = DxFFCDestroy(handle);
```

8.4.53. DxFFCGetCoefficientsSize

Declaration:

VxInt32 DHDECL DxFFCGetCoefficientsSize(DX_FLAT_FIELD_CORRECTION_HANDLE handle, const FLAT_FIELD_CORRECTION_PARAMETER *pstFFCParam, VxInt32 *pnFFCCoefficientsSize);

Descriptions:

This function is used to get the size of the FFC coefficients.

Formal parameter:

handle	FFC handle. Cannot be NULL.
pstFFCParam	FFC parameters structure.
pnFFCCoefficientsSize	The size of the FFC coefficients, unit byte.

Note: If dark field images are not captured, the pointer pointing to the dark field image should be set to NULL when calculating the FFC coefficients.

Returns:

Returns DX_OK if successful, otherwise see the DX_STATUS related definition.

Code Sample:

```
// Declaration of FFC handle.
DX_FLAT_FIELD_CORRECTION_HANDLE handle = NULL;
DX_STATUS emDxStatus = DX_OK;

// Create FFC handle.
DxFFCCreate(&handle);
if (handle == NULL)
{
      return;
}

GX_PIXEL_FORMAT_ENTRY emPixelFormat = GX_PIXEL_FORMAT_BAYER_RG8;
int nWidth = 720;
int nHeight = 540;
int nFFCBlockSize = -1;
int nFFCExpectedGray = -1;
int nFFCCoefficientsSize = 0;
unsigned char *pFFCCoefficients = NULL;

FLAT_FIELD_CORRECTION_PARAMETER stFFCParam;
memset(&stFFCParam, 0, sizeof(FLAT_FIELD_CORRECTION_PARAMETER));

stFFCParam.pBrightBuf = pBrightBuf;
stFFCParam.pDarkBuf = NULL;
stFFCParam.emPixelFormat = emPixelFormat;
StFFCParam.nImgWid = nWidth;
StFFCParam.nImgHei = nHeight;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
StFFCParam.nFFCBlockSize = nFFCBlockSize;
StFFCParam.nFFCExpectedGray = nFFCExpectedGray;

//Get the size of the FFC coefficients.
emDxStatus = DxFFCGetCoefficientsSize(handle, &stFFCParam,
                                      &nFFCCoefficientsSize);
if (emDxStatus != DX_OK)
{
   DxFFCDestroy(handle);
   handle = NULL;
   return;
}

//Others.
//.....
DxFFCDestroy(handle);
handle = NULL;
```

8.4.54. DxFFCSetFrameCount

Declaration:

VxInt32 DHDECL DxFFCSetFrameCount(DX_FLAT_FIELD_CORRECTION_HANDLE handle, VxUint16 nFFCFrameCount);

Descriptions:

This function is used to set the number of frames to be captured for calculating the FFC coefficients. .

Formal parameter:

handle	FFC handle.
nFFCFrameCount	The number of capturing frames, with value of 1, 2, 4, 8, 16

Returns:

Returns DX_OK if successful, otherwise see the DX_STATUS related definition.

Code Sample:

```
// Declaration of FFC handle.
DX_FLAT_FIELD_CORRECTION_HANDLE handle = NULL;
DX_STATUS emDxStatus = DX_OK;
VxUint16 nFFCFrameCount = 8;

// Create FFC handle.
DxFFCCreate(&handle);
if (handle == NULL)
{
       return;
}

// If the bright field image is obtained by averaging multiple images, the
// number of frames to be captured must be set.
emDxStatus = DxFFCSetFrameCount(handle, nFFCFrameCount);
if (emDxStatus != DX_OK)
{
   DxFFCDestroy(handle);
   handle = NULL;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
        return;
    }

    //Others.
     //.....................

    DxFFCDestroy(handle);
    handle = NULL;
```

8.4.55. DxDecompression

Declaration:

VxInt32 DHDECL DxDecompression (DX_DECOMPRESSION_HANDLE handle,
void *pCompressionBuffer,
VxInt32 nCompressionSize,
void *pDecompressionBuffer,
VxInt32 *pnDecompressionSize,
GX_PIXEL_FORMAT_ENTRY emPixelFormat,
VxUint32 nImgWidth,
VxUint32 nImgHeight,
VxInt32 nCompressionMethod);

Descriptions:

This function is used to decompresse acquired compressed data into raw data while supporting frame information parsing.(refer to the GxDecompression code sample for usage details)

Formal parameter:

handle	Decompression handle. Cannot be NULL.
pCompressionBuffer	Compressed Data.
nCompressionSize	The size of compressed data.
pDecompressionBuffer	Decompressed Data.
pnDecompressionSize	The size of decompressed data.
emPixelFormat	The pixel format.
nImgWidth	The image width.
nImgHeight	The image height.
nCompressionMethod	The compression method.

If the size of decompressed data entered by the user is less than the current actual size, return FAULT.

If the size of decompressed data entered by the user is more than the current actual size, return TRUTH.The pnDecompressionSize return actual size.

Returns:

Returns DX_OK if successful, otherwise see the DX_STATUS related definition.

Code Sample:

```
    // Declaration of Decompression handle.
    DX_DECOMPRESSION_HANDLE handle = NULL;
    DX_STATUS emDxStatus = DX_OK;

    // Create Decompression handle
    DxDecompressionCreate(&handle);
    if (handle == NULL)
    {
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
        return;
    }

    GX_PIXEL_FORMAT_ENTRY emPixelFormat = GX_PIXEL_FORMAT_BAYER_RG8;
    VxUint32 nWidth = 720;
    VxUint32 nHeight = 540;

    // Obtain the compression method from the camera.
    VxInt32 nCompressionMethod = 1;

    // Obtain the compressed data and the size of compressed data from
    // the camera.
    unsigned char *pui8CompressionBuffer = pImgBuf;
    VxInt32 nCompressionSize = nImgSize;

    // Retrieve PayloadSize from the camera as the size of the decompressed
    // data.
    unsigned char *pui8DecompressionBuffer = NULL;
    VxInt32 nDecompressionSize = nPayloadSize;

    // Allocate memory.
    pui8DecompressionBuffer = new unsigned char[nDecompressionSize];
    if (pui8DecompressionBuffer == NULL)
    {
       DxDecompressionDestroy(handle);
       handle = NULL;
       return;
    }

    //Decompress the image.
    emDxStatus = DxDecompression(handle,
                                 pui8CompressionBuffer,
                                 nCompressionSize,
                                 pui8DecompressionBuffer,
                                 nDecompressionSize,
                                 emPixelFormat,
                                 nWidth,
                                 nHeight,
                                 nCompressionMethod);

    // Others.
    //……
    // Complete.
    delete[] pui8DecompressionBuffer;
    pui8DecompressionBuffer = NULL;

    DxDecompressionDestroy(handle);
    handle = NULL;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

8.4.56. DxDecompressionCreate

Declaration:

VxInt32 DHDECL DxDecompressionCreate(DX_DECOMPRESSION_HANDLE *phandle);

Descriptions:

This function is used to create the decompression handle.

Formal parameter:

phandle	decompression handle.

Returns:

Returns DX_OK if successful, otherwise see the DX_STATUS related definition.

Code Sample:

```
// Declaration of decompression handle.
DX_DECOMPRESSION_HANDLE handle;
DX_STATUS emDxStatus = DX_OK;

// Create decompression handle.
DxStatus = DxDecompressionCreate(&handle);
```

8.4.57. DxDecompressionDestroy

Declaration:

VxInt32 DHDECL DxDecompressionDestroy(DX_DECOMPRESSION_HANDLE handle);

Descriptions:

This function is used to destroy the decompression handle.

Formal parameter:

phandle	decompression handle.

Returns:

Returns DX_OK if successful, otherwise see the DX_STATUS related definition.

Code Sample:

```
// Declaration of decompression handle.
DX_DECOMPRESSION_HANDLE handle;
DX_STATUS emDxStatus = DX_OK;

// Create decompression handle.
DxStatus = DxDecompressionCreate(&handle);
if (handle == NULL)
    {
        return;
    }
// Destroy decompression handle.
emDxStatus = DxDecompressionDestroy(handle);
handle = NULL;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

8.5. Function

8.5.1. Image Quality Enhancement Function

This function can realize the color correction function, the contrast adjustment and the Gamma adjustment function of any combination.

8.5.1.1. Related Functions

- Set the contrast look-up table function

VxInt32 DHDECL DxGetContrastLut (int nContrastParam, void *pContrastLut, int *pLutLength);

- Set the Gamma look-up table function

VxInt32 DHDECL DxSetGammatLut (double dGammaParam, void *pGammaLut, int *pLutLength);

- The function of image quality promotion

VxInt32 DHDECL DxImageImprovment (void *pInputBuffer, void *pOutputBuffer, VxUint32 nWidth, VxUint32 nHeight, VxInt64 nColorCorrectionParam, void *pContrastLut, void *pGammaLut);

For the use of the functions, refer to the interface section.

8.5.1.2. ColorTransformationControl

- Terms

Color correction (Color transformation): Improves the color reduction of the camera to make the image closer to human visual perception.

Color transformation mode: Sets the mode of the color transformation to be performed. 0: Set to the default mode, the color correction factor uses the coefficient provided from factory; 1: Set to the user-defined mode. The user can input the color correction factor according to the actual application. The user can modify the value of this function during the acquisition.

Color transformation matrix value selection: Sets the value to be inputted in the color transformation matrix to customize the color transformation. Note: Depending on the camera model, some values of the color transformation matrix may be pre-set and cannot be changed. The user can modify the value of this function during the acquisition.

Color transformation matrix value: Gets the value in the color transformation matrix, which is used to customize the current value of color transformation.

The user expects that the camera can output the precision color, but the color world is a dynamic one, everyone sees some difference in the color. Same to the sensor, different sensors have different interpretations of color. But what is the precision color and who determine it? So, the user needs a color template. The color template contains 24 colors, and each color has fixed RGB value, as shown in Figure 8-17:



Figure 8-17 Color template

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

With this color template, the user can base on it to shoot the color template with a camera, the RGB value of each color may be different from the standard RGB value of the color template, the vendor can use the software or hardware to convert the RGB value that is read to the standard RGB value. Because the color space is continuous, all the other RGB values read can be converted to the standard RGB values by using the 24 colors.

- Related Parameters

  `GX_ENUM_COLOR_TRANSFORMATION_MODE`: The color transformation mode, refer to `GX_COLOR_TRANSFORMATION_MODE_ENTRY`

  `GX_BOOL_COLOR_TRANSFORMATION_ENABLE`: Color transformation enable

  `GX_ENUM_COLOR_TRANSFORMATION_VALUE_SELECTOR`: The color transformation matrix value selection, refer to `GX_COLOR_TRANSFORMATION_VALUE_SELECTOR_ENTRY`

  `GX_FLOAT_COLOR_TRANSFORMATION_VALUE`: The color transformation matrix value

  `GX_INT_COLOR_CORRECTION_PARAM`: The color correction parameter

- Effect Images



Figure 8-18 Before color correction



Figure 8-19 After color correction

- Sample Code

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
//Enables color transformation.
emStatus = GXSetBool(hDevice, GX_BOOL_COLOR_TRANSFORMATION_ENABLE,
                     true);

//Sets the color transformation mode to user-defined mode.
GX_COLOR_TRANSFORMATION_MODE_ENTRY emValue;
emValue = GX_COLOR_TRANSFORMATION_SELECTOR_USER;
emStatus = GXSetEnum(hDevice, GX_ENUM_COLOR_TRANSFORMATION_MODE,
                     emValue);

//Gets the color correction parameter value.
int64_t i64ColorParam = 0;
emStatus = GXGetInt(hDevice, GX_INT_COLOR_CORRECTION_PARAM,
                    &i64ColorParam);
```

8.5.1.3. Contrast Adjustment

- Terms

Contrast: The brightness ratio of the bright part and the dark part of the image is called contrast. The image with high contrast is clear and the profile of the object which is captured by the camera is clear. Conversely,

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

the image with low contrast is not clear and the profile of the object is not clear.

- Related Parameters

  GX_INT_CONTRAST_PARAM:the contrast parameter.

- Effect Images



Figure 8-20 Before contrast adjustment



Figure 8-21 After contrast adjustment

8.5.1.4. Gamma Adjustment

- Terms

Gamma adjustment: The Gamma adjustment is to make the output of the display as close as possible to the input.
Gamma: The value of Gamma adjustment. A nonlinear transformation of the pixel value is performed on the current image in the form of a power function.
Gamma mode: Manual adjustment mode 0: set to default mode, 1: set to user-defined mode.

- Related Parameters

  GX_BOOL_GAMMA_ENABLE: Gamma enable

  GX_ENUM_GAMMA_MODE: Gamma mode, refer to GX_GAMMA_MODE_ENTRY

  GX_FLOAT_GAMMA: Gamma

  GX_FLOAT_GAMMA_PARAM: Gamma parameter

- Effect Images

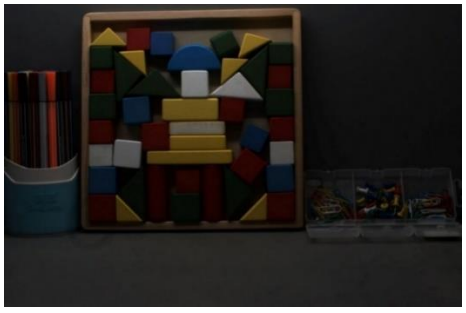


Figure 8-22 Before Gamma adjustment



Figure 8-23 After Gamma adjustment

- Sample Code

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//Enables Gamma.
emStatus = GXSetBool(hDevice, GX_BOOL_GAMMA_ENABLE, true);
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
//Sets Gamma mode to user-defined mode.
GX_GAMMA_MODE_ENTRY emValue;
emValue = GX_GAMMA_SELECTOR_USER;
emStatus = GXSetEnum(hDevice, GX_ENUM_GAMMA_MODE, emValue);

//Gets the Gamma parameter value.
double dColorParam = 0.0;
emStatus = GXGetFloat(hDevice, GX_FLOAT_GAMMA_PARAM, &dColorParam);
```

8.5.1.5. Sharpen

- Terms

Sharpen: Sharpening is to improve the definition of the image edges. The higher the definition, the clearer the outline of the image.

Sharpness: Adjust the sharpness value to adjust the camera's sharpness to the image. The adjustment range is 0-3.0. The larger the value, the higher the sharpness.

Sharpen mode: Decide whether to enable the sharpening function. ON means that the sharpening function is enabled; OFF means that the sharpening function is disabled.

- Related Parameters

`GX_FLOAT_SHARPNESS`: Sharpness

`GX_ENUM_SHARPNESS_MODE`: Sharpen mode, refer to `GX_SHARPNESS_MODE_ENTRY`

- Effect Images

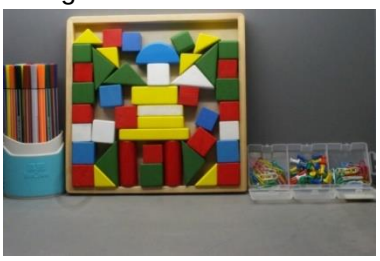


Figure 8-24 Before sharpen adjustment



Figure 8-25 After sharpen adjustment

- Sample Code

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//Enables sharpening.
GX_SHARPNESS_MODE_ENTRY emValue;
emValue = GX_SHARPNESS_MODE_ON;
emStatus = GXSetEnum(hDevice, GX_ENUM_SHARPNESS_MODE, emValue);

//Gets the value of sharpness.
double dColorParam = 0.0;
emStatus = GXGetFloat(hDevice, GX_FLOAT_SHARPNESS, &dColorParam);
```

8.5.1.6. Noise reduction

- Terms

Noise reduction: During the digitization and transmission of an image, it is often disturbed by the noise of the imaging device and the external environment, which will cause the image with noise. The process of reducing or suppressing the noise in the image is called image noise reduction.

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Noise reduction: Adjust the noise reduction value to adjust the camera's noise reduction intensity to the image. The adjustment range is 0-4.0. The larger the value, the higher the degree of noise reduction.

Noise reduction mode: Decide whether to enable the noise reduction. ON means that the noise reduction function is enabled. OFF means that the noise reduction function is disabled.

- Related Parameters

  `GX_FLOAT_NOISE_REDUCTION`: Noise reduction

  `GX_ENUM_NOISE_REDUCTION_MODE`: Noise reduction mode, refer to `GX_NOISE_REDUCTIONSHARPNESS_MODE_ENTRY`

- Effect Images

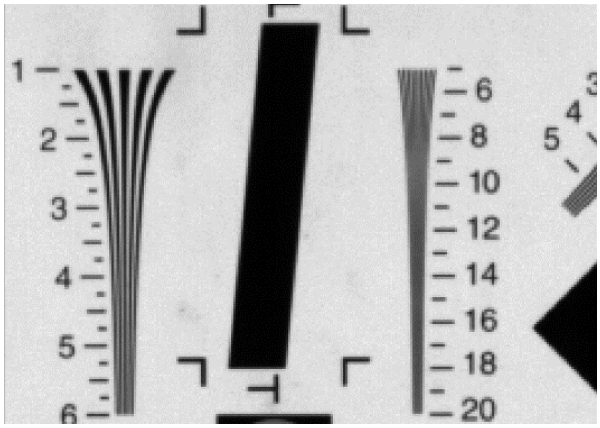


Figure 8-26 Before noise reduction adjustment

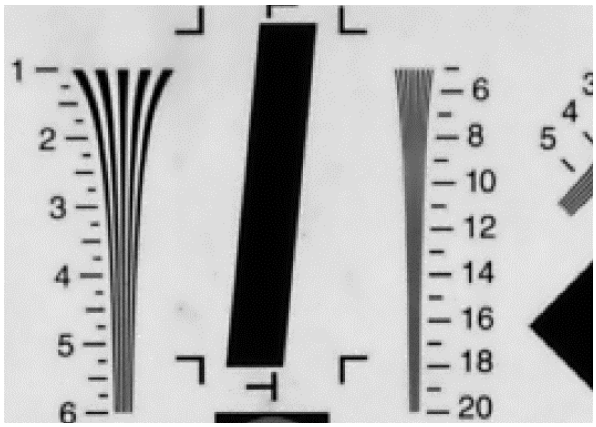


Figure 8-27 After noise reduction adjustment

- Sample Code

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//Enables noise reduction.
GX_NOISE_REDUCTION_MODE_ENTRY emValue;
emValue = GX_NOISE_REDUCTION_MODE_ON;
emStatus = GXSetEnum(hDevice, GX_ENUM_NOISE_REDUCTION_MODE,
                     emValue);

//Gets the value of noise reduction.
double dNoiseReductionParam = 0.0;
emStatus = GXGetFloat(hDevice, GX_FLOAT_NOISE_REDUCTION,
                      &dNoiseReductionParam);
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

8.5.1.7. Saturation

- Terms

Saturation is one of the constituent elements of color. Higher saturation parameter values result in more vivid, vibrant colors. Lower saturation parameter values result in more muted, dim colors.

Saturation: Adjusting the saturation parameter changes the colorfulness (intensity) of the colors. Enter a value for the saturation parameter. The parameter's value range is 0-128. By default, the parameter is set to 64 (no saturation perform). The parameter is set higher than 64 (increase saturation) and set lower than 64 (decrease saturation). 128: Saturation is twice the current. 0: monochrome image.

Saturation mode: Decide whether to enable the saturation function. ON means that the saturation function is enabled. OFF means that the saturation function is disabled.

- Related Parameters

`GX_INT_SATURATION`: Saturation

`GX_ENUM_SATURATION_MODE`: Saturation mode, refer to: `GX_ENUM_SATURATION_MODE_ENTRY`

- Effect Images



Figure 8-28 Before saturatuin adjustment



Figure 8-29 After saturatuin adjustment

- Sample Code

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//Enables saturation function.
GX_ENUM_SATURATION_MODE_ENTRY emValue;
emValue = GX_ENUM_SATURATION_ON;
emStatus = GXSetEnum(hDevice, GX_ENUM_SATURATION_MODE, emValue);

//Gets the value of saturation.
int64_t i64Saturation = 0;
emStatus = GXGetInt(hDevice, GX_INT_SATURATION, &i64Saturation);
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

8.5.1.8. Static Defect Correction

- Terms

Static defect is the defect pixel generated due to manufacturing technique during sensor manufacturing, which will not change with time, gain, etc. If the sensor has defect pixels, it will directly affect the quality of the image.

Static defect correction mode: Decide whether to enable the static defect correction function. ON means that the static defect correction function is enabled. OFF means that the static defect correction function is disabled.

- Related Parameters

`GX_ENUM_STATIC_DEFECT_CORRECTION`: Static defect correction mode, refer to `GX_ENUM_STATIC_DEFECT_CORRECTION_ENTRY`

- Sample Code

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//Enables static defect correction.
GX_ENUM_STATIC_DEFECT_CORRECTION_ENTRY emValue;
emValue = GX_ENUM_STATIC_DEFECT_CORRECTION_ON;
emStatus = GXSetEnum(hDevice, GX_ENUM_STATIC_DEFECT_CORRECTION,
                     emValue);
```

8.5.1.9. 2D/3D Noise Reduction Mode

- Terms

Noise Reduction: Image noise can be produced by the image sensor and the external environment during the image digitization and transmission process. The noise reduction reduces random variations in brightness or color information in images.

Noise Reduction Mode: Divided into three noise reduction modes: LOW, MIDDLE and HIGH.

- Related Parameters

`GX_ENUM_2D_NOISE_REDUCTION_MODE`: 2D Noise Reduction Mode, refer to `GX_2D_NOISE_REDUCTION_MODE_ENTRY`

`GX_ENUM_3D_NOISE_REDUCTION_MODE`: 3D Noise Reduction Mode, refer to `GX_3D_NOISE_REDUCTION_MODE_ENTRY`

- Sample Code

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//Set 2D Noise Reduction Mode: LOW.
//The mode can be set according to the needs.
GX_2D_NOISE_REDUCTION_MODE_ENTRY emValue;

//Disabled 2D Noise Reduction Mode.
//emValue = GX_2D_NOISE_REDUCTION_MODE_OFF;

//2D Noise Reduction Mode: Low.
emValue = GX_2D_NOISE_REDUCTION_MODE_LOW;

//2D Noise Reduction Mode: Middle.
//emValue = GX_2D_NOISE_REDUCTION_MODE_MIDDLE;
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

```
//2D Noise Reduction Mode: High.

//emValue = GX_2D_NOISE_REDUCTION_MODE_HIGH;
emStatus =GXSetEnum(hDevice, GX_ENUM_2D_NOISE_REDUCTION_MODE, emValue);

//Set 3D Noise Reduction Mode: High.
//The mode can be set according to the needs.
GX_3D_NOISE_REDUCTION_MODE_ENTRY emValue;

//Disabled 3D Noise Reduction Mode.
//emValue = GX_3D_NOISE_REDUCTION_MODE_OFF;

//3D Noise Reduction Mode: Low.
//emValue = GX_3D_NOISE_REDUCTION_MODE_LOW;

//3D Noise Reduction Mode: Middle.
//emValue = GX_3D_NOISE_REDUCTION_MODE_MIDDLE;

//3D Noise Reduction Mode: High.
emValue = GX_3D_NOISE_REDUCTION_MODE_HIGH;
emStatus =GXSetEnum(hDevice, GX_ENUM_3D_NOISE_REDUCTION_MODE,emValue);
```

8.5.1.10. HDR Mode

- Terms

HDR stands for “high dynamic range”. For industrial cameras, it means that the camera directly generates high dynamic range digital images. In contrast, acquiring images using traditional methods is called LDR (low dynamic range).

The dynamic range of an image (ie, contrast) is the ratio of light to dark. In other words, if the image contains both brighter and darker areas, the dynamic range is higher (e.g., an image of a person stands with his back to the sun). If the dynamic range is extremely small, the brightness can also be high because there is no darker area (e.g., facing the sun).

The HDR mode acquires two images at different exposures.Long-exposure image and short-exposure image merged into one HDR image. The HDR image retains both dark details in the high-brightness image and the bright details in the low-brightness image.

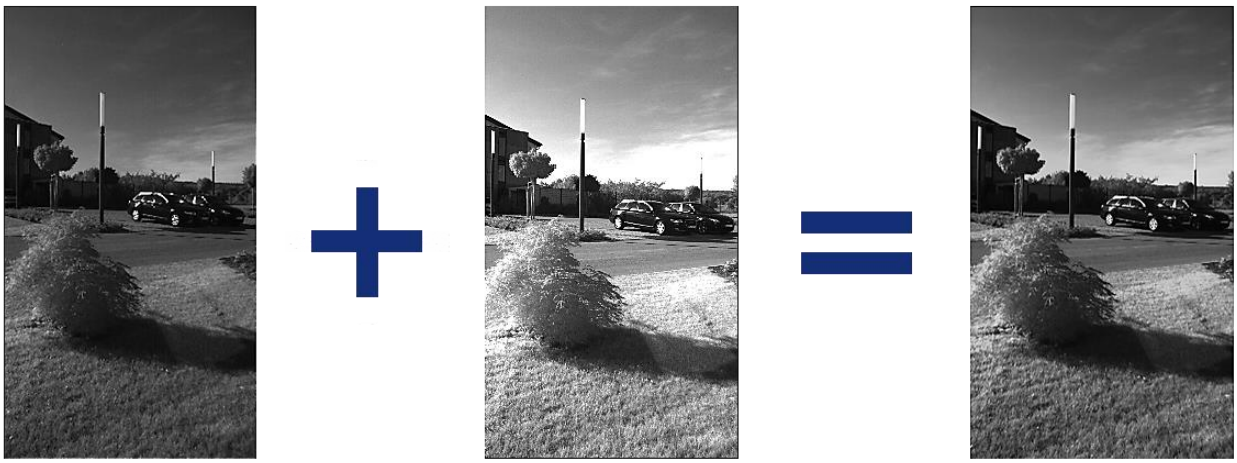


Figure 8-30 HDR diagram

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

Camera model that support this feature:

Model
MER-203-30GC-P-L

- Related Parameters

  GX_ENUM_HDR_MODE：HDR Mode, refer to GX_HDR_MODE_ENTRY

  GX_INT_HDR_TARGET_LONG_VALUE：Long-exposure target gray value

  GX_INT_HDR_TARGET_SHORT_VALUE：Short-exposure target gray value

  GX_INT_HDR_TARGET_MAIN_VALUE：High dynamic image target gray value

- Sample Code

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//Enable continuous mode.
GX_HDR_MODE_ENTRY emValue = GX_HDR_MODE_CONTINUOUS;
emStatus =GXSetEnum(hDevice, GX_ENUM_HDR_MODE, emValue);

//Sample code, please set your own according to your needs.
//Set the target gray value of the long-exposure image.
emStatus = GXSetInt(hDevice, GX_INT_HDR_TARGET_LONG_VALUE, 80);

//Set the target gray value of the short-exposure image.
emStatus = GXSetInt(hDevice, GX_INT_HDR_TARGET_SHORT_VALUE, 28);

//Set the target gray value of the HDR image
emStatus = GXSetInt(hDevice, GX_INT_HDR_TARGET_MAIN_VALUE, 255);
```

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

9. Revision History

No.	Version	API Version	Changes	Date
1	V1.0.0	1.0.1303.x or above	1) Initial release	2013-03
2	V2.0.0	1.0.1901.x or above	1) Initial release for Windows & Linux	2019-01-11
3	V2.0.1	1.0.1901.x or above	1) Modify a bug of Sample Code	2019-01-23
4	V2.0.2	1.0.1902.x or above	1) Modify the mistake found in the system test	2019-02-12
5	V2.0.3	1.0.1902.x or above	1) Modify some descriptions	2019-02-14
6	V2.0.4	1.0.1902.x or above	1) Modify some formats	2019-02-21
7	V2.0.5	1.0.1902.x or above	1) Add section 3.5	2019-07-17
8	V2.0.6	1.0.1907.x or above	1) Add interface description for device reset and device reconnection	2019-07-18
9	V2.0.7	1.0.1908.x or above	1) Modify a bug of inconsistent between Chinese and English versions 2) Add the interface description of obtain the optimal packet size 3) Add sample code of get optimal package size in section 2.2.9, 3.3.2, 7.4.4, 7.4.4.2	2019-08-15
10	V2.0.8	1.0.1908.x or above	1) Add related parameters description of section 3.9 2) Add section 3.13.6, 3.13.7, 3.13.8	2019-08-29
11	V2.0.9	1.0.1910.x or above	1) Add section 8.3.3, 8.3.4, 8.4.4, 8.4.19, 8.4.23, 8.4.24, 8.4.31, 8.4.32, 8.4.33	2019-10-12
12	V2.0.10	1.0.1910.x or above	1) Add node information in section 3.1.1	2019-10-22
13	V2.1.0	1.0.1912.x or above	1) Add related parameters of StreamBufferHandlingMode in Section 5.2.1	2020-01-14
14	V2.1.1	1.0.2008.x or above	1) Add section 8.2.7 2) Add section 8.5.1.6 3) Add ExposureTimeMode description in section 3.3.3 4) Add the GX_ENUM_COUNTER_TRIGGER_SOURCE and GX_INT_COUNTER_DURATION function codes in section 3.5.2	2020-08-18

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

No.	Version	API Version	Changes	Date
			5) Add GX_BUFFER_FFCLOAD and GX_BUFFER_FFCSAVE function codes in section 3.13.8	
15	V2.1.2	1.0.2008.x or above	1) Add related description for the function of sensor shutter mode in section 3.2.5	2020-08-21
16	V2.1.3	1.0.2008.x or above	1) Add the description of how to get the sample code for feature read & write in section 2.2.7	2020-08-28
17	V2.1.4	1.0.2010.x or above	1) Add the description of LightSourcePreset in section 3.6.3	2020-10-12
18	V2.1.5	1.0.2011.x or above	1) Add the description of windows only in section 7.4.5.12	2020-11-17
19	V2.1.6	1.0.2012.x or above	1) Add the description of static defect correction in section 8.3.5 and 8.4.42	2020-12-23
20	V2.1.7	1.0.2101.x or above	1) Add the description of saturation function in section 8.5.1.7 2) Add the description of static defect correction in section 8.5.1.8	2021-01-07
21	V2.1.8	1.0.2104.x or above	1) Add the description of MGC function in section 3.3.10 2) Add the description of 2D/3D Noise Reduction Mode in section 8.5.1.9 3) Add the description of HDR Mode in section 8.5.1.10	2021-04-07
22	V2.1.9	1.0.2110.x or above	1) Add the description of GX_STRING_DEVICE_ISP_FIRMWARE_VERSION function code in section 3.1.1 2) Add the description of GX_INT_DECIMATION_LINENUMBER function code in section 3.2.2	2021-10-18
23	V2.1.10	1.0.2111.x or above	1) Add the description of Sensor Horizontal Binning and Sensor Vertical Binning function code in section 3.2.2	2021-11-24
24	V2.1.11	1.0.2201.x or above	1) Add the description of caculate camera lookup table interface in section 8.4.46	2022-01-17
25	V2.1.12	1.0.2203.x or above	1) Modify 8.4.46, add lookup table file parsing interface description in section 8.4.47	2022-03-23
26	V2.1.13	1.0.2209.x or above	1) Add 3.3.5 acquisition burst mode description 2) Modify 3.3.6 frame store mechanism description	2022-09-19

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

No.	Version	API Version	Changes	Date
27	V2.1.14	1.0.2209.x or above	1) Add 7.3.3 GX_REGISTER_STACK_ENTRY section 2) Add 7.5.4.1, 7.5.4.2, 7.5.4.3, 7.5.4.4 sections, about reading and setting camera attribute register interface 3) Modify 3.3.5 Acquisition Burst Mode precautions	2022-09-23
28	V2.1.15	1.0.2301.x or above	1) Add 2.1.3 macOS	2023-02-03
29	V3.0.0	2.0.2403.x or above	1) Add 7.4.2 Initialization and Cancellation functional interface Enumerating device interfaces, it is recommended to use the newly added interface GXUpdateAllDeviceListEx, while enumerating interface GXUpdateDeviceList is not recommended. Get the device information interface, It is recommended to use the newly added interface GXGetDeviceInfo, while other interfaces such as GXGetAllDeviceBaseInfo and GXGetDeviceIPInfo are not recommended. 2) Add interface type in 7.1.2 Handle Type: Add GX_IF_HANDLE, obtained through the GXGetInterfaceHandle interface. Used for read and write access to interface layer devices. Add GX_LOCAL_DEV_HANDLE, obtained through the GXGetLocalDeviceHandleFromDev interface. It is used to read and write the local Mechanical floor devices. Add GX_DS_HANDLE, obtained through the GXGetDataStreamHandleFromDev interface. Used for read and write access to tropospheric devices. 3) Modify 7.4.3 Device Parameter Settings: It is recommend to use the newly added string for read/write access interface: GXGetIntValue, GXSetIntValue, GXGetEnumValu, GXSetEnumValue, GXSetEnumValueByString, GXGetFloatValue, GXSetFloatValue, GXGetBoolValue, GXSetBoolValue, GXGetStringValue, GXSetStringValue, GXGetRegisterLength, GXGetRegisterValue, GXSetRegisterValue, GXRegisterFeatureCallbackByString, GXUnregisterFeatureCallbackByString It is not recommended to use function codes for reading and writing.	2024-03-20
30	V3.0.1	2.0.2405.x or above	1) Add 7.2.10 section, GX_LOG_TYPE_LIST structure	2024-06-12

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

No.	Version	API Version	Changes	Date
			2) Add 7.4.2.3 section GXSetLogType and 7.4.2.4 section GXGetLogType 3) Modify 8.2.3 section, add RAW2RGB_WEIGHT 4) Add 8.2.9 section DX_SYSTEM_PARAM constant 5) Add 8.4.48 section DxSetSystem and 8.4.49 section DxGetSystem	
31	V3.0.2	2.0.2406.x or above	1) Windows support DQBuf Interface	2024-07-09
32	V3.0.3	2.0.2412.x or above	1) Add 7.3.10.GX_GIGE_ACTION_COMMAND_RESULT 2) Add 7.4.4.14.GXGigEIssueActionCommand 7.4.4.15.GXGigEIssueScheduledActionCommand	2024-12-10
33	V3.0.4	2.0.2412.x or above	1) Add 2.1.2.1.Recommended configuration 2) Add 2.2.3.Get Interface Information 2.2.4.Get Interface Object 3) Add 7.4.4.12.GXExportConfigFileW 7.4.4.13.GXImportConfigFileW Add 7.4.5.144.GXRegisterBuffer 7.4.5.155.GXUnRegisterBuffer 4) Add 7.5.4.35.GXGetFeatureNameList 7.5.4.36.GXGetFeatureName 7.5.4.37.GXGetFeatureType (Not Recommended) 5) Add 8.2.7.Color image RGB order 6) Add 8.4.7.DxRotate90CW16B 8.4.8.DxRotate90CCW16B 8.4.12.DxSharpenMono8 8.4.26.DxImageMirror16B 7) Optimize section description information	2025-01-08
34	V3.0.5	2.0.2412.x or above	1) Add section 7.1.3 Callback Function Type 2) Add introduction to disconnection and reconnection event functions and examples related content 7.4.6.3 GXRegisterDeviceReconnectCallback 7.4.6.4 GXUnregisterDeviceReconnectCallback 7.4.6.5 GXRegisterDeviceDisconnectCallback 7.4.6.6 GXUnregisterDeviceDisconnectCallback	2025-04-17
35	V3.0.6	2.0.2412.x or above	1) Add section 3.3.11 Frame Information 2) Update section GX_FRAME_CALLBACK_PARAM, GX_FRAME_DATA and GX_FRAME_BUFFER add frame information handle related content	2025-04-30

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch

No.	Version	API Version	Changes	Date
36	V3.0.6	2.0.2412.x or above	1) Add section 8.2.10 Flat Field Correction Handle 2) Add section 8.3.6 Flat field correction function set structure 3) Add section 8.4.50 DxFFCCalculate, 8.4.51 DxFFCCreate, 8.4.52 DxFFCDestroy, 8.4.53 DxFFCGetCoefficientsSize, 8.4.54 DxFFCSetFrameCount	2025-06-10
37	V3.0.7	2.0.2412.x or above	1) Add section 2.2.10 Reconnect 2) Update section 4.1 Related Attribute, add the auto reconnection related attribute	2025-09-02
38	V3.0.8	2.0.2412.x or above	1) Add section 8.2.11 Decompression Handle 2) Add section 8.4.55 DxDecompression, section 8.4.56 DxDecompressionCreate, 8.4.57 DxDecompressionDestroy	2025-11-13
39	V3.0.9	2.0.2412.x or above	1) Add section 2.2.11 Properties Setting UI 2) Modify section 7.1.2 Handle Type 3) Add section 7.2.11 GX_WINDOWS_ID, section 7.2.12 GX_WINDOWS_SHOW_MODE, section 7.2.13 GX_CFA_METHOD, section 7.2.14 GX_IMAGE_FORMAT_TYPE, section 7.2.15 GX_VIDEO_FORMAT_TYPE, 4) Add section 7.4.4.16 GXCreateWnd, section 7.4.4.17 GXDestroyWnd, section 7.4.4.18 GXSetShowPosition, section 7.4.4.19 GXSetShowMode, section 7.4.4.20 GXSetWndTitle, section 7.4.4.21 GXShowWnd, 5) Add section 7.4.7 Image and Video Saving Interface	2026-01-13
40	V3.1.0	2.0.2412.x or above	1) Update section 7.3 Structure, add GX_FRAME_DATA_EX，delete GX_FRAME_BUFFER, GX_FRAME_CALLBACK_PARAM and GX_FRAME_DATA 2) Update section 7.5.3 Structure, add GX_FRAME_BUFFER, GX_FRAME_CALLBACK_PARAM and GX_FRAME_DATA 3) Update section 7.5.4 Interface, add GXRegisterCaptureCallback, GXGetImage, GXDQBuf, GXQBuf and GXDQAllBufs (only Linux) 4) Update the relevant description	2026-05-07

© 2026 China Daheng Group, Inc. Beijing Image Vision Technology Branch